author1hash=dea95a303b85b48b0e1f424dded5271cfamily=Elgabry, familyi=E., given=Omar, giveni=O. author1hash=b6aea1a416c89509a7df1cbb69249cb6family=Wikipedia contributors, familyi=W. author1hash=b6aea1a416c89509a7df1cbb69249cb6family=Wikipedia contributors, familyi=W.

Machine Learning Methods

Siger

November 12, 2026

Si Dieu est infini, alors je suis une partie de Dieu sinon je serai sa limite...

Contents

I Collect and Pre-process Data **9**

1 Data Cleaning **10**
1.1 Validity . . . 10
1.2 Accuracy . . . 10
1.3 Completeness . . . 10
1.4 Consistency . . . 10
1.5 Uniformity . . . 10
1.6 Inspection . . . 10
1.7 Cleaning . . . 11
1.8 Verifying . . . 11
1.9 Reporting . . . 11

II Statistics **12**

2 Fundamental Probability Concepts **13**
2.1 Basic Probability Properties . . . 13
2.1.1 What is a probability? . . . 13
2.1.2 Properties . . . 13
2.1.3 Moments . . . 14
2.1.4 Asymptotic properties . . . 15
2.1.5 Central Limit Theorem . . . 16
2.1.6 Convergence in distribution . . . 16
2.1.7 Lévy Continuity Theorem . . . 16
2.2 Bivariate case . . . 16
2.3 Distribution Function . . . 17
2.3.1 Definition of probability density function (pdf): . . . 17
2.3.2 Definition of cumulative density function (cdf): . . . 17
2.3.3 Percentile for continuous random variables. . . . 17

3 Distributions **19**
3.1 Discrete Distributions with Finite Support . . . 19
3.1.1 Bernoulli . . . 19
3.1.2 Binomial . . . 19
3.1.3 Hypergeometric . . . 20
3.1.4 Negative Hypergeometric . . . 20
3.1.5 Benford's law . . . 21
3.1.6 Zipf's law . . . 22
3.2 Discrete Distributions with Infinite Support . . . 22
3.2.1 Beta Negative . . . 22
3.2.2 Boltzmann . . . 23
3.2.3 Borel . . . 24
3.2.4 Discrete phase-type . . . 24
3.2.5 Extended negative binomial . . . 24
3.2.6 Geometric . . . 24
3.2.7 Mixed Poisson . . . 24
3.2.8 Negative binomial . . . 24

3.2.9 compound Poisson . . . 24
3.2.10 Poisson . . . 24
3.2.11 Skellam . . . 24
3.2.12 Zeta . . . 24
3.3 Continuous Distributions with Bounded Interval . . . 24
3.4 Continuous Distributions with Semi-bounded Interval . . . 24
3.5 Continuous Distributions on the Whole Real Line . . . 24
3.6 Continuous Distributions with Variable Interval . . . 24
3.7 Mixed Discrete-Continuous Distributions . . . 24
3.8 Joint Distributions: Two or More RV on the Same Sample Space . . . 24
3.9 Joint Distributions: Matrix-valued RV . . . 24

4 Bayesian Approach **25**
4.1 Components . . . 25
4.1.1 Bayesian concept learning . . . 25
4.1.2 Likelihood . . . 25
4.1.3 Prior . . . 25
4.1.4 Posterior . . . 25
4.2 Summarizing Posterior Distributions . . . 25
4.2.1 MAP (Maximum A Posteriori) estimation . . . 25
4.2.2 Credible intervals . . . 26
4.3 Bayesian Model Selection . . . 26
4.3.1 Baysian Occam's razor . . . 26
4.3.2 Computing the marginal likelihood (evidence) . . . 27
4.3.3 Bayes Factors . . . 27
4.3.4 Jeffreys-Lindley paradox . . . 28
4.4 Priors . . . 28
4.4.1 Uninformative priors . . . 28
4.4.2 Jeffreys priors . . . 28
4.4.3 Robust priors . . . 29
4.4.4 Mixture of conjugate priors . . . 29
4.5 Hierarchical and Empirical Bayes . . . 29
4.5.1 Hierarchical Bayes . . . 29
4.5.2 Empirical Bayes . . . 29
4.6 Bayesian Decision Theory . . . 30
4.6.1 Bayes estimators for common loss functions . . . 31
4.6.2 Model evaluation metrics . . . 31

5 Frequentist Approach **35**
5.1 Sampling Distribution . . . 35
5.1.1 Sampling Distributions of an estimator . . . 35
5.1.2 Bootstrap . . . 35
5.2 Frequentist Decision Theory . . . 36
5.2.1 Bayes risk . . . 37
5.2.2 Admissible estimators . . . 37
5.3 Desirable Properties of Estimators . . . 37
5.3.1 Consistent estimators . . . 37
5.3.2 Unbiased estimator . . . 37
5.3.3 Minimum variance estimators . . . 38
5.3.4 Bias-Variance Trade-off . . . 38
5.4 Empirical Risk Minimization . . . 38
5.4.1 Frequentist issue . . . 38
5.4.2 Regularized risk minimization . . . 39
5.4.3 Practical applications in Machine Learning . . . 39
5.5 Components . . . 39
5.5.1 Introduction . . . 39
5.5.2 Hypothesis Testing . . . 39
5.5.3 *p-value* . . . 40

5.5.4 Confidence intervals . . . 40
5.5.5 Multiple comparisons . . . 40
5.5.6 Effect size . . . 41
5.5.7 Sample size . . . 41
5.6 Power Analysis . . . 41
5.6.1 Purpose . . . 41
5.6.2 Error types and power . . . 41
5.6.3 Key quantities . . . 42
5.6.4 Power calculation . . . 42
5.6.5 Sample size determination . . . 43
5.6.6 Conducting a power analysis . . . 43
5.6.7 Example: two-sample t-test . . . 44

6 Common Statistical Tests **45**
6.1 Use of Statistical Tests . . . 45
6.1.1 Terms . . . 45
6.1.2 Table of statistical hypothesis test . . . 45
6.2 List of Common Statistical Tests . . . 45
6.2.1 Binomial . . . 45
6.2.2 χ^2 test . . . 46
6.2.3 Exact test of goodness-of-fit . . . 48
6.2.4 Fisher's exact test . . . 49
6.2.5 G-test . . . 51
6.2.6 Cochran's Q test . . . 54
6.2.7 Sign test . . . 56
6.2.8 Contingency coefficient: Cramér's V . . . 58
6.2.9 Contingency table from a Bayesian perspective . . . 60
6.2.10 Wilcoxon signed-rank test . . . 63
6.2.11 Mann-Whitney test . . . 67
6.2.12 Kruskal-Wallis test . . . 69
6.2.13 Friedman test . . . 72
6.2.14 Spearman test . . . 73
6.2.15 Pearson correlation coefficient . . . 75
6.2.16 Repeated-measures ANOVA . . . 77
6.2.17 1-way ANOVA . . . 80
6.2.18 T-test . . . 83

7 Data Recovery **86**
7.1 Sampling Methods . . . 86
7.1.1 Monte Carlo approximation . . . 86
7.1.2 Bootstrap . . . 86
7.1.3 Monte Carlo Inference . . . 87
7.1.4 Particle filtering . . . 87
7.1.5 Annealing methods . . . 87
7.2 Information Theory . . . 87
7.2.1 Shannon entropy . . . 87
7.2.2 Kullback-Leibler (KL) divergence . . . 88
7.2.3 Mutual information . . . 88
7.3 Key Mathematical Functions . . . 88
7.3.1 Softmax function . . . 88

8 Markov Models **90**
8.1 Markov Models and Hidden Markov Models . . . 90
8.1.1 Markov Models . . . 90
8.1.2 Hidden Markov Models . . . 92
8.1.3 Inference HMMs . . . 93
8.1.4 Learning for HMMs . . . 95
8.2 State Models . . . 96
8.2.1 Basics . . . 96

8.2.2 Time Series Forecasting . . . 97
8.2.3 Inference in LG-SSM (Linear Gaussian - State Space Models) . . . 97
8.3 Markov Random Fields . . . 98
8.3.1 Basics . . . 98

III Classical Learning **100**

9 Supervised Learning **102**
9.1 Classification . . . 102
9.1.1 Naive Bayes classifiers . . . 102
9.1.2 Linear/Quadratic Discriminant Analysis . . . 103
9.1.3 Nearest Shrunken Centroids Classifier . . . 105
9.1.4 Fisher's Linear Discriminant Analysis (FLDA) . . . 105
9.1.5 Logistic Regression . . . 106
9.2 Regression . . . 107
9.2.1 Linear Regression . . . 107
9.2.2 Generalized Linear Models (GLMs) . . . 108
9.2.3 Learning to rank . . . 109
9.2.4 Supervised PCA . . . 110
9.2.5 Partial Least Squares . . . 110
9.3 Classification and Regression . . . 111
9.3.1 Multi-task . . . 111
9.3.2 Mixture models . . . 111
9.3.3 ARD: Automatic Relevance Determination . . . 113
9.3.4 Sparse coding . . . 113
9.3.5 Support Vector Machines (SVMs) . . . 114
9.3.6 MODEL COMPARISON . . . 115
9.3.7 Gaussian Processes . . . 115
9.3.8 Classification And Regression Trees (CART) . . . 116
9.3.9 Generalized Additive models . . . 117
9.3.10 Multi-layer Perceptron . . . 118
9.4 Model Selection . . . 119
9.4.1 Bayesian Information Criterion (BIC) . . . 119
9.4.2 Akaike Information Criterion (AIC) . . . 119
9.4.3 Bayesian Variable Selection . . . 119
9.4.4 Least Angle Regression Shrinkage . . . 120
9.5 Regularization . . . 121
9.5.1 l_2 regularization . . . 121
9.5.2 l_1 regularization . . . 122
9.5.3 Noise Robustness . . . 123
9.5.4 Regularization path . . . 123
9.5.5 Coordinate Descent . . . 123
9.6 Ensemble Learning . . . 124
9.6.1 Boosting . . . 124
9.6.2 Bayes Model Averaging (that is not an Ensemble Method) . . . 125
9.6.3 Stacking . . . 126
9.6.4 Boostrap Aggregating (Bagging) . . . 126

10 Unsupervised Learning **127**
10.1 Clustering . . . 127
10.1.1 Clustering in general . . . 127
10.1.2 *K-means* algorithm . . . 128
10.1.3 Vector Quantization . . . 128
10.1.4 Factor Analysis . . . 129
10.1.5 Mixture of Factor Analyzers . . . 129
10.1.6 Canonical Correlation Analysis . . . 129
10.1.7 Independent Component Analysis (ICA) . . . 130
10.1.8 DBSCAN: Density-Based Spatial Clustering of Applications with Noise . . . 130

10.1.9 Affinity Propagation . . . 131
10.1.10 Dirichlet Process . . . 131
10.1.11 Spectral Clustering . . . 132
10.1.12 Agglomerative Clustering . . . 132
10.1.13 Bayesian Hierarchical Clustering . . . 132
10.2 Association . . . 133
10.2.1 Directed graphical models (Bayes nets) . . . 133
10.2.2 Kernels . . . 133
10.2.3 Latent Dirichlet Allocation . . . 135
10.3 Dimensionality Reduction . . . 135
10.3.1 Principal Components Analysis . . . 135
10.3.2 PCA for categorical data . . . 136

11 Semi-supervised Learning **137**

12 Reinforcement Learning **138**
12.1 Markov Decision Processes . . . 138
12.1.1 Setting . . . 138
12.1.2 Value Functions and Bellman Equations . . . 138
12.2 Value-based Methods . . . 139
12.2.1 Purpose . . . 139
12.2.2 Monte Carlo and Temporal Difference . . . 139
12.2.3 SARSA and Q-learning . . . 139
12.3 Policy Gradient Methods . . . 140
12.3.1 Purpose . . . 140
12.3.2 Policy Gradient Theorem . . . 140
12.3.3 Variance Reduction . . . 140
12.4 Proximal Policy Optimization (PPO) . . . 141

13 Optimization Methods **142**
13.1 Optimization of Loss Functions . . . 142
13.1.1 Gradient Descent . . . 142
13.1.2 Newton's method . . . 142
13.1.3 Online learning and stochastic optimization . . . 143
13.1.4 EM Algorithm . . . 143
13.1.5 Backpropagation Algorithm . . . 144
13.2 Variational Inference . . . 144
13.2.1 Basics . . . 144
13.2.2 Variational Bayes . . . 145
13.2.3 Variational Bayes EM . . . 145
13.2.4 Gibbs sampling . . . 145
13.3 Numerical Computation . . . 145
13.3.1 Numerical Computation . . . 145
13.3.2 Jacobian and Hessian matrices . . . 145
13.3.3 Optimization technique . . . 145

IV Deep Learning 146

14 Optimization **147**
14.1 Back-propagation . . . 147
14.1.1 Purpose . . . 147
14.1.2 Modeling . . . 147
14.2 Optimizers . . . 148
14.2.1 Purpose . . . 148
14.2.2 Momentum . . . 148
14.2.3 Adaptive learning rates: RMSProp and Adam . . . 148
14.2.4 Learning-rate schedules . . . 149
14.2.5 Gradient clipping . . . 149

15 Regularization **150**
15.1 Norm Penalties . . . 150
15.1.1 L^2 parameter regularization . . . 150
15.1.2 L^1 parameter regularization . . . 150
15.2 Data Augmentation . . . 150
15.3 Early Stopping . . . 151
15.4 Drop out . . . 151

16 Types of Neural Networks **153**
16.1 Feed Forward Neural Networks . . . 153
16.2 Convolutional Neural Networks . . . 154
16.2.1 Components of CNN . . . 154
16.3 Recurrent and Recursive Neural Networks . . . 155
16.3.1 Recurrent Neural Networks . . . 155
16.3.2 Encoder-Decoder Sequence-to-Sequence Architectures . . . 156
16.3.3 The Challenge of Long-Term Dependencies . . . 157
16.3.4 Leaky Units . . . 157
16.3.5 The Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) . . . 157
16.4 Transformer Networks . . . 159
16.4.1 Motivations . . . 159
16.4.2 Encoder and Decoder stacks . . . 159
16.4.3 Attention . . . 160
16.4.4 Masking . . . 161
16.4.5 Positional Encoding . . . 161
16.4.6 Residual Connections and Layer Normalization . . . 161
16.4.7 Feed-Forward Sub-layer . . . 162
16.4.8 Embeddings and Output Layer . . . 162
16.4.9 Why Self-Attention instead of Recurrence . . . 162
16.4.10 Training Recipe of the Original Paper . . . 162
16.4.11 Model Families . . . 163
16.5 Autoencoders . . . 163
16.6 Generative Adversarial Networks . . . 163
16.7 Graph Neural Networks . . . 163

V Large Language Models **164**

17 Tokens and Embeddings **165**
17.1 Tokenization . . . 165
17.2 Embeddings . . . 165

18 Language Modeling **167**
18.1 Autoregressive Factorization . . . 167
18.2 Training Objective . . . 167
18.3 Perplexity . . . 167

19 Decoder-only Transformers (GPT) **169**
19.1 Architecture . . . 169
19.2 Number of Parameters . . . 169
19.3 Parameters versus Context . . . 169

20 Text Generation **170**
20.1 Autoregressive Decoding . . . 170
20.2 Decoding Strategies . . . 170
20.3 KV Cache . . . 171

21 Training Large Language Models **172**
21.1 Pretraining . . . 172
21.2 Scaling Laws . . . 172
21.3 Supervised Fine-Tuning (SFT) . . . 173
21.4 Parameter-Efficient Fine-Tuning: LoRA . . . 173

22 Alignment with Human Feedback **174**
22.1 Why Human Feedback . . . 174
22.2 Reward Model . . . 174
22.3 RL Fine-Tuning with PPO . . . 174
22.4 Direct Preference Optimization (DPO) . . . 175
22.5 Verifiable Rewards and GRPO . . . 175
22.6 Limitations . . . 175

23 Retrieval-Augmented Generation **176**
23.1 Principle . . . 176
23.2 Embeddings for Retrieval . . . 176
23.3 Vector Search . . . 176
23.4 Chunking . . . 177
23.5 Hybrid Search and Reranking . . . 177
23.6 Evaluation . . . 177
23.7 Failure Modes . . . 177
23.8 RAG, Fine-tuning and Long Context . . . 178

24 LLM Agents **179**
24.1 From a Model to an Agent . . . 179
24.2 Tool Use . . . 179
24.3 The Agent Loop: ReAct . . . 179
24.4 Planning, Reflection and Multiple Agents . . . 180
24.5 Memory and Context . . . 180
24.6 Reliability and Evaluation . . . 180
24.7 Safety . . . 180

VI Overall Issues **181**

25 High Dimensions **182**
25.0.1 Dimension Increase . . . 182
25.0.2 Curse of dimensionality . . . 182

Part I

Collect and Pre-process Data

Chapter 1

Data Cleaning

[**omar_elgabry_the_ultimate_guide_to_data_cleaning**]

1.1 Validity

- **Data-Type Constraints**: for a given column a fixed data-type must be associated with.
- **Range Constraints**: only a range of values should be taken.
- **Mandatory Constraints**: some columns cannot be empty.
- **Unique Constraints**: across a given dataset a field or a combination of variables.
- **Foreign-key constraints**: a foreign key column cannot have a value that does not exist in the primary key.
- **Regular expression patterns**: text fields that have to follow a given alphanumerical pattern.
- **Cross-field validation**: consistency of values, for example considering a given man, his birth date have to be older than his death date.

1.2 Accuracy

The degree to which the data is close to the true value.

1.3 Completeness

The degree to which the all the required data is known.

1.4 Consistency

The degree to which the data is consistent, within the same data set or across multiple data sets.

1.5 Uniformity

The degree to which the data is specified using the same unit of measure.

1.6 Inspection

Detect unexpected behavior in the data.

- **Data profiling**: summary statistics about the data, see *ydata-profiling* in Python.
- **Visualizations**: visualize the data using statistical metrics, see *plotly*
- **Software packages**: to note and check the constraints regarding the data see *pydeequ*

1.7 Cleaning

Fix or remove anomalies discovered in the above phase.

- **Irrelevant Data**: ask to the expert what can be the unnecessary columns, check them and remove them if they are not useful.
- **Duplicates**
- **Type conversion**: make sure the appropriate data type is associated with a given column.
- **Syntax errors**: white spaces, pad strings ...
- **Standardize**: same unit across the dataset, same pattern for text.
- **Scaling/Transformation**: in order to compare different scores for example.
- **Normalization**: useful for some statistical methods.
- **Missing values**:
  - Drop: only if the missing values in a column rarely and randomly occur.
  - Impute: many methods,
    - *mean* is relevant when data is not skewed
    - *median* otherwise
    - *hot-deck* imputes from a randomly selected similar record
    - *polynomial interpolation* approximate the data with a polynomial
    - *linear regression* imputes from prediction
    - *k-nearest* imputes depending on a set of closet observations
  - Flag: let the missing value as it is.
- Outliers: Remove outliers only if they are harmful for the chosen model.
- In-record & cross-datasets errors: fix non-consistent situations like married kids, quantity being different of the one when we compute using other columns.

1.8 Verifying

Check correctness of the cleaning phase.

1.9 Reporting

Report about changes made, using one of the software summarising the data quality for example.

Part II

Statistics

Chapter 2

Fundamental Probability Concepts

2.1 Basic Probability Properties

2.1.1 What is a probability?

It is a mathematical measure of the uncertainty of a given event.

Objectivist interpretation [wiki:probability_interpretation] : assigns numbers describing some objective state, *Frequentist* interpretation claiming that the probability of a random event is quantified by the relative frequency in a given experiment.

Subjectivist interpretation [wiki:probability_interpretation] : assigns numbers quantifying the degree of belief that a given event occurs. *Bayesian* interpretation uses expert knowledge considered as subjective and represented by the prior, as well as experimental data represented by the likelihood. The normalized product of the 2 above quantity is the posterior probability distribution containing both expert knowledge and experimental data.

2.1.2 Properties

Event and its opposite $\mathbb{P}(\{A\}) + \mathbb{P}\left(\{\overline{A}\}\right) = 1$

Not necessary mutually exclusive events $\mathbb{P}(\{A \cup B\}) = \mathbb{P}(\{A\}) + \mathbb{P}(\{B\}) - \mathbb{P}(\{A \cap B\})$

Independent events $\mathbb{P}(\{A \cap B\}) = \mathbb{P}(\{A\}) \times \mathbb{P}(\{B\})$

Conditional Probability $\mathbb{P}(A|B) = \dfrac{\mathbb{P}(A \cap B)}{\mathbb{P}(B)}$, keep in mind that it comes from the fact that considering case where A occurs knowing that B occurred means reducing probability space Ω to B. In combinatorics we could think about $\mathbb{P}(A|B) = \dfrac{|A \cap B|}{|B|} = \dfrac{\frac{|A \cap B|}{|\Omega|}}{\frac{|B|}{|\Omega|}} = \dfrac{\mathbb{P}(A \cap B)}{\mathbb{P}(B)}$

Note on conditioning on hypotheses The notation $\mathbb{P}(A|B)$ assumes both A and B are events (or outcomes of random variables) with well-defined probabilities in the sample space. However, in hypothesis testing, the interpretation of conditioning differs between paradigms:

- **Frequentist framework**: A hypothesis H_0 is not a random variable and has no probability — it is either true or false. The notation $\mathbb{P}(\text{data}|H_0)$ means "probability of the data computed in the probability space where H_0 holds", i.e., assuming H_0 is true. This is NOT decomposable as $\frac{\mathbb{P}(\text{data} \cap H_0)}{\mathbb{P}(H_0)}$ because $\mathbb{P}(H_0)$ is undefined. **The conditioning is on an *assumption*, not an event**.
- **Bayesian framework**: Hypotheses can be assigned probabilities via prior distributions. Here, $\mathbb{P}(H_0|\text{data})$ is meaningful and can be computed using Bayes' theorem:

$$\mathbb{P}(H_0|\text{data}) = \frac{\mathbb{P}(\text{data}|H_0) \times \mathbb{P}(H_0)}{\mathbb{P}(\text{data})}$$

In this case, H_0 is treated as a random variable (or event) and $\mathbb{P}(H_0)$ represents our prior belief about the hypothesis.

In summary: $\mathbb{P}(A|B)$ with the decomposition $\mathbb{P}(A|B) = \frac{\mathbb{P}(A \cap B)}{\mathbb{P}(B)}$ applies when both A and B are events with defined probabilities. When conditioning on a *hypothesis* **in frequentist statistics, the conditioning symbol | represents an assumption rather than an event**, and Bayes' decomposition does not apply.

Law of Total Probability $\begin{cases} (B_i)_{1 \leq i \leq n} : \text{ partition of a sample } \mathcal{S} \\ \forall i \in [\![1, n]\!],\ \mathbb{P}(\{B_i\}) \neq 0 \end{cases} \Rightarrow \mathbb{P}(A) = \sum_{i=1}^{n} \mathbb{P}(B_i)\,\mathbb{P}(A|B_i)$

Bayes' Theorem Using Law of Total Probability:

$$\begin{cases} (B_i)_{1 \leq i \leq n} : \text{ partition of a sample } \mathcal{S} \\ \forall i \in [\![1, n]\!],\ \mathbb{P}(\{B_i\}) \neq 0 \end{cases} \Rightarrow \mathbb{P}(B_i|A) = \frac{\mathbb{P}(A|B_i) \times \mathbb{P}(B_i)}{\sum_{k=1}^{n} \mathbb{P}(A|B_k)\,\mathbb{P}(B_k)}$$

It is used to compute the probability of a cause given its effect.

2.1.3 Moments

They are certain quantitative measures related to the shape of the function's graph [**wiki:moments_mathematics**]. Recall that in physics to compute moment you need to store for each point the product resulting from the distance between mass center and that point with force applied on that point and finally sum them each other. $(\mu = \sum_x d_c(x) \times F(x))$.

n^{th} moments of a random variable: The nth moment about the origin of a random variable X as denoted by $E(X^n)$, is defined to be:

$$\mathbb{E}(X^n) = \begin{cases} \sum_{x \in R_X} x^n f(x) \text{ if } X \text{ is discrete} \\ \int_{-\infty}^{\infty} x^n f(x) dx \text{ if } X \text{ is continuous} \end{cases}$$

Expected value: The expected value of a random variable X as denoted by $E(X)$, is defined to be:

$$\mathbb{E}(X) = \begin{cases} \sum_{x \in R_X} x f(x) \text{ if } X \text{ is discrete} \\ \int_{-\infty}^{\infty} x f(x) dx \text{ if } X \text{ is continuous} \end{cases}$$

After normalized this moment by total mass we have the center of mass.

Variance : Let X be a random variable with mean μ_X. The variance of X denoted by $\mathbb{V}(X)$ or σ_X^2 is defined by:

$$\mathbb{V}(X) = \mathbb{E}\left([X - \mu_X]^2\right)$$

After normalized this moment by total mass we have the moment of inertia.
If X is a random variable with mean μ_X and variance σ_X^2 then:

$$\sigma_X^2 = \mathbb{E}(X^2) - \mu_X^2$$

And:

$$\mathbb{V}\left(\sum_{i=1}^{n} a_i X_i\right) = \mathbb{E}\left(\left(\sum_{i=1}^{n} a_i X_i\right)^2\right) - \left(\mathbb{E}\left(\sum_{i=1}^{n} a_i X_i\right)\right)^2 = \mathbb{E}\left(\sum_{i=1}^{n}\sum_{j=1}^{n} a_i a_j X_i X_j\right) - \left(\sum_{i=1}^{n} a_i \mathbb{E}(X_i)\right)^2$$

$$= \sum_{i=1}^{n}\sum_{j=1}^{n} a_i a_j \mathbb{E}(X_i X_j) - \sum_{i=1}^{n}\sum_{j=1}^{n} a_i a_j \mathbb{E}(X_i)\mathbb{E}(X_j) = \sum_{i=1}^{n}\sum_{j=1}^{n} a_i a_j \left(\mathbb{E}(X_i X_j) - \mathbb{E}(X_i)\mathbb{E}(X_j)\right)$$
$$= \sum_{i=1}^{n}\sum_{j=1}^{n} a_i a_j \mathrm{Cov}(X_i, X_j) = \sum_{i=1}^{n} a_i^2 \mathbb{V}(X_i) + 2\sum_{i=1}^{n}\sum_{j>i}^{n} a_i a_j \mathrm{Cov}(X_i, X_j)$$

Remember that

$$Cov(X,Y) = \mathbb{E}([X_i - \mu_X]) \times \mathbb{E}([Y_i - \mu_Y]) = \mathbb{E}(XY) - \mu_X \mu_Y$$

Skewness and Kurtosis

- Skewness: $\mathbb{E}\left(\left[\frac{X - \mu_X}{\sigma_X}\right]^3\right)$, indicates the direction (negative → left tail is longer, positive → right tail is longer) and relative magnitude of a distribution's deviation from the normal distribution.
- Kurtosis: $\mathbb{E}\left(\left[\frac{X - \mu_X}{\sigma_X}\right]^4\right)$, measures the outliers, data within one standard deviation will not contribute a lot to the kurtosis values conversely data exceeding one standard deviation will contribute a lot because of the fourth power.

2.1.4 Asymptotic properties

Markov inequality

$$X \geq 0 \Rightarrow \mathbb{P}(\{X \geq t\}) \leq \frac{\mathbb{E}(X)}{t}$$

Chebychev inequality allows to find an estimate of the area between the values $\mu - k\sigma$ and $\mu + k\sigma$ for some given $k > 0$, showing that the area under $f(x)$ on the interval $[\mu - k\sigma, \mu + k\sigma]$ is at least $1 - \frac{1}{k^2}$. Let X be a random variable with probability density function $f(x)$. If μ and $\sigma > 0$ are the mean and standard deviation of X then:

$$\boxed{\mathbb{P}(\{|X - \mu| < k\sigma\}) \geq 1 - \tfrac{1}{k^2}}$$

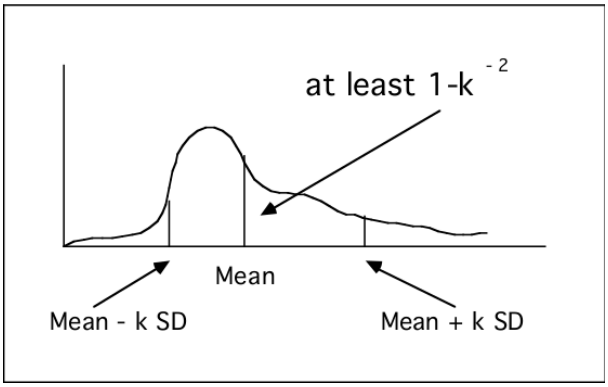


Figure 2.1: Illustration of Chebychev inequality

Theorem weak law of large numbers: Let $(X_i)_{1 \leq i \leq n}$: independent & identically distributed RV

$$\boxed{\forall \epsilon \in \mathbb{R}_+ : \lim_{n \to \infty} \mathbb{P}\left(\left\{\left|\overline{S}_n - \mu\right| \geq \epsilon\right\}\right) = 0 \text{ with } \overline{S}_n = \frac{1}{n}\sum_{i=1}^{n} X_i}$$

Convergence in probability Suppose $(X_i)_{1 \leq i \leq n}$ is a sequence of random variables defined on a sample space S. The sequence "converges in probability" to the random variable X if, for any $\epsilon > 0$

$$\lim_{n \to \infty} \mathbb{P}(\{|X_n - X| < \epsilon\}) = 1$$

Convergence almost surely Suppose the RV X and $(X_i)_{1\leq i\leq n}$ is a sequence of random variables defined on a sample space S. The sequence $X_n(\omega)$ "converges almost surely" to $X(\omega)$ if

$$\mathbb{P}\left(\left\{w \in S | \lim_{n\to\infty} X_n(\omega) = X(\omega)\right\}\right) = 1$$

Properties

- For a Bernoulli distribution, $\overline{S}_n$ converges in probability to p
- For a Normal distribution, $\overline{S}_n$ converges almost surely to μ

2.1.5 Central Limit Theorem

The central limit therorem (Lindeberg-Levy Theorem) states that for any population distribution, the distribution of the standardized sample mean is approximately standard normal with better approximations obtained with the larger sample size.

$$\begin{cases} (X_i)_{1\leq i\leq n} \hookrightarrow ?(\mu, \sigma^2) \\ n \to \infty \end{cases} \Rightarrow \frac{\overline{X} - \mu}{\frac{\sigma}{\sqrt{n}}} \hookrightarrow \mathcal{N}(0,1)$$

2.1.6 Convergence in distribution

Consider X with its cumulative density function F and $(X_i)_{1\leq i\leq n}$ with their cdf $(F_i)_{1\leq i\leq n}$:

$$\lim_{n\to\infty} F_n(x) = F(x) \Rightarrow X_n \text{ "converges in distribution" to } X$$

2.1.7 Lévy Continuity Theorem

$$\begin{cases} (X_i)_{1\leq i\leq n} \text{ RV} \\ (F_i)_{1\leq i\leq n} \text{ distribution functions} \\ (M_{X_i})_{1\leq i\leq n} \text{ moment generating function} \end{cases} \quad \forall t \in [-h,h] \lim_{n\to\infty} M_{X_n}(t) = M_X(t) \Rightarrow \lim_{n\to\infty} F_n(x) = F(x)$$

2.2 Bivariate case

Joint probability density function Let $(X,Y) : (\Omega_X, \Omega_Y) \to (R_X, R_Y)$ and $f : R_X \times R_Y \to \mathbb{R}$

$$\forall (x,y) \in R_X \times R_Y, f(x,y) = \mathbb{P}(\{X = x, Y = y\}) \Leftrightarrow$$
f is the joint probability density function for X and Y

Marginal probability density function Let for all $(x,y) \in R_X \times R_Y$: $f(x,y)$ be the joint probability density of X and Y

$$\begin{cases} f_X(x) = \int_{-\infty}^{\infty} f(x,y)dy \text{ is the marginal probability density of } X \\ f_Y(y) = \int_{-\infty}^{\infty} f(x,y)dx \text{ is the marginal probability density of } Y \end{cases}$$

Joint cumulative probability distribution function Let $F : \mathbb{R}^2 \to \mathbb{R}$

$$\forall (x,y) \in \mathbb{R}^2, F(x,y) = \mathbb{P}(\{X \leq x, Y \leq y\}) = \int_{-\infty}^{y}\int_{-\infty}^{x} f(u,v)dudv \Leftrightarrow$$
F is the joint cumulative probability density function for X and Y

From the fundamental theorem of calculus: $f(x,y) = \dfrac{\partial^2 F(x,y)}{\partial x \partial y}$

Conditional expectation The conditional mean of X given $Y = y$ is defined as:

$$\mathbb{E}(X|y) = \begin{cases} \sum_{x \in R_X} xg(x/y) \Leftarrow X \text{ discrete} \\ \int_{-\infty}^{\infty} xg(x/y)dx \Leftarrow X \text{ continuous} \end{cases}$$

Properties:

$$\begin{cases} \mathbb{E}_X\left(\mathbb{E}_{y|x}(Y|X)\right) = \mathbb{E}_y(Y) \\ \mathbb{E}(Y|\{X = x\}) = \mu_Y + \rho\frac{\sigma_Y}{\sigma_X}(x - \mu_X) \end{cases}$$

Conditional Variance

$$\begin{cases} \mathbb{V}(Y|x) = \mathbb{E}\left(Y^2|x\right) - \mathbb{E}(Y|x)^2 \\ \mathbb{E}_x\left(\mathbb{V}(Y|X) = (1 - \rho^2)\mathbb{V}(Y)\right) \end{cases}$$

Conditional Independence $X \perp Y[Z \Leftrightarrow \mathbb{P}(X, Y|Z) = \mathbb{P}(X|Z)\,\mathbb{P}(Y|Z)$

The chain rule of conditional probabilities Any joint probability distribution over many random variables may be decomposed into conditional distribution over only one variable:

$$\mathbb{P}(x_{1:T}) = \mathbb{P}(x_1)\prod_{t=2}^{T}\mathbb{P}(x_t|x_{1:t-1})$$

2.3 Distribution Function

2.3.1 Definition of probability density function (pdf):

Let R_X be the space of the random variable X. The function: $f : R_X \to \mathbb{R}$ defined by:

$f(x) = \mathbb{P}(\{X = x\})$ if X is discrete.

$f(x) = \mathbb{P}(\{X \in A\}) = \int_A f(x)dx$ if X is continuous, with A a set of real numbers.

is called probability density function of X.

2.3.2 Definition of cumulative density function (cdf):

Let R_X be the space of the random variable X. The function: $F : R_X \to \mathbb{R}$ defined by:

$F(x) = \mathbb{P}(\{X \leq x\})$ if X is discrete.

$F(x) = \mathbb{P}(\{X \leq x\}) = \int_{-\infty}^{x} f(t)dt$ if X is continuous, with A a set of real numbers.

2.3.3 Percentile for continuous random variables.

Let $p \in [0; 1]$, a $100p^{th}$ percentile of the distribution of a random variable X is $q \in \mathbb{R}$ satisfying:

$$\mathbb{P}(\{X \leq q\}) = p$$

(Recall that the F is a monotonically increasing function, then it has an inverse F^{-1})

$$q = F^{-1}(p)$$

A $100p^{th}$ is a measure of location for the probability distribution in the sense that q divides the distribution of the probability mass into 2 parts, one having probability mass p and other having probability mass $1 - p$

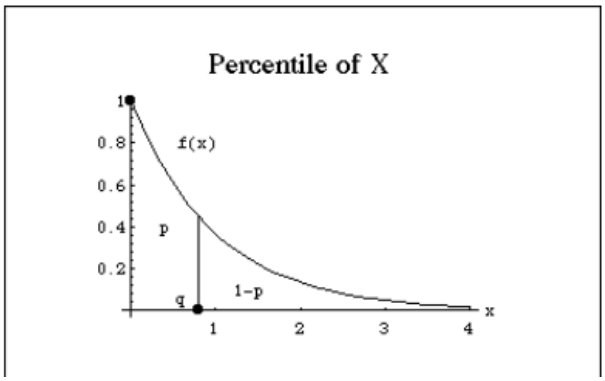


Figure 2.2: Percentile

The 50th percentile of any distribution is called median of the distribution.

Chapter 3

Distributions

3.1 Discrete Distributions with Finite Support

3.1.1 Bernoulli

Purpose To estimate the outcome of an experience providing binary outcomes.

Theory

Parameters p such that $\mathbb{P}(X = 1) = p$.

3.1.2 Binomial

Purpose It estimates the number of successes in a sequence of n independent experiments having binary outcomes.

Parameters

- n: number of independent experiments
- p: probability of success, $\mathbb{P}(X = 1) = p$
- $f(k, n, p) = \mathbb{P}(X = k) = \binom{n}{k} p^k (1-p)^{n-k}$ the probability mass function giving the probability to get exactly k successes over n independent experiments.

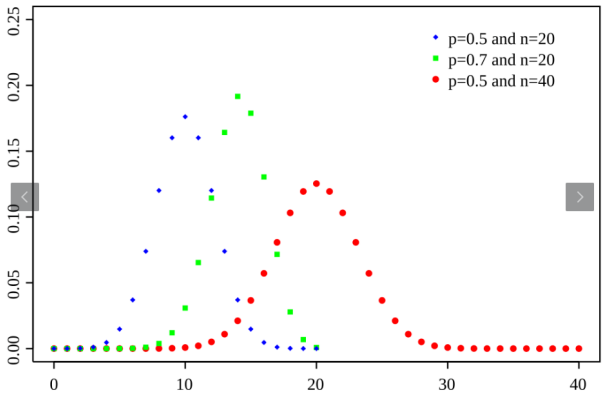


Figure 3.1: Binomial probability mass function

Approximation

- *Normal distribution*: when n becomes large enough the *skew* gets lower and a reasonable approximation of $\mathcal{B}(n,p) \sim \mathcal{N}\left(np, np(1-p)\right)$
- *Poisson distribution*: when $n \to \infty \wedge np \to C$ (n should be sufficiently large and p sufficiently small) $\mathcal{B}(n,p) \sim \mathcal{P}(np)$
- *Beta distribution*: when $\alpha = k+1 \wedge \beta = n-k+1$ we have $Beta(p,\alpha,\beta) = (n+1)Binomial(k,n,p)$

3.1.3 Hypergeometric

Purpose It describes the probability of k successes (random draws for which an object has a given feature) in n draws *without replacement* from a finite population of size N that contains exactly K objects with that feature.

Theory

Parameters

- N (population size) $\in \mathbb{N}$
- K (count of objects having the sought feature) $\in [\![0, N]\!]$
- n (count of draws) $\in [\![0, N]\!]$

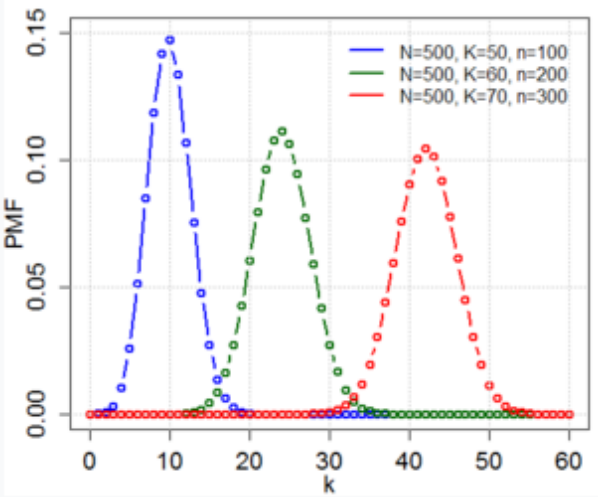


Figure 3.2: Hypergeometric probability mass function

Probability mass function $f(k;N,n,K) = \mathbb{P}(X=k) = \frac{\binom{K}{k}\binom{N-K}{n-k}}{\binom{N}{n}}$

3.1.4 Negative Hypergeometric

Paragraph It describes the likelihood of k successes in a sample with exactly r failures.

Theory

Parameters

- N (population size) $\in \mathbb{N}$
- K (count of objects having the sought feature) $\in [\![0, N]\!]$
- r (count of failures) $\in [\![0, N-K]\!]$

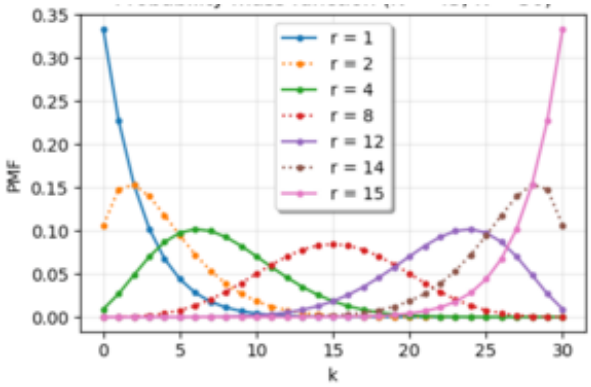


Figure 3.3: Negative Hypergeometric probability mass function

Probability mass function $f(k; N, r, K) = \mathbb{P}(X = k) = \dfrac{\binom{k+r-1}{k}\binom{N-r-k}{K-k}}{\binom{N}{K}}$

3.1.5 Benford's law

Purpose It tends to describe the probability that a number, in a real-life set, starts with a given digit (0-9), it follows an observation that in many real-life sets of numerical data the leading digit is likely to be small.
It seems that real-world distributions that span several orders of magnitude rather uniformly like stock-market prices are likely to satisfy Benford's law very accurately, however distribution mostly or entirely within one order of magnitude like IQ scores is unlikely to satisfy Benford's law very accurately.

Theory

Parameters $d \in [\![1, 9]\!]$ the leading digit

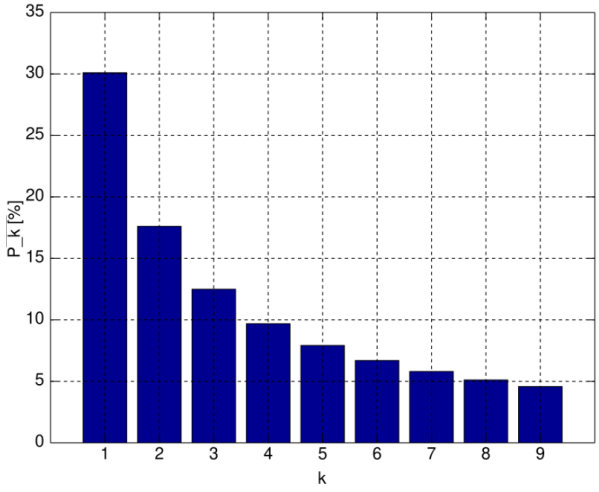


Figure 3.4: Benford's law probability density function

Probability mass function $f(d) = \mathbb{P}(X = d) = \log_{10}(d+1) - \log_{10}(d) = \log_{10}(\frac{d+1}{d}) = \log_{10}(1 + \frac{1}{d})$

3.1.6 Zipf's law

Purpose This empirical law states that when a list of measured values is sorted in decreasing order the value of the n-th entry is often approximately inversely proportional to n.

Theory

Parameters Let's consider the generalized case where we have an inverse power law with exponent s instead of 1

- $s \in \mathbb{R}_+$ exponent of generalization
- $N \in \mathbb{N}_*$ count of elements at which we assign a rank k

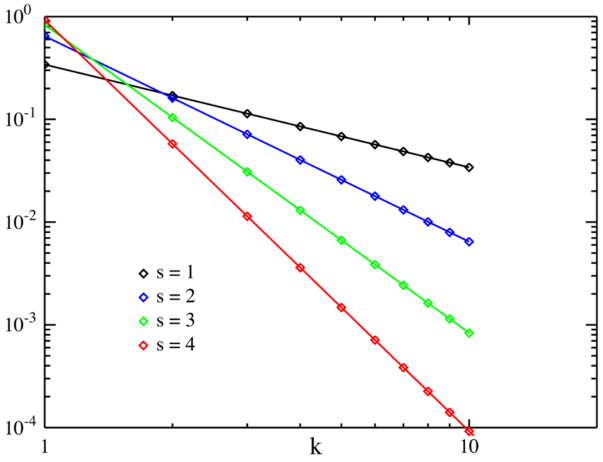


Figure 3.5: Zipf's law probability mass function

Consider $H_{N,s} = \sum_{k=1}^{N} \frac{1}{k^s}$ the Nth harmonic number.

$$f(k; N, s) = \begin{cases} \frac{1}{H_{N,s}} \times \frac{1}{k^s} & \Leftarrow k \in [\![1, N]\!] \\ 0 & \Leftarrow (k < 1) \vee (N < k) \end{cases}$$

Application

- Natural Language: $word_frequency \propto \frac{1}{word_rank}$

3.2 Discrete Distributions with Infinite Support

3.2.1 Beta Negative

Purpose It describes the probability of failures needed to get r successes in a sequence of independent Bernoulli trials. The probability p remains constant within a given a Bernoulli experiment however varies across different experiments following a beta distribution.

Theory

Parameters

- $\alpha \in \mathbb{R}_+$ parameter of Beta distribution
- $\beta \in \mathbb{R}_+$ parameter of Beta distribution

Probability mass function Consider $f_{X|p}(k|r,q)$ the negative binomial density and $f_p(q|\alpha,\beta)$ the beta density we obtain the probability mass function $f(k|\alpha,\beta,r)$ by marginalisation

$$\begin{aligned}
f(k|\alpha,\beta,r) &= \int_0^1 f_{X|p}(k|r,q) \times f_p(q|\alpha,\beta)dq \\
&= \int_0^1 \binom{k+r-1}{k}(1-q)^k q^r \times \frac{q^{\alpha-1}(1-q)^{\beta-1}}{B(\alpha,\beta)}dq \\
&= \frac{1}{B(\alpha,\beta)}\binom{k+r-1}{k}\int_0^1 q^{\alpha+r-1}(1-q)^{\beta+k-1}dq \\
&= \frac{1}{B(\alpha,\beta)}\binom{k+r-1}{k}\frac{\Gamma(\alpha+r)\Gamma(\beta+k)}{\Gamma(\alpha+\beta+k+r)} \\
&= \frac{1}{B(\alpha,\beta)}\binom{k+r-1}{k}B(\alpha+r,\beta+k)
\end{aligned}$$

More generally $f(x) = \frac{\Gamma(r+k)}{k!\Gamma(r)} \times \frac{B(\alpha+r,\beta+k)}{B(\alpha,\beta)}$

3.2.2 Boltzmann

Purpose This distribution known as well as Gibbs distribution describes the probability that a system will be in certain state as a function of that state's energy and the temperature of the system.

Theory

Parameters

- ϵ_i the energy of the state i
- k the Boltzmann constant
- T the temperature

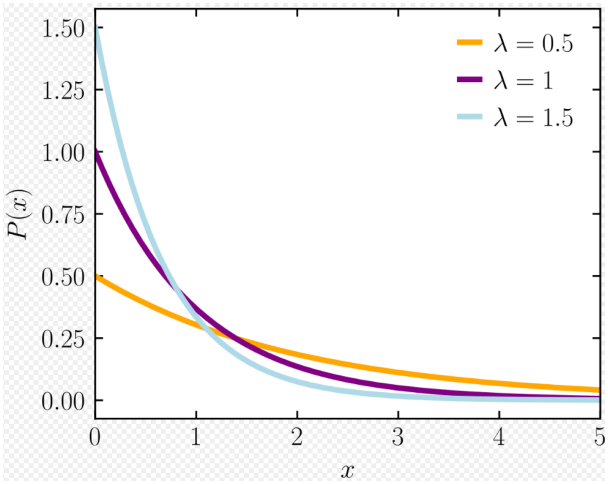


Figure 3.6: Boltzmann probability function, note that $lambda = kT$

Probability mass function Assume p_i the probability of the system being in state i, $p_i \propto e^{-\frac{\epsilon_i}{kT}}$. More precisely we have $f(i) = \mathbb{P}(X = i) = \dfrac{e^{-\frac{\epsilon_i}{kT}}}{\sum_{j=1}^{M} e^{-\frac{\epsilon_j}{kT}}}$

3.2.3 Borel

3.2.4 Discrete phase-type

3.2.5 Extended negative binomial

3.2.6 Geometric

3.2.7 Mixed Poisson

3.2.8 Negative binomial

3.2.9 compound Poisson

3.2.10 Poisson

3.2.11 Skellam

3.2.12 Zeta

3.3 Continuous Distributions with Bounded Interval

3.4 Continuous Distributions with Semi-bounded Interval

3.5 Continuous Distributions on the Whole Real Line

3.6 Continuous Distributions with Variable Interval

3.7 Mixed Discrete-Continuous Distributions

3.8 Joint Distributions: Two or More RV on the Same Sample Space

3.9 Joint Distributions: Matrix-valued RV

Chapter 4

Bayesian Approach

4.1 Components

4.1.1 Bayesian concept learning

Let be $\mathcal{D}$ the data and in

- discrete approach h the hypothesis taken in account (Binomial, ...)
- in continuous approach: m, θ the model (Gaussian, ...) ant its parameters associated with

4.1.2 Likelihood

the probability to get the observed data:

- discrete approach: $p\left(\mathcal{D}|h\right)$
- continuous approach: $p\left(\mathcal{D}|\theta, m\right)$

4.1.3 Prior

the probability of our hypothesis, many prior can be used, and this **subjective** aspect of Bayesian reasoning is a source of much controversy.

- in discrete approach: $p(h)$
- in continuous approach: $p(\theta|m)$

4.1.4 Posterior

The posterior is simply the likelihood times the prior, normalized.

- in discrete approach: $p\left(h|\mathcal{D}\right) = \dfrac{p\left(\mathcal{D}|h\right) \times p(h)}{\sum\limits_{h' \in \mathcal{H}} p\left(\mathcal{D}|h'\right) p(h')}$
- in continuous approach: $p\left(\theta|\mathcal{D}, m\right) = \dfrac{p\left(\mathcal{D}|\theta, m\right) \times p(\theta|m)}{\int_{\theta' \in \Theta} p\left(\mathcal{D}|\theta', m\right) p(\theta'|m) d\theta'}$

4.2 Summarizing Posterior Distributions

4.2.1 MAP (Maximum A Posteriori) estimation

Although most appropriate choice for:

$$\begin{cases} \text{Real valued quantity} & \rightarrow \textit{posterior median or mean} \\ \text{Discrete} & \rightarrow \textit{vector of posterior marginals} \end{cases}$$

The most popular choice is *posterior mode* aka MAP, because it reduces to optimization problems for which efficient algorithms often exist.
Some point to be aware about MAP:

- No measure of uncertainty
- Plugging in the MAP estimate can result in overfitting
- The mode is an untypical point, unlike the mean or median the mode is a point of measure 0, it does not take the volume of the space into account.
- MAP estimation is not invariant to reparameterization, for example passing from centimeters to inches can break things.)
  The MLE does not suffer from this since the likelihood is a function not a probability density

4.2.2 Credible intervals

With point estimates, we want a measure of confidence.

$$C_\alpha(\mathcal{D}) = (l, u) : \mathbb{P}(\{l \leq \theta \leq u | \mathcal{D}\}) \geq 1 - \alpha$$

In general, credible intervals are usually what people want to compute but confidence intervals are usually what they actually compute, because most people are taught frequentist statistics but not Bayesian statistics.
Sometimes with central intervals there might be points be outside the CI which have higher probability density.
More formally p^* such that:

$$1 - \alpha = \int_{\theta : p(\theta|\mathcal{D}) > p^*} p(\theta|\mathcal{D}) d\theta$$

Then the HPD such that:

$$\mathcal{D} = \{\theta : p(\theta|\mathcal{D}) \geq p^*\}$$

4.3 Bayesian Model Selection

A more efficient approach than cross-validation, meaning fitting k times each model, is to compute the posterior over models.

$$p(m|\mathcal{D}) = \frac{p(\mathcal{D}|m)p(m)}{\sum_{m \in \mathcal{M}} p(m, \mathcal{D})}$$

From this we can compute the MAP model $\hat{m} = \arg\max_m p(m|\mathcal{D})$

Then we have the marginal likelihood: $p(\mathcal{D}|\hat{m}) = \int p(\mathcal{D}|\theta)p(\theta|\hat{m})d\theta$. This equation means that considering the hyper-parameter $\hat{m}$, that governs value taken by parameter θ we do the weighted sum of likelihood over all possible values of θ with a weight of $p(\theta|\hat{m})$

4.3.1 Baysian Occam's razor

In integrating out the parameters rather than maximizing them we are automatically protected from overfitting: model with more parameters do not necessarily have higher marginal likelihood.
A way to understand the Bayesian Occam's razor effect is to remember that probabilities must sum to one, meaning $\sum_{\mathcal{D}'} p(\mathcal{D}'|m) = 1$. Complex models, which can predict many things, must spread their probability mass thinly, and hence will not obtain as large a probability for any given data set as simpler models.

4.3.2 Computing the marginal likelihood (evidence)

Use of conjugate prior For a fixed model we often write:

$$p(\boldsymbol{\theta}|\mathcal{D}, m) \propto p(\mathcal{D}|\boldsymbol{\theta}, m) \times p(\boldsymbol{\theta}|m)$$

We ignore the normalization constant $\mathbb{P}(\mathcal{D}|m)$ as it is constant with reference to $\boldsymbol{\theta}$. However when comparing models we need to know how to compute the marginal likelihood, $p(\mathcal{D}|m)$. In general this can be quite hard, since we have to integrate over all possible parameter values, but when we have a conjugate prior, it is easy to compute.

Let $p(\boldsymbol{\theta}) = \frac{q(\boldsymbol{\theta})}{Z_0}$ be our prior, where $q(\boldsymbol{\theta})$ is an unnormalized distribution, and Z_0 is the normalization constant of the prior. Let $p(\mathcal{D}|\boldsymbol{\theta}) = \frac{q(\mathcal{D}|\boldsymbol{\theta})}{Z_l}$ be the likelihood, where Z_l contains any constant factors in the likelihood. Finally let $p(\boldsymbol{\theta}|\mathcal{D}) = \frac{q(\boldsymbol{\theta}|\mathcal{D})}{Z_N}$ be our posterior where $q(\boldsymbol{\theta}|\mathcal{D}) = q(\mathcal{D}|\boldsymbol{\theta})q(\boldsymbol{\theta})$ is the unnormalized posterior, and Z_N is the normalization constant of the posterior.

We have: $\begin{cases} p(\boldsymbol{\theta}|\mathcal{D}) & = \frac{p(\mathcal{D}|\boldsymbol{\theta})p(\boldsymbol{\theta})}{p(\mathcal{D})} \\ \frac{q(\boldsymbol{\theta}|\mathcal{D})}{Z_N} & = \frac{q(\mathcal{D}|\boldsymbol{\theta})q(\boldsymbol{\theta})}{Z_l Z_0 p(\mathcal{D})} \\ p(\mathcal{D}) & = \frac{Z_N}{Z_0 Z_l} \end{cases}$

BIC and AIC Simpler approach

- **BIC** In general $p(\mathcal{D}|m) = \int p(\mathcal{D}|\boldsymbol{\theta})p(\boldsymbol{\theta}|m)d\boldsymbol{\theta}$ can be quite difficult to compute. A popular approximation is: $BIC \triangleq \log(p(\mathcal{D}|\hat{\boldsymbol{\theta}}_{\boldsymbol{MLE}})) - \frac{dof(\hat{\boldsymbol{\theta}}_{\boldsymbol{MLE}})}{2}\log(N) \approx \log p(\mathcal{D})$
- **AIC**: $AIC(m, \mathcal{D}) \triangleq \log(p(\mathcal{D}|\hat{\boldsymbol{\theta}}_{MLE})) - dof(m)$
  This is derived from Frequentist framework and cannot be interpreted as an approximation to the marginal likelihood. The penalty of AIC is less than BIC, it causes AIC pick more complex models. That can be better for predictive accuracy.

Use of Empirical Bayes The way to chose the prior is not always clear, for posterior inference it is not an issue as the likelihood often overwhelm the prior, but when computing the marginal likelihood the prior plays a much more important role. Indeed, we are averaging likelihood over all possible parameter settings as weighted by the prior.

If the prior is unknown the correct Bayesian procedure is to to put a prior on the prior (hierarchical Bayes).

In the case of a linear regression we can use a prior of the form $\mathbb{P}(\boldsymbol{w}) = \mathcal{N}(\mathbf{0}, \alpha^{-1}\boldsymbol{I})$, then

$$\mathbb{P}(\mathcal{D}|m) = \int\int \mathbb{P}(\mathcal{D}|\boldsymbol{w})\,\mathbb{P}(\boldsymbol{w}|\alpha, m)\,\mathbb{P}(\alpha|m)\,d\boldsymbol{w}d\alpha$$

The higher up we go in the Bayesian hierarchy the less sensitive are the results to the prior settings. A computational shortcut is to optimize α rather than integrating it out. That is, we use $p(\mathcal{D}|m) \approx \int p(\mathcal{D}|\boldsymbol{w})p(\boldsymbol{w}|\hat{\alpha}, m)d\boldsymbol{w}$. where $\hat{\alpha} = \arg\max_{\alpha} p(\mathcal{D}|\alpha, m) = \arg\max_{\alpha} \int p(\mathcal{D}|\boldsymbol{w})p(\boldsymbol{w}|\alpha, m)d\boldsymbol{w}$ This is Empirical Bayes.

4.3.3 Bayes Factors

When prior on models is uniform, then model selection is equivalent to picking the model with the highest marginal likelihood. Now suppose we just have two models we are considering, call them the null hypothesis, M_0 and the alternative hypothesis, M_1.

$$BF_{1,0} \triangleq \frac{p(\mathcal{D}|M_1)}{p(\mathcal{D}|M_0)} = \frac{\left\{\frac{p(M_1|\mathcal{D})}{p(M_0|\mathcal{D})}\right\} \textit{Posterior odds}}{\left\{\frac{p(M_1)}{p(M_0)}\right\} \textit{Prior odds}}$$

- *Posterior odds*: quantifies relative plausibility of the rival hypotheses **after** having seen the data.
- *Bayes Factor*, $BF_{1,0}$, quantifies the evidence provided by the data, this is like a likelihood ratio, except we integrate out the parameters, which allows us to compare models of different complexity.
- *Prior odds*: quantifies relative plausibility of the rival hypotheses **before** seeing the data.

Bayes Factor $BF(1,0)$	**Interpretation**
$BF < \frac{1}{100}$	Decisive evidence for M_0
$BF < \frac{1}{10}$	Strong evidence for M_0
$\frac{1}{10} < BF < \frac{1}{3}$	Modest evidence for M_0
$\frac{1}{3} < BF < 1$	Weak evidence for M_0
$1 < BF < 3$	Weak evidence for M_1
$3 < BF < 10$	Modest evidence for M_1
$BF > 10$	Strong evidence for M_1
$BF > 100$	Decisive evidence for M_1

4.3.4 Jeffreys-Lindley paradox

Problems can arise when we use improper priors (i.e. priors that do not integrate to 1) for model selection/ hypothesis testing, even though such priors may be acceptable for other purposes. In particular the Bayes Factor will always favor the simplest model since the probability of the observed data under a complex model with a very diffuse prior will be very small. Thus it is important to use proper priors when doing model selection.

4.4 Priors

The most controversial aspect of Bayesian statistics is its reliance on priors

4.4.1 Uninformative priors

If we do not have strong evidence on what θ should be, it is common to use an uninformative priors, to "let the data speak for itself".
One might think that, for Bernoulli parameter θ the most uninformative prior would be the uniform distribution: $Beta(1,1)$, but the posterior mean would then be: $\mathbb{E}(\theta|\mathcal{D}) = \frac{N_1+1}{N_1+N_0+2}$, whereas the MLE is $\frac{N_1}{N_1+N_0}$.
As by decreasing the magnitude of the pseudo counts, we can lessen the impact of the prior, we can argue that the most non-informative prior is:

$$\lim_{\epsilon \to 0} Beta(\epsilon, \epsilon) = Beta(0,0)$$

Called the *Haldane prior*, it is an improper prior.
In general it is advisable to perform a some kind of **sensitivity analysis**, in which one checks how much one's conclusions or prediction change in response to change in the modelling assumptions which includes the choice of the prior and the likelihood as well. If the conclusion are relatively insensitive to the modelling assumption, one can have more confidence in the results.

4.4.2 Jeffreys priors

Harold Jeffreys designed a general purpose technique for creating non-informative priors. The key observation is that if $p(\phi)$ is non-informative then any re-parametrization of the prior, such as $\theta = h(\phi)$ for some function h should also be non-informative.

- Start with a variable change: $p_\theta(\theta) = p_\phi(\phi)\left|\frac{d\phi}{d\theta}\right|$

- Consider the following constraint: $p_\phi(\phi) \propto \sqrt{\mathcal{I}(\phi)}$, where $\mathcal{I}(\phi)$ is the Fisher information. $\mathcal{I}(\phi) \triangleq -\mathbb{E}\left(2 \times \frac{d \log\left(p(X|\phi)\right)}{d\phi}\right)$. This a measure of the curvature of the expected negative log likelihood and hence a measure of stability of the MLE.
- Now $\frac{d \log(p(x|\theta))}{d\theta} = \frac{d \log(p(X|\phi))}{d\phi}\frac{d\phi}{d\theta}$
- $\mathcal{I}(\theta) = \mathcal{I}(\phi)\left(\frac{d\phi}{d\theta}\right)^2$
- $\sqrt{\mathcal{I}(\theta)} = \sqrt{\mathcal{I}(\phi)}\left|\frac{d\phi}{d\theta}\right|$
- Finally $p_\theta(\theta) = p_\phi(\phi)\left|\frac{d\phi}{d\theta}\right| \propto \sqrt{\mathcal{I}(\phi)}\left|\frac{d\phi}{d\theta}\right| = \sqrt{\mathcal{I}(\theta)}$

4.4.3 Robust priors

To prevent an undue influence on the result, we build priors having heavy tails, which avoids forcing things to be too close to the prior mean.

4.4.4 Mixture of conjugate priors

Conjugate priors simplify the computation of robust priors, but are often not robust, and not flexible enough to encode our prior knowledge. However it turns out that a mixture (linear combination) of conjugate priors is also conjugate, and seem to be a good compromise.
We can represent a mixture by introducing a latent indicator variable z, where $z = k$ means that θ comes from mixture component k.

4.5 Hierarchical and Empirical Bayes

4.5.1 Hierarchical Bayes

A key requirement for computing the posterior $p(\theta|\mathcal{D})$ is the specification of a prior $p(\theta|\eta)$ where η are the hyper-parameters. A Bayesian approach is to put a prior on our priors. This is an example of a **hierarchical Bayesian Model**.

4.5.2 Empirical Bayes

In hierarchical Bayesian models, we need to compute the posterior on multiple levels of latent variables. For example, in a two-level model, we need to compute: $p(\eta, \theta|\mathcal{D}) \propto p(\mathcal{D}|\theta)p(\theta|\eta)p(\eta)$
We can approximate the posterior on the hyper-parameters with a point-estimate, $p(\eta|\mathcal{D}) \approx \delta_{\hat{\eta}}(\eta)$ where $\hat{\eta} = \arg\max_\eta p(\eta|\mathcal{D})$. Since η is typically much smaller than θ in dimensionality, it is less prone to overfitting, so we can safely use a uniform prior on η. Then the estimate becomes:

$$\hat{\eta} = \arg\max_\eta p(\mathcal{D}|\eta) = \arg\max_\eta \int p(\mathcal{D}|\theta)p(\theta|\eta)d\theta$$

This overall approach is called **Empirical Bayes**
Empirical Bayes violates the principle that the prior should be chosen independently of the data. However, we can just view it as a computationally cheap approximation to inference in a hierarchical Bayesian model, just as we viewed MAP estimation as an approximation to inference in the one level model $\theta \to \mathcal{D}$. In fact, we can construct a hierarchy in which the more integrals one performs, the "more Bayesian" one becomes:

Method	Definition
Maximum likelihood	$\hat{\theta} = \arg\max_{\theta} p(\mathcal{D}\|\theta)$
MAP estimation	$\hat{\theta} = \arg\max_{\theta} p(\mathcal{D}\|\theta)p(\theta\|\eta)$
ML-II (Empirical Bayes)	$\hat{\eta} = \arg\max_{\eta} \int p(\mathcal{D}\|\theta)p(\theta\|\eta)d\theta = \arg\max_{\eta} p(\mathcal{D}\|\eta)$
MAP-II	$\hat{\eta} = \arg\max_{\eta} \int p(\mathcal{D}\|\theta)p(\theta\|\eta)p(\eta)d\theta = \arg\max_{\eta} p(\mathcal{D}\|\eta)p(\eta)$
Full Bayes	$p(\theta, \eta\|\mathcal{D}) \approx p(\mathcal{D}\|\theta)p(\theta\|\eta)p(\eta)$

4.6 Bayesian Decision Theory

We can formalize any given statistical decision problem as a game against nature (as opposed to a game against other strategic players, which is the topic of game theory). In this game, nature picks a state or parameter θ or label, $y \in \mathcal{Y}$, unknown to us at prediction time, and then generates an observation, $\boldsymbol{x} \in \mathcal{X}$ which we get to see. We then have to make a decision, that is, we have to choose an action a from some **action space** $\mathcal{A}$. Finally we incur some **loss**, $L(y, a)$, which measures how compatible our action a is with nature's hidden state y.
Our goal is to derive a decision procedure or policy, $\delta : \mathcal{X} \to \mathcal{A}$ which specifies the optimal action for each possible input which specifies the optimal action for each possible input, meaning the action that minimizes the expected loss:

$$\delta(\boldsymbol{x}) = \arg\min_{a \in \mathcal{A}} \mathbb{E}_{p(y|\boldsymbol{x})} \left(L(y, a) \right)$$

In the Bayesian vision, we mean the expected value of y given the data we have seen so far, whereas in the frequentist vision we mean the expected value refers to x and y that we expect to see in the future.
In the Bayesian vision the optimal action having observed $\boldsymbol{x}$ is defined as the action a that minimizes the **posterior expected loss**:

$$\rho(a|\boldsymbol{x}) \triangleq \mathbb{E}_{p(y|x)} \left(L(y, a) \right) = \begin{cases} \sum_y L(y, a)p(y|x) & \text{discrete} \\ \int L(y, a)p(y|x)dy & \text{continuous} \end{cases}$$

Hence the Bayes estimator also called Bayes decision rule is given by:

$$\delta(\boldsymbol{x}) = \arg\min_{a \in \mathcal{A}} \rho(\boldsymbol{a}|\boldsymbol{x})$$

Note that here we adopted a predictive point of view, as we took y as reference. Alternatively, adopting an inferential point of view (estimating parameters), the posterior expected loss becomes:

$$\rho(a|\mathcal{D}) \triangleq \mathbb{E}_{p(\theta|\mathcal{D})} \left(L(\theta, a) \right) = \begin{cases} \sum_\theta L(\theta, a)p(\theta|\mathcal{D}) & \text{discrete} \\ \int L(\theta, a)p(\theta|\mathcal{D})d\theta & \text{continuous} \end{cases}$$

where we average over the posterior distribution of the parameter θ given data $\mathcal{D}$.

Examples of posterior expected loss

- **0-1 loss** (predictive):
  - Support: $\mathcal{A} = \{0, 1\}$
  - Loss: $L(y, a) = \mathbb{1}_{\{y \neq a\}}$
  - Posterior expected loss: $\rho(a|\boldsymbol{x}) = \mathbb{P}(y \neq a|\boldsymbol{x}) = 1 - \mathbb{P}(y = a|\boldsymbol{x})$
  - Optimal action: MAP $a^* = \arg\max_y \mathbb{P}(y|\boldsymbol{x})$

  Note: here ρ coincides with a probability because 0-1 loss is binary. The expected value of an indicator function equals the probability of the event: $\mathbb{E}[\mathbb{1}_{\{y \neq a\}}|\boldsymbol{x}] = \mathbb{P}(y \neq a|\boldsymbol{x})$. This is a special property of 0-1 loss; for other losses (e.g., squared error), ρ is not a probability.

- **Squared error** (inferential):
  - Support: $\mathcal{A} = \mathbb{R}$
  - Loss: $L(\theta, a) = (\theta - a)^2$
  - Posterior expected loss: $\rho(a|\mathcal{D}) = \mathbb{E}[(\theta - a)^2|\mathcal{D}]$
  - Optimal action: posterior mean $a^* = \mathbb{E}[\theta|\mathcal{D}]$

Practical applications in Machine Learning The Bayesian posterior expected loss is used when uncertainty quantification matters:

- **Bayesian Neural Networks**: instead of a single prediction $\delta(\boldsymbol{x})$, average predictions over the posterior distribution of weights: $\mathbb{E}_{p(\boldsymbol{\theta}|\mathcal{D})}[\delta(\boldsymbol{x}; \boldsymbol{\theta})]$
- **Uncertainty quantification**: $\rho(a|\boldsymbol{x})$ provides a measure of confidence in predictions, useful for safety-critical applications
- **Active learning**: select samples with high posterior uncertainty (high ρ) to label next, improving data efficiency
- **Model averaging**: combine predictions from multiple models weighted by their posterior probabilities
- **Gaussian Processes**: naturally provide posterior predictive distributions with uncertainty estimates

4.6.1 Bayes estimators for common loss functions

- **MAP** estimate minimizes 0-1 loss: $L(y, a) = \mathbb{1}_{\{y \neq a\}} \begin{cases} 0 \text{ if } a = y \\ 1 \text{ else} \end{cases}$
- **Reject option**, in classification problems where $p(y|\boldsymbol{x})$ is very uncertain we may prefer to choose a reject action, in which we refuse to classify the example as any of the specified classes. Let choosing $a = C + 1$ correspond to picking the reject action, and choosing $a \in \{1, ..., C\}$ correspond to picking one of the classes.
  $$L(y = j, a = i) = \begin{cases} 0 & \text{if } i = j \text{ and } i, j \in \{1, ..., C\} \\ \lambda_r & \text{if } i = C + 1 \\ \lambda_s & \text{otherwise} \end{cases}$$
  where λ_r is the cost of the reject action, and λ_s is the cost of a substitution error.
- **Squared Error (l_2)** for a continuous parameters. $L(y, a) = (y - a)^2$
- **Absolute Error (l_1)** more robust against outliers. $L(y, a) = |y - a|$. The optimal point is the median.
- **Supervised learning** considering a prediction function $\delta : \mathcal{X} \to \mathcal{Y}$ and some cost function $l(y, \delta(x))$. Then the loss incurred by taking action δ when the unknown state of nature is θ (the parameters of the data generating the mechanism). $L(\boldsymbol{\theta}, \delta) \triangleq \mathbb{E}_{(\boldsymbol{x},y)\ p(\boldsymbol{x},y|\boldsymbol{\theta})}\left(l(y, \delta(\boldsymbol{x}))\right) = \sum_{\boldsymbol{x}} \sum_{y} L\left(y, \delta(\boldsymbol{x}) p(\boldsymbol{x}, y|\boldsymbol{\theta})\right)$

4.6.2 Model evaluation metrics

- **False positive vs False negative trade-off** for binary decision problems three are 2 types of errors:
  1. false positive (false alarm) if $\hat{y} = 1 \wedge y = 0$
  2. false negative (missed detection) if $\hat{y} = 0 \wedge y = 1$

We can consider the loss matrix:

Headers	$y=1$	$y=0$
$\hat{y}=1$	0	L_{FP}
$\hat{y}=0$	L_{FN}	0

where L_{FN} is the cost of a false negative and L_{FP} the cost of a false positive.

- **Precision recall curves** When trying to detect a rare event the number of negatives is very large, hence comparing *sensitivity* and *the error of type I* is not very informative. We would then like to use a measure that only talks about positives.
  - **precision** $= \dfrac{TP}{\hat{N}_+}$
  - **recall** $= \dfrac{TP}{N_+}$

  A **precision recall curve** is a plot of *precision* vs *recall*.
- **F-scores** is the *harmonic mean of precision and recall*:
  $F_1 \triangleq \dfrac{2}{\frac{1}{precision}+\frac{1}{recall}}$

Receiver Operating Characteristic (ROC) curve From the below table

Headers		Truth		Count
Estimate	1	TP	FP	$\hat{N}_+ = TP + FP$
	0	FN	TN	$\hat{N}_- = FN + TN$
Count		$N_+ = TP + FN$	$N_- = FP + TN$	$N = N_+ + N_- = \hat{N}_+ + \hat{N}_-$

we can generate the *confusion matrix* is the below table

Headers	$y=1$	$y=0$
$\hat{y}=1$	$\dfrac{TP}{N_+}$ (sensitivity/recall)	$\dfrac{FP}{N_-}$ (error type I/ false alarm)
$\hat{y}=0$	$\dfrac{FN}{N_+}$ (error type II/ missed detection)	$\dfrac{TN}{N_-}$ (specificity)

Consider $f(\boldsymbol{x})$ a measure of our confidence that $y=1$ like $f=\sigma$, however it does not need to be probabilistic simply being monotonically related to $\mathbb{P}(y=1|\boldsymbol{x})$. Let τ be some threshold parameter, then our classification rule is $\mathbb{1}_{\{f(\boldsymbol{x})>\tau\}}$. We can then compute TPR(True Positive Rate) also known as sensibility recall by using $TPR = \frac{TP}{N_+} \approx \mathbb{P}(\hat{y}=1|y=1)$. As well we are able to compute FPR(False Positive Rate) by using $FPR = \frac{FP}{N_-} \approx \mathbb{P}(\hat{y}=1|y=0)$
ROC is achieved by computing the above TPR and FPR for different value of τ Note that

- classifying everything as negative ($\tau=1$) leads to $TPR=0$ and $FPR=0$
- classifying everything as positive ($\tau=0$) leads to $TPR=1$ and $FPR=1$
- a system working randomly will be close to the diagonal line $TPR=FPR$

AUC (Area Under the Curve) summarizes the ROC curve as a single scalar. It admits a probabilistic interpretation:

$$AUC = \mathbb{P}(f(\boldsymbol{x}) > f(\boldsymbol{x}') \mid y=1,\ y'=0)$$

where $\boldsymbol{x}$ and $\boldsymbol{x}'$ are two independently drawn observations with labels $y=1$ and $y'=0$ respectively. AUC measures the probability that the model ranks a random positive higher than a random negative. A random classifier achieves $AUC=0.5$ (the diagonal line) and a perfect classifier achieves $AUC=1$.
Connection to the loss matrix. Each point on the ROC curve corresponds to a threshold τ. The Bayesian decision framework yields the optimal threshold τ^* directly from the loss matrix. Using the false positive/false negative loss matrix, the posterior expected losses for each action are:

$$\rho(1|\boldsymbol{x}) = L_{FP}\cdot(1-f(\boldsymbol{x}))$$
$$\rho(0|\boldsymbol{x}) = L_{FN}\cdot f(\boldsymbol{x})$$

Predicting $\hat{y}=1$ when $\rho(1|\boldsymbol{x}) < \rho(0|\boldsymbol{x})$ gives the optimal threshold:

$$\tau^* = \frac{L_{FP}}{L_{FP}+L_{FN}}$$

This selects the operating point on the ROC curve that minimizes expected loss under the given cost structure. A higher cost for false negatives ($L_{FN} \gg L_{FP}$) pushes τ^* lower, increasing recall at the expense of precision.

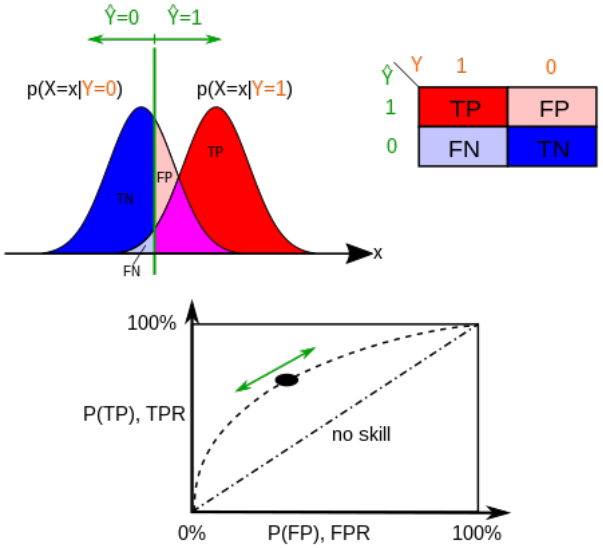


Figure 4.1: Receiver Operating Characteristic curve with distribution illustration

In binary classification the class prediction for each instance is often made based on a continuous random variable $S = f(\boldsymbol{x})$, the score computed for the observation $\boldsymbol{x}$ (e.g. the estimated probability in logistic regression). Note that S is a scalar distinct from the observation vector $\boldsymbol{x}$.
S follows a probability density $f_1(s) = p(S = s \mid y = 1)$ if the instance belongs to the class "positive" and $f_0(s) = p(S = s \mid y = 0)$ otherwise.
Therefore $\begin{cases} TPR(\tau) = \int_{\tau}^{\infty} f_1(s)\, ds \\ FPR(\tau) = \int_{\tau}^{\infty} f_0(s)\, ds \end{cases}$ and ROC curve plots parametrically $TPR(\tau)$ versus $FPR(\tau)$ with τ a varying parameter.
Note that this τ is fundamentally the same that the one above which was squash in the interval $[0, 1]$

AUC derivation. By definition, AUC is the area under the ROC curve, i.e. the integral of TPR over FPR:

$$AUC \triangleq \int_0^1 TPR\; d(FPR)$$

Changing variable from FPR to τ and noting that $-\dfrac{d(FPR)}{d\tau} = f_0(\tau)$:

$$\begin{aligned} AUC &= \int_{-\infty}^{+\infty} TPR(\tau) \cdot f_0(\tau)\; d\tau \\ &= \int_{-\infty}^{+\infty} \left[\int_{\tau}^{\infty} f_1(s)\, ds \right] f_0(\tau)\; d\tau \\ &= \iint_{s>\tau} f_1(s)\, f_0(\tau)\, ds\, d\tau \\ &= \mathbb{P}(S_1 > S_0) \end{aligned}$$

where $S_1 \sim f_1$ and $S_0 \sim f_0$ are independently drawn scores, which is exactly $\mathbb{P}(f(\boldsymbol{x}) > f(\boldsymbol{x}') \mid y = 1,\ y' = 0)$.
Note that f_1 and f_0 are the PDFs of the score for each class; the ROC curve itself is not a PDF.

	A	B	C	D	E	F	G	H
3	ROC Table							
4								
5		Observed		Cumulative				
6	*Dosage*	*Lives*	*Dies*	*Lives*	*Dies*	*FPR*	*TPR*	*AUC*
7				0	0	1	1	0.064516
8	less than 2.00	34	3	34	3	0.935484	0.989247	0.118259
9	2.00 - 3.99	63	7	97	10	0.815939	0.964158	0.160998
10	4.00 - 5.99	88	11	185	21	0.648956	0.924731	0.184244
11	6.00 - 7.99	105	14	290	35	0.449715	0.874552	0.204117
12	8.00 - 9.99	123	23	413	58	0.216319	0.792115	0.142791
13	10.00 - 11.99	95	60	508	118	0.036053	0.577061	0.009855
14	12.00 - 13.99	9	75	517	193	0.018975	0.308244	0.003509
15	14.00 - 15.99	6	41	523	234	0.00759	0.16129	0.001224
16	16.00 -17.99	4	30	527	264	0	0.053763	0
17	18.00 or more	0	15	527	279	0	0	0
18		527	279					0.889515

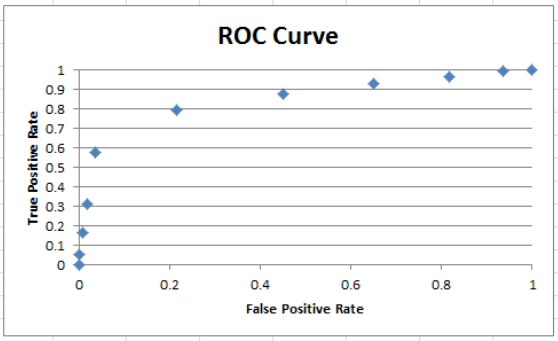


Figure 4.2: Receiver Operating Characteristic curve

In the above table we have the following formula :

- $D9:\ SUM(\$B\$7:B9) \rightarrow$ Cumulative Lives
- $E9:\ SUM(\$C\$7:C9) \rightarrow$ Cumulative Dies
- $F9:\ 1 - D9/\$D\$17 \rightarrow$ False Positive Rate $(1 - \frac{TN}{N_-})$
- $G9:\ 1 - E9/\$E\$17 \rightarrow$ True Positive Rate $(1 - \frac{FN}{N_+})$
- $H9:\ (F10 - F9) * G9 \rightarrow$ Area Under the Curve

Chapter 5

Frequentist Approach

5.1 Sampling Distribution

5.1.1 Sampling Distributions of an estimator

In frequentist statistic a parameter estimate $\hat{\theta}$ is computed by applying an estimator δ to some data $\mathcal{D}$, so $\hat{\theta} = \delta(\mathcal{D})$. The uncertainty in the parameter estimate can be measured by computing the *sampling distribution of the estimator*. Imagine sampling many different datasets $\mathcal{D}^{(s)}$ from some true model $p(\cdot|\theta^*)$ meaning $\mathcal{D}^{(s)} = \left\{x_i^{(s)} \hookrightarrow p(\cdot|\theta^*)\right\}_{1 \leq i \leq N}$ for $1 \leq s \leq S$ and θ^* is the true parameter. Now apply the estimator $\hat{\theta}(\cdot)$ to each $\mathcal{D}^{(s)}$ to get a set of estimates $\{\delta(\mathcal{D}^{(s)})\}_{1 \leq s \leq S}$.
As we late $S \to \infty$, the distribution induced on $\hat{\theta}(\cdot)$ is the **sampling distribution of the estimator**.

Examples of estimators

- **Sample Mean** (estimator for population mean μ):

  $\hat{\mu} = \bar{x} = \frac{1}{N}\sum_{i=1}^{N} x_i$

  Sampling distribution: If $X \sim \mathcal{N}(\mu, \sigma^2)$, then $\bar{X} \sim \mathcal{N}\left(\mu, \frac{\sigma^2}{N}\right)$.
  By CLT, approximately normal for large N even for non-normal data.

- **Sample Variance** (estimator for population variance σ^2):

  - Biased: $\hat{\sigma}^2 = \frac{1}{N}\sum_{i=1}^{N}(x_i - \bar{x})^2$
  - Unbiased: $s^2 = \frac{1}{N-1}\sum_{i=1}^{N}(x_i - \bar{x})^2$

  Sampling distribution: $\frac{(N-1)s^2}{\sigma^2} \sim \chi^2_{N-1}$

5.1.2 Bootstrap

It is a simple *Monte Carlo* technique to approximate the sampling distribution. The idea is that if we knew the true parameters θ^*, we could generate S fake datasets of size N, from the true distribution. We could then compute our estimator from each sample, and use the empirical distribution of the resulting samples as our estimate of the sampling distribution.
Since θ^* is unknown, the idea of the **parametric bootstrap** is to generate the samples using $\hat{\theta}(\mathcal{D})$ instead. An alternative, called **non-parametric bootstrap** is to sample the x_i^s (with replacement) from the original data $\mathcal{D}$ and then compute the induced distribution as before.

5.2 Frequentist Decision Theory

In Frequentist decision theory there is a loss function and a likelihood, but there is no prior and hence no posterior or posterior expected loss. Thus there is no automatic way of deriving an optimal estimator, unlike the Bayesian case.
Instead, we are free to choose any estimator or decision procedure $\delta : \mathcal{X} \to \mathcal{A}$ we want.
Having chosen an estimator, we define its expected loss or risk as follows

$$R(\theta^*, \delta(\tilde{\mathcal{D}})) \triangleq \mathbb{E}_{p(\tilde{\mathcal{D}}|\theta^*)}\left(L(\theta^*, \delta(\tilde{\mathcal{D}}))\right) = \begin{cases} \sum_{\tilde{\mathcal{D}}} L\left(\theta^*, \delta(\tilde{\mathcal{D}})\right) p(\tilde{\mathcal{D}}|\theta^*) & \text{discrete} \\ \int L\left(\theta^*, \delta(\tilde{\mathcal{D}})\right) p(\tilde{\mathcal{D}}|\theta^*) d\tilde{\mathcal{D}} & \text{continuous} \end{cases}$$

where $\tilde{\mathcal{D}}$ is data sampled from 'nature's distribution' represented by parameter θ^*. Note that here we adopted an inferential point of view (estimating parameters). Alternatively, adopting a predictive point of view (predicting labels), the expected loss becomes:

$$R(y, \delta(\boldsymbol{x})) \triangleq \mathbb{E}_{p(\tilde{\boldsymbol{x}}, \tilde{y}|\theta^*)}\left(L(\tilde{y}, \delta(\tilde{\boldsymbol{x}}))\right) = \begin{cases} \sum_{\tilde{\boldsymbol{x}}, \tilde{y}} L\left(\tilde{y}, \delta(\tilde{\boldsymbol{x}})\right) p(\tilde{\boldsymbol{x}}, \tilde{y}|\theta^*) & \text{discrete} \\ \int L\left(\tilde{y}, \delta(\tilde{\boldsymbol{x}})\right) p(\tilde{\boldsymbol{x}}, \tilde{y}|\theta^*) d\tilde{\boldsymbol{x}} d\tilde{y} & \text{continuous} \end{cases}$$

where we average over future data $(\tilde{\boldsymbol{x}}, \tilde{y})$ sampled from the true distribution.
Whereas the Bayesian posterior expected loss:

$$\rho(a|\mathcal{D}, \pi) \triangleq \mathbb{E}_{p(\theta|\mathcal{D}, \pi)}\left(L(\theta, a)\right) = \begin{cases} \sum_{\theta} L(\theta, a) p(\theta|\mathcal{D}, \pi) & \text{discrete} \\ \int_{\Theta} L(\theta, a) p(\theta|\mathcal{D}, \pi) d\theta & \text{continuous} \end{cases}$$

The tilde notation $(\tilde{\mathcal{D}})$ highlights a fundamental difference between the two paradigms:

- **Frequentist** (pre-data perspective):
  - Conditions on: θ^* (the true but unknown parameter)
  - Averages over: $\tilde{\mathcal{D}}$ (hypothetical future datasets that *could* be sampled)

  The tilde indicates these are imagined/potential samples, not observed data. "How would my estimator perform across many hypothetical repetitions of the experiment?"
- **Bayesian** (post-data perspective):
  - Conditions on: $\mathcal{D}$ (the actual observed data)
  - Averages over: θ (the uncertain parameter)

  No tilde because we use the actual observed data. "Given what I observed, what do I believe about the unknown quantity?"

Examples of frequentist expected loss (risk)

- **Mean Squared Error (MSE)** (inferential):
  - Loss: $L(\theta^*, \delta(\mathcal{D})) = (\theta^* - \delta(\mathcal{D}))^2$
  - Risk: $R(\theta^*, \delta) = \mathbb{E}[(\theta^* - \delta(\tilde{\mathcal{D}}))^2] = \text{Var}(\delta) + \text{Bias}^2(\delta)$
- **Misclassification rate** (predictive):
  - Loss: $L(y, \delta(\boldsymbol{x})) = \mathbb{1}_{\{y \neq \delta(\boldsymbol{x})\}}$
  - Risk: $R = \mathbb{E}[\mathbb{1}_{\{\tilde{y} \neq \delta(\tilde{\boldsymbol{x}})\}}] = \mathbb{P}(\tilde{y} \neq \delta(\tilde{\boldsymbol{x}}))$

5.2.1 Bayes risk

How to chose amongst the estimators? We need some way to convert $R(\theta^*, \delta)$ into single measure of quality, $R(\delta)$ which does not depend on knowing θ^*. One approach is to put a prior on θ^* and then to define **Bayes risk** of an estimator as follows:

$$R_B(\delta) \triangleq \mathbb{E}_{p(\theta^*)}\left(R(\theta^*, \delta)\right) = \int R(\theta^*, \delta) p(\theta^*) d\theta^*$$

A **Bayes estimator** or **Bayes decision rule** is one which minimizes the expected risk: $\delta_B \triangleq \arg\min_{\delta} R_B(\delta)$

Connection Bayesian and Frequentist approaches to decision theory.

- *Theorem 1* A Bayes estimator can be obtained by minimizing the posterior expected loss for each $\boldsymbol{x}$
- *Theorem 2* Every admissible frequentist decision rule is a Bayes decision rule with respect to some possibly improper prior distribution.

Minimax risk Some frequentist statistic users avoid using Bayes risk since it requires the choice of a prior, although this is only in the evaluation of the estimator, not necessarily as part of its construction. An alternative approach is as follows:

1. Define the maximum risk of an estimator as:
   $R_{max}(\delta) \triangleq \max_{\theta^*} R(\boldsymbol{\theta}^*, \delta)$
2. A **minimax rule** is one which minimizes the maximum risk: $\delta_{MM} \triangleq \arg\min_{\delta} R_{\max}(\delta)$

Minimax estimators have a certain appeal, however computing them can be hard and furthermore they are very pessimistic. In most statistical situations, excluding games theoretic ones, assuming nature is an adversary is not a reasonable assumption.

5.2.2 Admissible estimators

The basic problem with frequentis decision theory is that it relies on knowing the true distribution $p(\cdot|\theta^*)$ in order to evaluate the risk. However it might be the case that some estimators are worse than others regardless of the value of θ^*.
In particular if for $\theta \in \Theta, R(\theta, \delta_1) \leq R(\theta, \delta_2)$ then we say that δ_1 **dominates** δ_2.
An estimator is said to be **admissible** if it is not strictly dominated by any other estimator.
Admissibility is not enough

5.3 Desirable Properties of Estimators

5.3.1 Consistent estimators

An estimator is said to be **consistent** if it eventually recovers the true parameters that generated the data as the sample size goes to infinity.

5.3.2 Unbiased estimator

The **bias** of an estimator is defined as

$$bias\left(\hat{\theta}(\cdot)\right) = \mathbb{E}_{p(\mathcal{D}|\theta^*)}\left(\hat{\theta}(\mathcal{D}) - \theta^*\right)$$

The estimator is unbiased when the bias is equal to 0.

5.3.3 Minimum variance estimators

A famous result called the **Cramerè-Rao lower bound** provides a lower bound on the variance of any unbiased estimator. More precisely: Let $(X_j)_{1\leq j\leq p} \hookrightarrow p(X|\theta_0)$ and $\hat{\theta}(\cdot)$ an unbiased estimator of θ^* Then, under various smoothness assumptions on $p(X|\theta_0)$ we have

$$\mathbb{V}(\hat{\theta}) \geq \frac{1}{nI(\theta^*)}$$

where $I(\theta^*)$ is the Fisher information matrix.

5.3.4 Bias-Variance Trade-off

As $MSE = variance + bias^2$
It might be wise to use a biased estimator, so long as it reduces our variance, assuming our goal is to minimize squared error.

5.4 Empirical Risk Minimization

5.4.1 Frequentist issue

Frequentist decision theory suffers from the fundamental problem that one cannot actually compute the risk function, since it relies on knowing the true data distribution. By contrast, the Bayesian posterior expected loss can always be computed since it conditions on the data rather that on θ^*.
However there is one setting which avoids this problem, it is when the task is to predict observable quantities, as opposed to estimating hidden variables or parameters.
Instead of looking at loss functions of the form $L(\boldsymbol{\theta^*}, \delta(\mathcal{D}))$ let us look at loss functions of the form $L(y, \delta(\boldsymbol{x}))$.

Then the risk becomes: $R(\theta^*, \delta) \triangleq \mathbb{E}_{(\boldsymbol{x},y)\hookrightarrow p(\cdot|\theta^*)}\left(L(y, \delta(\boldsymbol{x}))\right) = \sum_{\boldsymbol{x}}\sum_{\boldsymbol{y}} L(y, \delta(\boldsymbol{x}))p(\boldsymbol{x}, y|\theta^*)$ This distribution is unknown, but a simple approach is to use the empirical distribution, derived from some training data to approximate $p(\boldsymbol{x}, y|\theta^*) \approx p_{emp}(\boldsymbol{x}, y) \triangleq \frac{1}{N}\sum_{i=1}^{N} \mathbb{1}_{\{\boldsymbol{x}=x_i\}}\mathbb{1}_{\{y=y_i\}}$ that it is a fundamental approach to model the true distribution based on frequencies considering the training dataset. From law of large numbers we expect that p_{emp} converges toward $p(\cdot|\theta^*)$.
where $\mathbb{1}_{\{\boldsymbol{x}=x_i\}}$ and $\mathbb{1}_{\{y=y_i\}}$ are **indicator functions** that equal 1 when the condition is satisfied and 0 otherwise. This places equal probability mass $(\frac{1}{N})$ on each observed data point. Substituting p_{emp} into the risk definition:

$$\begin{aligned}
R_{emp}(\mathcal{D}, \delta) &= \sum_{\boldsymbol{x}}\sum_{y} L(y, \delta(\boldsymbol{x}))p_{emp}(\boldsymbol{x}, y) \\
&= \sum_{\boldsymbol{x}}\sum_{y} L(y, \delta(\boldsymbol{x})) \cdot \frac{1}{N}\sum_{i=1}^{N} \mathbb{1}_{\{\boldsymbol{x}=x_i\}}\mathbb{1}_{\{y=y_i\}} \\
&= \frac{1}{N}\sum_{i=1}^{N}\sum_{\boldsymbol{x}}\sum_{y} L(y, \delta(\boldsymbol{x}))\mathbb{1}_{\{\boldsymbol{x}=x_i\}}\mathbb{1}_{\{y=y_i\}} \\
&= \frac{1}{N}\sum_{i=1}^{N} L(y_i, \delta(\boldsymbol{x}_i))
\end{aligned}$$

The last step follows because the indicator functions select only the terms where $\boldsymbol{x} = x_i$ and $y = y_i$. Hence the empirical risk:

$$R_{emp}(\mathcal{D}, \delta) = \frac{1}{N}\sum_{i=1}^{N} L(y_i, \delta(\boldsymbol{x}_i))$$

5.4.2 Regularized risk minimization

$$R'(\mathcal{D}, \delta) = R_{emp}(\mathcal{D}, \delta) + \lambda C(\delta)$$

where $C(\delta)$ measures the complexity of the prediction function $\delta(\boldsymbol{x})$ and λ controls the strength of the complexity penalty. This approach is known as **regularized risk minimization**.

5.4.3 Practical applications in Machine Learning

Empirical Risk Minimization (ERM) Training a model is minimizing empirical risk:

- Loss function in training (MSE, cross-entropy, etc.) corresponds to $L(y, \delta(\boldsymbol{x}))$
- Training objective: $R_{emp} = \frac{1}{N}\sum_{i=1}^{N} L(y_i, \delta(\boldsymbol{x}_i))$
- Gradient descent minimizes R_{emp} over model parameters

ML Algorithm	**Loss** $L(y, \delta(\boldsymbol{x}))$
Linear regression	$(y - \delta(\boldsymbol{x}))^2$
Logistic regression	$-y \log(\delta(\boldsymbol{x})) - (1-y)\log(1-\delta(\boldsymbol{x}))$
SVM	$\max(0, 1 - y \cdot \delta(\boldsymbol{x}))$
Neural networks	cross-entropy, MSE, etc.

Regularized Risk in practice $R'(\mathcal{D}, \delta) = R_{emp} + \lambda C(\delta)$ is used in:

- **Ridge regression**: $C(\delta) = ||\boldsymbol{w}||_2^2$
- **Lasso**: $C(\delta) = ||\boldsymbol{w}||_1$
- **Neural network weight decay**: $C(\delta) = ||\boldsymbol{\theta}||_2^2$
- **Dropout**: implicit regularization

True Risk vs Empirical Risk gap This gap explains fundamental ML phenomena:

- **Overfitting**: low R_{emp} but high $R(\theta^*, \delta)$
- **Generalization**: want $R(\theta^*, \delta) \approx R_{emp}$
- **Validation/test sets**: estimate true risk since we cannot compute it directly
- **Bias-variance tradeoff**: decomposition of true risk

5.5 Components

5.5.1 Introduction

Avoid treating parameters as random variables. The notion of variation across repeated trials forms the basis for modelling uncertainity.

5.5.2 Hypothesis Testing

In frequentist statistics, probabilities represent frequencies of events across repeated trials.

- **Null hypothesis** H_0: default assumption (e.g., no effect, $\theta = \theta_0$)
- **Alternative hypothesis** H_1: what we want to test (e.g., $\theta \neq \theta_0$)
- **Test statistic** $T = T(\mathcal{D})$: function of data used for decision
- **Critical region** $\mathcal{C}_\alpha$: values of T leading to rejection of H_0

Decision rule: reject H_0 if $T \in \mathcal{C}_\alpha$.

5.5.3 *p-value*

The probability of observing a test statistic as extreme or more extreme than the observed value, assuming H_0 is true:

$$p\text{-value} \triangleq \mathbb{P}(T \geq t_{obs} \mid H_0)$$

For two-sided tests: $p\text{-value} = \mathbb{P}(|T| \geq |t_{obs}| \mid H_0)$.
Decision rule:

$$\begin{cases} \text{Reject } H_0 & \text{if } p\text{-value} < \alpha \\ \text{Fail to reject } H_0 & \text{if } p\text{-value} \geq \alpha \end{cases}$$

where α is the **significance level** (typically $\alpha = 0.05$).

5.5.4 Confidence intervals

A $(1-\alpha)$ **confidence interval** for parameter θ is an interval $[\hat{\theta}_L, \hat{\theta}_U]$ such that:

$$\mathbb{P}(\hat{\theta}_L \leq \theta \leq \hat{\theta}_U) = 1 - \alpha$$

For a sample mean with known variance: $\text{CI}_{1-\alpha} = \bar{x} \pm z_{\alpha/2}\frac{\sigma}{\sqrt{n}}$

For unknown variance (using t-distribution): $\text{CI}_{1-\alpha} = \bar{x} \pm t_{\alpha/2,n-1}\frac{s}{\sqrt{n}}$

Frequentist interpretation: if we repeat the experiment many times, $(1-\alpha)\%$ of the constructed intervals will contain the true parameter θ.

Link to hypothesis testing: if the $(1-\alpha)$ CI for θ does not include θ_0, then we reject $H_0 : \theta = \theta_0$ at significance level α.

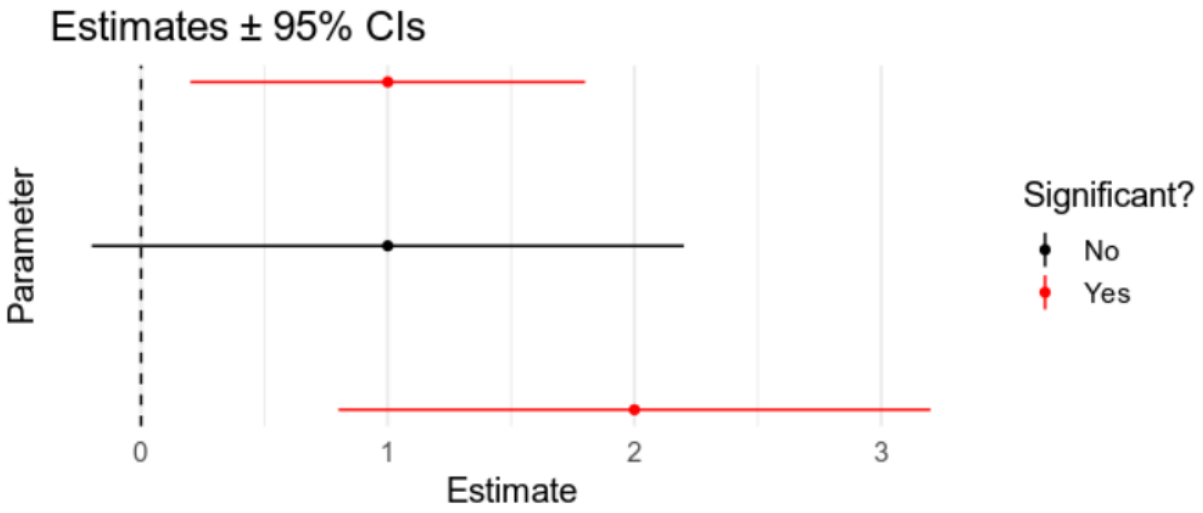


Figure 5.1: Confidence interval

5.5.5 Multiple comparisons

Running k tests increases the **family-wise error rate** (FWER):

$$\mathbb{P}(\text{at least one false positive}) = 1 - (1-\alpha)^k \approx k\alpha \text{ for small } \alpha$$

Bonferroni correction: controls FWER by adjusting either:

- the threshold: $\alpha_{adj} = \frac{\alpha}{k}$, or
- the p-values: $p_{adj} = \min(k \times p_{values}, 1)$

Reject $H_0^{(i)}$ if $p_i < \alpha_{adj}$ (equivalently, if $p_{adj,i} < \alpha$).

Note: Bonferroni is conservative. Alternatives include Holm-Bonferroni (less conservative) and False Discovery Rate (FDR) control for large-scale testing.

5.5.6 Effect size

The **effect size** measures the magnitude of a phenomenon independently of sample size. Unlike p-values, effect sizes indicate practical significance.

- **Cohen's** d (difference between means): $d = \dfrac{\bar{x}_1 - \bar{x}_2}{s_{pooled}}$ where $s_{pooled} = \sqrt{\dfrac{(n_1-1)s_1^2 + (n_2-1)s_2^2}{n_1+n_2-2}}$
- **Pearson's** r (correlation): $r \in [-1, 1]$
- η^2, ω^2 (variance explained in ANOVA)

Effect size	**Small**	**Medium**	**Large**
Cohen's d	0.2	0.5	0.8
Pearson's r	0.1	0.3	0.5

5.5.7 Sample size

Sample size n is related to **effect size** d, **power** $(1-\beta)$, and **significance level** α through:

$$n \propto \frac{(z_{1-\alpha/2} + z_{1-\beta})^2}{d^2}$$

For a two-sample t-test (per group):

$$n = 2\left(\frac{z_{1-\alpha/2} + z_{1-\beta}}{d}\right)^2$$

Key relationships:

- Larger effect size $d \Rightarrow$ smaller n needed
- Higher power $(1-\beta) \Rightarrow$ larger n needed
- Smaller α (stricter significance) $\Rightarrow$ larger n needed

Trade-off: with large n, even tiny (practically irrelevant) effects become statistically significant. Always report effect sizes alongside p-values.

5.6 Power Analysis

5.6.1 Purpose

Power analysis aims to determine the required sample size n to detect an effect of a given magnitude with specified confidence.

5.6.2 Error types and power

	H_0 **is True**	H_1 **is True**
Do not reject H_0	Correct: $1-\alpha$	Type II Error: β
Reject H_0	Type I Error: α	Correct: $1-\beta$ (Power)

- **Type I error** (α): $\mathbb{P}$(reject H_0 | H_0 true) — false positive
- **Type II error** (β): $\mathbb{P}$(fail to reject H_0 | H_1 true) — false negative
- **Power** $(1-\beta)$: $\mathbb{P}$(reject H_0 | H_1 true) — true positive rate

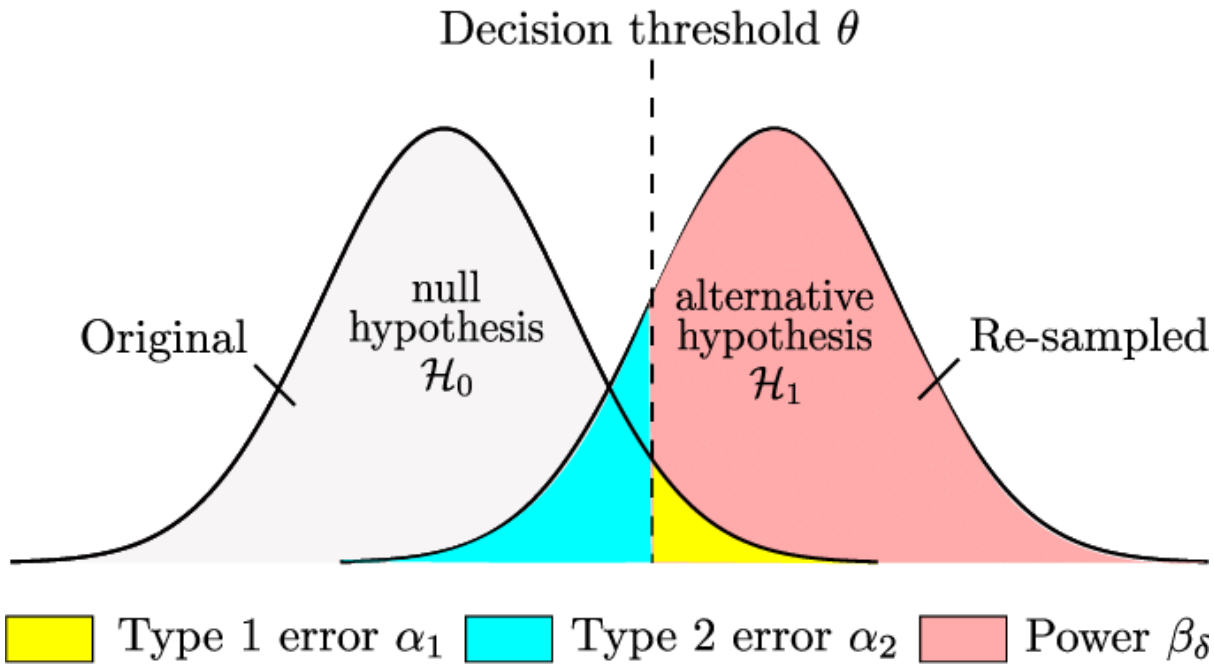


Figure 5.2: Distribution under H_0 and H_1 showing α, β, and power

5.6.3 Key quantities

Four interdependent quantities — fixing any three determines the fourth:

Symbol	Description	Typical value
α	Significance level (Type I error rate)	0.05
$1-\beta$	Power (1 - Type II error rate)	0.80
d	Effect size (standardized)	problem-dependent
n	Sample size	to be determined

5.6.4 Power calculation

For a **one-sample z-test**: $H_0 : \mu = \mu_0$ vs $H_1 : \mu = \mu_1$ (one-sided, $\mu_1 > \mu_0$).

Step 1: Test statistic and its distributions

$$Z = \frac{\bar{X} - \mu_0}{\sigma/\sqrt{n}}$$

- Under H_0: $Z \sim \mathcal{N}(0,1)$
- Under H_1: $Z \sim \mathcal{N}\left(\frac{\mu_1 - \mu_0}{\sigma/\sqrt{n}}, 1\right) = \mathcal{N}(\delta, 1)$ where $\delta = \frac{\mu_1 - \mu_0}{\sigma/\sqrt{n}} = d\sqrt{n}$ is the **non-centrality parameter**

Step 2: Rejection region under H_0
Reject H_0 if $Z > z_{1-\alpha}$ (critical value from standard normal).

Step 3: $Power = \mathbb{P}(\textbf{reject } H_0 | H_1 \textbf{ true})$

$$\begin{aligned}\text{Power} &= \mathbb{P}(Z > z_{1-\alpha} \mid H_1) \\ &= P\left(\frac{\bar{X} - \mu_0}{\sigma/\sqrt{n}} > z_{1-\alpha} \mid \mu = \mu_1\right)\end{aligned}$$

Standardizing under H_1 (subtract δ to get standard normal):

$$\begin{aligned}\text{Power} &= P\left(Z - \delta > z_{1-\alpha} - \delta\right) \\ &= P\left(Z' > z_{1-\alpha} - \delta\right) \quad \text{where } Z' \sim \mathcal{N}(0,1)\end{aligned}$$

Step 4: Express using CDF
Let $\Phi(z) = \mathbb{P}(Z' \leq z)$ be the standard normal CDF:

$$\begin{aligned}\text{Power} &= 1 - \mathbb{P}(Z' \leq z_{1-\alpha} - \delta) \\ &= 1 - \Phi(z_{1-\alpha} - \delta) \\ &= \Phi(\delta - z_{1-\alpha}) \quad \text{(using symmetry: } 1 - \Phi(x) = \Phi(-x)\text{)}\end{aligned}$$

Substituting $\delta = d\sqrt{n}$:

$$\boxed{\text{Power} = \Phi\left(d\sqrt{n} - z_{1-\alpha}\right)}$$

For two-sided test ($H_1 : \mu \neq \mu_0$): use $z_{1-\alpha/2}$ instead of $z_{1-\alpha}$:

$$\text{Power} \approx \Phi\left(d\sqrt{n} - z_{1-\alpha/2}\right)$$

5.6.5 Sample size determination

Solving for n given target power $(1-\beta)$, effect size d, and significance α:

$$n = \left(\frac{z_{1-\alpha/2} + z_{1-\beta}}{d}\right)^2$$

where $d = \dfrac{|\mu_1 - \mu_0|}{\sigma}$ is the standardized effect size.

For common tests:

- **Two-sample t-test** (per group), assuming equal variances ($\sigma_1^2 = \sigma_2^2 = \sigma^2$):

$$n = 2\left(\frac{z_{1-\alpha/2} + z_{1-\beta}}{d}\right)^2 \quad \text{where } d = \frac{|\mu_1 - \mu_2|}{\sigma}$$

- **Two-sample t-test** (per group), unequal variances (Welch's t-test):

$$n_1 = \left(1 + \frac{\sigma_1^2}{\sigma_2^2}\right)\left(\frac{z_{1-\alpha/2} + z_{1-\beta}}{d}\right)^2 \quad \text{where } d = \frac{|\mu_1 - \mu_2|}{\sqrt{\sigma_1^2 + \sigma_2^2}}$$

  For equal sample sizes with unequal variances: $n = \left(\dfrac{\sigma_1^2 + \sigma_2^2}{(\mu_1 - \mu_2)^2}\right)(z_{1-\alpha/2} + z_{1-\beta})^2$

- **Paired t-test**: $n = \left(\dfrac{z_{1-\alpha/2} + z_{1-\beta}}{d}\right)^2$ where $d = \dfrac{\mu_d}{\sigma_d}$

- **Correlation test**: $n = \left(\dfrac{z_{1-\alpha/2} + z_{1-\beta}}{0.5 \ln \frac{1+r}{1-r}}\right)^2 + 3$

5.6.6 Conducting a power analysis

A priori (before data collection): determine n given α, power, and expected d

$$n = f(\alpha, 1-\beta, d) \quad \Rightarrow \quad \text{plan experiment}$$

Post hoc (after data collection): compute achieved power given n, α, and observed d

$$1 - \beta = g(\alpha, n, d) \quad \Rightarrow \quad \text{interpret results}$$

Sensitivity: determine minimum detectable effect size given n, α, and power

$$d = h(\alpha, 1-\beta, n) \quad \Rightarrow \quad \text{assess study limitations}$$

5.6.7 Example: two-sample t-test

Given: $\alpha = 0.05$, power $= 0.80$, expected Cohen's $d = 0.5$ (medium effect)

$$z_{1-\alpha/2} = z_{0.975} = 1.96$$
$$z_{1-\beta} = z_{0.80} = 0.84$$
$$n = 2\left(\frac{1.96 + 0.84}{0.5}\right)^2 = 2 \times (5.6)^2 \approx 63 \text{ per group}$$

Total sample size: $N = 2n \approx 126$ participants.

Chapter 6

Common Statistical Tests

6.1 Use of Statistical Tests

6.1.1 Terms

- *Paired* samples: one-to-one correspondence between data in the first and second set.
- *Matched* samples: every subject in one group with an equivalent in another.

6.1.2 Table of statistical hypothesis test

Statistical method table.

	Binomial/Discrete	**Continuous, from Normal distribution**	**Continuous measurement (Score/Rank), from non-Normal distribution**
Example of data sample	List of patients recovering or not after a treatment	Reading of heart pressure from several patients	Ranking of several treatment efficiency
Describe one data sample	Proportions	Mean, Standard Deviation	Median
Compare one data sample to a hypothetical distribution	χ^2 / *G-test* or Binomial test	1-sample t-test	Sign test or Wilconox test
Compare 2 paired samples	Sign test	Paired t-test	Sign test or Wilconox test
Compare 2 unpaired samples	χ^2 / *G-test* or Fisher's extract test	Unpaired t-test	Mann-Whitney test
Compare 3 or more unmatched samples	χ^2 / *G-test*	1-way ANOVA	Kruksal-Wallis test
Compare 3 or more matched samples	Cochrane Q test	Repeated-measures ANOVA	Friedman test
Quantify association between 2 paired samples	Contingency coefficients	Pearson correlation	Spearman correlation

6.2 List of Common Statistical Tests

6.2.1 Binomial

To check if the deviations from a theoretically expected distribution of observations into 2 categories.

Assumptions

- Sample items are independent.

- Items are dichotomous and nominal.
- The sample size is significantly less than the population size
- The sample is a fair representation of the population

Frequentist Let define a user-defined probability p_0, with $H_0 : p = p_0$ and $\begin{cases} H_1 : p \neq p_0 \text{: two-tailed test} \\ H_1 : p < p_0 \text{: left-tailed test} \\ H_1 : p > p_0 \text{: right-tailed test} \end{cases}$

Bayesian Define the prior distribution with a $Beta(a, b)$ distribution

Return to the table.

6.2.2 χ^2 test

Either used to test *goodness-of-fit* or *independence* between 2 variables. It checks either if there is a significant difference between the expected and observed frequencies.

- *goodness-of-fit*: expected frequencies are computed with a theoretical relationship between observed frequencies
- *independence*: expected frequencies are computed with observed frequencies from the other sample

Assumptions

- simple random sample
- sample with a sufficiently large size is assumed, for small sample size see Fisher's exact test
- expected cell count has to be adequate, a rule of thumb is at least 5 for 2-by-2 table and 5 or more in 80% of cells in larger tables.
- Independence of the observations

Frequentist $\chi^2 = \sum_{j=1}^{k} \frac{(O_j - E_j)^2}{E_j}$ where $E_j = Np_j$ is the expected count and: $\begin{cases} O_j \text{: observed count of category } j \\ E_j \text{: expected count of category } j \\ N \text{: total number of observations} \\ k \text{: number of categories} \\ p_j \text{: expected proportion of category } j \text{ u} \end{cases}$

From trials to counts Consider N independent trials, each resulting in one of k categories. Define the **indicator variable**:

$$X_{ij} = \mathbb{1}_{\{\text{trial } i \text{ results in category } j\}} = \begin{cases} 1 & \text{if trial } i \to \text{category } j \\ 0 & \text{otherwise} \end{cases}$$

	Cat. 1	**Cat. 2**	$\cdots$	**Cat.** j	$\cdots$	**Cat.** k	**Row sum**
Trial 1	X_{11}	X_{12}	$\cdots$	X_{1j}	$\cdots$	X_{1k}	1
Trial 2	X_{21}	X_{22}	$\cdots$	X_{2j}	$\cdots$	X_{2k}	1
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\ddots$	$\vdots$	$\vdots$
Trial i	X_{i1}	X_{i2}	$\cdots$	X_{ij}	$\cdots$	X_{ik}	1
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\ddots$	$\vdots$	$\vdots$
Trial N	X_{N1}	X_{N2}	$\cdots$	X_{Nj}	$\cdots$	X_{Nk}	1
Col sum	$O_1 = \sum_{i=1}^{N} X_{i1}$	$O_2 = \sum_{i=1}^{N} X_{i2}$	$\cdots$	$O_j = \sum_{i=1}^{N} X_{ij}$	$\cdots$	$O_k = \sum_{i=1}^{N} X_{ik}$	N

Key relationships:

- Each row sums to 1: $\sum_{j=1}^{k} X_{ij} = 1$ (each trial falls in exactly one category)
- **Observed count**: $O_j = \sum_{i=1}^{N} X_{ij}$ (sum of column j)
- $X_{ij} \sim \text{Bernoulli}(p_j)$ with $\mathbb{E}[X_{ij}] = p_j$
- **Expected count**: $E_j = \mathbb{E}[O_j] = \mathbb{E}\left[\sum_{i=1}^{N} X_{ij}\right] = \sum_{i=1}^{N} \mathbb{E}[X_{ij}] = \sum_{i=1}^{N} p_j = Np_j$

Derivation (goodness-of-fit) The χ^2 test statistic is derived from the normal approximation to the multinomial distribution.

1. **Multinomial distribution**: Under H_0, observed counts $(O_i)_{1\leq i\leq k}$ follow a multinomial distribution with:
   - $\mathbb{E}[O_j] = Np_j = E_j$
   - $\text{Var}(O_j) = Np_j(1-p_j) = E_j(1-p_j)$

   Covariance derivation: Using $\text{Cov}(O_j, O_{j'}) = \mathbb{E}[O_j O_{j'}] - \mathbb{E}[O_j]\mathbb{E}[O_{j'}]$:
   - $\mathbb{E}[O_j]\mathbb{E}[O_{j'}] = N^2 p_j p_{j'}$
   - $\mathbb{E}[O_j O_{j'}] = \sum_{i=1}^{N}\sum_{l=1}^{N} \mathbb{E}[X_{ij}X_{lj'}] = \underbrace{N \times 0}_{\text{same trial}} + \underbrace{(N^2-N) \times p_j p_{j'}}_{\text{different trials}}$
     Same trial ($i = l$): $X_{ij}X_{ij'} = 0$ (mutually exclusive).
     Different trials ($i \neq l$): $\mathbb{E}[X_{ij}X_{lj'}] = p_j p_{j'}$ (independent).
   - $\text{Cov}(O_j, O_{j'}) = (N^2-N)p_j p_{j'} - N^2 p_j p_{j'} = -Np_j p_{j'}$

2. **Why $\sqrt{E_j}$ and not $\sqrt{E_j(1-p_j)}$?**
   The puzzle: The true standard deviation of O_j is $\sigma_j = \sqrt{E_j(1-p_j)}$, yet the χ^2 statistic uses $\sqrt{E_j}$ in the denominator. Why does this work?

   Step 1 – Define Z_j: Let $Z_j = \dfrac{O_j - E_j}{\sqrt{E_j}}$, so $\chi^2 = \sum_{j=1}^{k} Z_j^2 = \boldsymbol{Z}^T\boldsymbol{Z}$ where $\boldsymbol{Z} = (Z_i)_{1\leq i\leq k}^T \in \mathbb{R}^k$ is a column vector.

   Step 2 – The Z_j are not standard normals: Since we divided by $\sqrt{E_j}$ instead of $\sqrt{E_j(1-p_j)}$:

$$\text{Var}(Z_j) = \frac{\text{Var}(O_j)}{E_j} = \frac{E_j(1-p_j)}{E_j} = 1 - p_j \neq 1$$

   Also, the Z_j are correlated (using $\text{Cov}(aX, bY) = ab\text{Cov}(X,Y)$):

$$\text{Cov}(Z_j, Z_{j'}) = \frac{\text{Cov}(O_j, O_{j'})}{\sqrt{E_j}\sqrt{E_{j'}}} = \frac{-Np_j p_{j'}}{\sqrt{Np_j}\sqrt{Np_{j'}}} = -\sqrt{p_j p_{j'}}$$

   Step 3 – Covariance matrix: Let $\sqrt{\boldsymbol{p}} = \left(\sqrt{p_i}\right)_{1\leq i\leq k}^T \in \mathbb{R}^k$ (column vector). The covariance matrix $\boldsymbol{\Sigma} \in \mathbb{R}^{k\times k}$ of $\boldsymbol{Z}$ is:

$$\boldsymbol{\Sigma} = \boldsymbol{I}_k - \sqrt{\boldsymbol{p}}\sqrt{\boldsymbol{p}}^T$$

   where $\sqrt{\boldsymbol{p}}\sqrt{\boldsymbol{p}}^T$ is the outer product (a $k \times k$ matrix).
   Entries: $\boldsymbol{\Sigma}_{jj} = 1 - p_j$ (diagonal), $\boldsymbol{\Sigma}_{jj'} = -\sqrt{p_j p_{j'}}$ (off-diagonal).

Step 4 – $\boldsymbol{\Sigma}$ is idempotent: Since $\sqrt{\boldsymbol{p}}^T\sqrt{\boldsymbol{p}} = \sum_{j=1}^{k} p_j = 1$ (a scalar):

$$\boldsymbol{\Sigma}^2 = (\boldsymbol{I}_k - \sqrt{\boldsymbol{p}}\sqrt{\boldsymbol{p}}^T)^2 = \boldsymbol{I}_k - 2\sqrt{\boldsymbol{p}}\sqrt{\boldsymbol{p}}^T + \sqrt{\boldsymbol{p}}\underbrace{(\sqrt{\boldsymbol{p}}^T\sqrt{\boldsymbol{p}})}_{=1}\sqrt{\boldsymbol{p}}^T = \boldsymbol{I}_k - \sqrt{\boldsymbol{p}}\sqrt{\boldsymbol{p}}^T = \boldsymbol{\Sigma}$$

Step 5 – Cochran's theorem: For $\boldsymbol{Z} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{\Sigma})$ with $\boldsymbol{\Sigma}$ idempotent: $\boldsymbol{Z}^T\boldsymbol{Z} \sim \chi^2_{\text{rank}(\boldsymbol{\Sigma})}$.
This is a special case of Cochran's theorem on quadratic forms of normal variables.
Here $\text{rank}(\boldsymbol{\Sigma}) = \text{tr}(\boldsymbol{\Sigma}) = \sum_{j=1}^{k}(1 - p_j) = k - 1$.

Conclusion: The "wrong" normalization by $\sqrt{E_j}$ combined with the specific covariance structure yields an idempotent $\boldsymbol{\Sigma}$, which guarantees $\chi^2 \sim \chi^2_{k-1}$.

3. **Chi-squared distribution**: Recall that if $(Z_i)_{1\leq i\leq m} \overset{iid}{\sim} \mathcal{N}(0,1)$, then $\sum_{l=1}^{m} Z_l^2 \sim \chi^2_m$

4. **Degrees of freedom**: $k-1$ (not k) due to the constraint $\sum_{j=1}^{k} O_j = N$, which introduces a linear dependency among the k counts.

This explains why expected counts must be sufficiently large ($E_j \geq 5$): the test relies on the normal approximation to the multinomial, which requires large samples.

Bayesian Does not exist, see *contingency table*
Return to the table.

6.2.3 Exact test of goodness-of-fit

Also known as the multinomial exact test, this is the exact version of the χ^2 goodness-of-fit test. Unlike asymptotic tests, we compute the exact p-value using the multinomial distribution under H_0 rather than relying on large-sample approximations.

Assumptions

- Observations are independent.
- Each observation falls into exactly one of k mutually exclusive categories
- Small to moderate sample size (computationally feasible, typically $N \lessapprox 1000$)
- For large samples, the χ^2 test is preferred as the exact test becomes computationally expensive

Frequentist Let N be the total number of observations, $(p_i)_{1\leq i\leq k}$ the theoretical probabilities under H_0 with $\sum_{j=1}^{k} p_j = 1$, and $(O_i)_{1\leq i\leq k}$ the observed counts with $\sum_{j=1}^{k} O_j = N$.

$$\begin{cases} H_0 : \text{The data follow the specified distribution with probabilities } (p_i)_{1\leq i\leq k} \\ H_1 : \text{The data do not follow the specified distribution} \end{cases}$$

Methodology:

1. Under H_0, the observed counts follow a multinomial distribution:

$$\mathbb{P}(\boldsymbol{O} = (O_i)_{1\leq i\leq k} \,|H_0) = \frac{N!}{\prod_{j=1}^{k} O_j!}\prod_{j=1}^{k} p_j^{O_j}$$

2. Compute the probability of the observed data: $P_{obs} = \mathbb{P}(\boldsymbol{O}_{obs}|H_0)$
3. Define "as extreme or more extreme" using a test statistic. Common choices:
   - **Probability criterion**: Tables with $\mathbb{P}(\boldsymbol{O}'|H_0) \leq P_{obs}$
   - **Chi-squared criterion**: Tables with $\chi^2(\boldsymbol{O}') \geq \chi^2(\boldsymbol{O}_{obs})$ where $\chi^2(\boldsymbol{O}) = \sum_{j=1}^{k} \frac{(O_j - Np_j)^2}{Np_j}$
4. The exact p-value is:
$$p\text{-value} = \sum_{\boldsymbol{O}' \in \mathcal{E}} \mathbb{P}(\boldsymbol{O}'|H_0)$$
where $\mathcal{E} = \{\boldsymbol{O}' : \boldsymbol{O}' \text{ is as or more extreme than } \boldsymbol{O}_{obs}\}$ and the sum is over all possible tables with $\sum_{j=1}^{k} O'_j = N$

Example with binomial case ($k = 2$): Suppose $N = 10$ tosses, $H_0 : p = 0.5$ (fair coin), observe $O_1 = 8$ heads.

- Probability of observing exactly 8 heads: $\mathbb{P}(O_1 = 8|H_0) = \binom{10}{8}(0.5)^{10} = 0.0439$
- Tables "as extreme or more extreme": $\{8, 9, 10, 0, 1, 2\}$ heads (two-sided test)
- Exact p-value: $p = \mathbb{P}(O_1 \in \{0, 1, 2, 8, 9, 10\}|H_0) = \sum_{x \in \{0,1,2,8,9,10\}} \binom{10}{x}(0.5)^{10} = 2 \times [0.0010 + 0.0098 + 0.0439] = 0.109$
- This is *exactly* the probability of observing an outcome as extreme or more extreme than what we got, computed by summing over discrete outcomes rather than integrating over a continuous distribution.

Special case - Binomial exact test: When $k = 2$, this reduces to the binomial test with $O_1 \sim \text{Binomial}(N, p_1)$.

Strengths

- Provides exact p-values without relying on asymptotic approximations
- Appropriate for small samples where χ^2 test may be unreliable
- No requirement for minimum expected counts (unlike χ^2 test requiring $E_j \geq 5$)

Weaknesses

- Computationally intensive for large N or many categories k
- Choice of "extreme" criterion (probability vs. χ^2) can affect results
- Conservative (actual Type I error rate may be below nominal α) due to discreteness

6.2.4 Fisher's exact test

This is the exact test of independence for 2×2 contingency tables. Unlike asymptotic tests (like the χ^2 test), we compute the exact p-value using the hypergeometric distribution under H_0 rather than relying on large-sample approximations. Initially Fisher used this test to distinguish drinks in which tea had been put before milk or vice-versa. For large samples, use the *G-test* or χ^2 test instead, as the exact test becomes computationally expensive.

Assumptions

- Observations are independent
- Each observation falls into exactly one cell of the 2×2 table
- Small to moderate sample size (computationally feasible, typically $n \lessapprox 1000$)
- For large samples, asymptotic tests (χ^2, G-test) are preferred

Frequentist Consider a 2×2 contingency table classifying n observations by two binary variables. For example, we might divide a population by gender (Men/Women) and study status (Studying/Non-Studying):

	Men	**Women**	**Row Total**
Studying	a	b	$a+b$
Non-Studying	c	d	$c+d$
Column Total	$a+c$	$b+d$	$n=a+b+c+d$

$\begin{cases} H_0 : \text{The two variables are independent (no association between gender and study status)} \\ H_1 : \text{The two variables are dependent (there is an association)} \end{cases}$

Methodology:

1. Under H_0 (independence), conditional on the row and column totals (margins), the count a in the upper-left cell follows a hypergeometric distribution:

$$\mathbb{P}(a|H_0, \text{margins fixed}) = \frac{\binom{a+b}{a}\binom{c+d}{c}}{\binom{n}{a+c}} = \frac{\binom{a+b}{b}\binom{c+d}{d}}{\binom{n}{b+d}}$$

   This represents drawing $a+c$ items (column 1 total) from a population of n items containing $a+b$ "successes" (row 1 total) and $c+d$ "failures" (row 2 total).

2. Compute the probability of the observed table: $P_{obs} = \mathbb{P}(a_{obs}|H_0)$

3. Define "as extreme or more extreme". For Fisher's exact test, the **probability criterion** is standard: tables with $\mathbb{P}(a'|H_0) \leq P_{obs}$ are considered as extreme or more extreme.

4. The exact p-value is:

$$p\text{-value} = \sum_{a' \in \mathcal{E}} \mathbb{P}(a'|H_0)$$

   where $\mathcal{E} = \{a' : \mathbb{P}(a'|H_0) \leq P_{obs}\}$ and the sum is over all possible values of a' consistent with the fixed margins (i.e., $\max(0, (a+b)-(b+d)) \leq a' \leq \min(a+b, a+c)$).

What is a'? a' is not an undefined symbol, but represents a specific possible value for the upper-left cell count:

- a' **is a possible count**: An integer value that the upper-left cell could take
- **Sample space**: Given fixed margins, a' can only take values in the range $[\max(0, (a+b)-(b+d)), \min(a+b, a+c)]$. Once a' is specified, all other cells are determined: $b' = (a+b)-a'$, $c' = (a+c)-a'$, $d' = (b+d)-b'$.
- **The set $\mathcal{E}$**: Contains all possible values a' that produce tables as extreme or more extreme (less probable) than the observed table.
- **Example**: With margins $a+b=10$, $c+d=10$, $a+c=8$, $b+d=12$, possible values are $a' \in \{0, 1, 2, \ldots, 8\}$ (since $\min(10, 8) = 8$). Each a' defines a complete table.

Connection to usual p-value definition: The p-value represents $\mathbb{P}(\text{observe table as extreme or more extreme}|H_0)$. Since the sample space (all possible tables with fixed margins) is discrete and finite, we sum the probabilities of individual outcomes rather than integrating. Each outcome is a specific configuration of the table, uniquely determined by the value of one cell (e.g., a) given the fixed margins.

Example: Suppose we observe the following table testing whether gender affects study status:

	Men	**Women**	**Row Total**
Studying	1	9	10
Non-Studying	11	3	14
Column Total	12	12	24

- Probability of observed table: $\mathbb{P}(a=1) = \frac{\binom{10}{1}\binom{14}{11}}{\binom{24}{12}} = \frac{10 \times 364}{2{,}704{,}156} \approx 0.00135$
- Possible values for a: $\max(0, 10-12) = 0$ to $\min(10, 12) = 10$, so $a \in \{0, 1, \ldots, 10\}$

- Tables as extreme or more extreme (one-sided test, $a \leq 1$): $a' \in \{0, 1\}$
- Exact p-value (one-sided): $p = \mathbb{P}(a = 0) + \mathbb{P}(a = 1) = \frac{\binom{10}{0}\binom{14}{12}}{\binom{24}{12}} + 0.00135 \approx 0.00003 + 0.00135 = 0.00138$
- This represents the exact probability of observing such an extreme association (or more extreme) if gender and study status were truly independent.

Bayesian Does not exist, see *contingency table*

Return to the table.

6.2.5 G-test

Also known as the likelihood-ratio test for goodness-of-fit, this is a statistical significance test based on maximum likelihood principles. It can be used to test both *goodness-of-fit* and *independence* between 2 variables, checking if there is a significant difference between expected and observed frequencies. This test tends to replace the χ^2 test as it provides better approximation to the theoretical χ^2 distribution, especially when observed counts are more than twice the expected counts.

Assumptions

- Observations are independent
- Each observation falls into exactly one of k mutually exclusive categories
- Expected count must be sufficiently large in each category (same guideline as χ^2 test: $E_i \geq 5$)
- For small samples where expected counts are low, use exact tests (Fisher's exact test for 2×2 tables, exact test of goodness-of-fit for larger tables)
- Large sample size (for asymptotic distribution to hold)

Frequentist The G-test comes in three variants depending on the experimental design:

- *Goodness-of-fit*: Expected frequencies E_i are computed from a theoretical distribution (e.g., testing if data follows a uniform distribution)
- *Independence*: Expected frequencies E_i are computed from marginal totals in a contingency table (e.g., testing if two categorical variables are independent)
- *Repeated tests* (heterogeneity): The first variable is analyzed for goodness-of-fit, and the second variable represents repetitions of the experiment. This allows assessment of goodness-of-fit across multiple subgroups simultaneously.

We focus on the **goodness-of-fit** case. Let N be the total number of observations, $(p_i)_{1 \leq i \leq k}$ the theoretical probabilities under H_0 with $\sum_{i=1}^{k} p_i = 1$, $(O_i)_{1 \leq i \leq k}$ the observed counts with $\sum_{i=1}^{k} O_i = N$, and $(E_i)_{1 \leq i \leq k}$ the expected counts with $E_i = Np_i$.

$$\begin{cases} H_0 : \text{The data follow the specified distribution with probabilities } (p_i)_{1 \leq i \leq k} \\ H_1 : \text{The data do not follow the specified distribution} \end{cases}$$

Methodology:

1. Compute expected counts: Under H_0, for each category i: $E_i = Np_i$
2. Calculate the G-statistic (also called G^2 or likelihood-ratio χ^2):

$$G = 2\sum_{i=1}^{k} O_i \ln\left(\frac{O_i}{E_i}\right)$$

3. Under H_0, G asymptotically follows a χ^2 distribution with $k-1$ degrees of freedom
4. **Compute p-value**: $p = \mathbb{P}(\chi^2_{k-1} \geq G_{obs})$
5. **Decision**: Reject H_0 if $p < \alpha$ (or equivalently, if $G > \chi^2_{k-1,1-\alpha}$)

Interpreting the components:

- **The ratio** O_i/E_i: Measures the fold-change or relative deviation for category i. Values near 1 indicate good fit; values far from 1 (either $\gg 1$ or $\ll 1$) indicate poor fit. Example: $O_i/E_i = 2$ means we observed twice what we expected.
- **The term** $O_i \ln(O_i/E_i)$: Represents category i's contribution to the log-likelihood ratio. It quantifies how much the data in category i favor the observed distribution over the null hypothesis, weighted by the sample size O_i (so categories with more observations contribute more). This is closely related to the Kullback-Leibler divergence, measuring information lost when approximating the observed distribution with the null hypothesis.

Derivation from likelihood ratio: The G-statistic derives from the likelihood-ratio test principle. We compare two models:

- **Null model**: Data follow the specified distribution with probabilities $(p_i)_{1 \leq i \leq k}$
- **Saturated model**: Each category has its own parameter estimated from the data: $\hat{p}_i = O_i/N$

The likelihood under the saturated model (maximum likelihood):

$$L(\hat{\boldsymbol{p}}|\boldsymbol{O}) = \prod_{i=1}^{k} \left(\frac{O_i}{N}\right)^{O_i}$$

The likelihood under the null hypothesis:

$$L(\boldsymbol{p}_0|\boldsymbol{O}) = \prod_{i=1}^{k} p_i^{O_i} = \prod_{i=1}^{k} \left(\frac{E_i}{N}\right)^{O_i}$$

The log-likelihood ratio is:

$$\begin{aligned}
\ln\left(\frac{L(\hat{\boldsymbol{p}}|\boldsymbol{O})}{L(\boldsymbol{p}_0|\boldsymbol{O})}\right) &= \ln\left(\frac{\prod_{i=1}^{k} \left(\frac{O_i}{N}\right)^{O_i}}{\prod_{i=1}^{k} \left(\frac{E_i}{N}\right)^{O_i}}\right) \\
&= \ln\left(\prod_{i=1}^{k} \left(\frac{O_i}{E_i}\right)^{O_i}\right) \\
&= \sum_{i=1}^{k} O_i \ln\left(\frac{O_i}{E_i}\right)
\end{aligned}$$

By Wilks' theorem, $-2\ln$(likelihood ratio) asymptotically follows a χ^2 distribution, giving us:

$$G = -2\ln\left(\frac{L(\boldsymbol{p}_0|\boldsymbol{O})}{L(\hat{\boldsymbol{p}}|\boldsymbol{O})}\right) = 2\sum_{i=1}^{k} O_i \ln\left(\frac{O_i}{E_i}\right) \overset{H_0}{\sim} \chi^2_{k-1}$$

Where does Wilks' theorem come from? Wilks' theorem (Wilks, 1938) is a fundamental result from asymptotic statistical theory. It derives from:

1. **Asymptotic normality of MLEs**: Under regularity conditions, $\sqrt{n}(\hat{\theta} - \theta_0) \xrightarrow{d} \mathcal{N}(0, I(\theta_0)^{-1})$ where $I(\theta)$ is the Fisher information matrix
2. **Taylor expansion of log-likelihood** around the MLE $\hat{\theta}$: $\ell(\theta_0) \approx \ell(\hat{\theta}) + \frac{1}{2}(\theta_0 - \hat{\theta})^T \ell''(\hat{\theta})(\theta_0 - \hat{\theta})$ (first-order term vanishes since $\ell'(\hat{\theta}) = 0$ at the MLE)

3. **Fisher information**: The Hessian relates to Fisher information as $-\ell''(\theta) \approx nI(\theta)$ where $I(\theta) = -\mathbb{E}[\ell''(\theta)]$

4. **Quadratic form of normals**: The likelihood ratio becomes $-2[\ell(\hat{\theta}_0) - \ell(\hat{\theta})] \approx n(\hat{\theta} - \theta_0)^T I(\theta_0)(\hat{\theta} - \theta_0) = Z^T Z$ where $Z = \sqrt{n} I(\theta_0)^{1/2}(\hat{\theta} - \theta_0) \sim \mathcal{N}(0, I_r)$, thus $Z^T Z \sim \chi^2_r$

The factor of 2 emerges naturally: the $\frac{1}{2}$ from the Taylor expansion multiplied by -2 gives the canonical chi-squared form. Without this factor, the statistic would not have the standard χ^2 distribution needed to compute p-values using standard tables.

Relationship to χ^2 test: Both tests are asymptotically equivalent and yield similar p-values for large samples. However, they differ in their derivation and finite-sample behavior:

Aspect	**Pearson's χ^2**	**G-test (Likelihood Ratio)**
Formula	$\chi^2 = \sum_{i=1}^{k} \frac{(O_i - E_i)^2}{E_i}$	$G = 2\sum_{i=1}^{k} O_i \ln\left(\frac{O_i}{E_i}\right)$
Deviation measure	Additive: $(O_i - E_i)$	Multiplicative: O_i/E_i
Metric	Squared difference	Log-ratio
Foundation	Heuristic/empirical (1900)	Likelihood principle (1930s)
Interpretation	Euclidean distance	Information/KL divergence
Development	Karl Pearson (1900)	Neyman, Pearson, Wilks

For most practical purposes, $G \approx \chi^2$ as $n \to \infty$, but G-test is preferred when $O_i > 2E_i$.

Example - Testing die fairness: Suppose we roll a die 60 times and want to test if it's fair ($H_0 : p_1 = \cdots = p_6 = 1/6$). Observed counts: $(8, 9, 7, 12, 13, 11)$.

- Expected counts under H_0: $E_i = 60 \times (1/6) = 10$ for all i
- G-statistic:

$$\begin{aligned} G &= 2[8\ln(8/10) + 9\ln(9/10) + 7\ln(7/10) + 12\ln(12/10) + 13\ln(13/10) + 11\ln(11/10)] \\ &= 2[8(-0.223) + 9(-0.105) + 7(-0.357) + 12(0.182) + 13(0.262) + 11(0.095)] \\ &= 2[-1.784 - 0.945 - 2.499 + 2.184 + 3.406 + 1.045] \\ &= 2[1.407] = 2.814 \end{aligned}$$

- Degrees of freedom: $k - 1 = 6 - 1 = 5$
- Critical value: $\chi^2_{5,0.95} = 11.07$ (for $\alpha = 0.05$)
- Decision: Since $G = 2.814 < 11.07$, we fail to reject H_0. The die appears fair.
- P-value: $\mathbb{P}(\chi^2_5 \geq 2.814) \approx 0.729$ (strong evidence for fairness)

For comparison, the χ^2 test on the same data gives $\chi^2 = 2.8$, nearly identical to $G = 2.814$.

Strengths

- Better approximation to the theoretical χ^2 distribution than Pearson's χ^2 test for many data configurations
- More accurate when $O_i > 2E_i$ (G-test is always superior in these cases)
- Theoretically grounded in likelihood ratio test framework
- Asymptotically equivalent to χ^2 test, giving similar results in most cases
- Can be decomposed additively for hierarchical models (useful for repeated tests design)

Weaknesses

- For small sample sizes, use exact tests instead (Fisher's exact test for 2×2 independence, exact test of goodness-of-fit for larger tables)
- Requires sufficiently large expected counts (same as χ^2 test: $E_i \geq 5$ guideline)
- Computationally slightly more complex than χ^2 test (requires logarithm calculations)
- Less familiar to many practitioners compared to χ^2 test

Bayesian Does not exist, see *contingency table*

Return to the table.

6.2.6 Cochran's Q test

Cochran's Q test is a non-parametric statistical test for comparing $k \geq 2$ related samples with binary (dichotomous) outcomes. It tests whether the proportion of "successes" differs across k treatments or conditions when each subject (or block) is measured under all conditions. This test is analogous to repeated measures ANOVA but specifically designed for binary data. It extends McNemar's test (which handles $k = 2$) to multiple treatments, and is related to the Friedman test (which handles ordinal data).

Assumptions

- The same subjects/blocks are measured under all k treatments (repeated measures design)
- Outcomes are dichotomous (binary): success/failure, yes/no, present/absent, coded as 1/0
- **Blocks are randomly selected** from the population or are representative
- **Observations within each block are independent** across treatments (no carryover effects)
- **Large sample size**: The test statistic follows a χ^2 distribution asymptotically; typically require at least $b \geq 10$ blocks and $k \geq 3$ treatments
- Each block should have **at least some variation** (blocks where all treatments give the same response contribute nothing to the test)

Frequentist Cochran's Q test is commonly used when:

- Testing if multiple diagnostic tests have equal sensitivity (each patient tested with all methods)
- Comparing effectiveness of k treatments in a crossover design with binary outcomes
- Analyzing survey data where b respondents each answer k yes/no questions
- Evaluating if several judges agree in their binary classifications

Data structure: The data are arranged in a $b \times k$ table where b is the number of blocks (subjects) and k is the number of treatments:

	Treatment 1	**Treatment 2**	$\cdots$	**Treatment k**
Block 1	X_{11}	X_{12}	$\cdots$	X_{1k}
Block 2	X_{21}	X_{22}	$\cdots$	X_{2k}
Block 3	X_{31}	X_{32}	$\cdots$	X_{3k}
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
Block b	X_{b1}	X_{b2}	$\cdots$	X_{bk}

where $\forall (i,j) \in [\![1,b]\!] \times [\![1,k]\!], X_{ij} \in \{0,1\}$ (0 = failure, 1 = success).

$$\begin{cases} H_0 : \text{The proportion of successes is equal across all } k \text{ treatments} \\ H_1 : \text{The proportion of successes differs for at least one treatment} \end{cases}$$

Methodology:

1. Arrange data in a $b \times k$ table with binary entries (0 or 1)
2. Calculate row and column totals:
   - Row totals: $X_{i\cdot} = \sum_{j=1}^{k} X_{ij}$ (number of successes for block i across all treatments)
   - Column totals: $X_{\cdot j} = \sum_{i=1}^{b} X_{ij}$ (number of successes for treatment j across all blocks)

- Grand total: $N = \sum_{i=1}^{b}\sum_{j=1}^{k} X_{ij}$

3. Compute Cochran's Q statistic:

$$Q = \frac{k(k-1)\sum_{j=1}^{k}\left(X_{\cdot j} - \frac{N}{k}\right)^2}{\sum_{i=1}^{b} X_{i\cdot}\left(k - X_{i\cdot}\right)}$$

where:

- **Numerator**: Measures variation in treatment totals (between-treatment variability)
- **Denominator**: Measures within-block variability
- The ratio compares treatment differences to within-subject variation

4. Under H_0, Q asymptotically follows a χ^2 distribution with $k-1$ degrees of freedom
5. **Compute p-value**: $p = \mathbb{P}(\chi^2_{k-1} \geq Q_{obs})$
6. **Decision**: Reject H_0 if $p < \alpha$ (or equivalently, if $Q > \chi^2_{k-1,1-\alpha}$)

What the test statistic measures:

- **Numerator** $k(k-1)\sum_{j=1}^{k}(X_{\cdot j} - N/k)^2$: Quantifies how much the treatment totals deviate from what we'd expect if all treatments were equally effective (expected total $= N/k$)
- **Denominator** $\sum_{i=1}^{b} X_{i\cdot}(k - X_{i\cdot})$: Represents the variability within blocks. For a block with row total $X_{i\cdot}$, the maximum variability occurs when $X_{i\cdot} = k/2$
- Large Q indicates treatment totals differ more than expected by chance given the within-block variation

Example - Testing pain relief methods: Suppose 6 patients each try 3 different pain relief methods (A, B, C), and we record whether pain was relieved (1) or not (0):

Patient	**Method A**	**Method B**	**Method C**	**Row total** $(X_{i\cdot})$
1	1	0	1	2
2	1	1	1	3
3	0	0	1	1
4	1	0	1	2
5	1	1	0	2
6	0	0	1	1
Col total $(X_{\cdot j})$	4	2	5	$N = 11$

- $b = 6$ patients, $k = 3$ methods
- Column totals: $X_{\cdot 1} = 4$, $X_{\cdot 2} = 2$, $X_{\cdot 3} = 5$
- Row totals: $X_{1\cdot} = 2$, $X_{2\cdot} = 3$, $X_{3\cdot} = 1$, $X_{4\cdot} = 2$, $X_{5\cdot} = 2$, $X_{6\cdot} = 1$
- Grand total: $N = 11$
- Expected count per treatment under H_0: $N/k = 11/3 \approx 3.67$
- **Numerator**:

$$\begin{aligned} k(k-1)\sum_{j=1}^{k}\left(X_{\cdot j} - \frac{N}{k}\right)^2 &= 3(2)[(4-3.67)^2 + (2-3.67)^2 + (5-3.67)^2] \\ &= 6[0.11 + 2.79 + 1.77] \\ &= 6[4.67] = 28.02 \end{aligned}$$

- **Denominator**:

$$\begin{aligned}\sum_{i=1}^{b} X_{i\cdot}(k - X_{i\cdot}) &= 2(3-2) + 3(3-3) + 1(3-1) + 2(3-2) + 2(3-2) + 1(3-1) \\ &= 2(1) + 3(0) + 1(2) + 2(1) + 2(1) + 1(2) \\ &= 2 + 0 + 2 + 2 + 2 + 2 = 10\end{aligned}$$

- **Q statistic**: $Q = \frac{28.02}{10} = 2.802$
- Degrees of freedom: $k - 1 = 3 - 1 = 2$
- Critical value: $\chi^2_{2,0.95} = 5.99$ (for $\alpha = 0.05$)
- Decision: Since $Q = 2.802 < 5.99$, we fail to reject H_0. The three methods appear equally effective.
- P-value: $\mathbb{P}(\chi^2_2 \geq 2.802) \approx 0.246$ (no significant difference)

Relationship to other tests:

- **McNemar's test**: When $k = 2$, Cochran's Q test reduces to McNemar's test for paired binary data
- **Friedman test**: Similar purpose but for ordinal/continuous data (ranks), not just binary
- **Repeated measures ANOVA**: For continuous outcomes with normal distribution assumption

Strengths

- Non-parametric: No assumption about distribution of outcomes (only that they're binary)
- Accounts for repeated measures structure (same subjects under multiple conditions)
- **Natural extension of McNemar's test** to $k > 2$ treatments
- More powerful than treating data as independent samples (uses within-subject correlation)
- Easy to compute and interpret for binary outcomes

Weaknesses

- Requires binary outcomes - cannot handle ordinal or continuous data directly
- **Asymptotic test**: Requires sufficient sample size ($b \geq 10$ typically) for chi-squared approximation to be valid
- **Does not identify which treatments differ** - only detects that at least one differs; post-hoc pairwise comparisons (McNemar tests with correction) needed to identify specific differences
- Blocks with no variation (all 0s or all 1s) contribute nothing to the denominator, which can reduce power
- Less efficient than parametric alternatives if stronger assumptions hold

Bayesian Does not exist, see *contingency table*

Return to the table.

6.2.7 Sign test

Also known as the binomial sign test, this is a non-parametric statistical method to test for consistent differences between pairs of observations, such as the weight of subjects before and after treatment. For comparisons of paired observations (x, y), the sign test is most useful if comparisons can only be expressed qualitatively as $x > y$, $x = y$, or $x < y$. If instead the differences can be quantified numerically, consider using the *paired t-test* or *Wilcoxon signed-rank test*, which will usually have greater power than the sign test to detect consistent differences.

Assumptions Let $\forall i \in [\![1, n]\!]$, $Z_i = X_i - Y_i$

- The pairs (X_i, Y_i) are independent across observations
- Each Z_i comes from a continuous distribution (to avoid ties)
- The values X_i and Y_i represent paired or matched observations
- The distribution of differences need not be symmetric (unlike Wilcoxon signed-rank test)

Frequentist The sign test examines whether the median of the pairwise differences is zero, equivalently whether $\mathbb{P}(\{X > Y\}) = 0.5$.

Let $p = \mathbb{P}(\{X > Y\})$, then:

$$\begin{cases} H_0 : p = 0.5 \text{ (each element in a pair is equally likely to be larger)} \\ H_1 : p \neq 0.5 \text{ (two-sided test)} \\ H_1 : p > 0.5 \text{ (one-sided: } X \text{ tends to be larger than } Y) \\ H_1 : p < 0.5 \text{ (one-sided: } Y \text{ tends to be larger than } X) \end{cases}$$

Methodology:

1. Compute differences: For each pair (x_i, y_i), determine the sign of $x_i - y_i$
2. Count signs: Let n be the number of pairs. Omit pairs where $x_i = y_i$ (ties), leaving $m \leq n$ pairs with non-zero differences:
   - $n_+ =$ number of pairs where $x_i > y_i$ (positive differences)
   - $n_- =$ number of pairs where $x_i < y_i$ (negative differences)
   - $m = n_+ + n_-$ (total non-zero differences)
3. Define test statistic: The test statistic is the number of positive differences:

$$W = \sum_{i=1}^{m} \mathbf{1}_{\{x_i > y_i\}} = n_+$$

   Under H_0, each difference is equally likely to be positive or negative, so:

$$W \sim \mathcal{B}(m, 0.5)$$

4. **Compute p-value**: Using the binomial distribution with $p = 0.5$:
   - **Two-sided**: p-value $= 2 \min(\mathbb{P}(W \leq n_+), \mathbb{P}(W \geq n_+))$
   - **One-sided** $(H_1 : p > 0.5)$: p-value $= \mathbb{P}(W \geq n_+)$
   - **One-sided** $(H_1 : p < 0.5)$: p-value $= \mathbb{P}(W \leq n_+)$
5. **Decision**: Reject H_0 if p-value $< \alpha$

Large sample approximation: For large m (typically $m \geq 20$), use the normal approximation to the binomial:

$$Z = \frac{W - \frac{m}{2}}{\frac{\sqrt{m}}{2}} \approx \mathcal{N}(0, 1)$$

with continuity correction: $Z = \dfrac{|W - \frac{m}{2}| - 0.5}{\frac{\sqrt{m}}{2}}$

Example - Weight loss study: Suppose 12 patients are weighed before and after a diet program. We observe the following signs of differences (After − Before):

Patient	1	2	3	4	5	6	7	8	9	10	11	12
Sign	−	−	+	−	−	0	−	−	−	−	−	−

- $n_+ = 1$ (one patient gained weight)

- $n_- = 10$ (ten patients lost weight)
- Ties: 1 patient had no change (excluded)
- $m = 11$ non-zero differences
- Test statistic: $W = n_+ = 1$
- Under H_0: $W \sim \mathcal{B}(11, 0.5)$
- P-value (two-sided): $p = 2\mathbb{P}(W \leq 1) = 2\left[\binom{11}{0}(0.5)^{11} + \binom{11}{1}(0.5)^{11}\right] = 2\left[\frac{1+11}{2048}\right] = 2 \times 0.00586 = 0.0117$
- Decision: At $\alpha = 0.05$, we reject H_0. The diet program appears effective at reducing weight.

Strengths

- Minimal assumptions: No normality or symmetry required
- Robust to outliers: Only uses sign information, not magnitude
- **Appropriate for ordinal data**: Works when only direction of difference is known
- **Simple to compute and interpret**
- Valid for small sample sizes (exact p-values from binomial distribution)

Weaknesses

- Low statistical power: Less powerful than parametric tests when their assumptions hold
- Discards magnitude information: Ignores how much larger one value is than another
- **Less powerful than Wilcoxon signed-rank test**: When differences can be ranked by magnitude
- **Ties must be excluded**: Reduces effective sample size
- **Conservative for small samples**: Discrete nature of binomial leads to conservative p-values

6.2.8 Contingency coefficient: Cramér's V

A measure of association between two categorical variables in a contingency table, based on χ^2. It varies from 0 (no association) to 1 (perfect association), making it analogous to R^2 for categorical data. Unlike the raw χ^2 statistic, Cramér's V is normalized and comparable across tables of different sizes.

Frequentist Consider n observations, each classified by two categorical variables A and B with r and c categories respectively. Let:

$$\begin{cases} n_{ij}\text{: count of observations where } A = i \text{ and } B = j \\ n_{i\cdot} = \sum_{j=1}^{c} n_{ij}\text{: row marginal total (sum of row } i) \\ n_{\cdot j} = \sum_{i=1}^{r} n_{ij}\text{: column marginal total (sum of column } j) \\ n = \sum_{i=1}^{r}\sum_{j=1}^{c} n_{ij}\text{: total sample size} \\ r\text{: number of rows (categories of } A) \\ c\text{: number of columns (categories of } B) \end{cases}$$

Step 1: Compute the χ^2 test statistic for independence:

$$\chi^2 = \sum_{i=1}^{r}\sum_{j=1}^{c} \frac{\left(n_{ij} - \frac{n_{i\cdot}n_{\cdot j}}{n}\right)^2}{\frac{n_{i\cdot}n_{\cdot j}}{n}}$$

where $\frac{n_{i\cdot}n_{\cdot j}}{n}$ is the expected count under independence (in frequentist approach if $A \perp B$, then $\mathbb{P}(A \cap B) = \frac{n_A \times n_B}{n}$).

Step 2: Compute Cramér's V. The standard formula is:

$$V = \sqrt{\frac{\chi^2/n}{\min(r-1, c-1)}}$$

The division by n normalizes for sample size, and $\min(r-1, c-1)$ adjusts for table dimensions, ensuring $V \in [0, 1]$.

Bias-corrected Cramér's V (Bergsma, 2013): For small samples, the standard formula has upward bias. The bias-corrected version is:

$$V_{\text{corrected}} = \sqrt{\frac{\max\left(0, \frac{\chi^2}{n} - \frac{(r-1)(c-1)}{n-1}\right)}{\min\left(r - \frac{(r-1)^2}{n-1} - 1, c - \frac{(c-1)^2}{n-1} - 1\right)}}$$

This correction is recommended when $n < 100$ or when χ^2/n is small.

Interpretation guidelines:

- $V \approx 0$: No association (variables are independent)
- $V \approx 0.1$: Weak association
- $V \approx 0.3$: Moderate association
- $V \approx 0.5$ **or higher**: Strong association
- $V = 1$: Perfect association (knowing one variable determines the other)

Note: These thresholds are approximate and context-dependent.

Example - Education and Employment Status: Survey of 200 people classified by education level (3 categories) and employment status (2 categories):

	Employed	**Unemployed**	**Row Total**
High School	40	30	70
Bachelor's	50	20	70
Graduate	55	5	60
Column Total	145	55	200

- Expected count for (High School, Employed): $\frac{70 \times 145}{200} = 50.75$
- Expected count for (High School, Unemployed): $\frac{70 \times 55}{200} = 19.25$
- Continuing for all cells, compute: $\chi^2 = \frac{(40-50.75)^2}{50.75} + \frac{(30-19.25)^2}{19.25} + \cdots \approx 18.44$
- $\min(r-1, c-1) = \min(3-1, 2-1) = 1$
- Cramér's V: $V = \sqrt{\frac{18.44/200}{1}} = \sqrt{0.0922} \approx 0.304$
- **Interpretation**: Moderate association between education level and employment status

Assumptions

- Observations are independent (each individual counted once)
- Both variables are categorical (nominal or ordinal)
- Note: For ordinal variables, consider using measures that leverage ordering (Kendall's tau, Spearman's ρ)
- Sufficient sample size for χ^2 validity (expected counts ≥ 5 in most cells)

Strengths

- Normalized and interpretable: Values always in $[0, 1]$, comparable across studies
- Good analog of R^2 for categorical variables
- **Scale-invariant**: Not affected by sample size (unlike raw χ^2)
- **Works for any table size**: $r \times c$ tables with $r, c \geq 2$
- Simple to compute from χ^2 test results

Weaknesses

- Does not indicate direction of association (unlike correlation coefficients)
- Upward bias in small samples (use bias-corrected version when $n < 100$)
- **Depends on marginal distributions**: Can be affected by unbalanced margins
- **Ignores ordering in ordinal variables**: Treats all categories as nominal
- **Inherits χ^2 limitations**: Requires adequate expected cell counts, sensitive to sparse tables
- Difficult to reach $V = 1$ in rectangular tables where $r \neq c$

Alternatives

- 2×2 **tables**: Use Phi coefficient ($\phi = \sqrt{\chi^2/n}$, equivalent to Cramér's V)
- **Ordinal variables**: Use Kendall's tau-b or Spearman's ρ to leverage ordering
- **Asymmetric relationships**: Use Goodman-Kruskal's lambda or tau
- **Small samples**: Use Fisher's exact test instead of χ^2, then compute V if needed

Bayesian Cramér's V is a descriptive measure, not a hypothesis test, so there is no direct Bayesian equivalent. For Bayesian inference about association in contingency tables, see *Contingency table from a Bayesian perspective*.

Return to the table.

6.2.9 Contingency table from a Bayesian perspective

Bayesian test of independence between two categorical variables in a contingency table. Instead of computing p-values, this approach uses Bayes factors to quantify the evidence for independence (H_0) versus association (H_1). The methodology depends critically on the *sampling scheme* used to collect the data.

Bayesian Hypotheses: $\begin{cases} H_0\text{: Independence - cell probabilities factorize as } \theta_{ij} = \theta_{i\cdot}\theta_{\cdot j} \\ H_1\text{: Association - cell probabilities are unrestricted} \end{cases}$

The four sampling schemes:

The sampling scheme determines which aspects of the table are fixed by design versus random. This fundamentally affects the likelihood and thus the Bayes factor computation.

1. **Poisson sampling**:
   - *What's fixed*: Nothing - study observes events over fixed time/space
   - *What's random*: All cell counts AND the grand total n
   - *Model*: Each cell $n_{ij} \sim \text{Poisson}(\lambda_{ij})$ independently
   - *Example*: Count traffic accidents by time-of-day and weather condition over one month - total number of accidents is random

2. **Joint multinomial sampling**:
   - *What's fixed*: Grand total n (sample size decided in advance)
   - *What's random*: All cell counts (margins are random)
   - *Model*: $(n_{11}, \ldots, n_{rc}) \sim \text{Multinomial}(n; \theta_{11}, \ldots, \theta_{rc})$
   - *Example*: Survey exactly 200 people, classify by education and employment - sample size fixed, but margins random
3. **Independent multinomial sampling**:
   - *What's fixed*: One set of margins (e.g., row totals $n_{i\cdot}$)
   - *What's random*: Cell counts within each row/column
   - *Model*: Each row independently: $(n_{i1}, \ldots, n_{ic}) \sim \text{Multinomial}(n_{i\cdot}; \theta_{i1}, \ldots, \theta_{ic})$
   - *Example*: Sample exactly 50 men and 50 women, measure political preference - gender margins fixed by design
4. **Hypergeometric sampling**:
   - *What's fixed*: Both row AND column margins
   - *What's random*: Only the arrangement of fixed totals into cells
   - *Model*: Hypergeometric distribution (as in Fisher's exact test)
   - *Example*: Fisher's tea tasting - 8 cups, 4 milk-first, 4 tea-first, subject identifies 4 as milk-first - all margins predetermined

Bayes Factor formulas (Gunel and Dickey, 1974):
For a 2×2 table with cells:

	Col 1	**Col 2**	**Row total**
Row 1	n_{11}	n_{12}	$n_{1\cdot}$
Row 2	n_{21}	n_{22}	$n_{2\cdot}$
Col total	$n_{\cdot 1}$	$n_{\cdot 2}$	n

Using uniform (Jeffreys) priors with hyperparameter $\alpha = 1/2$, the Bayes factors are:

1. **Poisson sampling**:

$$BF_{01} = \frac{(n_{1\cdot} + n_{2\cdot} + 1)(n_{\cdot 1} + n_{\cdot 2} + 1)}{2(n+1)}$$

2. **Joint multinomial sampling**:

$$BF_{01} = \frac{n!}{(n_{1\cdot})!(n_{2\cdot})!} \cdot \frac{(n_{\cdot 1})!(n_{\cdot 2})!}{n!} \cdot \frac{B(n_{1\cdot} + \frac{1}{2}, n_{2\cdot} + \frac{1}{2}) \cdot B(n_{\cdot 1} + \frac{1}{2}, n_{\cdot 2} + \frac{1}{2})}{B(n_{11} + \frac{1}{2}, n_{12} + \frac{1}{2}, n_{21} + \frac{1}{2}, n_{22} + \frac{1}{2})}$$

   where $B(\cdot)$ is the multivariate beta function: $B(a_{1:k}) = \dfrac{\prod_{i=1}^{k} \Gamma(a_i)}{\Gamma(\sum_{i=1}^{k} a_i)}$

3. **Independent multinomial sampling** (row margins fixed):

$$BF_{01} = \frac{\prod_{i=1}^{2} \Gamma(n_{i\cdot} + 1)}{\Gamma(n+2)} \cdot \frac{\Gamma(n+2) \prod_{j=1}^{2} \Gamma(n_{\cdot j} + \frac{1}{2})}{\Gamma(n+1) \prod_{i,j} \Gamma(n_{ij} + \frac{1}{2})}$$

4. **Hypergeometric sampling**:

$$BF_{01} = \frac{\binom{n_{1\cdot}}{n_{11}}\binom{n_{2\cdot}}{n_{21}}}{\binom{n}{n_{\cdot 1}}} \cdot \frac{B(n_{11} + \frac{1}{2}, n_{12} + \frac{1}{2}) \cdot B(n_{21} + \frac{1}{2}, n_{22} + \frac{1}{2})}{B(n_{\cdot 1} + \frac{1}{2}, n_{\cdot 2} + \frac{1}{2})}$$

Interpretation of Bayes factors (Jeffreys, 1961; Kass and Raftery, 1995):

Bayes Factor BF_{01}	**Evidence for H_0 (independence)**
$< 1/100$	Decisive evidence against H_0 (strong association)
$1/100$ to $1/10$	Strong evidence against H_0
$1/10$ to $1/3$	Moderate evidence against H_0
$1/3$ to 3	Weak/inconclusive evidence
3 to 10	Moderate evidence for H_0 (independence)
10 to 100	Strong evidence for H_0
> 100	Decisive evidence for H_0

Note: $BF_{10} = 1/BF_{01}$ gives evidence for association.

Example - Gender and Political Preference:

Suppose we survey people about political preference, using *independent multinomial sampling* with fixed gender margins: 60 men and 40 women.

	Left	**Right**	**Row Total**
Men	35	25	60
Women	15	25	40
Column Total	50	50	100

Using the independent multinomial formula with $\alpha = 1/2$:

- $\Gamma(61)/\Gamma(102) = 1/101!$ (simplifies in ratio)
- $\Gamma(102) \cdot [\Gamma(50.5)]^2/[\Gamma(101) \cdot \Gamma(35.5) \cdot \Gamma(25.5) \cdot \Gamma(15.5) \cdot \Gamma(25.5)]$
- After computation: $BF_{01} \approx 0.23$
- **Interpretation**: $BF_{01} = 0.23 < 1/3$ indicates moderate evidence against independence, suggesting political preference differs between men and women
- Equivalently: $BF_{10} = 1/0.23 \approx 4.3$, moderate evidence for association

Comparison with frequentist approach:

- **Frequentist** (χ^2 test): Computes p-value, tests null hypothesis at fixed α
- **Bayesian**: Quantifies *relative evidence* for H_0 vs H_1, no arbitrary threshold
- Advantage: Can support H_0 (independence), not just reject it
- Bayesian approach naturally handles small samples (no asymptotic approximations needed)

Computational implementation:

- **R package**: `BayesFactor` - function `contingencyTableBF()`
- **Python**: `scipy.special.gamma` for computing gamma functions manually
- For large factorials, use log-gamma: $\log(BF_{01}) = \sum \log\Gamma(\cdot) - \sum \log\Gamma(\cdot)$ to avoid numerical overflow

Extension to $r \times c$ tables:

For larger tables, the formulas generalize but become more complex. The general principle:

$$BF_{01} = \frac{P(\text{data}|H_0, \text{prior})}{P(\text{data}|H_1, \text{prior})}$$

where marginal likelihoods are computed by integrating over Dirichlet priors. Modern software handles this automatically.

Assumptions

- Observations are independent (each individual counted once)
- Correct sampling scheme must be identified - different schemes give different Bayes factors
- **Prior specification**: Results depend on choice of prior hyperparameters (default: $\alpha = 1/2$)
- Both variables are categorical

Strengths

- Quantifies evidence for both hypotheses: Can support independence, not just reject it
- No asymptotic approximations: Exact for any sample size (unlike χ^2 test)
- **Coherent evidence accumulation**: Can update Bayes factor as data arrives
- **No arbitrary significance threshold**: Provides continuous measure of evidence
- Handles multiple sampling schemes in a principled way
- More interpretable than p-values for non-experts

Weaknesses

- Requires identifying correct sampling scheme - misspecification affects results
- Sensitive to prior specification (though uniform priors are often reasonable)
- **Computational complexity**: Formulas involve gamma/beta functions, require software
- **Less familiar to most researchers** compared to frequentist tests
- Different software may use different default priors, affecting comparability
- For very large tables, computation can be intensive

When to use

- When you want to quantify evidence *for* independence (not just test against it)
- Small sample sizes where χ^2 approximations are unreliable
- When combining evidence across studies
- When prior information about cell probabilities is available

Return to the table.

6.2.10 Wilcoxon signed-rank test

Also known as the Wilcoxon test for paired samples or one-sample Wilcoxon test, this is a non-parametric test for the location (median) of a distribution. It is used in two contexts: (1) testing if a population median equals a specified value (one-sample), or (2) testing if the median of paired differences equals zero (paired samples). It is a good alternative to the t-test when the normality assumption does not hold, as it uses ranks instead of raw values.

Note: The *Wilcoxon rank-sum test* (for two independent samples) is equivalent to the *Mann-Whitney U test*, covered separately.

Frequentist

One-sample test :
Let $x_{1:n}$ be a sample from a continuous distribution with median θ.

$$\begin{cases} H_0 : \theta = \theta_0 \text{ (median equals hypothesized value)} \\ H_1 : \theta \neq \theta_0 \text{ (two-sided)} \\ H_1 : \theta > \theta_0 \text{ (one-sided upper)} \\ H_1 : \theta < \theta_0 \text{ (one-sided lower)} \end{cases}$$

Paired-sample test :
Let $(x_1, y_1), \ldots, (x_n, y_n)$ be n paired observations. Let $d_i = x_i - y_i$ be the differences.

$$\begin{cases} H_0 : \theta_d = 0 \text{ (median of differences is zero)} \\ H_1 : \theta_d \neq 0 \text{ (two-sided)} \\ H_1 : \theta_d > 0 \text{ (one-sided: } x \text{ tends to be larger)} \\ H_1 : \theta_d < 0 \text{ (one-sided: } y \text{ tends to be larger)} \end{cases}$$

Test procedure :

1. **Compute differences**:
   - One-sample: $d_i = x_i - \theta_0$
   - Paired: $d_i = x_i - y_i$
2. **Remove zeros**: Exclude any $d_i = 0$. Let m = number of non-zero differences. (Zeros provide no information about direction)
3. **Rank absolute values**: Compute $|d_i|$ for all non-zero differences, then rank them from 1 (smallest) to m (largest). Let R_i be the rank of $|d_i|$.
   - **Handling ties**: If $|d_i| = |d_j|$ (*tied* means equal values), assign both the average rank.
     Example: Suppose you have these absolute differences to rank:

Value $\lvert d_i \rvert$	Natural position	Rank assigned
2	1st	1
5	2nd	3 (average of 2, 3, 4)
5	3rd	3 (average of 2, 3, 4)
5	4th	3 (average of 2, 3, 4)
8	5th	5

     The three values of 5 are *tied* (equal), so they share positions 2, 3, 4. We assign them all the average rank: $(2 + 3 + 4)/3 = 3$. This ensures fairness and preserves the sum of ranks: $1 + 3 + 3 + 3 + 5 = 15 = \frac{5 \times 6}{2}$
4. **Compute signed-rank statistic**:
   - Let $\text{sgn}(d_i) = \begin{cases} +1 & \text{if } d_i > 0 \\ -1 & \text{if } d_i < 0 \end{cases}$
   - Compute: $W = \sum_{i=1}^{m} \text{sgn}(d_i) \cdot R_i$
   - Alternatively, use: $W^+ = \sum_{d_i > 0} R_i$ (sum of positive ranks only)
   - Note: $W = W^+ - W^-$ where $W^- = \sum_{d_i < 0} R_i$, and $W^+ + W^- = \frac{m(m+1)}{2}$
5. **Compute p-value**:
   - **Small samples** ($m \leq 20$): Use exact distribution from tables (critical values of W^+ under H_0)
   - **Large samples** ($m > 20$): Use normal approximation

Large-sample normal approximation:
Under H_0, for large m:

$$Z = \frac{W - 0}{\sqrt{\frac{m(m+1)(2m+1)}{6}}} \approx \mathcal{N}(0, 1)$$

Or equivalently, using W^+:

$$Z = \frac{W^+ - \frac{m(m+1)}{4}}{\sqrt{\frac{m(m+1)(2m+1)}{24}}} \approx \mathcal{N}(0,1)$$

Continuity correction: For better accuracy, use:

$$Z = \frac{|W^+ - \frac{m(m+1)}{4}| - 0.5}{\sqrt{\frac{m(m+1)(2m+1)}{24}}}$$

Then compute:

- **Two-sided**: $p = 2\Phi(-|Z|)$ where Φ is the standard normal CDF
- **One-sided upper**: $p = \Phi(-Z)$
- **One-sided lower**: $p = \Phi(Z)$

Why this test works:
Under H_0 (median $= \theta_0$), positive and negative differences are equally likely and symmetric. Thus:

- Each rank is equally likely to be positive or negative
- Expected value: $\mathbb{E}[W] = 0$ and $\mathbb{E}[W^+] = \frac{m(m+1)}{4}$
- Large deviation from expected value indicates asymmetry, evidence against H_0

The test is more powerful than the sign test because it uses *magnitude information* (ranks of $|d_i|$), not just signs.

Example - Blood pressure reduction study:
10 patients measured before and after treatment. Test if treatment reduces blood pressure (one-sided: median difference > 0, meaning Before $>$ After).

Patient	**Before**	**After**	d_i	$\lvert d_i \rvert$	**Rank** R_i
1	145	140	5	5	2.5
2	160	155	5	5	2.5
3	138	140	-2	2	1
4	150	142	8	8	7
5	155	148	7	7	5.5
6	142	142	0	-	-
7	148	138	10	10	8
8	152	140	12	12	9
9	156	150	6	6	4
10	162	155	7	7	5.5

Step-by-step:

1. Differences: $d_i = \text{Before} - \text{After}$
2. Exclude patient 6 ($d_6 = 0$), leaving $m = 9$
3. Rank absolute values (note ties for $|d| = 5$ and $|d| = 7$):
   - $|d| = 2$ (patient 3): rank 1
   - $|d| = 5$ (patients 1,2): average rank $(2 + 3)/2 = 2.5$ each
   - $|d| = 6$ (patient 9): rank 4
   - $|d| = 7$ (patients 5,10): average rank $(5 + 6)/2 = 5.5$ each
   - $|d| = 8$ (patient 4): rank 7
   - $|d| = 10$ (patient 7): rank 8
   - $|d| = 12$ (patient 8): rank 9

4. Compute W^+ (sum of positive ranks):

$$W^+ = 2.5 + 2.5 + 7 + 5.5 + 8 + 9 + 4 + 5.5 = 44$$

   Only patient 3 has negative difference, contributing $W^- = 1$

5. Check: $W^+ + W^- = 44 + 1 = 45 = \frac{9 \times 10}{2}$

6. Normal approximation ($m = 9$ is borderline, but we'll use it):

$$Z = \frac{44 - \frac{9 \times 10}{4}}{\sqrt{\frac{9 \times 10 \times 19}{24}}} = \frac{44 - 22.5}{\sqrt{71.25}} = \frac{21.5}{8.44} \approx 2.55$$

7. One-sided p-value (upper): $p = \Phi(-2.55) \approx 0.0054$

8. **Conclusion**: At $\alpha = 0.05$, reject H_0. Strong evidence that treatment reduces blood pressure.

Assumptions

- Observations are independent (or pairs are independent in paired design)
- Distribution is symmetric around its median (for test to be about the median)
  - Without symmetry, the test is about the *pseudo-median* (median of pairwise averages), a less interpretable quantity
- **Data is at least ordinal**: Can rank differences
- **Continuous distribution**: Ties should be rare (though methods exist for handling ties)

Strengths

- Non-parametric: No normality assumption required
- More powerful than the sign test: Uses magnitude information (ranks), not just direction
- **Robust to outliers**: Ranks are insensitive to extreme values
- **Exact test available**: For small samples, no asymptotic approximations needed
- Can be used with ordinal data where means are not meaningful
- Efficiency relative to t-test is about 95% when normality holds

Weaknesses

- Requires symmetry assumption: For test to be about the median
- Less powerful than t-test when normality holds: 95% efficient at best
- **Sensitive to ties**: Tied values complicate the test (though corrections exist)
- **More complex than sign test**: Requires ranking, not just counting signs
- Cannot easily extend to multivariate settings

Comparison with alternatives

- **vs. Sign test**: Wilcoxon is more powerful (uses ranks), but sign test requires no symmetry assumption
- **vs. Paired t-test**: t-test is more powerful under normality; Wilcoxon is more robust and works for non-normal data
- **vs. Mann-Whitney U test**: Mann-Whitney is for *independent* samples; Wilcoxon signed-rank is for *paired* samples or one-sample tests

Bayesian alternative: For Bayesian paired comparisons, use hierarchical models with priors on the difference distribution, though no direct non-parametric equivalent exists.

Return to the table.

6.2.11 Mann-Whitney test

Also known as the Mann-Whitney U test or Wilcoxon rank-sum test, this is a non-parametric test for comparing two independent groups. It tests whether one population tends to produce larger values than the other. It is the non-parametric alternative to the two-sample t-test when the normality assumption does not hold.

Equivalently, it tests the hypothesis that, for randomly selected x from group 1 and y from group 2:

$$H_0 : \mathbb{P}(\{x > y\}) = \mathbb{P}(\{x < y\}) = \tfrac{1}{2}$$

Note: The *Wilcoxon rank-sum test* is mathematically equivalent to the Mann-Whitney U test (they yield the same p-value). The *Wilcoxon signed-rank test* is a different test, for paired samples.

Frequentist Let $n_1, n_2 \in \mathbb{N}^*$, and let $(x_i)_{1 \leq i \leq n_1}$ and $(y_j)_{1 \leq j \leq n_2}$ be two independent samples.

$$\begin{cases} H_0 : \text{both populations have the same distribution} \\ H_1 : \text{one population tends to produce larger values (two-sided)} \\ H_1 : \mathbb{P}(\{x > y\}) > \frac{1}{2} \text{ (one-sided: group 1 tends larger)} \\ H_1 : \mathbb{P}(\{x > y\}) < \frac{1}{2} \text{ (one-sided: group 2 tends larger)} \end{cases}$$

Test procedure :

1. **Combine and rank**: Pool all $N = n_1 + n_2$ observations and rank them from 1 (smallest) to N (largest), keeping track of group membership.
   - **Handling ties**: Assign the average of the tied ranks.
2. **Compute rank sums**: Let R_1 and R_2 be the sums of ranks for groups 1 and 2 respectively. Note: $R_1 + R_2 = \dfrac{N(N+1)}{2}$.
3. **Compute U statistics**:

$$\begin{cases} U_1 = n_1 n_2 + \dfrac{n_1(n_1+1)}{2} - R_1 \\ U_2 = n_1 n_2 + \dfrac{n_2(n_2+1)}{2} - R_2 \end{cases}$$

   Note: $U_1 + U_2 = n_1 n_2$. The test statistic is $U = \min(U_1, U_2)$.

   Intuition: U_1 counts the number of times an observation from group 1 *precedes* an observation from group 2 in the combined ranking (i.e., the number of pairs (x_i, y_j) such that $x_i < y_j$):

$$U_1 = \sum_{i=1}^{n_1} \sum_{j=1}^{n_2} \mathbf{1}[x_i < y_j]$$

   Under H_0, each ordering is equally likely, so $\mathbb{E}([]\, U_1] = n_1 n_2 / 2$.
4. **Compute p-value**:
   - **Small samples** ($n_1, n_2 \leq 8$): Use exact distribution tables.
   - **Large samples**: Use normal approximation (see below).

Large-sample normal approximation (recommended when $\min(n_1, n_2) \geq 8$):
Under H_0:

$$Z = \frac{U_1 - \dfrac{n_1 n_2}{2}}{\sqrt{\dfrac{n_1 n_2 (N+1)}{12}}} \approx \mathcal{N}(0,1)$$

Tie correction: When ties are present, replace the variance with:

$$\mathbb{V}(U_1) = \frac{n_1 n_2}{12}\left[(N+1) - \frac{\sum_k (t_k^3 - t_k)}{N(N-1)}\right]$$

where t_k is the number of observations sharing the k-th tied rank.

Then compute:

- **Two-sided**: $p = 2\Phi(-|Z|)$
- **One-sided** (group 1 larger): $p = \Phi(-Z)$
- **One-sided** (group 2 larger): $p = \Phi(Z)$

Relationship to AUC: The U statistic has a direct probabilistic interpretation:

$$\mathbb{P}\left(\widehat{\{x > y\}}\right) = \frac{U_1}{n_1 n_2}$$

This is exactly the empirical AUC (Area Under the ROC Curve), making the Mann-Whitney test directly connected to classifier evaluation in machine learning.

Example — Comparing two training algorithms:

Two algorithms are trained on 6 and 5 independent runs. Test scores:

- Algorithm A ($n_1 = 6$): 82, 75, 90, 68, 85, 78
- Algorithm B ($n_2 = 5$): 72, 80, 65, 88, 70

Score	Group	Rank
65	B	1
68	A	2
70	B	3
72	B	4
75	A	5
78	A	6
80	B	7
82	A	8
85	A	9
88	B	10
90	A	11

Step-by-step:

1. $N = 11$, $R_1 = 2 + 5 + 6 + 8 + 9 + 11 = 41$, $R_2 = 1 + 3 + 4 + 7 + 10 = 25$
2. Check: $R_1 + R_2 = 66 = \frac{11 \times 12}{2}$ ✓
3. $U_1 = 6 \times 5 + \frac{6 \times 7}{2} - 41 = 30 + 21 - 41 = 10$
4. $U_2 = 6 \times 5 + \frac{5 \times 6}{2} - 25 = 30 + 15 - 25 = 20$
5. Check: $U_1 + U_2 = 30 = n_1 n_2$ ✓
6. $Z = \dfrac{10 - 15}{\sqrt{\frac{6 \times 5 \times 12}{12}}} = \dfrac{-5}{\sqrt{30}} \approx -0.91$
7. Two-sided $p = 2\Phi(-0.91) \approx 0.36$
8. **Conclusion**: No significant difference at $\alpha = 0.05$.

Assumptions

- All observations from both groups are independent of each other
- Values are at least ordinal: observations can be ranked
- **Similar shape**: For interpreting as a location shift (median test), both distributions should have the same shape (but possibly different locations). Without this, the test is about stochastic dominance rather than medians.

Strengths

- Non-parametric: No normality assumption required
- Related to AUC: $U_1/(n_1 n_2)$ estimates $\mathbb{P}(\{x > y\})$, directly useful in classification evaluation
- **Robust to outliers**: Uses ranks, not raw values
- **Works with ordinal data**: Does not require a continuous scale
- **Exact test available**: Valid for small samples without asymptotic approximations
- High efficiency relative to the t-test ($\approx 95\%$) even when normality holds

Weaknesses

- Less powerful than the two-sample t-test when normality holds
- Sensitive to ties: Many ties reduce test validity (though tie corrections exist)
- **Shape assumption**: Without equal-shape distributions, it tests stochastic dominance rather than a location shift, which is harder to interpret
- Cannot directly estimate a location parameter (unlike the t-test which estimates the mean difference); requires the Hodges-Lehmann estimator for a point estimate

Comparison with alternatives

- **vs. Two-sample t-test**: t-test is more powerful under normality; Mann-Whitney is robust and valid for non-normal or ordinal data
- **vs. Wilcoxon signed-rank test**: Mann-Whitney is for *independent* samples; Wilcoxon signed-rank is for *paired* samples or one-sample tests
- **vs. Kruskal-Wallis test**: Kruskal-Wallis generalises Mann-Whitney to $k > 2$ independent groups
- **vs. Permutation test**: Permutation test is more flexible (any test statistic), but computationally heavier; Mann-Whitney has a closed-form distribution

Bayesian alternative: Model observations with a non-parametric prior (e.g., Dirichlet process) or use a robust location model with heavy-tailed distributions (e.g., Student-t likelihood). The posterior of $\mathbb{P}(\{x > y\})$ provides a Bayesian analogue of the AUC interpretation.

Return to the table.

6.2.12 Kruskal-Wallis test

The Kruskal-Wallis test is a non-parametric generalisation of one-way ANOVA to $g \geq 2$ independent groups. It tests whether the groups originate from the same distribution. It is the non-parametric alternative to one-way ANOVA when the normality assumption does not hold, and reduces to the Mann-Whitney U test when $g = 2$.

Frequentist Let N be the total number of observations, g the number of groups, n_i the number of observations in group i, and r_{ij} the rank of observation j from group i in the combined ranking of all N observations. Define: $\begin{cases} \overline{r}_{i\cdot} = \dfrac{\sum_{j=1}^{n_i} r_{ij}}{n_i} & \text{(mean rank of group } i\text{)} \\ \overline{r} = \dfrac{N+1}{2} & \text{(grand mean rank)} \end{cases}$

$\begin{cases} H_0 : \text{all } g \text{ populations have the same distribution} \\ H_1 : \text{at least one population tends to produce larger values} \end{cases}$

Test procedure :

1. **Combine and rank**: Pool all N observations and rank them 1 to N, assigning average ranks to ties. Record r_{ij} for each observation.
2. **Compute the H statistic**:

$$H = (N-1)\frac{\sum_{i=1}^{g} n_i \left(\overline{r}_{i\cdot} - \overline{r}\right)^2}{\sum_{i=1}^{g}\sum_{j=1}^{n_i} \left(r_{ij} - \overline{r}\right)^2}$$

   This simplifies (when there are no ties) to the equivalent form:

$$H = \frac{12}{N(N+1)}\sum_{i=1}^{g}\frac{R_i^2}{n_i} - 3(N+1)$$

   where $R_i = \sum_{j=1}^{n_i} r_{ij}$ is the rank sum of group i.

   Intuition: H measures the weighted variance of group mean ranks around the grand mean rank. Under H_0, all groups have similar mean ranks; large H indicates that at least one group's ranks are systematically higher or lower.
3. **Tie correction**: When ties are present, divide H by:

$$C = 1 - \frac{\sum_k (t_k^3 - t_k)}{N^3 - N}$$

   giving $H_c = H/C$, where t_k is the size of the k-th tied group.
4. **Compute p-value**: Under H_0, for large samples, $H \sim \chi^2(g-1)$, so $p = \mathbb{P}\left(\left\{\chi^2(g-1) > H\right\}\right)$. For small samples ($n_i < 5$), use exact tables.

Post-hoc tests: A significant H only tells you that *some* groups differ. To identify *which* pairs differ, use:

- **Dunn's test**: pairwise comparisons using the rank sums, with Bonferroni or Benjamini-Hochberg correction for multiple testing
- **Pairwise Mann-Whitney** with adjusted α

Example — Comparing three diet plans:
Weight loss (kg) after 8 weeks for three groups ($n_1 = n_2 = n_3 = 4$):

- Diet A: 3, 5, 7, 4 Diet B: 6, 9, 8, 7 Diet C: 2, 4, 3, 1

Combined ranking (N = 12):

Value	Group	Rank
1	C	1
2	C	2
3	A	3.5
3	C	3.5
4	A	5.5
4	C	5.5
5	A	7
6	B	8
7	A	9.5
7	B	9.5
8	B	11
9	B	12

$R_1 = 3.5 + 5.5 + 7 + 9.5 = 25.5, \quad R_2 = 8 + 9.5 + 11 + 12 = 40.5, \quad R_3 = 1 + 2 + 3.5 + 5.5 = 12$

$$H = \frac{12}{12 \times 13}\left(\frac{25.5^2}{4} + \frac{40.5^2}{4} + \frac{12^2}{4}\right) - 3 \times 13 \approx 7.65$$

$p = \mathbb{P}\left(\{\chi^2(2) > 7.65\}\right) \approx 0.022$. Reject H_0 at $\alpha = 0.05$: at least one diet plan differs significantly.

Assumptions

- All observations are independent across and within groups
- Values are at least ordinal: observations can be ranked
- **Similar shape**: For interpreting as a location shift, all groups should have the same distributional shape (but possibly different locations). Otherwise the test measures stochastic dominance only.

Strengths

- Non-parametric: no normality assumption required
- Generalises Mann-Whitney to $k > 2$ independent groups
- Robust to outliers; works with ordinal data
- Exact test available for small samples

Weaknesses

- Less powerful than one-way ANOVA when normality holds
- Only detects whether groups differ, not which ones; requires post-hoc tests
- Sensitive to ties (though the correction H_c mitigates this)
- Cannot model interactions or covariates (unlike ANOVA extensions)

Comparison with alternatives

- **vs. One-way ANOVA**: ANOVA is more powerful under normality; Kruskal-Wallis is robust for non-normal or ordinal data
- **vs. Mann-Whitney**: Mann-Whitney is the special case $g = 2$; Kruskal-Wallis is the multi-group extension
- **vs. Friedman test**: Friedman is for *repeated measures* (matched groups); Kruskal-Wallis is for *independent* groups
- **vs. Permutation ANOVA**: More flexible but computationally heavier; Kruskal-Wallis has a closed-form χ^2 approximation

Bayesian alternative: Use a hierarchical model with group-level location parameters and heavy-tailed likelihoods (e.g. Student-t), or a non-parametric prior (Dirichlet process mixture). Posterior contrasts between groups replace post-hoc tests.

Return to the table.

6.2.13 Friedman test

The Friedman test is a non-parametric analogue of repeated-measures (one-way) ANOVA. Data is arranged in n *blocks* (e.g. subjects) and k *treatments* (conditions), with one observation per cell. It is the non-parametric alternative to repeated-measures ANOVA when normality or sphericity cannot be assumed, and generalises the Wilcoxon signed-rank test to $k > 2$ treatments.

Frequentist Let n be the number of blocks, k the number of treatments, x_{ij} the observation in block i and treatment j, and r_{ij} its rank *within block* i (ranked 1 to k). Define: $\begin{cases} R_j = \sum_{i=1}^{n} r_{ij} & \text{(rank sum of treatment } j\text{)} \\ \overline{r}_{\cdot j} = R_j/n & \text{(mean rank of treatment } j\text{)} \end{cases}$

$\begin{cases} H_0 : \text{all } k \text{ treatments have the same effect (identical distributions)} \\ H_1 : \text{at least one treatment tends to produce different values} \end{cases}$

Test procedure :

1. **Rank within blocks**: For each block i, rank the k observations from 1 (smallest) to k (largest), assigning average ranks to ties.
2. **Compute the Q statistic**:

$$Q = \frac{12n}{k(k+1)} \sum_{j=1}^{k} \left(\overline{r}_{\cdot j} - \frac{k+1}{2} \right)^2$$

   Equivalently, using rank sums:

$$Q = \frac{12}{nk(k+1)} \sum_{j=1}^{k} R_j^2 - 3n(k+1)$$

   Intuition: Q measures how much the mean rank of each treatment deviates from the grand mean rank $\frac{k+1}{2}$. Under H_0, all treatments are equally likely to receive any rank within a block, so mean ranks should be close to $\frac{k+1}{2}$; large Q indicates systematic rank differences between treatments.
3. **Tie correction**: When ties occur within a block, divide Q by:

$$C = 1 - \frac{\sum_{i=1}^{n} \sum_{s} (t_{is}^3 - t_{is})}{nk(k^2-1)}$$

   giving $Q_c = Q/C$, where t_{is} is the size of the s-th tied group in block i.
4. **Compute p-value**: For large n or k ($n > 15$ or $k > 4$), $Q \sim \chi^2(k-1)$, so $p = \mathbb{P}\left(\{\chi^2(k-1) > Q\}\right)$. For small samples, use exact Friedman tables.

Post-hoc tests: A significant Q only tells you that *some* treatments differ. To identify *which* pairs differ, use:

- **Nemenyi test**: pairwise comparisons based on rank sum differences, controlling the family-wise error rate
- **Pairwise Wilcoxon signed-rank tests** with Bonferroni or Benjamini-Hochberg correction

Example — Comparing three analgesic treatments on 5 patients:
Pain relief scores (higher = better) for $n = 5$ patients under $k = 3$ treatments:

Block	A	B	C	Rank A	Rank B	Rank C
1	2	8	5	1	3	2
2	4	9	3	2	3	1
3	1	7	6	1	3	2
4	6	8	4	2	3	1
5	3	9	5	1	3	2
R_j				7	15	8

$$Q = \frac{12}{5 \times 3 \times 4}\left(7^2 + 15^2 + 8^2\right) - 3 \times 5 \times 4 = 0.2 \times 338 - 60 = 67.6 - 60 = 7.6$$

$p = \mathbb{P}\left(\{\chi^2(2) > 7.6\}\right) \approx 0.022$. Reject H_0 at $\alpha = 0.05$: at least one treatment differs significantly. Post-hoc tests would reveal that B differs from A and C.

Assumptions

- Blocks are mutually independent: observations across subjects are independent
- Observations can be ranked within each block: data is at least ordinal
- One observation per block-treatment cell (no replication within cells)

Strengths

- Non-parametric: no normality or sphericity assumptions required
- Controls for block effects: ranking within blocks removes between-subject variability, increasing sensitivity
- Works with ordinal data; robust to outliers
- Exact test available for small n and k

Weaknesses

- Less powerful than repeated-measures ANOVA when normality and sphericity hold
- Only detects whether treatments differ, not which ones; requires post-hoc tests
- Restricted to one observation per cell; cannot handle within-block replication
- Sensitive to ties within blocks (though the correction Q_c mitigates this)

Comparison with alternatives

- **vs. Repeated-measures ANOVA**: ANOVA is more powerful under normality and sphericity; Friedman is robust when these fail
- **vs. Kruskal-Wallis**: Kruskal-Wallis is for *independent* groups; Friedman is for *matched/repeated* measures — it controls for block effects
- **vs. Wilcoxon signed-rank**: Wilcoxon is the special case $k = 2$; Friedman is the multi-treatment extension
- **vs. Cochran's Q**: Cochran's Q is the special case for binary (0/1) data

Bayesian alternative: Use a hierarchical model with block-level random effects and treatment-level fixed effects, with heavy-tailed or non-parametric likelihoods. Posterior contrasts between treatment parameters replace post-hoc tests.

Return to the table.

6.2.14 Spearman test

The Spearman rank correlation coefficient r_s measures the strength and direction of the monotonic association between two variables. It is Pearson's correlation applied to the ranks of the data, making it robust to outliers and applicable to ordinal variables. It does not assume linearity — only that the relationship is monotone.

Frequentist Let $(x_1, y_1), \ldots, (x_n, y_n)$ be n paired observations. Let $R(x_i)$ and $R(y_i)$ denote the ranks of x_i and y_i in their respective samples.

$$\begin{cases} H_0 : \rho_s = 0 & \text{(no monotonic association)} \\ H_1 : \rho_s \neq 0 & \text{(two-sided)} \\ H_1 : \rho_s > 0 & \text{(one-sided: positive monotonic association)} \\ H_1 : \rho_s < 0 & \text{(one-sided: negative monotonic association)} \end{cases}$$

Test procedure :

1. **Rank each variable separately**: Assign ranks 1 to n to the x_i values, and independently to the y_i values. Use average ranks for ties.
2. **Compute the correlation**: The Spearman coefficient is Pearson's r applied to the ranks:

$$r_s = \rho_{R(X),\,R(Y)} = \frac{Cov(R(X),\,R(Y))}{\sigma_{R(X)}\,\sigma_{R(Y)}}$$

Simplified formula (exact when there are no ties):

$$r_s = 1 - \frac{6\sum_{i=1}^{n} d_i^2}{n(n^2-1)}$$

where $d_i = R(x_i) - R(y_i)$ is the rank difference for pair i.

Intuition: If the rankings of X and Y agree perfectly, all $d_i = 0$ and $r_s = 1$. If they are perfectly reversed, $r_s = -1$. Random agreement gives $r_s \approx 0$.

3. **Assess significance**: For $n \geq 10$, use the t-approximation:

$$t = r_s \sqrt{\frac{n-2}{1-r_s^2}} \sim t(n-2) \text{ under } H_0$$

For small samples ($n < 10$), use exact Spearman tables.

Interpretation of r_s:

- $r_s = +1$: perfect positive monotonic relationship (ranks agree exactly)
- $r_s = -1$: perfect negative monotonic relationship (ranks are reversed)
- $r_s = 0$: no monotonic association
- $|r_s| > 0.7$: strong, 0.4–0.7: moderate, < 0.4: weak (rules of thumb)

Example — Study hours and exam scores:
7 students, ranked by study hours (X) and exam score (Y):

Student	**Hours**	**Score**	**Rank X**	**Rank Y**	d_i	d_i^2
1	2	55	1	1	0	0
2	4	65	2	3	-1	1
3	6	60	3	2	1	1
4	8	70	4	4	0	0
5	10	80	5	6	-1	1
6	12	75	6	5	1	1
7	14	90	7	7	0	0
					$\sum d_i^2$	4

$$r_s = 1 - \frac{6 \times 4}{7 \times 48} = 1 - \frac{24}{336} \approx 0.929$$

$$t = 0.929 \times \sqrt{\frac{5}{1-0.929^2}} = 0.929 \times \sqrt{\frac{5}{0.137}} \approx 5.61$$

$p = 2\,\mathbb{P}(\{t(5) > 5.61\}) \approx 0.002$. **Conclusion**: Strong evidence of a positive monotonic association between study hours and exam score.

Assumptions

- Observations are independent: each pair (x_i, y_i) is drawn independently
- Variables are at least ordinal: observations can be meaningfully ranked
- The relationship between X and Y is **monotonic** (not necessarily linear) — if the true relationship is non-monotone, r_s may be misleading

Strengths

- Non-parametric: no normality assumption required
- Detects any monotonic association, not just linear relationships
- **Robust to outliers**: ranks dampen the influence of extreme values
- Works with ordinal data where numerical differences are not meaningful
- Efficiency $\approx 91\%$ relative to Pearson's r under bivariate normality

Weaknesses

- Less powerful than Pearson's r for detecting linear relationships in normal data
- Insensitive to non-monotonic relationships: a U-shaped association may yield $r_s \approx 0$ even when X and Y are strongly dependent
- **Sensitive to ties**: many ties require the general Pearson formula on ranks and can reduce accuracy of the simplified d_i^2 formula
- Only captures pairwise monotonic association; does not extend naturally to partial or multiple correlation settings

Comparison with alternatives

- **vs. Pearson's** r: Pearson is more powerful for linear relationships under normality; Spearman is robust and works for ordinal or non-normal data
- **vs. Kendall's** τ: Kendall's τ counts concordant/discordant pairs and has better statistical properties for small samples and many ties; Spearman is more common and easier to compute
- **vs. Distance correlation**: Distance correlation detects any dependence (including non-monotonic); Spearman only detects monotonic associations

Bayesian alternative: Place a prior on ρ_s (e.g. a uniform or Beta-transformed prior) and update with the observed rank data. The posterior distribution of ρ_s replaces the p-value with a full uncertainty quantification over the association strength.

Return to the table.

6.2.15 Pearson correlation coefficient

The Pearson correlation coefficient r measures the strength and direction of the linear association between two continuous variables. It is a normalised covariance, bounded in $[-1, 1]$, and is the most widely used measure of association when both variables are continuous and the relationship is expected to be linear.

Frequentist Let $(x_1, y_1), \ldots, (x_n, y_n)$ be n paired observations. The population parameter and its sample estimator are:

$$\rho_{X,Y} = \frac{Cov(X,Y)}{\sigma_X \, \sigma_Y} \qquad r = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \bar{x})^2 \sum_{i=1}^{n}(y_i - \bar{y})^2}}$$

$$\begin{cases} H_0 : \rho = 0 & \text{(no linear association)} \\ H_1 : \rho \neq 0 & \text{(two-sided)} \\ H_1 : \rho > 0 & \text{(one-sided: positive linear association)} \\ H_1 : \rho < 0 & \text{(one-sided: negative linear association)} \end{cases}$$

Test procedure :

1. **Compute** r from the sample using the formula above.

   Intuition: r measures how tightly the data cluster around a straight line. It is the cosine of the angle between the mean-centred vectors $(x_i - \bar{x})$ and $(y_i - \bar{y})$: perfect alignment gives $|r| = 1$, orthogonality gives $r = 0$.

2. **Assess significance**: Under H_0 and bivariate normality,

$$t = r\sqrt{\frac{n-2}{1-r^2}} \sim t(n-2)$$

3. **Confidence interval**: Use Fisher's z-transformation, which stabilises the variance:

$$z = \operatorname{arctanh}(r) = \tfrac{1}{2}\ln\left(\tfrac{1+r}{1-r}\right) \quad \sim \mathcal{N}\left(\operatorname{arctanh}(\rho),\ \tfrac{1}{n-3}\right)$$

   A $(1-\alpha)$ CI for ρ: back-transform $\operatorname{arctanh}(r) \pm z_{\alpha/2}/\sqrt{n-3}$ with $\tanh(\cdot)$.

Interpretation of r:

- $r = +1$: perfect positive linear relationship
- $r = -1$: perfect negative linear relationship
- $r = 0$: no linear relationship (non-linear dependence may still exist)
- r^2: *coefficient of determination* — proportion of variance in Y explained by a linear function of X
- $|r| > 0.7$: strong, 0.4–0.7: moderate, < 0.4: weak (rules of thumb)

Example — Advertising spend and sales:
$n = 6$ observations of weekly advertising spend (x, \$000s) and units sold (y):

i	x	y	$x - \bar{x}$	$y - \bar{y}$	$(x-\bar{x})(y-\bar{y})$	$(x-\bar{x})^2$	$(y-\bar{y})^2$
1	2	3	-5	-5	25	25	25
2	4	7	-3	-1	3	9	1
3	6	5	-1	-3	3	1	9
4	8	9	1	1	1	1	1
5	10	11	3	3	9	9	9
6	12	13	5	5	25	25	25
	$\bar{x} = 7$	$\bar{y} = 8$		Σ	66	70	70

$$r = \frac{66}{\sqrt{70 \times 70}} = \frac{66}{70} \approx 0.943 \qquad r^2 \approx 0.889$$

$$t = 0.943 \times \sqrt{\frac{4}{1 - 0.943^2}} = 0.943 \times \sqrt{\frac{4}{0.111}} \approx 5.66$$

$p = 2\,\mathbb{P}(\{t(4) > 5.66\}) \approx 0.005$. **Conclusion**: Strong evidence of a positive linear association; advertising spend explains $\approx 89\%$ of the variance in sales.

Assumptions

- Both variables are continuous and measured on an interval or ratio scale
- Linear relationship: r only captures linear association; a non-linear relationship can give $r \approx 0$ even with strong dependence
- **Bivariate normality**: required for the t-test and CI to be exact; robust in large samples by the CLT
- **Homoscedasticity**: the variance of Y should be constant across values of X
- **Independence**: each pair (x_i, y_i) is drawn independently

Strengths

- Optimal for linear relationships under bivariate normality
- r^2 has a direct variance-explained interpretation
- Well-understood theoretical properties; closed-form inference via t and Fisher's z
- Foundation for linear regression (r equals the standardised regression slope)

Weaknesses

- Not robust to outliers: a single extreme point can drastically shift r
- Only detects linear relationships: $r = 0$ does not imply independence
- **Sensitive to non-normality** for small samples
- **Spurious correlations**: two variables can be correlated through a common confounder; r does not imply causation

Comparison with alternatives

- **vs. Spearman's** r_s: Spearman is robust to outliers and detects monotonic (not just linear) associations; Pearson is more powerful under normality and linearity
- **vs. Kendall's** τ: Kendall's τ is better for small samples and heavy ties; Pearson assumes continuous, normally distributed data
- **vs. Distance correlation**: Distance correlation detects any form of dependence (including non-linear); Pearson only detects linear association
- **vs. Partial correlation**: Partial correlation removes the effect of confounding variables; Pearson measures marginal (unadjusted) association

Bayesian alternative: Place a prior on ρ (e.g. a uniform prior or the Jeffreys prior $p(\rho) \propto (1-\rho^2)^{-3/2}$) and derive the posterior from the bivariate normal likelihood. The posterior of ρ gives a full uncertainty quantification over the association strength, replacing the p-value.

Return to the table.

6.2.16 Repeated-measures ANOVA

Repeated-measures ANOVA is a parametric test for comparing $k \geq 2$ conditions when the same subjects are measured under each condition (within-subject design). By explicitly modelling individual differences, it removes between-subject variability from the error term, yielding a more sensitive test than independent-groups ANOVA.

Frequentist Let n be the number of subjects, k the number of conditions, x_{ij} the observation for subject i under condition j, and $N = nk$ the total number of observations. Define:

$$\begin{cases} \bar{x} = \dfrac{1}{N}\displaystyle\sum_{i,j} x_{ij} & \text{(grand mean)} \\ \bar{x}_{i\cdot} = \dfrac{1}{k}\displaystyle\sum_{j=1}^{k} x_{ij} & \text{(subject } i \text{ mean)} \\ \bar{x}_{\cdot j} = \dfrac{1}{n}\displaystyle\sum_{i=1}^{n} x_{ij} & \text{(condition } j \text{ mean)} \end{cases}$$

$$\begin{cases} H_0 : \mu_1 = \mu_2 = \cdots = \mu_k & \text{(all condition means are equal)} \\ H_1 : \text{at least one condition mean differs} \end{cases}$$

SS partitioning :
The key advantage over between-subjects ANOVA is the additional partition of $SS_{subjects}$:

$$\begin{aligned} &\textbf{Between-subjects:} && SS_{total} = SS_{treatment} + SS_{error} \\ &\textbf{Repeated-measures:} && SS_{total} = SS_{treatment} + SS_{subjects} + SS_{error} \end{aligned}$$

Intuition: Since the same subjects appear in every condition, the variability due to individuals ($SS_{subjects}$) can be subtracted from what would otherwise inflate the error. This shrinks MS_{error} and increases the F-statistic.

Formulas :

$$\begin{aligned} SS_{treatment} &= n\sum_{j=1}^{k} (\bar{x}_{\cdot j} - \bar{x})^2 & df_{treatment} &= k - 1 \\ SS_{subjects} &= k\sum_{i=1}^{n} (\bar{x}_{i\cdot} - \bar{x})^2 & df_{subjects} &= n - 1 \\ SS_{error} &= SS_{total} - SS_{treatment} - SS_{subjects} & df_{error} &= (k-1)(n-1) \\ SS_{total} &= \sum_{i=1}^{n}\sum_{j=1}^{k} (x_{ij} - \bar{x})^2 & df_{total} &= nk - 1 \end{aligned}$$

The test statistic is:

$$F = \frac{MS_{treatment}}{MS_{error}} = \frac{SS_{treatment}/(k-1)}{SS_{error}/\left[(k-1)(n-1)\right]} \sim F(k-1,\ (k-1)(n-1)) \text{ under } H_0$$

Sphericity and corrections :
Sphericity requires that the variances of all pairwise difference scores are equal. Mauchly's test checks this assumption. When violated:

- **Greenhouse-Geisser correction**: multiply both df by $\hat{\varepsilon}_{GG} \in [1/(k-1),\ 1]$
- **Huynh-Feldt correction**: less conservative than GG; preferred when $\hat{\varepsilon} > 0.75$

Post-hoc tests: Pairwise t-tests on the condition means with Bonferroni or Holm correction for multiple comparisons.

Example — Effect of three teaching methods on 4 students:
Test scores ($n = 4$ students, $k = 3$ methods):

Subject	**A**	**B**	**C**	$\bar{x}_{i\cdot}$
1	3	7	5	5
2	5	9	7	7
3	4	6	5	5
4	6	8	7	7
$\bar{x}_{\cdot j}$	4.5	7.5	6	$\bar{x} = 6$

$$
\begin{aligned}
SS_{treatment} &= 4\left[(4.5-6)^2+(7.5-6)^2+(6-6)^2\right] = 4\times 4.5 = 18\\
SS_{subjects} &= 3\left[(5-6)^2+(7-6)^2+(5-6)^2+(7-6)^2\right] = 3\times 4 = 12\\
SS_{total} &= \sum(x_{ij}-6)^2 = 9+1+1+1+9+1+4+0+1+0+4+1 = 32\\
SS_{error} &= 32-18-12 = 2
\end{aligned}
$$

$$
F = \frac{18/2}{2/6} = \frac{9}{0.333} = 27.0 \sim F(2,6)
$$

$p = \mathbb{P}(\{F(2,6) > 27\}) \approx 0.001$. **Conclusion**: Strong evidence that teaching method affects test scores. Post-hoc tests would identify method B as significantly better than A and C.

Assumptions

- Normality: the dependent variable is normally distributed within each condition (robust in large samples)
- Sphericity: variances of all pairwise difference scores are equal; test with Mauchly's test and apply GG or HF correction if violated
- **Independence**: subjects are sampled independently from each other
- **No carry-over effects**: in practice, condition order should be counterbalanced to avoid learning or fatigue effects

Strengths

- Higher statistical power than between-subjects ANOVA: removes between-subject variability from the error term
- Fewer subjects needed to achieve the same power as independent-groups ANOVA
- Each subject serves as their own control, eliminating confounding from individual differences

Weaknesses

- Sphericity assumption: violation inflates Type I error; requires correction or switch to mixed-effects models
- Sensitive to missing data: a single missing cell removes the entire subject from the analysis
- **Carry-over effects**: repeated exposure to conditions can introduce order or practice effects
- Not appropriate when conditions cannot be applied to the same subject (use between-subjects or mixed design instead)

Comparison with alternatives

- **vs. One-way ANOVA**: one-way ANOVA assumes independent groups; RM-ANOVA exploits the within-subject structure for greater power
- **vs. Friedman test**: Friedman is the non-parametric equivalent; use it when normality or sphericity cannot be assumed
- **vs. Linear mixed-effects model (LMM)**: LMM handles missing data, unbalanced designs, and more complex correlation structures; RM-ANOVA is a special case of LMM with compound-symmetry covariance
- **vs. MANOVA**: MANOVA makes no sphericity assumption by modelling the full covariance matrix; preferred when sphericity is badly violated

Bayesian alternative: Use a hierarchical (mixed-effects) model with subject-level random intercepts and condition-level fixed effects. Posterior contrasts between conditions replace post-hoc t-tests, and the full posterior naturally quantifies uncertainty without requiring sphericity.

Return to the table.

6.2.17 1-way ANOVA

One-way ANOVA tests equality of means across $k \geq 2$ independent groups using a single categorical explanatory variable. Despite its name, the term "analysis of variance" is a misnomer: we use variance-like quantities to study the equality of population means, not variances. It is the parametric generalisation of the two-sample t-test to $k > 2$ groups (and reduces to it when $k = 2$, with $F = t^2$).

The general linear model is:

$$y_{ij} = \mu + \alpha_j + \epsilon_{ij}, \quad \epsilon_{ij} \overset{iid}{\sim} \mathcal{N}(0, \sigma^2)$$

where μ is the grand mean, $\alpha_j = \mu_j - \mu$ is the treatment effect of group j, and $\sum_{j=1}^{k} n_j \alpha_j = 0$ (sum-to-zero constraint).

Frequentist Let k be the number of groups, n_j the number of observations in group j, $N = \sum_{j=1}^{k} n_j$ the total, Y_{ij} the i-th observation in group j, $\overline{Y}_{\bullet j}$ the group mean, and $\overline{Y}$ the grand mean.

$$\begin{cases} H_0 : \mu_1 = \mu_2 = \cdots = \mu_k \quad \text{(all group means are equal, i.e. } \alpha_j = 0 \ \forall j) \\ H_1 : \text{at least one } \mu_j \text{ differs} \end{cases}$$

SS partitioning :

In ANOVA, all variance-like quantities are computed as SS/df and called *mean squares*. The total variability decomposes as:

$$SS_{total} = SS_{between} + SS_{within}$$

Within-group SS measures variability of observations around their group mean (see Figure 6.1). The within-group deviation for observation i in group j is $Y_{ij} - \overline{Y}_{\bullet j}$.

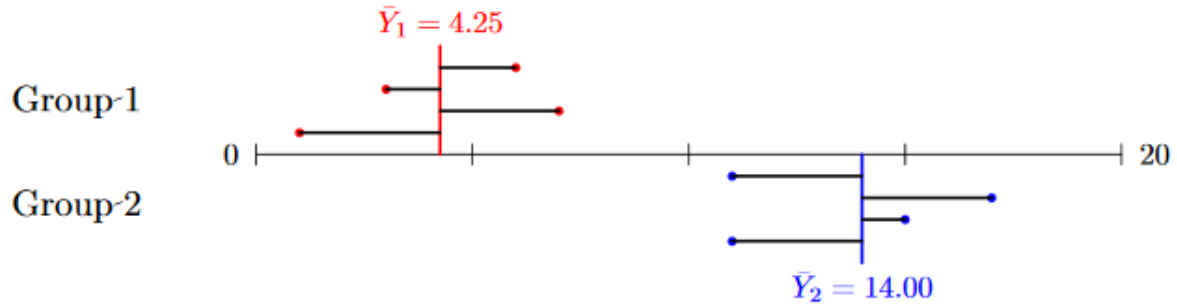


Figure 6.1: Deviations for within-group sum of squares

$$MS_{within} = \frac{SS_{within}}{df_{within}} \begin{cases} SS_{within} = \sum_{j=1}^{k} \sum_{i=1}^{n_j} \left(Y_{ij} - \overline{Y}_{\bullet j}\right)^2 \\ df_{within} = N - k \end{cases}$$

MS_{within} is always a valid estimate of σ^2, regardless of whether H_0 is true.

Between-group SS measures variability of group means around the grand mean (see Figure 6.2). The between-group deviation for group j is $\overline{Y}_{\bullet j} - \overline{Y}$.

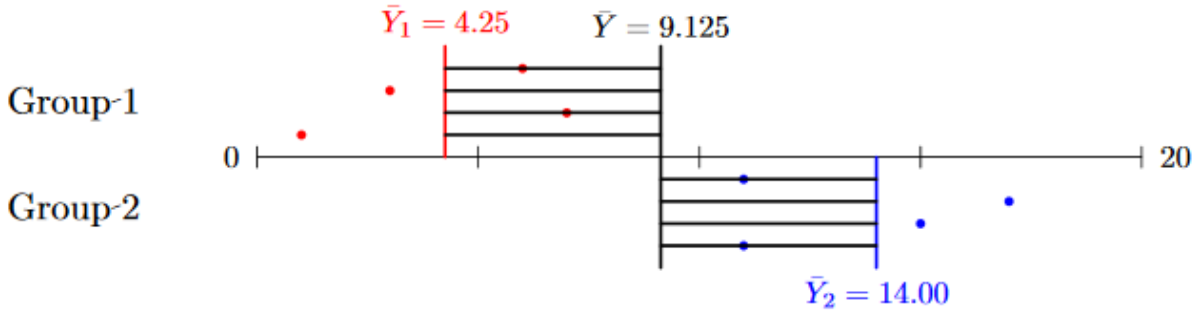


Figure 6.2: Deviations for between-group sum of squares

$$MS_{between} = \frac{SS_{between}}{df_{between}} \begin{cases} SS_{between} = \sum_{j=1}^{k} n_j \left(\overline{Y}_{\bullet j} - \overline{Y}\right)^2 \\ df_{between} = k - 1 \end{cases}$$

$MS_{between}$ estimates σ^2 only under H_0; otherwise it is inflated by the treatment effects. This motivates the F-statistic:

$$F = \frac{MS_{between}}{MS_{within}} \sim F(k-1,\, N-k) \text{ under } H_0$$

Intuition: Under H_0, both $MS_{between}$ and MS_{within} estimate the same σ^2, so $F \approx 1$. Under H_1, group means differ, inflating $MS_{between}$ while MS_{within} remains unchanged, so $F \gg 1$.

More precisely:

- Under H_0: $\mathbb{E}\left(MS_{between}\right) = \mathbb{E}\left(MS_{within}\right) = \sigma^2$
- Under H_1: $\mathbb{E}\left(MS_{between}\right) = \sigma^2 + \dfrac{\sum_{j=1}^{k} n_j \alpha_j^2}{k-1} > \sigma^2, \quad \mathbb{E}\left(MS_{within}\right) = \sigma^2$

The standard error for pairwise contrasts between group means is $\sqrt{MS_{within}\left(\frac{1}{n_i} + \frac{1}{n_j}\right)}$.

ANOVA table :

Source	SS	df	MS	F
Between groups	$SS_{between}$	$k-1$	$SS_{between}/(k-1)$	$MS_{between}/MS_{within}$
Within groups	SS_{within}	$N-k$	$SS_{within}/(N-k)$	—
Total	SS_{total}	$N-1$	—	—

Special case $k = 2$: equivalence with the t-test :
With two groups ($n = n_1 + n_2$, common variance σ^2):

$$SS_{between} = \frac{n_1 n_2}{n_1 + n_2} \left(\overline{y}_1 - \overline{y}_2\right)^2$$
$$SS_{within} = (n_1 - 1)s_1^2 + (n_2 - 1)s_2^2$$

where $s_p = \sqrt{\dfrac{(n_1-1)s_1^2 + (n_2-1)s_2^2}{n-2}}$ is the pooled standard deviation. One can show that $F = t^2$, confirming that one-way ANOVA with $k = 2$ is identical to the two-sample t-test.

Post-hoc tests: A significant F only tells you that *some* group means differ. To identify *which* pairs differ, use (with appropriate correction for multiple comparisons):

- **Tukey's HSD**: controls family-wise error rate for all pairwise comparisons

- **Bonferroni**: conservative; good for a small number of planned comparisons
- **Scheffé**: most conservative; valid for any linear contrast, not just pairwise

Example — Effect of three fertilizers on plant yield:
$k = 3$ fertilizer groups, $n_j = 4$ plants each ($N = 12$):

Group	Observations				$\overline{Y}_{\bullet j}$
A	5	7	6	8	6.5
B	9	11	10	12	10.5
C	3	5	4	6	4.5
				Grand mean $\overline{Y}$	7.17

$$SS_{between} = 4\left[(6.5-7.17)^2 + (10.5-7.17)^2 + (4.5-7.17)^2\right] = 4 \times 18.67 = 74.67$$
$$SS_{within} = 5+5+5 = 15 \quad \text{(each group: } (5-6.5)^2 + \cdots = 5\text{)}$$

Source	SS	df	MS	F
Between	74.67	2	37.33	22.4
Within	15.00	9	1.67	—
Total	89.67	11	—	—

$p = \mathbb{P}(\{F(2,9) > 22.4\}) \approx 0.0003$. **Conclusion**: Strong evidence that fertilizer type affects yield. Post-hoc tests (e.g. Tukey's HSD) would reveal that B differs significantly from both A and C.

Assumptions

- Normality: observations within each group are normally distributed (robust in large samples by CLT)
- Homoscedasticity: all groups share a common variance σ^2; check with Levene's or Bartlett's test; use Welch's ANOVA if violated
- **Independence**: observations are independent within and across groups

Strengths

- Simultaneously compares $k > 2$ groups while controlling the family-wise Type I error rate (unlike running multiple t-tests)
- Optimal under normality and homoscedasticity
- Well-developed post-hoc procedures (Tukey, Bonferroni, Scheffé)
- Generalises naturally to factorial (two-way, three-way) and mixed designs

Weaknesses

- Sensitive to non-normality with small samples
- Sensitive to unequal variances: use Welch's ANOVA instead
- Only detects that *some* groups differ, not which ones; requires post-hoc tests
- Assumes independent groups; use RM-ANOVA for repeated measures

Comparison with alternatives

- **vs. Kruskal-Wallis**: Kruskal-Wallis is the non-parametric equivalent; use it when normality or homoscedasticity cannot be assumed
- **vs. Welch's ANOVA**: Welch's ANOVA relaxes the equal-variance assumption; preferred when group variances differ
- **vs. Repeated-measures ANOVA**: RM-ANOVA is for within-subject (matched) designs; one-way ANOVA assumes independent groups
- **vs. Linear regression**: one-way ANOVA with k groups is equivalent to linear regression with $k-1$ dummy variables; they give identical F-tests

Bayesian alternative: Use a hierarchical model with group-level means drawn from a common prior (e.g. $\mu_j \sim \mathcal{N}(\mu, \tau^2)$). The posterior of each μ_j shrinks estimates toward the grand mean, and posterior contrasts between groups replace post-hoc tests.

Return to the table.

6.2.18 T-test

The t-test tests hypotheses about means using a test statistic that follows Student's t-distribution when the population variance is unknown and estimated from the data. The general form is $t = Z/s$, where Z is sensitive to the alternative hypothesis and s is the estimated scaling parameter. There are three main variants depending on the experimental design.

Frequentist

One-sample t-test :
Tests whether the population mean equals a specified value μ_0.
$\begin{cases} H_0 : \mu = \mu_0 \\ H_1 : \mu \neq \mu_0 \text{ (two-sided)}, \quad H_1 : \mu > \mu_0 \text{ or } H_1 : \mu < \mu_0 \text{ (one-sided)} \end{cases}$

$$t = \frac{\overline{x} - \mu_0}{s/\sqrt{n}} \sim t(n-1) \text{ under } H_0$$

where $\overline{x}$ is the sample mean, s the sample standard deviation, and n the sample size. A $(1-\alpha)$ confidence interval for μ is:

$$\overline{x} \pm t_{n-1,\,1-\alpha/2}\,\frac{s}{\sqrt{n}}$$

Two-sample t-test (independent groups) :
Tests whether two independent population means are equal.
$\begin{cases} H_0 : \mu_1 = \mu_2 \\ H_1 : \mu_1 \neq \mu_2 \text{ (two-sided)}, \quad \text{or directional (one-sided)} \end{cases}$
Student's t-test (equal variances, $\sigma_1^2 = \sigma_2^2$):

$$t = \frac{\overline{X}_1 - \overline{X}_2}{s_p\sqrt{\frac{1}{n_1} + \frac{1}{n_2}}} \sim t(n_1 + n_2 - 2) \text{ under } H_0$$

where the pooled standard deviation is $s_p = \sqrt{\frac{(n_1-1)s_1^2 + (n_2-1)s_2^2}{n_1 + n_2 - 2}}$.

Welch's t-test (unequal variances; preferred in practice):

$$t = \frac{\overline{X}_1 - \overline{X}_2}{\sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}} \sim t(\nu) \text{ under } H_0$$

with Satterthwaite degrees of freedom:

$$\nu = \frac{\left(\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}\right)^2}{\frac{(s_1^2/n_1)^2}{n_1 - 1} + \frac{(s_2^2/n_2)^2}{n_2 - 1}}$$

Intuition: t measures the observed difference in means relative to the uncertainty (standard error). Large $|t|$ means the groups are far apart relative to within-group variability — evidence against H_0. Under H_0 with $k = 2$, one-way ANOVA and the two-sample t-test are equivalent: $F = t^2$.

Paired t-test :

Tests whether the mean of within-pair differences equals zero. Let $d_i = x_{i1} - x_{i2}$ be the difference for pair i.

$$\begin{cases} H_0 : \mu_d = 0 \\ H_1 : \mu_d \neq 0 \text{ (two-sided)}, \quad \text{or directional (one-sided)} \end{cases}$$

$$t = \frac{\overline{d} - \mu_0}{s_d/\sqrt{n}} \sim t(n-1) \text{ under } H_0$$

where $\overline{d}$ and s_d are the mean and standard deviation of the differences. This is mathematically identical to a one-sample t-test on the differences.

t-test for a regression slope :

In a simple linear model $Y = \alpha + \beta x + \epsilon$ with $\epsilon \sim \mathcal{N}(0, \sigma^2)$, testing $H_0 : \beta = \beta_0$:

$$t = \frac{\hat{\beta} - \beta_0}{SE_{\hat{\beta}}} \sim t(n-2) \text{ under } H_0 \qquad SE_{\hat{\beta}} = \frac{\sqrt{\frac{1}{n-2}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}}{\sqrt{\sum_{i=1}^{n}(x_i - \overline{x})^2}}$$

Example — One-sample: A machine is calibrated to fill bottles to 500ml. A sample of $n = 10$ bottles gives $\overline{x} = 504$ml, $s = 6$ml. Test $H_0 : \mu = 500$.

$$t = \frac{504 - 500}{6/\sqrt{10}} = \frac{4}{1.897} \approx 2.11, \quad df = 9$$

$p = 2\,\mathbb{P}\left(\{t(9) > 2.11\}\right) \approx 0.064$. Fail to reject H_0 at $\alpha = 0.05$.

Example — Two-sample (Student's): Comparing exam scores under two teaching methods ($n_1 = n_2 = 6$, similar variances):

Group	Scores						$\overline{x}$	s
A (new)	80	75	82	78	74	79	78	3.0
B (traditional)	70	68	72	74	66	70	70	2.8

$$s_p = \sqrt{\frac{5 \times 9 + 5 \times 7.84}{10}} = \sqrt{8.42} \approx 2.90$$

$$t = \frac{78 - 70}{2.90\sqrt{2/6}} = \frac{8}{1.674} \approx 4.78, \quad df = 10$$

$p = 2\,\mathbb{P}\left(\{t(10) > 4.78\}\right) \approx 0.001$. Strong evidence that method A yields higher scores.

Assumptions

- Independence: observations (or pairs) are drawn independently
- Normality: the population (or differences, for paired) is normally distributed; robust in large samples by CLT
- **Equal variances** (Student's two-sample only): use Welch's t-test when variances differ; check with Levene's test

Strengths

- Uniformly most powerful test for means under normality (by Neyman-Pearson)
- Exact for any n under normality; robust for large n by CLT
- Paired design increases power by removing between-subject variability
- Closed-form confidence intervals for the mean difference

Weaknesses

- Sensitive to outliers: a single extreme value can distort $\overline{x}$ and s
- Only tests means: does not detect differences in variance or shape
- Student's two-sample version is sensitive to unequal variances; prefer Welch's in practice
- Requires normality for small samples; use Wilcoxon tests as non-parametric alternatives

Comparison with alternatives

- **vs. Wilcoxon signed-rank / Mann-Whitney**: non-parametric alternatives; more robust to outliers and non-normality, at the cost of some power under normality
- **vs. One-way ANOVA**: ANOVA generalises the two-sample t-test to $k > 2$ groups; with $k = 2$, $F = t^2$ and they are equivalent
- **vs. z-test**: z-test requires known σ^2; t-test estimates it from data and is appropriate for unknown variance (always in practice)
- **vs. Welch's t-test**: Welch's is strictly more general; Student's gains a small amount of power only when equal variances truly hold

Bayesian alternative: Place a prior on μ (e.g. $\mathcal{N}(\mu_0, \tau^2)$) and σ^2 (e.g. inverse-Gamma). The posterior of μ — or of the difference $\mu_1 - \mu_2$ — replaces the p-value with a full uncertainty quantification. The Bayes factor comparing H_0 to H_1 can also be computed (e.g. via the JZS prior).

Return to the table.

Chapter 7

Data Recovery

7.1 Sampling Methods

7.1.1 Monte Carlo approximation

Purpose computing the distribution of a random variable's function using the change of variables formula can be difficult.
As simple and powerful alternative is to generate S samples $(x_s)_{1\leq s\leq S}$ from the distribution.

Theory Given the samples we can approximate the distribution of $f(X)$ by using the empirical distribution of $\{f(x_s)\}_{1\leq s\leq S}$
We can use Monte Carlo to approximate the expected value of any function of a random variable:

$$\mathbb{E}\left(f(X)\right) = \int f(x)p(x)dx \approx \frac{1}{S}\sum_{s=1}^{S} f(x_s)$$

where $x_s \hookrightarrow p(X)$. This called Monte Carlo integration

Accuracy It increases with sample size.
In denoting $\mu = \mathbb{E}\left(f(X)\right)$ the exact mean and $\hat{\mu}$ the MC approximation, one can show that

$$(\hat{\mu} - \mu) \hookrightarrow \mathcal{N}\left(0, \frac{\sigma^2}{S}\right)$$

where $\sigma^2 = \mathbb{V}\left(f(X)\right)$ this is a consequence of central-limit theorem, of course σ^2 is unknown.
$\sqrt{\frac{\hat{\sigma}^2}{S}}$ is called the *standard error* and is an estimate of our uncertainty about our estimate μ

Strengths

- function only evaluated in places where there is non-negligible probability: advantage over numerical integration

Examples

- estimating π

7.1.2 Bootstrap

Purpose In practice 32 bootstrap seems to be a good compromise between speed and accuracy.
It is a Monte Carlo technique to approximate the sampling distribution.

Theory If we knew the true parameters θ^* we could generate S fake datasets of size N from the distribution $\forall (i,s) \in [\![1,n]\!] \times [\![1,s]\!], x_i^s \hookrightarrow p(\cdot|\theta^*)$
We could then compute our estimator from each sample: $\hat{\theta}^s = f(x^s)$.
Then 2 approaches:

- *parametric*: generate the samples using $\hat{\theta}(\mathcal{D})$ as θ is unknown
- *non-parametric*: sample the $(x^s)_{1 \leq s \leq S}$ with replacement from the original $\mathcal{D}$ and then compute induced distribution

Connection between $\hat{\theta}^s = \hat{\theta}(x^s)$ and $\theta^s \hookrightarrow p(\cdot|\mathcal{D})$ Conceptually quite different, but in the common case the prior is not very strong they can be quite similar. One can think of the bootstrap distribution distribution as a "poor man's" posterior.

Strengths

- Useful when the estimator is a complex function of of the true parameter

7.1.3 Monte Carlo Inference

Sampling from a Gaussian (Box-Muller method) The idea is we sample uniformly from a unit radius circle and then use the change of variables to derive samples from spherical 2d Gaussian.

Rejection sampling

Basic idea we create a *proposal distribution* $q(x)$ which satisfies

Importance sampling

Basic idea The idea is to draw samples $\boldsymbol{x}$ in regions which have high probability $\mathbb{P}(\boldsymbol{x})$

7.1.4 Particle filtering

it is a Monte Carlo algorithm for recursive Bayesian inference

7.1.5 Annealing methods

COMPLETE

7.2 Information Theory

Originally developed to study sending messages from discrete alphabets over a noisy channel (like communication via radio transmission). The basic intuition behind information theory is that learning that an unlikely event has occurred is more informative than learning that a likely event has occurred.

7.2.1 Shannon entropy

Purpose It is a measure of the uncertainty in a random variable.

Theory We have self-information

$$I(x) = -\log(\mathbb{P}(x))$$

Depending of the basis of the used logarithm, the unit can be expressed in *nats* if the base is e or *bits* if the basis is 2.
One *nat* is the amount of information gained by observing an event of probability $\frac{1}{e}$

$$\mathbb{H}(X) \triangleq -\mathbb{E}_{X \hookrightarrow P}(I(x)) = -\sum_{k=1}^{K} \mathbb{P}(X=k) \log(\mathbb{P}(X=k))$$

It gives a lower bound on the number of bits needed on average to encode symbols drawn from a distribution P. The discrete distribution with maximum entropy is the uniform distribution, conversely the one with minimum entropy is any delta-function that puts all its mass on one state.

7.2.2 Kullback-Leibler (KL) divergence

Purpose It is a measure of dissimilarity of 2 probability distribution p and q.

Theory

$$\begin{aligned}\mathbb{KL}(p||q) &= \sum_{k=1}^{K} p_k \log\left(\frac{p_k}{q_k}\right) \\ &= \sum_{k=1}^{K} p_k \log(p_k) - \sum_{k=1}^{K} p_k \log(q_k) \\ &= -\mathbb{H}(p) + \mathbb{H}(p,q)\end{aligned}$$

It gives the extra amount of information needed to send a message containing symbols drawn from probability distribution P when we use a code that was designed to minimize the length of messages drawn from probability distribution Q Where $\mathbb{H}(p,q)$ is called *cross entropy*, being the average number of bits needed to encode data coming from a source with distribution p when we use model q to define our codebook.

7.2.3 Mutual information

Purpose Correlation coefficient is quite restrictive, a more general approach is to determine how similar the joint distribution $p(X,Y)$ is to the factorized distribution $p(X)p(Y)$

Theory

Discrete

$$\boxed{\mathbb{I}(X,Y) \triangleq \mathbb{KL}\left(p(X,Y)||p(X)p(Y)\right) = \mathbb{H}(X) - \mathbb{H}(X|Y) = \mathbb{H}(Y) - \mathbb{H}(Y|X)}$$

where $\mathbb{H}(Y|X)$ is the *conditional entropy* defined as $\mathbb{H}(Y|X) = \Sigma_x p(x)\mathbb{H}(Y|X=x)$ It is the extra amount of information needed to send a message containing symbols drawn from probability distribution P when use a code that was designed to minimize the length of messages drawn from probability distribution Q. Thus we can interpret the *Mutual Information* between X and Y as the reduction in uncertainty about X after observing Y, or, by symmetry about Y after observing X.

Continuous Quantizing can have significant impact on the results, we could then try to estimate many different bin sizes and locations to finally compute the maximum MI achieved called *maximal information coefficient*:

$$m(x,y) = \frac{\max_{G\in\mathcal{G}(x,y)} \mathbb{I}\left(X(G), Y(G)\right)}{\log\left(min(x,y)\right)}$$

where $\mathcal{G}(x,y)$ is the set of 2d grids of size $x \times y$ and $X(G), Y(G)$ represents a discretization of the variable onto the grid. $MIC \triangleq \max_{x,y:xy<B} m(x,y)$

7.3 Key Mathematical Functions

7.3.1 Softmax function

Purpose The softmax function takes as input a vector z of K real numbers, and normalizes it into a probability distribution consisting of K probabilities proportional to the exponentials of the input numbers.

$$\sigma(\boldsymbol{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

Interpretation it is rather a smooth approximation of the *argmax* function, meaning the function returning the index of the maximum value of a given vector.

Chapter 8

Markov Models

8.1 Markov Models and Hidden Markov Models

8.1.1 Markov Models

Definition Andrey Markov introduced these models in 1906 to extend the law of large numbers beyond fully independent variables, by keeping only a weaker form of independence: the future is independent of the past once the present is known.
It assumes that X_t captures all the relevant information for predicting the future. If we assume discrete time steps the joint distribution is:

$$\mathbb{P}(X_{1:T}) = \mathbb{P}(X_1) \prod_{t=2}^{T} \mathbb{P}(X_t|X_{t-1})$$

if the transition function $\mathbb{P}(X_t|X_{t-1})$ is independent of time then the chain is called *homogeneous*, *stationary* or *time-invariant*.
This assumption allows us to model an arbitrary of variables using a fixed number of parameters, such models are called **stochastic process**.

Assumptions $x_{t+1} \perp x_{1:t-1}|x_t$, meaning the future is independent of the past given the present, is called the *first order Markov assumption.*

Choosing the order An order-m model assumes X_t depends only on $X_{t-m:t-1}$ and needs K^{m+1} parameters.

	Order too low	**Order too high**
Error type	Bias (underfitting)	Variance (overfitting)
Assumption made	Dependent events treated as independent beyond the last m states	Noise in the counts treated as real dependence
Model capacity	Too small: different histories are merged into one state	Contains all lower orders, but needs K^{m+1} parameters
Effect on data	Long-range context is lost	Contexts rarely or never seen, unreliable estimates (0 or 1 probabilities)
Symptom	Locally plausible but incoherent sequences	Memorized, verbatim sequences and poor generalization
Fix	Raise the order	Lower the order, smooth (backoff, interpolation) or regularize
Order selection	Choose m by validation likelihood or AIC/BIC: take the smallest order whose validation score stops improving.	

Transition matrix When X_t is discrete the conditional distribution $\mathbb{P}(X_t|X_{t-1})$ can be written as $K \times K$ matrix known as **transition matrix** $\boldsymbol{A}$, where $A_{ij} = \mathbb{P}(X_t = j|X_{t-1} = i)$ being the probability of going from state i to j. Each row of the matrix sums to one $\Sigma_j A_{ij} = 1$ so it is called *stochastic matrix*.

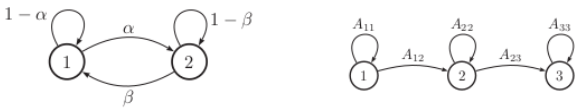


Figure 8.1: State transition diagrams for, 2-state chain and 3-state chain.

The n-step transition matrix $\boldsymbol{A}(n)$ is defined as $\boldsymbol{A}_{ij}(n) = \mathbb{P}(X_{t+n} = j|X_t = i)$
For a steps $m + n$, $\boldsymbol{A}(m + n) = \boldsymbol{A}(m) \cdot \boldsymbol{A}(n)$

Transition matrix for higher orders For an order-m model, define the augmented state $Y_t = X_{t-m+1:t}$, which takes K^m values. Since Y_t depends only on Y_{t-1}, Y is a first-order chain with a $K^m \times K^m$ transition matrix. For $m = 2$:

$$\boxed{\mathbb{P}(Y_t = (j,k)|Y_{t-1} = (i,j)) = \mathbb{P}(X_t = k|X_{t-2} = i, X_{t-1} = j)}$$

- the matrix is sparse: from (i,j) one can only reach the states whose first component is j, so each row has at most K non-zero entries out of K^m
- each row still sums to one, so it is a stochastic matrix
- everything above (n-step matrix, stationary distribution, MLE by counting) applies to Y
- only K^{m+1} of its K^{2m} entries can be non-zero, which is the parameter count of the model

Example: with $K = 2$ and $m = 2$ the states are $00, 01, 10, 11$ and the matrix is 4×4; state 01 can only go to 10 or 11, and state 00 only to 00 or 01.

Application

- Language modeling
  - Sentence completion
  - Data compression
  - Text classification
  - Automatic essay writing

MLE for Markov language models The probability of any particular sequence of length T is given by:

$$\mathbb{P}(x_{1:T}|\boldsymbol{\theta}) = \pi(x_1)\prod_{t=2}^{T} A(x_{t-1}, x_t) = \prod_{i=1}^{K} \pi_i^{\mathbb{1}_{\{x_1=i\}}} \prod_{t=2}^{T}\prod_{j=1}^{K}\prod_{i=1}^{K} A_{ij}^{\mathbb{1}_{\{x_t=j, x_{t-1}=i\}}}$$

Then the log-likelihood of a set of sequences $\mathcal{D} = \{\boldsymbol{x}_i\}_{1 \le i \le n}$:

$$\boxed{\log\left(\mathbb{P}(\mathcal{D}|\boldsymbol{\theta})\right) = \sum_{n=1}^{N} \log\left(\mathbb{P}(\boldsymbol{x}_n|\boldsymbol{\theta})\right) = \sum_i N_i^{(1)} \log(\pi_i) + \sum_i \sum_j N_{ij} \log(a_{ij})}$$

where $\hat{\pi}_i = \dfrac{N_i^{(1)}}{\sum_{i'}^{K} N_{i'}^{(1)}}$ and $\hat{a}_{ij} = \dfrac{N_{ij}}{\sum_{j'}^{K} N_{ij'}}$

Stationary distribution of a Markov chain We can interpret Markov models as stochastic dynamical systems, where we hop from one state to another at each time step, we are then often interested in the long term distribution over states, called *stationary distribution*.
Considering the transition matrix $\boldsymbol{A}$ defined by $\boldsymbol{A}_{ij} = \mathbb{P}(X_t = j | X_{t-1} = i)$ and the probability to be at a given state at time t $\boldsymbol{\pi}$ defined by $\pi_t(j) = \mathbb{P}(X_t = j)$.
If we ever reach a stage where $\boldsymbol{\pi} = \boldsymbol{\pi}\boldsymbol{A}$ then we have reached the *stationary distribution*.

8.1.2 Hidden Markov Models

Definition It consists of a discrete-time, discrete-state Markov chain, with hidden states $z_t \in [\![1, K]\!]$. The corresponding joint distribution has the form:

$$\mathbb{P}(\boldsymbol{z}_{1:T}, \boldsymbol{x}_{1:T}) = \mathbb{P}(\boldsymbol{z}_{1:T})\,\mathbb{P}(\boldsymbol{x}_{1:T}|\boldsymbol{z}_{1:T}) = \left[\mathbb{P}(z_1)\prod_{t=2}^{T}\mathbb{P}(z_t|z_{t-1})\right]\prod_{t=1}^{T}\mathbb{P}(\boldsymbol{x}_t|z_t)$$

It's common to have for:

- *discrete* observations: $\mathbb{P}(\boldsymbol{x}_t = l | z_t = k, \boldsymbol{\theta}) = Beta(k, l)$
- *continuous* observations: $\mathbb{P}(\boldsymbol{x}_t | z_t = k, \boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{x}_t | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$

Strengths

- can represent long-range dependencies between observations compared to Markov models
- useful for time series prediction
- can define class-conditional densities inside a generative classifier

Examples

- Automatic speech recognition: $\boldsymbol{x}_t$ represents features extracted from the speech signals and z_t represents the word that is being spoken
- Activity recognition: $\boldsymbol{x}_t$ represents feature extracted from a video frame and z_t is the class of activity the person is engaged (running, waking, sitting)

Types of inferences

- **Filtering**: computing the *belief state* $\mathbb{P}(z_t|\boldsymbol{x}_{1:t})$ online or recursively as the data streams in. The name "filtering" is due to the noise reduction induced with the hidden state estimating using the current $\mathbb{P}(z_t|\boldsymbol{x}_t)$
- **Smoothing**: computing $\mathbb{P}(z_t|\boldsymbol{x}_{1:T})$ offline given all the evidence
- **Fixed Lag Smoothing**: computing $\mathbb{P}(z_{t-l}|\boldsymbol{x}_{1:t})$ that is an interesting compromise between online and offline estimation. It provides better performance than filtering but incurs a slight delay. Changing the lag size, one can trade off *accuracy* vs *delay*
- **Prediction**: computing $\mathbb{P}(z_{t+h}|\boldsymbol{x}_{1:t})$ where $h > 0$ is called the *prediction horizon*. For example $h = 2$ implies $\mathbb{P}(z_{t+2}|\boldsymbol{x}_{1:t}) = \sum_{z_{t+1}}\sum_{z_t}\mathbb{P}(z_{t+2}|z_{t+1})\,\mathbb{P}(z_{t+1}|z_t)\,\mathbb{P}(z_t|\boldsymbol{x}_{1:t})$
- **MAP estimation**: computing $\underset{(\boldsymbol{z}_k)_{1\leq k\leq t}}{\arg\min}$ which is most probable state sequence.
- **Posterior samples**: if there is more that one plausible interpretation of the data, it can be useful to sample from the posterior $\boldsymbol{z}_{1:t} \hookrightarrow \mathbb{P}(\boldsymbol{z}_{1:t}|\boldsymbol{x}_{1:t})$
- **Probability of the evidence**: compute $\mathbb{P}(\boldsymbol{x}_{1:t}) = \sum_{\boldsymbol{z}_{1:t}}\mathbb{P}(\boldsymbol{z}_{1:t}, \boldsymbol{x}_{1:t})$ meaning summing up over all hidden paths. Can be used to classify sequences for model-based clustering, anomaly detection.

8.1.3 Inference HMMs

The forward algorithm

Purpose recursively computing the filtered marginals: for $j \in [\![1, K]\!]$ $\mathbb{P}(z_t = j|\boldsymbol{x}_{1:t})$ in a hidden Markov model.

Steps

1. *Prediction step*: computing the *one-step-ahead predictive density* acting as the new prior for t:
   $\mathbb{P}(z_t = j|\boldsymbol{x}_{1:t-1}) = \sum_i \mathbb{P}(z_t = j|z_{t-1} = i)\,\mathbb{P}(z_{t-1} = i|\boldsymbol{x}_{1:t-1})$
2. *Update step*: we absorb the observed data from t using Bayes rule:
   $\alpha_t(j) \triangleq \mathbb{P}(z_t = j|\boldsymbol{x}_{1:t}) = \mathbb{P}(z_t = j|\boldsymbol{x}_t, \boldsymbol{x}_{1:t-1}) = \frac{1}{Z_t}\mathbb{P}(\boldsymbol{x}_t|z_t = j)\,\mathbb{P}(z_t = j|\boldsymbol{x}_{1:t-1})$ where the normalization constant is $Z_t \triangleq \mathbb{P}(\boldsymbol{x}_t|\boldsymbol{x}_{1:t-1}) = \sum_j \mathbb{P}(\boldsymbol{x}_t|z_t = j)\,\mathbb{P}(z_t = j|\boldsymbol{x}_{1:t-1})$

   Hint: consider $\boldsymbol{x}_t \perp \boldsymbol{x}_{1:t-1}|z_t$

The distribution $\mathbb{P}(z_t|\boldsymbol{x}_{1:t})$ is called the (filtered) belief state at time t, it is a vector of K numbers denoted $\boldsymbol{\alpha}_t$ Considering $\odot$ being the *Hadamard product* representing element-wise vector multiplication.

$$\boxed{\boldsymbol{\alpha}_t \propto \boldsymbol{\psi}_t \odot \boldsymbol{\Psi}^T \boldsymbol{\alpha}_{t-1}}$$

where $\psi_t(j) = \mathbb{P}(\boldsymbol{x}_t|z_t = j)$ being the local evidence at time t and $\boldsymbol{\Psi}(i, j) = \mathbb{P}(z_t = j|z_{t-1} = i)$ is the transition matrix.

Forwards-Backwards algorithm

Purpose Computing smoothed marginals $\mathbb{P}(z_t = j|\boldsymbol{x}_{1:T})$ using offline inference.

Basic idea The key decompositions relies on the fact that we can break the chain into two parts: the *past* and the *future* by conditioning on z_t
Let $\alpha_t(j) \triangleq \mathbb{P}(z_t = j|\boldsymbol{x}_{1:t})$ be the filtered belief state as before. Define as well $\beta_t(j) \triangleq \mathbb{P}(\boldsymbol{x}_{t+1:T}|z_t = j)$ being the conditional likelihood of the future evidence given that the hidden state at time t is j.
Finally define $\gamma_t(j) \triangleq \mathbb{P}(z_t = j|\boldsymbol{x}_{1:T})$ as the desired smoothed posterior marginal.
We previously described how to recursively compute the α's in a *left-to-right* fashion. We now describe how to recursively compute the β's in a *right-to-left* fashion.

$$\begin{aligned}
\beta_{t-1}(i) &= \mathbb{P}(\boldsymbol{x}_{t:T}|z_{t-1} = i) \\
&= \sum_j \mathbb{P}(z_t = j, \boldsymbol{x}_t, \boldsymbol{x}_{t+1:T}|z_{t-1} = i) \\
&= \sum_j \mathbb{P}(\boldsymbol{x}_{t+1:T}|z_t = j, \cancel{z_{t-1} = i}, \cancel{\boldsymbol{x}_t})\,\mathbb{P}(z_t = j, \boldsymbol{x}_t|z_{t-1} = i) \\
&= \sum_j \mathbb{P}(\boldsymbol{x}_{t+1:T}, z_t = j)\,\mathbb{P}(\boldsymbol{x}_t|z_t = j, \cancel{z_{t-1} = i})\,\mathbb{P}(z_t = j|z_{t-1} = i) \\
&= \sum_j \beta_t(j)\psi_t(j)\Psi(i, j)
\end{aligned}$$

In matrix-vector form: $\boldsymbol{\beta} = \boldsymbol{\Psi}(\boldsymbol{\psi}_t \odot \boldsymbol{\beta}_t)$ We can think of this algorithm as passing "messages" from left to right and then from right to left and then combining them at each node.

Complexity It takes $O(K^2T)$ time since at each step we perform a $K \times K$
We can think of this algorithm as passing "messages" from left to right and then from right to left, and then combining them at each node. Finally

$$\boxed{\gamma_t(j) \propto \alpha_t(j)\beta_t(j)}$$

The Viterbi Algorithm

Purpose Compute the most probable sequence of states in a chain-structured graphical model: $\boldsymbol{z}^* = \arg\max_{\boldsymbol{z}_{1:T}} \mathbb{P}(\boldsymbol{z}_{1:T}|\boldsymbol{x}_{1:T})$.
This is equivalent to computing a shortest path through *trellis diagram* where the nodes are possible states at each time step and the node and edge weights are log probabilities.

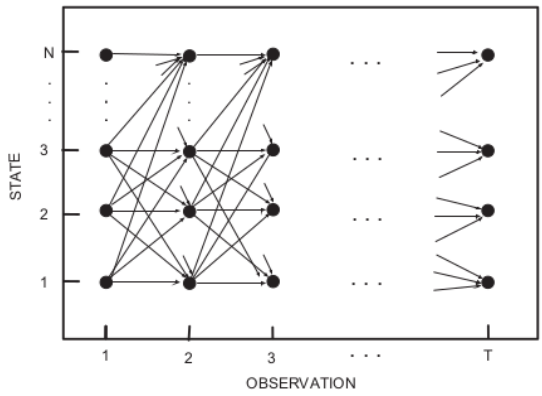


Figure 8.2: Trellis diagram: states vs time for a Markov chain.

MAP vs MPE The (*jointly*) most probable sequence of states: $\boldsymbol{z}^*$ is not necessarily the same as the sequence of (*marginally*) most probable states, *Maximize of the Posterior Marginals* (MPM): $\hat{\boldsymbol{z}} = \left(\arg\max_z \mathbb{P}(z|\boldsymbol{x}_{1:T})\right)_{z_1 \le z \le z_T}$.
In one hand the joint MAP estimate is that is always globally consistent on the other hand the MPM estimate can be more robust

Detail of the algorithm Although is tempting to implement Viterbi by replacing the sum-operator with max-operator in the forward-backwards, it can lead to incorrect results if there are multiple equally probably joint assignments. Therefore Viterbi uses max-product in the forward pass and a traceback procedure in the backwards pass.
Let's define the probability of ending up in state j at time t given that we take the most probable path:

$$\delta_t(j) \triangleq \max_{\boldsymbol{z}_{1:t-1}} \mathbb{P}(\boldsymbol{z}_{1:t-1}, z_t = j|\boldsymbol{x}_{1:t})$$

The key insight is that the most probable path to state j at time t must consist of the most probable path to some other state i at time $t-1$, followed by a transition from i to j.

1. Hence $\delta_t(j) = \max_i \delta_{t-i}(i)\Psi(i,j)\psi_t(j)$
2. Keeping track of the most likely previous state for each possible state that we end up: $a_t(j) = \arg\max_i \delta_{t-1}(i)\Psi(i,j)\psi_t(j)$. It tells us the most likely previous state on the most probable path to $z_t = j$.
3. initialize $\delta_1(j) = \pi_j\psi_1(j)$
4. compute the most probable final state $z_T^* = \arg\max_j \delta_T(j)$
5. using traceback we can compute the most probable sequence of states: $z_t^* = a_{t+1}(z_{t+1}^*)$

Complexity It is $O(K^2T)$

Forwards filtering, backwards sampling

Purpose Sample paths from posterior: $\boldsymbol{z}_{1:T}^s \hookrightarrow \mathbb{P}(\boldsymbol{z}_{1:T}|\boldsymbol{x}_{1:T})$ Without being obliged to perform forward-backwards pass and then an additional forward sampling pass.

Steps The key insight is that to do forward pass and then perform sampling in the backward pass we can write the joint from right to left using $\mathbb{P}(\boldsymbol{z}_{1:T}|\boldsymbol{x}_{1:T}) = \mathbb{P}(z_T|\boldsymbol{x}_{1:T}) \prod_{t=T-1}^{1} \mathbb{P}(z_t|z_{t+1}, \boldsymbol{x}_{1:T})$
Then we can sample z_t given future sampled states using:
$z_t^s \hookrightarrow \mathbb{P}(z_t|z_{t+1:T}, \boldsymbol{x}_{1:T}) = \mathbb{P}(z_t|z_{t+1}, \cancel{\boldsymbol{z_{t+2:T}}}, \cancel{\boldsymbol{x_{t+1:T}}}) = \mathbb{P}\left(z_{t+1}|z_{t+1}^s, \boldsymbol{x}_{1:t}\right)$

The sampling distribution is given by:

$$\begin{aligned}
\mathbb{P}(z_t = i|z_{t+1} = j, \boldsymbol{x}_{1:t}) &= \mathbb{P}(z_t|z_{t+1}, \boldsymbol{x}_{1:t}, \cancel{\boldsymbol{x_{t+1}}}) \\
&= \frac{\mathbb{P}(z_{t+1}, z_t|\boldsymbol{x}_{1:t+1})}{\mathbb{P}(z_{t+1}|\boldsymbol{x}_{1:t+1})} \\
&\propto \frac{\mathbb{P}(\boldsymbol{x}_{t+1}|z_{t+1}, \cancel{z_t}, \cancel{\boldsymbol{x_{1:t}}})\,\mathbb{P}(z_{t+1}, z_t|\boldsymbol{x}_{1:t})}{\mathbb{P}(z_{t+1}, z_t|\boldsymbol{x}_{1:t+1})} \\
&= \frac{\mathbb{P}(\boldsymbol{x}_{t+1}|z_{t+1})\,\mathbb{P}(z_{t+1}|z_t, \cancel{\boldsymbol{x_{1:t}}})\,\mathbb{P}(z_t|\boldsymbol{x}_{1:t})}{\mathbb{P}(z_{t+1}, z_t|\boldsymbol{x}_{1:t+1})} \\
&= \frac{\phi_{t+1}(j)\psi(i,j)\alpha_t(i)}{\alpha_{t+1}(j)}
\end{aligned}$$

8.1.4 Learning for HMMs

We want to estimate the parameters $\boldsymbol{\theta} = (\boldsymbol{\pi}, \boldsymbol{A}, \boldsymbol{B})$ where $\pi(i) = \mathbb{P}(z_i = i)$ is the initial state distribution, $\boldsymbol{A}(i,j) = \mathbb{P}(z_t = j|z_{t-1} = i)$ the transition matrix and $\boldsymbol{B}$ the parameters of the class-conditional densities $\mathbb{P}(\boldsymbol{x}_t|z_t = j)$

Training with fully observed data If we observe the hidden state sequences, we cna compute the MLEs for $\boldsymbol{A}$ and $\boldsymbol{\pi}$ exactly.
In using a conjugate prior we can easily compute the posterior.
The details on how to estimate B depend on the form of the observation model.

EM for HMMs (Baum-Welch Algorithm) If z_t are not observed we are in a situation analogous to fitting a mixture model. The most common approach is to use EM algorithm to find the MLE or MAP parameters.

E step It is straightforward to show that the expected complete data log likelihood is given by:

$$Q(\boldsymbol{\theta}, \boldsymbol{\theta}^{old}) = \sum_{k=1}^{K} \mathbb{E}\left(N_k^1\right) \log(\pi_k) \sum_{j=1}^{K}\sum_{k=1}^{K} \mathbb{E}\left(N_{jk}\right) \log(A_{jk}) + \sum_{i=1}^{N}\sum_{t=1}^{T_i}\sum_{k=1}^{K} \mathbb{P}\left(z_t = k|\boldsymbol{x}_i, \boldsymbol{\theta}^{old}\right) \log\left(\mathbb{P}\left(\boldsymbol{x}_i|\phi_k\right)\right)$$

where $\begin{cases} \mathbb{E}\left(N_k^1\right) &= \sum_{i=1}^{N} \mathbb{P}\left(z_{i1} = k|\boldsymbol{x}_i, \boldsymbol{\theta}^{old}\right) \\ \mathbb{E}\left(N_{jk}\right) &= \sum_{i=1}^{N}\sum_{t=1}^{T_i} \mathbb{P}\left(z_{i,t-1} = j, z_{i,t} = k|\boldsymbol{x}_i, \boldsymbol{\theta}^{old}\right) \\ \mathbb{E}\left(N_j\right) &= \sum_{i=1}^{N}\sum_{t=1}^{T_i} \mathbb{P}\left(z_{i,t} = k|\boldsymbol{x}_i, \boldsymbol{\theta}^{old}\right) \end{cases}$ These expected sufficient statistics can be computed by running the forward-backward algorithm on each sequence.
In particular, this algorithm computes the following smoothed node and edge marginals:
$\begin{cases} \gamma_{i,t}(j) \triangleq \mathbb{P}(z_t = j|\boldsymbol{x}_{i,1:T_i}, \boldsymbol{\theta}) \\ \xi_{i,t}(j,k) \triangleq \mathbb{P}(z_{t-1} = j, z_t = k|\boldsymbol{x}_{i,1:T}, \boldsymbol{\theta}) \end{cases}$

M step From results in mixture of Multinoulli: $\hat{A}_{jk} = \dfrac{\mathbb{E}(N_k)}{\sum_{k'} \mathbb{E}(N_{jk'})}, \hat{\pi}_k = \dfrac{\mathbb{E}(N_k^1)}{N}$

For a Multinoulli observation model the expected sufficient statistics are: $\mathbb{E}(M_{jl}) = \sum_{i=1}^{N}\sum_{t=1}^{T_i} \gamma_{i,t}(j)\mathbb{1}_{\{x_{i,t}=l\}} = \sum_{i=1}^{N}\sum_{t:x_{i,t}=l}^{\gamma}{}_{i,t}(j)$.

The M step has the form $\hat{B}_{jl} = \dfrac{\mathbb{E}(M_{jl})}{\mathbb{E}(N_j)}$.

For a Gaussian observation model: $\begin{cases} \mathbb{E}(\overline{x}_k) = \sum_{i=1}^{N}\sum_{t=1}^{T_i} \gamma_{i,t}(k)\boldsymbol{x}_{i,t} \\ \mathbb{E}\left((\overline{xx})_k^T\right) = \sum_{i=1}^{N}\sum_{t=1}^{T_i} \gamma_{i,t}(k)\boldsymbol{x}_{i,t}\boldsymbol{x}_{i,t}^T \end{cases}$

The M step becomes :

$$\begin{cases} \hat{\boldsymbol{\mu}}_k = \dfrac{\mathbb{E}(\overline{\boldsymbol{x}}_k)}{\mathbb{E}(N_k)} \\ \hat{\boldsymbol{\Sigma}}_k = \dfrac{\mathbb{E}\left((\overline{xx})_k^T\right) - \mathbb{E}(N_k)\hat{\boldsymbol{\mu}}_k\boldsymbol{\mu}_k^T}{\mathbb{E}(N_k)} \end{cases}$$

Bayesian methods for 'fitting' HMMs We can use variational Bayes EM method, the E step uses forwards-backwards but where we plug in the posterior mean parameters instead of the MAP estimates. The M step updates the parameters o the conjugate posteriors instead of updating the parameters themselves.

Model selection

Choosing the number of hidden states

- using grid-search over range of K's
- Variational Bayes to "extinguish" unwanted components
- Use an "infinite HMM" which is based on the hierarchical Dirichlet process

8.2 State Models

8.2.1 Basics

Definition It is an hidden Markov models, except the hidden states are continuous. The model can be written in the following generic form: $\begin{cases} \boldsymbol{z}_t = g(\boldsymbol{u}_t, \boldsymbol{z}_{t-1}, \boldsymbol{\epsilon}_t) \\ \boldsymbol{y}_t = h(\boldsymbol{z}_t, \boldsymbol{u}_t, \boldsymbol{\delta}_t) \end{cases}$ where $\boldsymbol{z}_t$ is the hidden state $\boldsymbol{u}_t$ is an optimal input or control signal, $\boldsymbol{y}_t$ is the observation, g is the *transition model*, h is the *observation model* and $\boldsymbol{\epsilon}_t$ is the system noise at time t.

- *transition model*: $\boldsymbol{z}_t = \boldsymbol{A}_t\boldsymbol{z}_{t-1} + \boldsymbol{B}_t\boldsymbol{u}_t + \boldsymbol{\epsilon}_t$
- *observation model* $\boldsymbol{y}_t = \boldsymbol{C}_t\boldsymbol{z}_t + \boldsymbol{D}_t\boldsymbol{u}_t + \boldsymbol{\delta}_t$
- *system noise*: $\boldsymbol{\epsilon}_t \hookrightarrow \mathcal{N}(\boldsymbol{0}, \boldsymbol{Q}_t)$
- *observation noise*: $\boldsymbol{\delta}_t \hookrightarrow \mathcal{N}(\boldsymbol{0}, \boldsymbol{R}_t)$

This model is called a Linear-Gaussian State Space Model (LG-SSM), if the parameters $\boldsymbol{\theta}_t = (\boldsymbol{A}_t, \boldsymbol{B}_t, \boldsymbol{C}_t, \boldsymbol{D}_t, \boldsymbol{Q}_t, \boldsymbol{R}_t)$ are independent of time, the model is called *stationary*.

8.2.2 Time Series Forecasting

The idea is to create a generative model of the data in terms of latent processes which capture different aspects of the signal.

Local level model The simplest latent process: $\begin{cases} y_t &= a_t + \epsilon_t^y,\ \epsilon_t^y \hookrightarrow \mathcal{N}(0, R) \\ a_t &= a_{t-1} + \epsilon_t^a,\ \epsilon_t^a \hookrightarrow \mathcal{N}(0, Q) \end{cases}$ where the hidden state is just $z_t = a_t$

Local linear trend Many time series exhibit linear trends upward or downward, at least locally. We can model this by letting the level a_t change by an amount b_t at each step : $\begin{cases} y_t &= a_t + \epsilon_t^y,\ \epsilon_t^y \hookrightarrow \mathcal{N}(0, R) \\ a_t &= a_{t-1} + b_{t-1} + \epsilon_t^a,\ \epsilon_t^a \hookrightarrow \mathcal{N}(0, Q_a) \\ b_t &= b_{t-1} + \epsilon_t^b,\ \epsilon_t^b \hookrightarrow \mathcal{N}(0, Q_b) \end{cases}$

Seasonality This can be modeled by adding a latent process consisting of a series offset terms c_t which sum to zero (on average) over a complete cycle of S steps: $c_t = -\sum_{s=1}^{S-1} c_{t-s} + \epsilon_t^c, \epsilon_t^c \hookrightarrow \mathcal{N}(0, Q_c)$

ARMA(Auto-Regressive Moving-Average) models $x_t = \sum_{i=1}^{p} \alpha_i x_{t-i} + \sum_{j=1}^{q} \beta_j w_{t-j} + v_t$, where v_t and w_t follow a $\mathcal{N}(0, 1)$

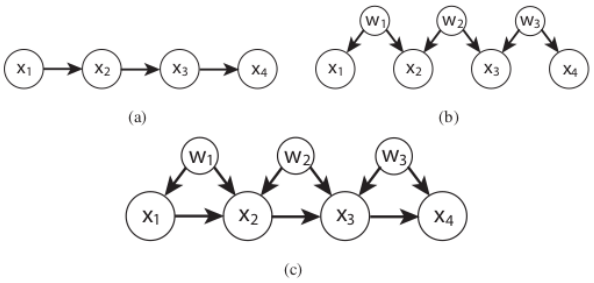


Figure 8.3: (a)→ $AR(1)$ (b)→ $MA(1)$ and (c)→ $ARMA(1, 1)$

The structural approach to time series is often easier to understand than the ARMA approach. In addition it allows the parameters to evolve over time, which makes the models more adaptive to non-stationary.

8.2.3 Inference in LG-SSM (Linear Gaussian - State Space Models)

Kalman filtering algorithm It is Kalman filter is an algorithm for exact Bayesian filtering for linear-Gaussian state space models that sequentially computes $\mathbb{P}(\boldsymbol{z}_t|\boldsymbol{y}_{1:t})$ for each t.

Prediction step

$$\begin{aligned} \mathbb{P}(\boldsymbol{z}_t|\boldsymbol{y}_{1:t-1}, \boldsymbol{u}_{1:t}) &= \int \mathcal{N}(\boldsymbol{z}_t|\boldsymbol{A}_t\boldsymbol{z}_{t-1} + \boldsymbol{B}_t\boldsymbol{u}_t, \boldsymbol{Q}_t)\, \mathcal{N}(\boldsymbol{z}_{t-1}|\boldsymbol{\mu}_{t-1}, \boldsymbol{\Sigma}_{t-1})\, d\boldsymbol{z}_{t-1} \\ &= \mathcal{N}(\boldsymbol{z}_t|\boldsymbol{\mu}_{t|t-1}, \boldsymbol{\Sigma}_{t|t-1}) \\ \boldsymbol{\mu}_{t|t-1} &\triangleq \boldsymbol{A}_t\boldsymbol{\mu}_{t-1} + \boldsymbol{B}_t\boldsymbol{u}_t \\ \boldsymbol{\Sigma}_{t|t-1} &\triangleq \boldsymbol{A}_t\boldsymbol{\Sigma}_{t-1}\boldsymbol{A}_t^T + \boldsymbol{Q}_t \end{aligned}$$

Measurement step Can be computed using Bayes rule:
$\mathbb{P}(\boldsymbol{z}_t|\boldsymbol{y}_t, \boldsymbol{y}_{1:t-1}, \boldsymbol{u}_{1:t}) \propto \mathbb{P}(\boldsymbol{y}_t|\boldsymbol{z}_t, \boldsymbol{u}_{1:t})\,\mathbb{P}(\boldsymbol{z}_t|\boldsymbol{y}_{1:t-1}, \boldsymbol{u}_{1:t})$.
We show that this is given by: $\begin{cases} \mathbb{P}(\boldsymbol{z}_t|\boldsymbol{y}_{1:t}, \boldsymbol{u}_t) & = \mathcal{N}(\boldsymbol{z}_t|\boldsymbol{\mu}_t, \boldsymbol{\Sigma_t}) \\ \boldsymbol{\mu}_t & = \boldsymbol{\mu}_{t|t-1} + \boldsymbol{K}_t\boldsymbol{r}_t \\ \boldsymbol{\Sigma}_t & = (\boldsymbol{I} - \boldsymbol{K}_t\boldsymbol{C}_t)\,\boldsymbol{\Sigma}_{t|t-1} \end{cases}$ where $\boldsymbol{r}_t$ is the residual given by the difference between our predicted observation and the actual observation: $\begin{cases} \boldsymbol{r}_t \triangleq \boldsymbol{y}_t - \hat{\boldsymbol{y}}_t \\ \hat{\boldsymbol{y}}_t \triangleq \mathbb{E}(\boldsymbol{y}_t|\boldsymbol{y}_{1:t-1}, \boldsymbol{u}_{1:t}) = \boldsymbol{C}_t\boldsymbol{\mu}_{t|t-1} + \boldsymbol{D}_t\boldsymbol{u}_t \end{cases}$
and K_t is the Kalman gain matrix given by $\boldsymbol{K}_t \triangleq \boldsymbol{\Sigma}_{t|t-1}\boldsymbol{C}_t^T\boldsymbol{S}_t^{-1}$ where $\boldsymbol{S}_t \triangleq Cov(\boldsymbol{r}_t|\boldsymbol{y}_{1:t-1}, \boldsymbol{u}_{1:t})$

The Kalman smoothing algorithm In offline stting we can wait until all the data has arrived and then compute $\mathbb{P}(\boldsymbol{z}_t|\boldsymbol{y}_{1:T})$. By conditioning on past and future data our uncertainty will be significantly reduced.

Comparison to the forwards-backwards algorithm for HMMs It turns out that we can rewrite the Kalman smoother in a modified form which makes it more similar to forwards-backwards for HMMs.

8.3 Markov Random Fields

8.3.1 Basics

Defintion Similar to Bayesian Network in its representation dependencies except that MRF are undirected and may be cyclic.

Conditional independence properties

- **Global Markov** property: for a sets of notes A, B and C, $\boldsymbol{x}_A \perp_G \boldsymbol{x}_B|\boldsymbol{x}_C$ *iff* C separates A from B in the graph G.
  Meaning that when we remove the nodes of C there is no way to connect any nodes in A to any nodes in B
- **Undirected local Markov** property: consider the set of nodes rending a given node N *conditionally independent* of all other nodes this is the Markov blanket denoted $mb(N) = N \perp \mathcal{V}\backslash cl(N)|mb(N)$, with $cl(N) \triangleq mb(N) \cup \{N\}$ is the *closure* of node N. One can show that in a UGM a node's Markov blanket is its set of immediate neighbors.
- **Pairwise Markov** property: two nodes are conditionally independent given the rest if there is no direct edge between them: $s \perp N|\mathcal{V}\backslash\{s, N\} \Leftrightarrow G_{sN} = 0$

Parametrization of MRFs

Hammersley-Clifford A positive distribution $p(\boldsymbol{y}) > 0$ satisfies the CI properties of an undirected graph G iff p can be represented as a product of factors:

$$p(\boldsymbol{y}|\boldsymbol{\theta}) = \frac{1}{Z(\boldsymbol{\theta})}\prod_{c\in\mathcal{C}}\psi_c(\boldsymbol{y}_c|\boldsymbol{\theta}_c)$$

where $\mathcal{C}$ is the set of all (maximal) cliques of G, and $Z(\boldsymbol{\theta})$ is the **partition function** given by $Z(\boldsymbol{\theta}) \triangleq \sum_{\boldsymbol{x}}\prod_{c\in\mathcal{C}}\psi_c(\boldsymbol{y}_c|\boldsymbol{\theta}_c)$

Connection with statistical physics There is a model known as the Gibbs distribution which can be written as follows: $p(\boldsymbol{y}|\boldsymbol{\theta}) = \frac{1}{Z(\boldsymbol{\theta})}\exp(E(\boldsymbol{y}_c|\boldsymbol{\theta}_c))$ where $E(\boldsymbol{y}_c) > 0$ is the energy associated with the variables in clique c. We can convert this to UGM by defining $\psi_c(\boldsymbol{y}_c|\boldsymbol{y}_c) = \exp(-E(\boldsymbol{y}_c|\boldsymbol{\theta}_c))$

Representing potential functions We define the log potentials as a linear function of the parameters: $\log\left(\psi_c(\boldsymbol{y}_c)\right) \triangleq \phi_c(\boldsymbol{y}_c)^T \boldsymbol{\theta}_c$ where $\phi_c(\boldsymbol{x}_c)$ is a feature vector derived from the values of the variables $\boldsymbol{y}_c$.

The resulting log probability has the form: $\log\left(p(\boldsymbol{y}|\boldsymbol{\theta})\right) \triangleq \sum_c \boldsymbol{\phi}_c(\boldsymbol{y}_c)^T \boldsymbol{\theta}_c - Z(\boldsymbol{\theta})$ this also known as a maximum entropy

Learning TO DO

Conditional Markov Random Field TO DO

Part III

Classical Learning

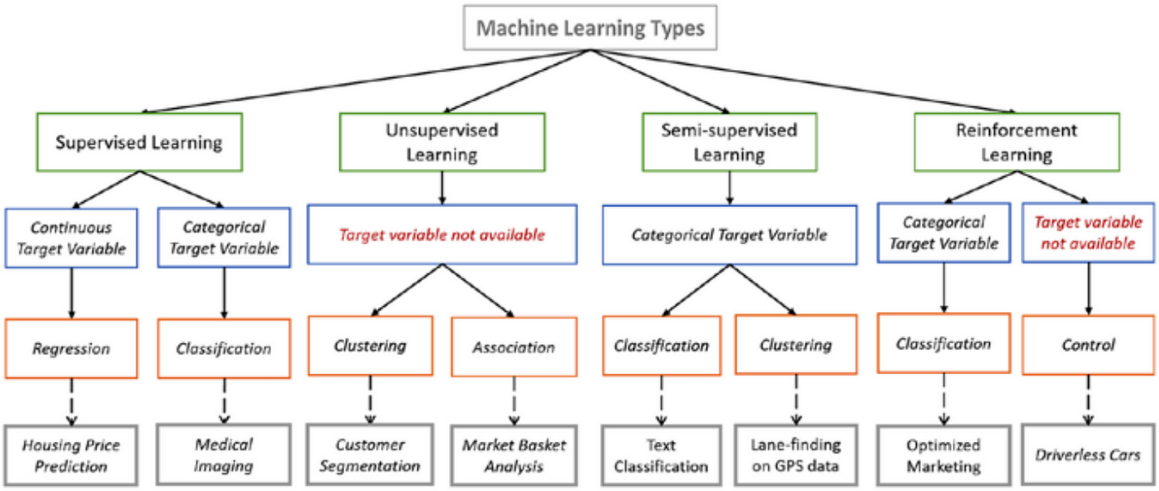


Figure 8.4: Types of machine learning methods

Chapter 9

Supervised Learning

With the *generative* approach we create a joint model of the form $\mathbb{P}(y, \boldsymbol{x})$, and then to condition on $\boldsymbol{x}$, thereby deriving $\mathbb{P}(y|\boldsymbol{x})$.
Alternatively, fitting directly a model of the form $\mathbb{P}(y|\boldsymbol{x})$ is a *discriminative* approach.

- *Easy to fit*: generative classifiers is usually very easy
- *Fit classes separately?*: in a generative classifier we estimate the parameters of each class conditional density independently
- *Handle missing features easily?*: in a generative classifier there is a natural way to handle missing data: $\mathbb{P}(\boldsymbol{x}_i, r_i|\boldsymbol{\theta}, \boldsymbol{\phi}) = \mathbb{P}(r_i|\boldsymbol{x}_i, \boldsymbol{\phi})\mathbb{P}(\boldsymbol{x}_i|\boldsymbol{\theta})$ where $\boldsymbol{\phi}$ are the parameters controlling whether the item is observed or not. Missing completely at random (MCAR): $\mathbb{P}(r_i|\boldsymbol{x}_i, \boldsymbol{\phi}) = \mathbb{P}\left(r_i|\boldsymbol{x}_i^0|\boldsymbol{\phi}\right)$, missing at random (MAR): $\mathbb{P}(r_i|\boldsymbol{x}_i, \boldsymbol{\phi}) = \mathbb{P}\left(r_i|\boldsymbol{x}_i^0, \boldsymbol{\phi}\right)$
- *Can handle unlabeled training data?*: Fairly easy in using generative models
- *Symmetric in inputs and outputs?*: We can run a generative model "backward" and infer probable inputs given the output by computing $\mathbb{P}(\boldsymbol{x}|y)$
- *Can handle feature preprocessing?*: discriminative methods allow to preprocess the input in arbitrary ways, by replacing $\boldsymbol{x}$ with $\phi(\boldsymbol{x})$
- *Well-calibrated probabilities?*: discriminative models are usually better calibrated in terms of their probability estimates.

9.1 Classification

9.1.1 Naive Bayes classifiers

Purpose Classifying vectors of discrete-valuated features $x \in [\![1, K]\!]^D$, where K is the number of values that each feature is able to take, and D the number of features.

Assumptions

- Features are conditionally independent given the class label: $\forall (j, j') \in [\![1, D]\!]^2, j \neq j' \Rightarrow \boldsymbol{x}_j \perp \boldsymbol{x}_{j'}|y = c$
- Independence of the observations.

Theory As a *generative* model, meaning of the form: $\mathbb{P}(y = c|\boldsymbol{x}, \boldsymbol{\theta}) \propto \mathbb{P}(\boldsymbol{x}|y = c, \boldsymbol{\theta})\mathbb{P}(y = c|\boldsymbol{\theta})$. The key of such models is the possibility to specify a suitable form for the class-conditional density $\mathbb{P}(\boldsymbol{x}|y = c, \boldsymbol{\theta})$ which defines what kind of data we expect to see in each class. And with the independence assumption we have:

$$\mathbb{P}(\boldsymbol{x}|y = c, \boldsymbol{\theta}) = \prod_{j=1}^{D} \mathbb{P}(x_j|y = c, \boldsymbol{\theta}_{jc})$$

with all $\mathbb{P}(x_j|y=c,\boldsymbol{\theta}_{jc})$ being able to follow a *normal*, *bernoulli* or *multinoulli* distribution.
Training a NBC consists in computing the MLE or the MAP estimate for the parameters.
For a single observation $\mathbb{P}(\boldsymbol{x}_i,y_i|\boldsymbol{\theta})=\mathbb{P}(y_i|\boldsymbol{\pi})\prod_j\mathbb{P}(x_{ij}|\boldsymbol{\theta}_j)=\prod_c\pi_c^{\mathbb{1}(y_i=c)}\prod_j\prod_c\mathbb{P}(x_{ij}|\boldsymbol{\theta}_{jc})^{\mathbb{1}(y_i=c)}$

Hence the *log-likelihooh*: $\log(\mathcal{D}|\boldsymbol{\theta})=\sum_{c=1}^{C}N_c\log(\pi_c)+\sum_{j=1}^{D}\sum_{c=1}^{C}\sum_{i:y_i=c}\log(\mathbb{P}(x_{ij}|\boldsymbol{\theta}_{jc}))$

By optimizing the above equation we are able to find the $(\boldsymbol{\theta}_{jc})_{\substack{1\leq j\leq D\\1\leq c\leq C}}$ and we can then use them to predict the output of an observation $\boldsymbol{x}$ as: $\mathbb{P}(y=c|\boldsymbol{x},\mathcal{D})\propto\mathbb{P}(y=c|\mathcal{D})\prod_{j=1}^{D}\mathbb{P}(x_j|y=c,\mathcal{D})$

Strengths

- Simple model, for C classes and D features, and hence relatively immune to overfitting

Weaknesses

- Unaccuracy because of the strong independence assumption

Relationships with other methods Logistic Regression: for discrete inputs *Naive Bayesian Classifiers* form a generative-discriminant pair with *Multinomial Logistic Regression*: each NBC can be considered a way of fitting a probability model that optimizes the joint likelihood $\mathbb{P}(C,\boldsymbol{x})$, while Multinomial Logistic Regression fits the same probability to optimize the conditional $\mathbb{P}(C|\boldsymbol{x})$

Examples of application

- Classifying documents using bag of words
- Determining the gender of a person, based on measured features

9.1.2 Linear/Quadratic Discriminant Analysis

Purpose It consists in defining the class conditional densities in a generative classifier: $\mathbb{P}(\boldsymbol{x}|y=c,\boldsymbol{\theta})=\mathcal{N}(\boldsymbol{x},\boldsymbol{\mu}_c,\boldsymbol{\Sigma}_c)$
As for a generative classifier we have the following equation:

$$\mathbb{P}(y=c|\boldsymbol{x},\boldsymbol{\theta})=\frac{\overbrace{\mathbb{P}(\boldsymbol{x}|y=c,\boldsymbol{\theta})}^{\textit{class-conditional density}}\overbrace{\mathbb{P}(y=c|\boldsymbol{\theta})}^{\textit{class prior}}}{\sum_{c'}\mathbb{P}(y=c'|\boldsymbol{\theta})\,\mathbb{P}(\boldsymbol{x}|y=c',\boldsymbol{\theta})}$$

Assumptions

- Features are normally distributed in each class grouping variable.
- Homoscedasticity for LDA: variances among group variables are the same across levels of predictors.
- Independence of the observations.

Theory of Quadratic Discriminant Analysis

$$\mathbb{P}(y=c|\boldsymbol{x},\boldsymbol{\theta})=\frac{|2\pi\det(\Sigma_c)|^{-\frac{1}{2}}\exp\left(-\frac{1}{2}[\boldsymbol{x}-\boldsymbol{\mu}_c]^T\Sigma_c^{-1}[\boldsymbol{x}-\boldsymbol{\mu}_c]\right)\pi_c}{\sum_{c'}|2\pi\det(\Sigma_{c'})|^{-\frac{1}{2}}\exp\left(-\frac{1}{2}[\boldsymbol{x}-\boldsymbol{\mu}_{c'}]^T\Sigma_{c'}^{-1}[\boldsymbol{x}-\boldsymbol{\mu}_{c'}]\right)\pi_{c'}}$$

The threshold of this results will be a quadratic function of $\boldsymbol{x}$.

Theory of Linear Discriminant Analysis

LDA Same equation than above but this time, $\forall c \in [\![1, C]\!]\, \Sigma_c = \Sigma$, then quadratic term $\boldsymbol{x}^T \Sigma^{-1} \boldsymbol{x}$ will cancel out from numerator and denominator. Also Then by considering the above cancellation and the fact that Σ is diagonal as well as the evidence is considered as a constant, we have:

$$\begin{aligned} \mathbb{P}(y = c|\boldsymbol{x}, \boldsymbol{\theta}) &\propto \exp\left(\log(\pi_c) + \boldsymbol{\mu}_c^T \Sigma^{-1} \boldsymbol{x}\right) \\ &= \exp\left(\boldsymbol{\beta}_c^T \boldsymbol{x} + \gamma_c\right) \end{aligned}$$

Note also that we have exactly: $\mathbb{P}(y = c|\boldsymbol{x}, \boldsymbol{\theta}) = \dfrac{e^{\boldsymbol{\beta}_c^T \boldsymbol{x} + \gamma_c}}{\sum_{c'} e^{\boldsymbol{\beta}_{c'}^T \boldsymbol{x} + \gamma_{c'}}} = S(\boldsymbol{\eta})_c$. With $\eta = \left(\boldsymbol{\beta}_c^T \boldsymbol{x} + \gamma_c\right)_{1 \leq c \leq C}$

We recognize the *softmax* function.

RDA Regularized Discriminant Analysisa, assuming the covariance matrix is shared across the classes as in LDA, we perform MAP estimation of $\boldsymbol{\Sigma} \hookrightarrow \mathit{InverseWishart}\left(diag\left(\hat{\boldsymbol{\Sigma}}_{mle}\right), \nu_0\right)$ then:

$$\boxed{\hat{\boldsymbol{\Sigma}} = \lambda diag\left(\hat{\boldsymbol{\Sigma}}_{mle}\right) + (1 - \lambda)\hat{\boldsymbol{\Sigma}}_{mle}}$$

where λ controls the amount of regularization which is related to the strength of the prior ν_0

MLE for discriminant analysis

$$\log\left(\mathbb{P}(\mathcal{D}|\boldsymbol{\theta})\right) = \sum_{i=1}^{n}\sum_{c=1}^{C} \mathbb{1}_{\{y_i = c\}} \log(\pi_c) + \sum_{c=1}^{C}\sum_{i:y_i=c} \log\left(\mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c)\right)$$

And we find $\begin{cases} \hat{\boldsymbol{\mu}}_c &= \dfrac{1}{N_c} \displaystyle\sum_{i:y_i=c} \boldsymbol{x}_i \\ \hat{\boldsymbol{\Sigma}}_c &= \dfrac{1}{N_c}(\boldsymbol{x}_i - \hat{\boldsymbol{\mu}}_c)(\boldsymbol{x}_i - \hat{\boldsymbol{\mu}}_c)^T \end{cases}$

Strategies for preventing overfitting

- use a diagonal covariance matrix for each class
- use a full/diagonal covariance matrix shared across the classes
- use a full covariance matrix but impose a prior and then integrate it out
- project the data into a low dimensional subspace and fit the Gaussian there

Strengths

- more common MVN(Multivariate Normal) model which is the most widely used joint probability density function for continuous variables.
- good simplicity-efficiently trade-off

Weaknesses

- Multicollinearity: predictive power can decrease with an increased correlation between predictor variables.

Relationships with other methods

- ANOVA
- Logistic Regression
- PCA: look for linear combination of variables which best explain the model
- Nearest Shrunken Centroids

Examples of application

- bankruptcy prediction
- dimension reduction for face recognition

9.1.3 Nearest Shrunken Centroids Classifier

Purpose The basic idea is to perform MAP estimation for diagonal *LDA* with a sparsity-promoting (Laplace) prior

Assumptions

Theory Define the class-specific feature mean μ_{cj} in terms of the class-independent feature mean m_j and a class-specific offset Δ_{cj} Thus:

$$\boxed{\mu_{jc} = m_j + \Delta_{cj}}$$

We will then put a prior on the Δ_{cj} terms to encourage them to be strictly zero and compute MAP estimate.
If for feature j $\forall c \in [\![1, C]\!]$ $\Delta_{cj} = 0$ then feature j will no play role in the classification decision, since μ_{cj} will be independent of c.
Thus feature that are not discriminative are automatically ignored.

Strengths

- Sparsity $\rightarrow$ more interpretable model

Weaknesses

Relationships with other methods

Examples of application

- Gene expression classification

9.1.4 Fisher's Linear Discriminant Analysis (FLDA)

Purpose An alternative way to the above discriminant analysis is to reduce the dimensionality of the features $\boldsymbol{x} \in \mathbb{R}^D$ and then fit a Multivariate Normal to the resulting low-dimensional features $\boldsymbol{z} \in \mathbb{R}^L$.
PCA would not be a good idea because it is unsupervised and the low-dimensional resulting features would not be optimal for the classification problem.
A better option would be to find a matrix $\boldsymbol{W}$ such that the low-dimensional data can be classified as well as possible using a Gaussian class-conditional density model, this is the *FLDA*.

Assumptions

- Gaussian class-conditional: reasonable since we are computing linear combination of potentially non-Gaussian features.

Theory For 2 classes Let us define $\begin{cases} \boldsymbol{\mu}_1 = \frac{1}{N_1} \sum_{i:y_i=1} \boldsymbol{x}_i \\ \boldsymbol{\mu}_2 = \frac{1}{N_2} \sum_{i:y_i=2} \boldsymbol{x}_i \end{cases}$, and $m_k = \boldsymbol{w}^T \boldsymbol{\mu}_k$ being the projection of each mean onto the line $\boldsymbol{w}$. Also let $z_i = \boldsymbol{w}^T \boldsymbol{x}_i$ be the projection of the data onto the line $\boldsymbol{w}$ and finally $s_k^2 = \sum_{i:y_i=k} (z_i - m_k)^2$.
The goal is to find $\boldsymbol{w}$ such that we maximize the distance between the means, $m_1 - m_2$, while also ensuring the projected clusters are "tight":

$$\boxed{\boldsymbol{J}(\boldsymbol{w}) = \frac{(m_2 - m_1)^2}{s_1^2 + s_2^2} = \frac{\boldsymbol{w}^T \boldsymbol{S}_B \boldsymbol{w}}{\boldsymbol{w}^T \boldsymbol{S}_W \boldsymbol{w}}}$$

with the *between-class scatter matrix*: $\boldsymbol{S}_B = (\mu_2 - \mu_1)(\mu_2 - \mu_1)^T$ and *within-class scatter matrix*: $\boldsymbol{S}_W = \sum_{i:y_i=1} (\boldsymbol{x}_i - \boldsymbol{\mu}_1)(\boldsymbol{x}_i - \boldsymbol{\mu}_1)^T + \sum_{i:y_i=2} (\boldsymbol{x}_i - \boldsymbol{\mu}_2)(\boldsymbol{x}_i - \boldsymbol{\mu}_2)^T$ One can show that $J(\boldsymbol{w})$ is maximized when $\boldsymbol{S}_B \boldsymbol{w} = \lambda \boldsymbol{S}_W \boldsymbol{w} \boldsymbol{w}$ where $\lambda = \frac{\boldsymbol{w}^T \boldsymbol{S}_B \boldsymbol{w}}{\boldsymbol{w}^T \boldsymbol{S}_W \boldsymbol{w}}$. If $\boldsymbol{S}_W$ is invertible we can convert it to a regular eigenvalue problem: $\boldsymbol{S}_W^{-1} \boldsymbol{S}_B \boldsymbol{w} = \lambda \boldsymbol{w}$

Strengths

- Classification in taking in account the response label.

Weaknesses

- FLDA is restricted to using $L \leq C - 1$ dimensions, regardless of D.

Relationships with other methods

- Linear Discriminant Analysis
- PCA
- ANOVA

Examples of application

- Speech recognition

9.1.5 Logistic Regression

Purpose For binary classification.

Assumptions

- Independence

Theory The data distribution is modelled by : $\boxed{\mathbb{P}(y|\boldsymbol{x}) = Bernoulli\left(y|\sigma\left(\boldsymbol{w}^T \boldsymbol{x}\right)\right)}$

With σ being the *sigmoid* function, such that $\sigma = \begin{cases} \mathbb{R} \longrightarrow [0,1] \\ x \mapsto \dfrac{e^x}{1+e^x} \end{cases}$

Maximum Likelihood Estimator

$$\begin{aligned} NLL(\boldsymbol{w}) &= -\sum_{i=1}^{N} \log\left(\hat{y}_i^{\mathbb{1}_{\{y_i=1\}}} \left[1 - \hat{y}_i\right]^{\mathbb{1}_{\{y_i=0\}}}\right) \\ &= -\sum_{i=1}^{N} y_i \log(\hat{y}_i) + [1 - y_i] \log(1 - \hat{y}_i) \end{aligned}$$

This called *cross-entropy*

Strengths

Weaknesses

Relationships with other methods

Examples of application

9.2 Regression

9.2.1 Linear Regression

Purpose

Assumptions

Theory

General It is a model for which the data distribution (likelihood) is described by:

$$\mathbb{P}\left(y|\boldsymbol{x},\boldsymbol{\theta}\right)=\mathcal{N}\left(y|\boldsymbol{w}^T\phi(\boldsymbol{x}),\sigma^2\right)$$

with ϕ that can be a non-linear function, in this case we talk about *basis function expansion*.
To estimate the parameters we can use the *maximum likelihood estimation*: $\hat{\theta}\triangleq\arg\max_\theta\log\left(\mathbb{P}\left(\mathcal{D}|\boldsymbol{\theta}\right)\right)$.
For computational purpose it is better to consider the minimization of the *Negative Log Likelihood* (NLL):

$$\begin{aligned}
NLL(\theta)&\triangleq-\log\left(p\left(\mathcal{D}|\theta\right)\right)\\
&=-\sum_{i=1}^{n}\log\left(\mathbb{P}\left(y_i|\boldsymbol{x}_i,\boldsymbol{\theta}\right)\right)\\
&=-\sum_{i=1}^{n}\log\left(\left[\frac{1}{2\pi\sigma^2}\right]^{\frac{1}{2}}\exp\left(-\frac{1}{2\sigma^2}\left[y_i-\boldsymbol{w}^T\boldsymbol{x}_i\right]^2\right)\right)\\
&=\frac{n}{2}\log\left(2\pi\sigma^2\right)+\frac{1}{2\sigma^2}\sum_{i=1}^{n}\left(y_i-\boldsymbol{w}^T\boldsymbol{x}_i\right)^2\\
&=\frac{n}{2}\log\left(2\pi\sigma^2\right)+\frac{1}{2\sigma^2}\;RSS(\boldsymbol{w})\\
&=\frac{n}{2}\log\left(2\pi\sigma^2\right)+\frac{1}{2\sigma^2}\left\|\epsilon\right\|_2^2
\end{aligned}$$

As the *MLE* for $\boldsymbol{w}$ is the one minimizing the *RSS* then this method is known as *least square*.

Derivation of the MLE it is better to use a matrix-vector representation.
$NLL(\boldsymbol{w})=\frac{1}{2}\left(y-\boldsymbol{X}\boldsymbol{w}\right)^T\left(y-\boldsymbol{X}\boldsymbol{w}\right)=\frac{1}{2}\boldsymbol{w}^T\left(\boldsymbol{X}^T\boldsymbol{X}\right)\boldsymbol{w}-\boldsymbol{w}^T\left(\boldsymbol{X}^Ty\right)$ Note that $\boldsymbol{X}^T\boldsymbol{X}$ is the *sum of squares matrix*. Then **gradient**, $g(\boldsymbol{w})=\boldsymbol{X}^T\boldsymbol{X}\boldsymbol{w}-\boldsymbol{X}^T\boldsymbol{y}$ that we have to equate to zero to get $\boldsymbol{X}^T\boldsymbol{X}\boldsymbol{w}=\boldsymbol{X}^T\boldsymbol{y}$ to conclude that:

$$\boxed{\hat{\boldsymbol{w}}_{OLS}=\left(\boldsymbol{X}^T\boldsymbol{X}\right)^{-1}\boldsymbol{X}^T\boldsymbol{y}}$$

Robust Linear Regression It is very common to model the noise in regression models using a *Gaussian distribution*, meaning $\epsilon_i=y_i-\boldsymbol{w}^T\boldsymbol{x}_i\hookrightarrow\prime,\sigma^{\in}$. One way to achieve *robustness* against *outliers* is to replace the Gaussian distribution for the response variable with a distribution having **heavy tails**.

Likelihood	Prior	Name
Gaussian	Uniform	*Least Squares*
Gaussian	Gaussian	*Ridge*
Gaussian	Laplace	*Lasso*

Ridge encourages parameters to be small by using a zero-mean Gaussian prior: $\mathbb{P}(\boldsymbol{w}) = \prod_{j=1}^{D} \mathcal{N}(\omega_j|0,\tau^2)$, where $\frac{1}{\tau^2}$ controls the strength of the prior.

The corresponding *MAP* estimation problem becomes: $\arg\max_{\boldsymbol{w}} \sum_{i=1}^{n} \log\left(\mathcal{N}\left(y_i|\omega_0 + \boldsymbol{w}^T\boldsymbol{x}_i, \sigma^2\right)\right) + \sum_{j=1}^{D} \log\left(\mathcal{N}\left(\omega_j|0,\tau^2\right)\right)$.

After some calculus and with where $\lambda \triangleq \frac{\sigma^2}{\tau^2}$ we deduce that:

$$\boxed{\hat{\boldsymbol{w}}_{Ridge} = \left(\lambda \boldsymbol{I}_D + \boldsymbol{X}^T\boldsymbol{X}\right)^{-1}\boldsymbol{X}\boldsymbol{y}}$$

Advantages of Ridge regression on OLS regression:

- $\left(\lambda \boldsymbol{I}_D + \boldsymbol{X}^T\boldsymbol{X}\right)$ is much better conditioned, and hence more likely to be invertible, than $\boldsymbol{X}^T\boldsymbol{X}$ at least for suitable large λ
- if we follow a *Singular Value Decomposition* $\boldsymbol{X} = \boldsymbol{U}\boldsymbol{S}\boldsymbol{V}^T$ we find that $\hat{\boldsymbol{y}} = \boldsymbol{X}\hat{\boldsymbol{w}}_{Ridge} = \sum_{j=1}^{D} \boldsymbol{u}_j \frac{\sigma_j^2}{\sigma_j^2+\lambda} \boldsymbol{u}_j^T\boldsymbol{y}$ with $(\sigma_j)_{1\leq j\leq D}$ the singular value of $\boldsymbol{X}$ whereas for OLS we have $\hat{\boldsymbol{y}} = \boldsymbol{X}\hat{\boldsymbol{w}}_{OLS} = \sum_{j=1}^{D} \boldsymbol{u}_j\boldsymbol{u}_j^T\boldsymbol{y}$. Meaning that with Ridge if σ_j^2 is small compared to λ then direction $\boldsymbol{u}_j$ will not have much effect on the prediction. In term of predictive accuracy *Ridge* regression is more interesting than *PCA* regression.

Lasso (Least Absolute Shrinkage and Absolute Selection Operator) $\hat{\boldsymbol{w}}_{Lasso} = sign(\hat{\boldsymbol{w}}_{OLS})\left[|\hat{\boldsymbol{w}}_{OLS}| - \frac{\lambda}{2}\right]$

Elastic-Net In practice Elastic-Net often performs best, since it provides a good combination of sparsity and regularization.

Strengths

- Simple
- Customizable to achieve robustness

Weaknesses

- Not very powerful for non-linear data

Relationships with other methods

- Ridge Regression has similitude with PCA

Examples of application

9.2.2 Generalized Linear Models (GLMs)

Models in which the output density is in the exponential family and in which the mean parameters are a linear combination of the inputs, passed through a possibly nonlinear function such as the logistic function.

Exponential family: $\mathbb{P}(\boldsymbol{x}|\boldsymbol{\theta}) = \frac{1}{Z(\boldsymbol{\theta})} h(x) e^{\boldsymbol{\theta}^T\phi(\boldsymbol{x})} = h(\boldsymbol{x}) e^{\boldsymbol{\theta}^T\phi(\boldsymbol{x}) - A(\boldsymbol{\theta})}$ where $\begin{cases} Z(\boldsymbol{\theta}) = \sum_{\mathcal{X}^m} h(\boldsymbol{x}) e^{\boldsymbol{\theta}^T\phi(\boldsymbol{x})} d\boldsymbol{x} \\ A(\boldsymbol{\theta}) = \log\left(Z(\boldsymbol{\theta})\right) \end{cases}$

with the *natural parameters* $\boldsymbol{\theta}$, the vector of *sufficient statistics* $\phi(\boldsymbol{x})$, the *partition function* $Z(\boldsymbol{\theta})$, the *log partition function* or *cumulant function* $A(\boldsymbol{\theta})$and the *scaling constant* $h(\boldsymbol{x})$.
The name *cumulant function* comes from the property of the exponential family being that derivatives of the log partition function can be used to generate cumulant of the sufficient statistics (the first and second cumulants of a distribution being its mean and variance).

Purpose We have the following data distribution: $\mathbb{P}\left(y_i|\boldsymbol{\theta}, \sigma^2\right) = \exp\left(\frac{y_i\boldsymbol{\theta} - A\left(\boldsymbol{\theta}\right)}{\sigma^2} + c(y_i, \sigma^2)\right)$ with c a normalization constant.
Let's consider an invertible mapping Φ such that $\boldsymbol{\theta} = \Phi(\mu)$ with μ being the mean parameter and $\boldsymbol{\theta}$ the natural parameter.
We have as well $\mu = \Phi^{-1}(\boldsymbol{\theta}) = A'(\boldsymbol{\theta})$.
We are free to chose any link function as long as the inverse have an appropriate range.

Distribution	**Link function**	**Natural parameter** $\theta = \Phi(\mu)$	**Mean parameter** $\mu = \Phi^{-1}(\theta) = \mathbb{E}(y)$
$\mathcal{N}(\mu, \sigma^2)$	*identity*	$\boldsymbol{\theta} = \boldsymbol{\mu}$	$\boldsymbol{\mu} = \boldsymbol{\theta}$
$\mathcal{B}(n, \mu)$	*logit*	$\boldsymbol{\theta} = \log\left(\frac{\mu}{1-\mu}\right)$	$\mu = sigm(\boldsymbol{\theta})$
$Poisson(\mu)$	*log*	$\boldsymbol{\theta} = \log(\mu)$	$\mu = e^{\theta}$

Assumptions

Theory

Strengths

- GLMs can be fit using methods like gradient descent.

Weaknesses

Relationships with other methods

Examples of application

9.2.3 Learning to rank

Purpose Modeling a function being able to rank a set of items. Suppose we have a query q and a set of documents $(d_i)_{1 \leq i \leq n}$ and we would like to sort these documents in decreasing order of relevance.

Assumptions

Theory Let us consider a document d and a query q, a standard way to measure the relevance between the both is to use $similitude(q, d) \triangleq \mathbb{P}(q|d) = \prod_{i=1}^{n} \mathbb{P}(q_i|d)$ with q_i being the ith word or term of q.

Pointwise approach for binary relevance labels, we can follow a standard binary classification scheme to estimate $\mathbb{P}(y = 1|\boldsymbol{x}(q, d))$, in the case of ordered relevancy labels we can use an *ordinal regression* to predict the rating $\mathbb{P}(y = r|\boldsymbol{x}(q, d))$.
However this method does not take into account the location of each document in the list.

Pairwise approach to check the relative relevance of two items rather than absolute relevance. We can model this kind of data using a binary classifier of the form $\mathbb{P}(y_{jk}|\boldsymbol{x}_j, \boldsymbol{x}_k) = \sigma\left(f(\boldsymbol{x}_j) - f(\boldsymbol{x}_k)\right)$ where f is a scoring function, often taken to be linear: $f(\boldsymbol{x}) = \boldsymbol{w}^T\boldsymbol{x}$.

Listwise approach we now consider methods looking at the entire list of items at the same time. We can define a total order on a list by specifying a permutation of its indices: $\boldsymbol{\pi}$. To model the uncertainty about $\boldsymbol{\pi}$ we can use the *Plackett-Luce* distribution. $\mathbb{P}(\boldsymbol{\pi}|\boldsymbol{s}) = \prod_{j=1}^{m} \frac{s_j}{\sum_{u=j}^{m} s_u}$ where $s_j = s\left(\pi^{-1}(j)\right)$, the score of the document ranked at the jth position.

Strengths

Weaknesses

Relationships with other methods

Examples of application

- *information retrieval* return a list of the top k more relevant documents, depending on a given query

9.2.4 Supervised PCA

Purpose Also called *Bayesian factor regression* this model take in account y_i when learning the low dimension embedding.

Assumptions

Theory $\begin{cases} \mathbb{P}(\boldsymbol{z}_i) = \mathcal{N}(\mathbf{0}, \boldsymbol{I}_L) \\ \mathbb{P}(y_i|\boldsymbol{z}_i) = \mathcal{N}\left(\boldsymbol{w}_y^T \boldsymbol{z}_i + \mu_y, \sigma_y^2\right) \\ \mathbb{P}(\boldsymbol{x}_i|\boldsymbol{z}_i) = \mathcal{N}\left(\boldsymbol{W}_x^T \boldsymbol{z}_i + \boldsymbol{\mu}_x, \sigma_x^2 \boldsymbol{I}_D\right) \end{cases}$
The basic idea compressing $\boldsymbol{x}_i$ to predict y_i can be formulated using information theory, in particular we might want to find an encoding distribution $\mathbb{P}(\boldsymbol{z}|\boldsymbol{x})$ such that we minimize $\mathbb{I}(X;Z) - \beta\mathbb{I}(X;Y)$. Where $\beta \geq 0$, the *information bottleneck* is some parameter controlling the trade-off between compression and predictive accuracy.

Strengths

Weaknesses

Relationships with other methods

- PCA

Examples of application

- predict the movies that you would like knowing who your friends are as well as the rating from other users

9.2.5 Partial Least Squares

Purpose The key idea is to allow some of the (co)variance in the input features to be explained by its own subspace $\boldsymbol{z}_i^x$ and to let the remaining of subspace $\boldsymbol{z}_i^s$ be shared between input and output.

Assumptions

Theory $\begin{cases} \mathbb{P}(\boldsymbol{z}_i) = \mathcal{N}(\boldsymbol{z}_i^s|\mathbf{0}, \boldsymbol{I}_{L_s})\mathcal{N}(\boldsymbol{z}_i^x|\mathbf{0}, \boldsymbol{I}_{L_x}) \\ \mathbb{P}(\boldsymbol{y}_i|\boldsymbol{z}_i) = \mathcal{N}\left(\boldsymbol{W}_y \boldsymbol{z}_i^s + \boldsymbol{\mu}_y, \sigma^2 \boldsymbol{I}_{D_y}\right) \\ \mathbb{P}(\boldsymbol{x}_i|\boldsymbol{z}_i) = \mathcal{N}\left(\boldsymbol{W}_x \boldsymbol{z}_i^s + \boldsymbol{B}_x \boldsymbol{z}_i^x + \boldsymbol{\mu}_x, \sigma^2 \boldsymbol{I}_{D_x}\right) \end{cases}$
We should choose L large enough so that the shared subspace does not capture covariate specific variations.

Strengths

Weaknesses

Relationships with other methods

Examples of application

9.3 Classification and Regression

9.3.1 Multi-task

Purpose To fit many related classification or regression models at the same time, as it is often reasonable to assume the input-output mappings is similar across these different models.

Assumptions

- Input/Output mapping

Theory

Hierarchical Bayes for multi-task learning Let consider y_{ij} the response of the ith item in group j with $(i,j) \in [\![1, N_j]\!] \times [\![1, J]\!]$.
Although some groups may have lot of data, there is often a long tail where the majority of groups have little data. Thus we can't reliably fit each model separately but we don't want to use the same model for all the groups. As a compromise, we can fit a separate model for each group but encourage the model parameters to be similar across the groups. More precisely:

$$\begin{cases} \mathbb{E}\left(y_{ij}|\boldsymbol{x}_{ij}\right) = g(\boldsymbol{x}_{ij}^T\boldsymbol{\beta}_j) \\ \boldsymbol{\beta}_j \hookrightarrow \mathcal{N}\left(\boldsymbol{\beta}_*, \sigma_j^2\boldsymbol{I}\right) \\ \boldsymbol{\beta}_* \hookrightarrow \mathcal{N}\left(\boldsymbol{\mu}, \sigma_*^2\boldsymbol{I}\right) \end{cases}$$

$\boldsymbol{\beta}_j$ is a coefficient vector associated with the group j.
Also groups with small sample size borrow statistical strength from the groups with larger sample size because the $\boldsymbol{\beta}_j$'s are correlated via the latent common parents $\boldsymbol{\beta}_*$.
The term σ_j^2 controls how much group j depends on the common parents and the σ_*^2 term controls the strength of the overall prior.

Strengths

- Performance improvement

Weaknesses

Relationships with other methods

Examples of application

- Determine the test scores for a given student in a given school without using as many models as school count
- Personal spam filtering:

9.3.2 Mixture models

Purpose It is the simplest form of *Latent Variable Models (LVMs)* is when $z_i \in [\![1, K]\!]$.
Two mains application of mixture models:

- use them as *black-box* density model, $p(\boldsymbol{x}_i)$, useful for data compression, outlier detection and creating generative classifiers
- use for clustering

Assumptions

Theory We use a discrete prior $p(z_i) = Cat(\boldsymbol{\pi})$ and the likelihood $p(\boldsymbol{x}_i|z_i = k)$, finally the **mixture model** is :

$$\mathbb{P}\left(\boldsymbol{x}_i|\boldsymbol{\theta}\right) = \sum_{k=1}^{K} \pi_k \mathbb{P}\left(\boldsymbol{x}_i|z_i = k, \boldsymbol{\theta}\right) = \sum_{k=1}^{K} \pi_k \mathbb{P}_k\left(\boldsymbol{x}_i|\boldsymbol{\theta}\right)$$

where $\mathbb{P}_k$ is the k'th *base distribution*.

Mixtures of Gaussian Each base distribution in the mixture is a multivariate Gaussian with mean μ_k and covariance matrix $\boldsymbol{\Sigma}_k$: $\mathbb{P}\left(\boldsymbol{x}_i|\boldsymbol{\theta}\right) = \sum_{k=1}^{K} \pi_k \mathcal{N}\left(\boldsymbol{x}_i|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k\right)$
Given a sufficiently large number of mixture components a Gaussian Mixture Models (GMMs) can be used to approximate any density defined on $\mathbb{R}^D$.

Mixtures of multinoullis can be used to define density models on data consisting of a D-dimensional bit vectors: $\mathbb{P}\left(\boldsymbol{x}_i|z_i = k, \boldsymbol{\theta}\right) = \prod_{j=1}^{D} Ber(x_{ij}|\mu_{jk}) = \prod_{j=1}^{D} \mu_{jk}^{x_{ij}} \left(1 - \mu_{jk}\right)^{1-x_{ij}}$
where μ_{jk} is the probability that bit j turns on in cluster k.

Mixture models for clustering We first fit the mixture model an then compute $\mathbb{P}\left(z_i = k|\boldsymbol{x}_i, \boldsymbol{\theta}\right)$ representing the posterior probability that point i belongs to cluster k.
This known as the *responsibility* of cluster k for point i and can be computed using Bayes:

$$r_{ik} \triangleq \mathbb{P}\left(z_i = k|\boldsymbol{x}_i, \boldsymbol{\theta}\right) = \frac{\mathbb{P}\left(z_i = k|\boldsymbol{\theta}\right)\mathbb{P}\left(\boldsymbol{x}_i|z_i = k, \boldsymbol{\theta}\right)}{\sum_{k'=1}^{K} \mathbb{P}\left(z_i = k'|\boldsymbol{\theta}\right)\mathbb{P}\left(\boldsymbol{x}_i|z_i = k', \boldsymbol{\theta}\right)}$$

Mixture of experts are build from a discriminative perspective, they relies on the idea that a good model can be achieved in using multiple different linear method each applying to a different part of the input space.
We can model this by allowing the mixing weights and the mixture densities to be input-dependent:

$$\begin{cases} \mathbb{P}\left(y_i|\boldsymbol{x}_i, z_i = k, \boldsymbol{\theta}\right) = \mathcal{N}(y_i|\boldsymbol{w}_k^T \boldsymbol{x}_i, \sigma_k^2) \\ \mathbb{P}\left(z_i|\boldsymbol{x}_i, \boldsymbol{\theta}\right) = Cat(z_i|S(\boldsymbol{V}^T \boldsymbol{x}_i)) \end{cases}$$

Useful in solving inverse problems, the ones in which we have to invert a many-to-one mapping, for example in robotics where the location of the end effector (hand) $\boldsymbol{y}$ is uniquely determined by the joint angles of the motors, $\boldsymbol{x}$.

The overall posterior is then

$$\mathbb{P}\left(y_i|\boldsymbol{x}_i, \boldsymbol{\theta}\right) = \sum_{k} \mathbb{P}\left(z_i = k|\boldsymbol{x}_i, \boldsymbol{\theta}\right)\mathbb{P}\left(y_i|\boldsymbol{x}_i, z_i = k, \boldsymbol{\theta}\right)$$

This model can be easily generalized to discrete data using the exponential family. The model is usually fit with *Expectation-Maximization* technique.

Strengths

Weaknesses

- Use of a single latent variable to generate the observation

Relationships with other methods

Examples of application

9.3.3 ARD: Automatic Relevance Determination

Purpose To get greater sparsity

Assumptions

Theory Relying on empirical Bayes, whereby we integrate out $\boldsymbol{w}$ and maximize the marginal likelihood τ.

Linear Regression Let's consider weight precisions by $\alpha_j = \frac{1}{\tau_j^2}$ and the measurement precision by $\beta = \frac{1}{\sigma^2}$:

$$\begin{cases} \mathbb{P}(y|\boldsymbol{x}, \boldsymbol{w}, \beta) = \mathcal{N}\left(y|\boldsymbol{w}^T\boldsymbol{x}, \frac{1}{\beta}\right) \\ \mathbb{P}(\boldsymbol{w}) = \mathcal{N}(\boldsymbol{w}|0, \boldsymbol{A}^{-1}) \end{cases}$$

where $\boldsymbol{A} = diag(\boldsymbol{\alpha})$ To regularize the problem we may put a conjugate prior on each precision $\alpha_j \hookrightarrow Gamma(a, b)$ and $\beta \hookrightarrow Gamma(c, d)$ When performing EB point estimation will use the improper prior $a = b = c = d = 0$ which results in maximal sparsity.

Sparsity If $\hat{\alpha}_j \approx 0$ we find $\hat{w}_j \approx \hat{w}_j^{mle}$ since the Gaussian prior shrinking w_j towards 0 has zero precision.
However if we find that $\hat{\alpha}_j \to \infty$ then the prior is very confident that $w_j = 0$ and hence that feature j is "irrelevant".

Logistic Regression uses iteratively reweighted l_1 algorithm.

Strengths

- Resilient against uninformative features

Weaknesses

Relationships with other methods

Examples of application

9.3.4 Sparse coding

Purpose Sparse priors for unsupervised learning, here we relax the constraint that $\boldsymbol{W}$ is orthogonal.

Assumptions

Theory

Strengths

Weaknesses

Relationships with other methods

Examples of application

9.3.5 Support Vector Machines (SVMs)

Purpose The combination of the *kernel trick* plus a modified loss function allowing *sparsity*, meaning that the prediction will only depend on a subset of the training data called *support vectors*. The overall process is known as a *Support Vector Machine*.
Because of sparsity encoding in the loss function instead of in the prior, kernel encoding through a trick instead of being an explicit part of the model,SVMs do not provide probabilistic outputs.

SVMs for regression with *kernalized ridge regression* the solution $\boldsymbol{w}$ depends on all the training inputs. We should then use a variant of *Huber loss* function: the *epsilon insensitive loss function*:

$$L_\epsilon(y-\hat{y}) = \begin{cases} 0 & \Leftarrow |y-\hat{y}| < \epsilon \\ |y-\hat{y}| - \epsilon & \Leftarrow |y-\hat{y}| \geq \epsilon \end{cases}$$

meaning that any point lying inside an ϵ-**tube** a around the prediction is not penalized: $J = C\sum_{i=1}^{n} L_\epsilon(y_i - \hat{y}_i) + \frac{1}{2}\|\boldsymbol{w}\|^2$, with $C = \frac{1}{\lambda}$ is a *regularization constant*.
It can be shown that optimal solution has the form $\hat{\boldsymbol{w}} = \sum_i \alpha_i \boldsymbol{x}_i$, where $\alpha_i \geq 0$. It turns out that $\boldsymbol{\alpha}$ is sparse as we don't care about errors which are smaller than ϵ. The $\boldsymbol{x}_i$ for which $\alpha_i > 0$ are the **support vectors**.
Then we have $\hat{y}(\boldsymbol{x}) = \hat{w}_0 + \hat{\boldsymbol{w}}\boldsymbol{x} = \hat{w}_0 + \sum_i \alpha_i \boldsymbol{x}_i^T \boldsymbol{x} = \sum_i \alpha_i k(\boldsymbol{x}, \boldsymbol{x}_i)$

Classification we consider now the *hinge loss*:

$$L_{hinge}(y,\eta) = \max(0, 1-y\eta) = (1-y\eta)_+$$

where $\eta = f(\boldsymbol{x})$ is our 'confidence' in choosing label $y = 1$ however it does not need to have any probabilistic semantics.
The overall objective has the form $\min_{\boldsymbol{w}:w_0} \frac{1}{2}\|\boldsymbol{w}\|^2 + C\sum_{i=1}^{n}(1-y_i f(\boldsymbol{x}_i))_+$. Same principle as in regression, but this time $\hat{y}(\boldsymbol{x}) = sgn(f(\boldsymbol{x})) = sgn(\hat{w}_0 + \hat{\boldsymbol{w}}^T\boldsymbol{x}) = sgn\left(\hat{w}_0 + \sum_{i=1}^{n}\alpha_i k(\boldsymbol{x}_i, \boldsymbol{x})\right)$

The large margin principle our goal is to derive a *discriminative function* $f(x)$ which will be linear in the feature space implied by the choice of kernel. Hence: $\boldsymbol{x} = \boldsymbol{x}_\perp + r\frac{\boldsymbol{w}}{\|\boldsymbol{w}\|}$ where r is the distance of $\boldsymbol{x}$ from the decision boundary whose normal vector is $\boldsymbol{w}$, and $\boldsymbol{x}_\perp$ is the orthogonal projection of $\boldsymbol{x}$ onto this boundary. Hence $f(\boldsymbol{x}) = \boldsymbol{w}^T\boldsymbol{x} + w_0 = (\boldsymbol{w}^T\boldsymbol{x}_\perp + w_0) + r\frac{\boldsymbol{w}^T\boldsymbol{w}}{\|\boldsymbol{w}\|}$. As $f(\boldsymbol{x}_\perp) = 0$, $\boldsymbol{w}^T\boldsymbol{x}_\perp + w_0 = 0$
Hence $f(\boldsymbol{x}) = r\frac{\boldsymbol{w}^T\boldsymbol{w}}{\|\boldsymbol{w}\|}$. Finally $r = \frac{f(\boldsymbol{x})}{\|\boldsymbol{w}\|}$, the distance that we would like to make as large as possible, in order to clearly separate the input.

Regularization parameter C it controls the number of errors we are willing to tolerate on the training set, it is chosen by cross-validation, an efficient way to chose C is to develop a path following algorithm in the spirit of *ARS*.

Assumptions

Theory

Strengths

- computational advantages over probabilistic model
- *kernel trick* → prevent underfitting: ensuring that the feature vector is sufficiently rich that a linear classifier can separate

- *sparsity & large margin principles* → prevent overfitting: ensure that we do not use all the basis functions

Weaknesses

- issues for multi-class classification due to the non-probabilistic aspect of the model: output scores are not on a calibrate scale

Relationships with other methods

Examples of application

9.3.6 MODEL COMPARISON

Comparison of discriminative kernel methods

- *L1VM*: l_1-regularized vector machine
- *L2VM*: l_2-regularized vector machine
- *SVM*: Support Vector Machine
- *RVM*: Relevance Vector Machine
- *GP*: Gaussian Process
- *speed* → use RVM
- *well-calibrated probabilistic outputs* → GP

the only circumstances under which using an SVM seems sensible is the structured output case, where likelihood-based methods can be slow.

9.3.7 Gaussian Processes

Purpose It defines a prior over functions, which can be converted into a posterior over functions once we have seen some data.
They can be thought of as a Bayesian alternative to the kernel methods. In Python we can use the library *GPflow*

Assumptions

Theory

Regression Let the prior on the regression function be GP denoted by:

$$\begin{cases} f(\boldsymbol{x}) & \hookrightarrow GP(m(\boldsymbol{x}), k(\boldsymbol{x}, \boldsymbol{x}')) \\ m(\boldsymbol{x}) & = \mathbb{E}\left(f(\boldsymbol{x})\right) \ \textit{the mean function} \\ k(\boldsymbol{x}, \boldsymbol{x}') & = \mathbb{E}\left(\left[f(\boldsymbol{x}) - m(\boldsymbol{x})\right]^T \left[f(\boldsymbol{x}') - m(\boldsymbol{x}')\right]\right) \ \textit{kernel or covariance function} \end{cases}$$

For any finite set of points, this process defines a joint Gaussian: $\mathbb{P}\left(\boldsymbol{f}|\boldsymbol{X}\right) = \mathcal{N}\left(\boldsymbol{f}[\boldsymbol{\mu}, \boldsymbol{K}\right)$.
Where $\boldsymbol{K}$ is defined by $k_{ij} = k(\boldsymbol{x}_i, \boldsymbol{x}_j)$ and $\boldsymbol{\mu} = (m(\boldsymbol{x}_i))_{1 \leq i \leq n}$
Let's consider the case where we predict using noise-free observation. Suppose we observe a training observation set $\mathcal{D} = \{(\boldsymbol{x}_i, f_i)\}_{1 \leq i \leq n}$, where $f_i = f(\boldsymbol{x}_i)$ is the noise-free observation of the function evaluated at $\boldsymbol{x}_i$. Given a test set $\boldsymbol{X}_*$ of size $n_* \times D$ we want to predict the function output $\boldsymbol{f}_*$. The Gaussian Process will act as an *interpolator* of the training data.

$$\begin{bmatrix} \boldsymbol{f} \\ \boldsymbol{f}_* \end{bmatrix} \hookrightarrow \mathcal{N}\left(\begin{bmatrix} \boldsymbol{\mu} \\ \boldsymbol{\mu}_* \end{bmatrix}, \begin{bmatrix} \boldsymbol{K} & \boldsymbol{K}_* \\ \boldsymbol{K}_*^T & \boldsymbol{K}_{**} \end{bmatrix}\right)$$

where $\boldsymbol{K} = k(\boldsymbol{X}, \boldsymbol{X})$, $\boldsymbol{K}_* = k(\boldsymbol{X}, \boldsymbol{X}_*)$ and $\boldsymbol{K}_{**} = k(\boldsymbol{X}_*, \boldsymbol{X}_*)$. By teh standard rules for conditioning Gaussians, the prosterior has the following form: $\begin{cases} \mathbb{P}(\boldsymbol{f}_*|\boldsymbol{X}_*, \boldsymbol{X}, \boldsymbol{f}) & = \mathcal{N}(\boldsymbol{f}_*|\boldsymbol{\mu}_*, \boldsymbol{\Sigma}_*) \\ \boldsymbol{\mu}_* & = \boldsymbol{\mu}(\boldsymbol{X}_*) + \boldsymbol{K}_*^T \boldsymbol{K}^{-1}(\boldsymbol{f} - \boldsymbol{\mu}(\boldsymbol{X})) \\ \boldsymbol{\Sigma}_* & = \boldsymbol{K}_{**} - \boldsymbol{K}_*^T \boldsymbol{K}^{-1} \boldsymbol{K}_* \end{cases}$

Strengths

Weaknesses

Relationships with other methods

Examples of application

- noise-free GP regression is a computationally cheap proxy for the behavior of a complex simulator such as a weather forecasting program.

9.3.8 Classification And Regression Trees (CART)

Purpose Learning useful features directly from the input data

Assumptions

Theory

$$f(\boldsymbol{x}) = \mathbb{E}(y|\boldsymbol{x}) = \sum_{m=1}^{M} w_m \mathbb{1}_{\{\boldsymbol{x} \in R_m\}} = \sum_{m=1}^{M} w_m \phi(\boldsymbol{x}; \boldsymbol{v}_m)$$

where R_m is the mth region w_m is the mean response in this region and $\boldsymbol{v}_m$ encodes the choice of variable to split on and the threshold value on the path from the root to the mth leaf.

Growing a tree

$$(j^*, t^*) = \underset{j \in [\![1, D]\!]}{\arg\min} \min_{t \in \mathcal{T}_j} cost(\{\boldsymbol{x}_i, y_i : x_{ij} \leq t\}) + \ cost(\{\boldsymbol{x}_i, y_i : x_{ij} > t\})$$

where the set of possible thresholds $\mathcal{T}_j$ for feature j can be obtained by sorting the unique values of x_{ij}

Cost

- *Regression*: $cost(\mathcal{D}) = \sum_{i \in \mathcal{D}} (y_i - \overline{y})^2$
- *Classification*: knowing that $\hat{\pi}_c = \frac{1}{|\mathcal{D}|} \sum_{i \in \mathcal{D}} \mathbb{1}_{\{y_i = c\}}$
  - *Misclassification Rate*: $\frac{1}{|\mathcal{D}|} \sum_{i \in \mathcal{D}} \mathbb{1}_{\{y_i \neq \hat{y}\}}$
  - *Entropy* $\mathbb{H}(\hat{\boldsymbol{\pi}}_c) = -\sum_{c=1}^{C} \hat{\pi}_c \log(\hat{\pi}_c)$
  - *Gini index*: $\sum_{c=1}^{C} \hat{\pi}_c (1 - \hat{\pi}_c)$,

Cross-entropy and Gini measures are very similar and more sensitive to changes in class probability than is the misclassification rate, finally they favor *pure nodes*

Pruning As stopping growing the tree to prevent overfitting would provoke to miss making any splits because of feature on its own has little predictive power.
Growing a "full" tree then performing *pruning* is better.

Random Forest to **reduce the variance of an estimate**, we can train M different trees on different subsets of the data, randomly chosen with replacement and compute:

$$f(\boldsymbol{x}) = \sum_{m=1}^{M} \frac{1}{M} f_m(\boldsymbol{x})$$

where f_m is the mth tree. This called **bagging (bootstrap aggregating)**
Simply re-running the sample learning algorithm on different subsets of the data can result in highly correlated predictors.
The **random forest** technique tries to decorrelate the base learners by learning trees based on a randomly chosen subset of input variables as well as a randomly chosen subset of data cases (observations).

Strengths

- easy to interpret
- handle mixed discrete and continuous input
- insensitive to monotone transformation of the inputs (because the splits are based on ranking the data points)
- automatic variable selection
- relatively robust to outliers
- scale well to large datasets
- can be modified to handle missing inputs

Weaknesses

- less accurate that other kinds of model (partly due to the greedy nature of the tree construction)
- unstable: small changes to the input data can have large effect on the structure of the tree (due to hierarchical nature of the tree-growing process)
- causing errors at the top can affect the rest of the tree

Relationships with other methods

Examples of application

9.3.9 Generalized Additive models

Purpose To create a nonlinear model with multiple inputs: $f(\boldsymbol{x}) = \alpha + \sum_{j=1}^{D} f_j(x_j)$

$f(\boldsymbol{x})$ can be mapped to $\mathbb{P}(y|\boldsymbol{x})$ using a *link funciton* as in Generalized Linear Models.

Assumptions

Theory

Smoothing splines The loss function in regression setting:

$$J(\alpha, (f_j)_{1\leq j\leq D}) = \sum_{i=1}^{n} \left(y_i - \alpha - \sum_{j=1}^{D} f_j(x_{ij}) \right)^2 + \sum_{j=1}^{D} \lambda_j \int f_j''(t_j)^2 dt_j$$

Backfitting As α is not uniquely identifiable since we can always add or substract constants to any of the f_j, the convention is $\forall j \in [\![1, D]\!]$, $\sum_{i=1}^{n} f_j(x_{ij}) = 0$, then $\hat{\alpha} = \frac{1}{n}\sum_{i=1}^{n} y_i$. Finally to fit the model:

1. $\hat{f}_j \leftarrow smoother\left(\left\{y_i - \sum_{k \neq j} \hat{f}_k(x_{ik})\right\}_{1 \leq i \leq n}\right)$
2. $\hat{f}_j \leftarrow \hat{f}_j - \frac{1}{n}\sum_{k \neq j} \hat{f}_k(x_{ik})$, ensuring the output is zero mean

where λ_j is the strength of the regularizer for f_j

Multivariate Adaptive Regression Splines (MARS) We create an ANOVA decomposition: $f(\boldsymbol{x}) = \beta_0 + \sum_{j=1}^{D} f_j(x_j) + \sum_{j,k} f_{jk}(x_j, x_k) + \sum_{j,k,l} f_{j,k,l}(x_j, x_k, x_l) + \cdots$

Strengths

Weaknesses

Relationships with other methods

Examples of application

9.3.10 Multi-layer Perceptron

Purpose It is a series of logistic regression models stacked on top of each other, with the final layer being another logistic regression or a linear regression model.

Assumptions

Theory For a three layers regression problem the model has the form: $\begin{cases} \mathbb{P}(y|\boldsymbol{x}, \boldsymbol{\theta}) & = \mathcal{N}\left(y|\boldsymbol{w}^T \boldsymbol{z}(\boldsymbol{x}), \sigma^2\right) \\ \boldsymbol{z}(\boldsymbol{x}) & = g(\boldsymbol{V}\boldsymbol{x}) \end{cases}$
where g is a non-linear activation function (usually the logistic function), $\boldsymbol{V}$ is the weight matrix from the input to the hidden nodes and $\boldsymbol{w}$ is the weight vector from hidden nodes to the output.

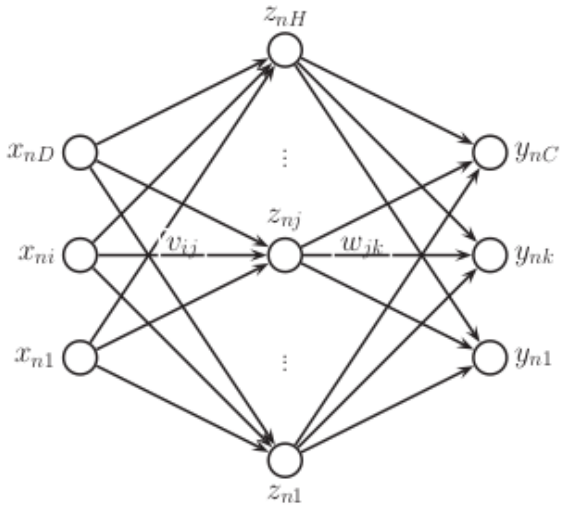


Figure 9.1: caption

Convolutional Neural Networks It is an MLP in which the hidden units have local receptive fields, and in which the weights are *tied* or *shared* across the image in order to reduce the number of parameters. Intuitively the effect of such spatial parameter tying is that any useful features that are "discovered" in some portion of the image can be re-used everywhere else without having to be independently learned. The resulting network then exhibits *translation invariance* meaning it can classify patterns no matter where they occur inside the input image.

Strengths

Weaknesses

Relationships with other methods

Examples of application

9.4 Model Selection

9.4.1 Bayesian Information Criterion (BIC)

Purpose Approximate the marginal likelihood $\mathbb{P}(D) = \sum_h \mathbb{P}(D|h)\,\mathbb{P}(h)$

Theory In Bayesian model selection we want to find the model for which the posterior is higher, let's consider M_k the kth model then: $\mathbb{P}(M_k|D) = \dfrac{\mathbb{P}(D|M_k)\,\mathbb{P}(M_k)}{\sum_j \mathbb{P}(D|M_j)\,\mathbb{P}(M_j)}$ as the key quantity is the marginal likelihood $\mathbb{P}(D|M_k) = \int_\Theta \mathbb{P}(D|\theta, M_k)\,\pi\theta|M_k d\theta$ We have then $\log(\mathbb{P}(D|M_k)) = \int_\Theta \log(\mathbb{P}(D|\theta, M_k)) + log(\pi\theta|M_k)\,d\theta$. Now from Laplace's Approximation an application $f \in \mathcal{C}^2(\mathbb{R}^d)$ such that $f : \mathbb{R}^d \longrightarrow \mathbb{R}$ be a smooth function. Consider as well that it has a unique maximum at $\hat{\theta}$

9.4.2 Akaike Information Criterion (AIC)

9.4.3 Bayesian Variable Selection

Purpose Let $\gamma_j : \begin{cases} \gamma_j = 1 \Leftarrow \text{feature } j \text{ is relevant} \\ \gamma_j = 0 \Leftarrow \text{otherwise} \end{cases}$ our goal is to compute the posterior over models:

$$\mathbb{P}(\gamma|\mathcal{D}) = \frac{e^{-f(\gamma)}}{\sum_{\gamma'} e^{-f(\gamma')}}$$

where the cost function is defined by $f(\gamma) \triangleq -[\log(\mathbb{P}(\mathcal{D}|\gamma)) + \log(\mathbb{P}(\gamma))]$

Assumptions

Theory

Spike and slab model remember that posterior is given by $\mathbb{P}(\boldsymbol{\gamma}|\mathcal{D}) \propto \mathbb{P}(\mathcal{D}|\boldsymbol{\gamma})\,\mathbb{P}(\boldsymbol{\gamma})$.
It is common to use the following prior $\mathbb{P}(\boldsymbol{\gamma}) = \prod_{j=1}^{D} Ber(\gamma_j|\pi_0) = \pi_0^{\|\gamma\|_0}(1-\pi_0)^{D-\|\gamma\|_0}$, where π_0 is the probability that a feature is relevant.
The likelihood is defined as follows: $\mathbb{P}(\mathcal{D}|\boldsymbol{X}, \boldsymbol{\gamma}) = \int\int \mathbb{P}(\boldsymbol{y}|\boldsymbol{X}, \boldsymbol{w}, \boldsymbol{\gamma})\,\mathbb{P}(\boldsymbol{w}|\boldsymbol{\gamma}, \sigma^2)\,\mathbb{P}(\sigma^2)\,d\boldsymbol{w}d\sigma^2$
Consider the prior $\mathbb{P}(\boldsymbol{w}|\boldsymbol{\gamma}, \sigma^2)$ in standardizing the inputs, a reasonable prior is $\mathcal{N}(0, \sigma^2\sigma_w^2)$, where σ_w^2

controls how big we expect the coefficients associated with the relevant variables to be, which is scaled by the overall noise. We can summarize this prior as follows: $\mathbb{P}\left(\boldsymbol{w}_j|\sigma^2,\gamma_j\right)\begin{cases}\delta_0(w_j) \Leftarrow \gamma_j = 0\\ \mathcal{N}(w_j|0,\sigma^2\sigma_w^2) \Leftarrow \gamma_j = 1\end{cases}$ the first term is a "spike" at the origin, as $\sigma_w^2 \to +\infty$ the distribution $\mathbb{P}\left(w_j|\gamma_j=1\right)$ approaches a uniform distribution which can be thought of as a "slab".

Bernoulli-Gaussian model we have $\begin{cases}\mathbb{P}\left(y_i|\boldsymbol{x}_i,\boldsymbol{w},\boldsymbol{\gamma},\sigma^2\right) = \mathcal{N}\left(\sum_j \gamma_j w_j x_{ij},\sigma^2\right)\\ \mathbb{P}\left(\gamma_j\right) = Ber(\pi_0)\\ \mathbb{P}\left(w_j\right) = \mathcal{N}(0,\sigma_w^2)\end{cases}$ we can think of the γ_j as a masking out the weights w_j. Unlike the spike and slab model we do not integrate the irrelevant coefficients, they always exists.
One interesting aspect of this model is that it can be used to derive objective function that is widely used in the non-Bayesian subset selection literature.

Algorithms assuming that we want to find the MAP model.

- Single best replacement: at each step, we define a neighborhood of the current model to be all models than can be reached by flipping a single bit of γ
- Orthogonal least squares: we start from an empty set of variables and we add the best feature $j^* = \arg\min_{j\notin\boldsymbol{\gamma}_t}\min_{\boldsymbol{w}}\|\boldsymbol{y}-\boldsymbol{X}_{\boldsymbol{\gamma}_t\cup j}\boldsymbol{w}\|^2$
- Orthogonal matching pursuits: as Orthogonal least squares is somewhat expensive, a simplification is to freeze the current weight and then pick the next feature to add by solving $j^* = \arg\min_{j\notin\boldsymbol{\gamma}_t}\min_{\beta}\|\boldsymbol{y}-\boldsymbol{X}\boldsymbol{w}_t-\beta\boldsymbol{x}_{\cdot j}\|^2$. And $\beta = \dfrac{\boldsymbol{x}_{\cdot j}^T\boldsymbol{r}_t}{\|\boldsymbol{x}_{\cdot j}\|^2}$, where $\boldsymbol{r} = \boldsymbol{y}-\boldsymbol{X}\boldsymbol{w}_t$
- Backward selection: starts with all variables in the model and deletes the
- Bayesian matching pursuits: similar to OMP except it uses a Bayesian marginal likelihood scoring criterion instead of a least square objective.

Strengths

Weaknesses

Relationships with other methods

Examples of application

9.4.4 Least Angle Regression Shrinkage

Purpose Instead of updating only one variable at a time, inducing slow converging, we can use *Active set* methods that update many variables at a time. They add or remove a few variables at a time so they can take a long time if they started far from the solution.
Suppose $\mathcal{A}_k$ is the active set of variables at the beginning of the kth step and let $\beta_{\mathcal{A}_k}$ be the coefficient vector for these variables at this step, there will be $k-1$ nonzero values and the one just entered will be zero.
If $\boldsymbol{r}_k = \boldsymbol{y}-\boldsymbol{X}_{\mathcal{A}_k}\beta_{\mathcal{A}_k}$ is the current residual, then the direction for this step is:

$$\delta_k = (\boldsymbol{X}_{\mathcal{A}_k}^T\boldsymbol{X}_{\mathcal{A}_k})^{-1}\boldsymbol{X}_{\mathcal{A}_k^T}^T\boldsymbol{r}_k$$

The name “least angle” arises from a geometrical interpretation of this process; $\boldsymbol{u}_k$ makes the smallest (and equal) angle with each of the predictors in $\mathcal{A}_k$

Assumptions

Theory Active methods exploit the fact that one can quickly compute $\hat{\boldsymbol{w}}(\lambda_k)$ from $\hat{\lambda}(\lambda_{k-1})$ if $\lambda_k \approx \lambda_{k-1}$ this is known as **warm starting**.

Least Angle Regression

1. Standardize the predictors to have mean zero an unit norm.
   Start with the residual $\boldsymbol{r} = \boldsymbol{y} - \overline{\boldsymbol{y}}$, and for all $j \in [\![1, p]\!]$, $\beta_j = 0$
2. Find the predictor x_j most correlated with $\boldsymbol{r}$
3. For $j \in [\![1, p]\!]$
   - Move β_j from 0 to its least-squares coefficient $\dfrac{\langle \boldsymbol{x}_j | \boldsymbol{r} \rangle}{\langle \boldsymbol{x}_j | \boldsymbol{x}_j \rangle}$, then recompute $\boldsymbol{r}$. Find the predictor x_k most correlated with the new $\boldsymbol{r}$
   - Move β_k from 0 to its least-squares coefficient $\dfrac{\langle \boldsymbol{x}_k | \boldsymbol{r} \rangle}{\langle \boldsymbol{x}_k | \boldsymbol{x}_k \rangle}$, then recompute $\boldsymbol{r}$. Find the predictor x_l most correlated with the new $\boldsymbol{r}\ldots$
   - Move β_j and β_k in the direction defined by their joint least squares coefficient of the current residual on (x_j, x_k), until some other competitor x_l has as much correlation with the current residual.
4. After $\min(N-1, p)$ steps, we arrive at the full least-squares solution.

Least Angle Regression: Lasso Modification

4a If a non-zero coefficient hits zero, drop its variable from the active set of varibalbes and recompute the current joint least squares direction

Strengths

- Numerically efficient when $p \gg n$
- As fast as forward selection and has the same order of complexity as OLS
- Produces a full piecewise linear solution path
- Stable
- Esealy modified to produce for other estimators like the Lasso

Weaknesses

- Because LARS is based upon iterative refitting of the residuals, it would appears to be especially sensitive to the effect noise.

Relationships with other methods

- Least Squares

Examples of application

9.5 Regularization

9.5.1 l_2 regularization

Purpose

- Prevent overfitting
- Improve condition number

Assumptions We assume $\mathbb{P}(\boldsymbol{w}) = \prod_{j=1}^{D} \mathcal{N}(w_j|0, \tau^2)$, where $\frac{1}{\tau^2}$ controls the strength of the prior!

Theory The corresponding MAP estimation problem is :

$$\arg\max \boldsymbol{w} \sum_{i=1}^{n} \log\left(\mathcal{N}(y_i|w_0 + \boldsymbol{w}^T\boldsymbol{x}_i, \sigma^2)\right) + \sum_{i=1}^{D} \log\left(\mathcal{N}(w_j|0, \tau^2)\right)$$

We can rewrite this as

$$\frac{1}{n}\|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{w}\|^2 + \lambda\|\boldsymbol{w}\|_2^2$$

The optimal solution is

$$\boldsymbol{w}_{ridge} = \left(\lambda \boldsymbol{I}_D + \boldsymbol{X}^T\boldsymbol{X}\right)^{-1}\boldsymbol{X}^T\boldsymbol{y}$$

Relationships with other methods

Connection with PCA Let $\boldsymbol{X} = \boldsymbol{U}\boldsymbol{S}\boldsymbol{V}^T$ be the *SVD* of $\boldsymbol{X}$. $\hat{\boldsymbol{w}}_{ridge} = \boldsymbol{V}\left(\boldsymbol{S}^{\check{s}} + \lambda \boldsymbol{I}\boldsymbol{j}\right)^{-1}\boldsymbol{S}\boldsymbol{U}^T\boldsymbol{y}$ Hence

$$\begin{aligned}
\hat{\boldsymbol{y}} &= \boldsymbol{X}\hat{\boldsymbol{w}}_{ridge} \\
&= \boldsymbol{U}\boldsymbol{S}\boldsymbol{V}^T\boldsymbol{V}\left(\boldsymbol{S}^{\check{s}} + \lambda \boldsymbol{I}\boldsymbol{j}\right)^{-1}\boldsymbol{S}\boldsymbol{U}^T\boldsymbol{y} \\
&= \boldsymbol{U}\tilde{\boldsymbol{S}}\boldsymbol{U}^T\boldsymbol{y} \\
&= \sum_{j=1}^{p} \boldsymbol{u}_j \tilde{\boldsymbol{S}}_{jj}\boldsymbol{u}_j^T\boldsymbol{y}
\end{aligned}$$

where $\tilde{\boldsymbol{S}}_{jj} \triangleq \left[\boldsymbol{S}\left(\boldsymbol{S}^2 + \lambda \boldsymbol{I}\right)^{-1}\boldsymbol{S}\right]_{jj} = \frac{\sigma_j^2}{\sigma_j^2 + \lambda}$ and σ_j are the singular values of $\boldsymbol{X}$. Therefore

$$\hat{\boldsymbol{y}} = \boldsymbol{X}\hat{\boldsymbol{w}}_{ridge} = \sum_{j=1}^{p} \boldsymbol{u}_j \frac{\sigma_j^2}{\sigma_j^2 + \lambda}\boldsymbol{u}_j^T\boldsymbol{y}$$

Whereas in ordinary least squares we have: $\hat{\boldsymbol{y}} = \boldsymbol{X}\hat{\boldsymbol{w}}_{ols} = \left(\boldsymbol{U}\boldsymbol{S}\boldsymbol{V}^T\right)\left(\boldsymbol{V}\boldsymbol{S}^{-1}\boldsymbol{U}^T\boldsymbol{y}\right) = \boldsymbol{U}\boldsymbol{U}^T\boldsymbol{y} = \sum_{j=1}^{p} \boldsymbol{u}_j\boldsymbol{u}_j^T\boldsymbol{y}$

9.5.2 l_1 regularization

Purpose

Assumptions We assume $\mathbb{P}(\boldsymbol{w}|\lambda) = \prod_{j=1}^{D} Lap(w_j|0, \frac{1}{\lambda}) \propto \prod_{j=1}^{D} e^{-\lambda|w_j|}$

Theory For penalized negative log likelihood has the form: $f(\boldsymbol{w}) = -\log\left(\mathbb{P}(\mathcal{D}|\boldsymbol{w})\right) - \log\left(\mathbb{P}(\boldsymbol{w}|\lambda)\right) = NLL(\boldsymbol{w}) + \lambda\|\boldsymbol{w}\|_1$.
Geometrically we understand that as we relax the constraint we grow l_1 ball until it meets the objective, the corners of the ball are more likely to intersect the ellipse than one of the sides especially in high dimensions because the corners "stick out". The corners correspond to sparse solutions which lie on the coordinate axes. By contrast when we grow the l_2 ball it can intersect the objective at any point, there are no "corners" so there is no preference for sparsity.

Strengths

Weaknesses

- can give quite different results if the data is slightly perturbed

Relationships with other methods

Examples of application

9.5.3 Noise Robustness

Purpose It can be much more powerful than shrinking the parameters

Assumptions

Theory

Strengths

Weaknesses

Relationships with other methods

Examples of application

9.5.4 Regularization path

Purpose As we increase λ, the solution vector $\hat{\boldsymbol{w}}(\lambda)$ will tend to get sparser, although not necessary monotonically. For each feature j we can plot the values $\hat{w}_j(\lambda)$ vs λ which is known as *regularization path*

Assumptions

Theory

Strengths

Weaknesses

Relationships with other methods

Examples of application

9.5.5 Coordinate Descent

Purpose These algorithms exploit the fact that one can quickly compute $\hat{\boldsymbol{w}}(\lambda_k)$ from $\hat{\boldsymbol{w}}(\lambda_{k-1})$ if $\lambda_k \approx \lambda_{k-1}$ this is known as *warm starting*.
Useful when it is hard to optimize all the variables simultaneously

Assumptions

Theory

Aim of Coordinate descent We solve for the jth coefficient with all the others held fixed:

$$w_j^* = \arg\min_z f(\boldsymbol{w} + z\boldsymbol{e}_j) - f(\boldsymbol{w})$$ with $\boldsymbol{e}_j$ is the j'th unit vector.

Algorithm for Lasso

1. Initialise $\boldsymbol{w} = \left(\boldsymbol{X}^T\boldsymbol{X} + \lambda\boldsymbol{I}\right)^{-1}\boldsymbol{X}^T y$
2. Repeat until convergence:
   for $j \in [\![1, D]\!]$
   - $\alpha_j = 2\sum_{i=1}^{n} x_{ij}^2$
   - $c_j = 2\sum_{i=1}^{n} x_{ij}(y_i - \boldsymbol{w}^T\boldsymbol{x}_i + w_j x_{ij})$
   - $w_j = soft\left(\frac{c_j}{\alpha_j}, \frac{\lambda}{\alpha_j}\right)$

Strengths

- if each one-dimensional optimization problem can be solved analitically

Weaknesses

Relationships with other methods

Examples of application

9.6 Ensemble Learning

The overall purpose is to learn a weighted combination of base models of the form:

$$f(y|\boldsymbol{x}, \boldsymbol{\pi}) = \sum_{m\in\mathcal{M}} w_m f_m(y|\boldsymbol{x})$$

where w_m are tunable parameters

9.6.1 Boosting

Purpose It aims to build a model with reduced bias and variance.

It is a greedy algorithm for fitting adaptive basis-function models $f(\boldsymbol{x}) = w_0 + \sum_{m=1}^{M} w_m\phi_m(\boldsymbol{x})$, where ϕ_m are generated by an algorithm called **weak learner**. This algorithm works by applying the weak learner sequentially to weighted versions of the data, where more weight is given to examples that were misclassified by earlier rounds.
This weak learner can be any classification or regression algorithm, but it common to use CART model. Boosting is very resistant to overfitting.
The goal of boosting is to solve the following optimization problem:

$$\min_f \sum_{i=1}^{n} L(y_i, f(\boldsymbol{x}_i))$$

Assumptions A family of *weak learners* (working slightly better that random guess) can be transformed in a family of *strong learners*

Theory

Forward stagewise additive modeling For squared error loss the optimal estimate is given by: $f^*(\boldsymbol{x}) = \arg\min_{f(\boldsymbol{x})} = \mathbb{E}_{y|\boldsymbol{x}}\left([y - f(\boldsymbol{X})]^2\right) = \mathbb{E}(y|\boldsymbol{x})$
For binary classification we can use the logloss which is a convex upper bound on 0-1 loss. One can show that $f^*(\boldsymbol{x}) = \frac{1}{2}\log\left(\frac{\mathbb{P}(\hat{y}=1|\boldsymbol{x})}{\mathbb{P}(\hat{y}=-1|\boldsymbol{x})}\right)$

1. initialise by defining $f_0(\boldsymbol{x}) = \arg\min_{\gamma} \sum_{i=1}^{n} L(y_i, f(\boldsymbol{x};\gamma))$, for example if we use squared error we can set $f_0(\boldsymbol{x}) = \overline{y}$ and if we use log-loss or exponential loss we can set $f_0 = \frac{1}{2}\log\left(\frac{\hat{\pi}}{1-\hat{\pi}}\right)$ where $\hat{\pi} = \frac{1}{n}\sum_{i=1}^{n} \mathbb{1}_{\{y_i=1\}}$
2. at iteration m we compute $(\beta_m, \gamma_m) = \arg\min_{\beta,\gamma} \sum_{i=1}^{n} L(y_i, f_{m-1}(\boldsymbol{x}_i) + \beta\phi(\boldsymbol{x}_i;\gamma))$ where ϕ is generated by the *weak learner* algorithm.
3. $f_m(\boldsymbol{x}) = f_{m-1}(\boldsymbol{x}) + \nu\beta_m\phi(\boldsymbol{x}_i;\gamma_m)$ where $\nu \in [0,1]$ is a step-size parameter.

Name	**Loss**	**Derivate**	$f*$	**Algorithm**
Squared error	$\frac{1}{2}(y_i - f(\boldsymbol{x}_i))^2$	$y_i - f(\boldsymbol{x}_i)$	$\mathbb{E}(y\|\boldsymbol{x}_i)$	*L2Boosting*
Absolute error	$\|y_i - f(\boldsymbol{x}_i\|$	$sgn(y_i - f(\boldsymbol{x}_i))$	$median(y\|\boldsymbol{x}_i)$	*Gradient Boosting*
Exponential loss	$\exp(-y_i, f(\boldsymbol{x}_i))$	$-y_i\exp(-y_i f(\boldsymbol{x}_i))$	$\frac{1}{2}\log\left(\frac{\pi_i}{1-\pi_i}\right)$	*AdaBoost*
Logloss	$\log\left(1 + e^{-y_i f(\boldsymbol{x}_i)}\right)$	$-y_i - \pi_i$	$\frac{1}{2}\log\left(\frac{\pi_i}{1-\pi_i}\right)$	*LogitBoost*

where $\pi = \sigma(2f(\boldsymbol{x}))$

Strengths - robust against overfitting - reduce bias and variance

Weaknesses

Relationships with other methods

Examples of application

9.6.2 Bayes Model Averaging (that is not an Ensemble Method)

Purpose Instead of picking the best model and then using this to make predictions we make a weighted average of the predictions made by each model:

Assumptions

Theory Let's compute:

$$\mathbb{P}(\{y|\boldsymbol{x}, \boldsymbol{\mathcal{D}}\}) = \sum_{m\in\mathcal{M}} \mathbb{P}(\{y|\boldsymbol{x}, m, \boldsymbol{\mathcal{D}}\})\,\mathbb{P}(\{m|\boldsymbol{\mathcal{D}}\})$$

Obviously averaging over all the models is computationally infeasible, then a simple approximation

Strengths

- Better results than using any single model

Weaknesses

Relationships with other methods

Examples of application

9.6.3 Stacking

Purpose Refers to learning a weighted combination of base models of the form: $f(y|\boldsymbol{x}, \boldsymbol{\pi}) = \sum_{m \in \mathcal{M}} w_m f_m(y|\boldsymbol{x})$ where w_m are the tunable parameters.

Assumptions

Theory To estimate the weights we can use :

$$\hat{\boldsymbol{w}} = \arg\min_{\boldsymbol{w}} \sum_{i=1}^{n} L\left(y_i, \sum_{m=1}^{M} w_m \hat{f}_m^{-i}(\boldsymbol{x})\right)$$

with $\hat{f}_m^{-i}(\boldsymbol{x})$ being the predictor obtained by training on data excluding $(\boldsymbol{x}_i, y_i)$. It would allow to avoid overfitting that would produce in using simply f_m. This know as **stacking** or **stacked generalization**.

Strengths

- Robustness when the "true" model is not in the model class.

Weaknesses

Relationships with other methods

Examples of application

- analogy with neural networks: f_m representing the mth hidden unit and w_m are the outputs layer weights
- analogy with boosting: where the weights on the base models are determined sequentially

9.6.4 Boostrap Aggregating (Bagging)

Purpose It is designed to improve the stability and accuracy of machine learning algorithms.

Assumptions

Theory Let's consider M different models $(f_m)_{1 \leq m \leq M}$ that we will train on random different subsets of data (*bootstrap step*).
We then compute the ensemble:

$$f(\boldsymbol{x}) = \sum_{m=1}^{M} \frac{1}{M} f_m(\boldsymbol{x})$$

Strengths

- Better results than using any single model

Weaknesses

- Variance reduction not very efficient because of rerunning same learning algorithm on different subsets of the data can result in highly correlated

Relationships with other methods

- Random forest: use bagging in randomly cutting the input space vertically (subsets of predictors) and horizontally (subset of observations)

Examples of application

Chapter 10

Unsupervised Learning

10.1 Clustering

10.1.1 Clustering in general

Purpose 2 kinds of inputs we might use, *similarity-based clustering* the input to the algorithm is an $N \times N$ dissimilarity matrix. In *feature-based clustering* the input to the algorithm is an $N \times D$ feature matrix or design matrix $\boldsymbol{X}$.

Assumptions

Theory

Measuring dissimilarity $\Delta(\boldsymbol{x}_i, \boldsymbol{x}_{i'}) = \sum_{j=1}^{D} \Delta_j(x_{ij}, x_{i'j})$

- *Squared distance* $\Delta_j(x_{ij}, x_{i'j}) = (x_{ij} - x_{i'j})^2$
- *correlation* (useful for time-series of real-valued data) $\Delta_j(x_{ij}, x_{i'j}) = Cor(\boldsymbol{x}_i, \boldsymbol{x}_{i'})$
- *l1 distance* $\Delta_j(x_{ij}, x_{i'j}) = |x_{ij} - x_{i'j}|$
- *ordinal variables* any dissimilarity functions for quantitative function
- *categorical distance* $\Delta_j(x_{ij}, x_{i'j}) = \sum_{j=1}^{D} \mathbb{1}_{\{x_{ij} \neq x_{i'j}\}}$

Measuring dis(similarity)

Evaluating the output of clustering methods The validation of clustering structures is the most difficult and frustrating part of cluster analysis

Purity $purity \triangleq \sum_i \frac{N_i}{N} p_i$

Rand index

Mutual information

Varation of information

Strengths

Weaknesses

Relationships with other methods

Examples of application

10.1.2 *K-means* algorithm

Purpose Consider a *Gaussian Mixture Models* in which we make the following assumptions $\boldsymbol{\Sigma}_k = \sigma^2 \boldsymbol{I}_D$ is fixed and $\pi_k = \frac{1}{K}$, then only the cluster centers $\mu_k \in \mathbb{R}^D$ have to be estimated.

Assumptions

Theory Now consider $\mathbb{P}(z_i = k|\boldsymbol{x}_i, \boldsymbol{\theta}) \approx \mathbb{1}_{\{k=z_i^*\}}$, where $z_i^* = \arg\max_k \mathbb{P}(z_i = k|\boldsymbol{x}_i, \boldsymbol{\theta})$.
As we are making a hard assignment of points to clusters, this is sometimes called *hard EM*. Since we assumed an equal spherical covariance matrix for each cluster we have $z_i^* = \arg\min_k \|\boldsymbol{x}_i - \boldsymbol{\mu}_k\|_2^2$
The *M step* updates each cluster center by computing the mean of all points assigned to it: $\boldsymbol{\mu}_k = \frac{1}{n_k} \sum_{i:z_i=k} \boldsymbol{x}_i$.
Since K-means is not a proper EM algorithm it is not maximizing likelihood, instead it can be interpreted as a greedy algorithm minimizing a loss function related to data compression.

Strengths

Weaknesses

Relationships with other methods

- Gaussian Mixture

Examples of application

10.1.3 Vector Quantization

Purpose Suppose performing lossy compression of some real-valued vectors $\boldsymbol{x}_i \in \mathbb{R}^D$.
The basic idea is to replace each real-valued vector $\boldsymbol{x}_i \in \mathbb{R}^D$ with a discrete symbol $z_i \in [\![1, K]\!]$ being an index into a *codebook* of K prototypes $\boldsymbol{\mu}_k \in \mathbb{R}^D$

Theory Each data vector is encoded by using the index of the most similar prototype where similarity is measured in terms of Euclidean distance:

$$encode(\boldsymbol{x}_i) = \arg\min_k \|\boldsymbol{x}_i - \boldsymbol{\mu}_k\|^2$$

Here a cost function measuring the quality of the codebook by computing the *reconstruction error*:

$$J(\boldsymbol{\mu}, \boldsymbol{z}|K, \boldsymbol{X}) \triangleq \frac{1}{N} \sum_{i=1}^{N} \sum_{i=1}^{N} \|\boldsymbol{x}_i - decode(encode(\boldsymbol{x}_i))\|^2 = \frac{1}{N} \sum_{i=1}^{N} \|\boldsymbol{x}_i - \boldsymbol{\mu}_{z_i}\|^2$$

Assumptions

Strengths

Weaknesses

Relationships with other methods

Examples of application

10.1.4 Factor Analysis

Purpose Getting a bigger representation power through latent variables than Mixture models in using real-valued latent variable $\boldsymbol{z}_i \in \mathbb{R}^L$.
It can be thought of as a way of specifying a joint density model on $\boldsymbol{x}$ using a small number of parameters.

Assumptions

Theory Let's consider $\begin{cases} \mathbb{P}(\boldsymbol{z}_i) = \mathcal{N}(\boldsymbol{z}_i|\boldsymbol{\mu}_0, \boldsymbol{\Sigma}_0) \\ \mathbb{P}(\boldsymbol{x}_i|\boldsymbol{z}_i, \boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{W}\boldsymbol{z}_i + \boldsymbol{\mu}, \boldsymbol{\Phi}) \end{cases}$ where $\boldsymbol{W}$ is a $D \times L$ matrix knows as *factor loading matrix* and Φ is a $D \times D$ covariance matrix, which is diagonal as we aim to force $\boldsymbol{z}_i$ to explain correlation.

10.1.5 Mixture of Factor Analyzers

Purpose While Factor Analysis assumes that data lives on a low dimensional *linear* manifold, in reality most data is better modeled by some form of low dimensional *curved* manifold.
We can approximate a curved manifold by a piecewise linear manifold

Assumptions

Theory Let the kth linear subspace of dimensionality L_k be represented by $\boldsymbol{W}_k$. Suppose we have a latent indicator $q_i \in [\![1, k]\!]$ specifying which subspace we should use to generate the data. Here the model:

$$\begin{cases} \mathbb{P}(\boldsymbol{x}_i|\boldsymbol{z}_i, q_i = k, \boldsymbol{\theta}) & = \mathcal{N}(\boldsymbol{x}_i|\boldsymbol{W}_k\boldsymbol{z}_i, \Psi) \\ \mathbb{P}(\boldsymbol{z}_i|\boldsymbol{\theta}) & = \mathcal{N}(\boldsymbol{z}_i|\boldsymbol{0}, \boldsymbol{I}) \\ \mathbb{P}(q_i|\boldsymbol{\theta}) & = Cat(q_i|\boldsymbol{\pi}) \end{cases}$$

Strengths

- it is a good density model for high-dimensional real-valued data

Weaknesses

Relationships with other methods

Examples of application

10.1.6 Canonical Correlation Analysis

Purpose It is a symmetric unsupervised version of PLS, it allows each view to have its own "private" subspace, but there is also a shared space.

Assumptions

Theory If we have 2 observed variables $\boldsymbol{x}_i$ and $\boldsymbol{y}_i$ then we have 3 latents variables: $\boldsymbol{z}_i^s \in \mathbb{R}^{L_0}$ which is shared, $\boldsymbol{z}_i^x \in \mathbb{R}^{L_x}$ and $\boldsymbol{z}_i^x \in \mathbb{R}^{L_x}$ which are private :

$$\begin{cases} \mathbb{P}(\boldsymbol{z}_i) & = \mathcal{N}(\boldsymbol{z}_i^s|\boldsymbol{0}, \boldsymbol{I}_{L_s})\mathcal{N}(\boldsymbol{z}_i^x|\boldsymbol{0}, \boldsymbol{I}_{L_x})\mathcal{N}(\boldsymbol{z}_i^y|\boldsymbol{0}, \boldsymbol{I}_{L_y}) \\ \mathbb{P}(\boldsymbol{x}_i|\boldsymbol{z}_i) & = \mathcal{N}(\boldsymbol{x}_i|\boldsymbol{B}_x\boldsymbol{z}_i + \boldsymbol{W}_x\boldsymbol{z}_i^s + \boldsymbol{\mu}_x, \sigma^2\boldsymbol{I}_{D_x}) \\ \mathbb{P}(\boldsymbol{y}_i|\boldsymbol{z}_i) & = \mathcal{N}(\boldsymbol{y}_i|\boldsymbol{B}_y\boldsymbol{z}_i + \boldsymbol{W}_y\boldsymbol{z}_i^s + \boldsymbol{\mu}_y, \sigma^2\boldsymbol{I}_{D_y}) \end{cases}$$

Strengths

Weaknesses

Relationships with other methods

Examples of application

10.1.7 Independent Component Analysis (ICA)

Purpose When we want to deconvolve mixed signals into their constituent parts.
Unlike PCA, we relax the Gaussian assumption about latent variable, now the distribution can be any non-Gaussian distribution. The reason the Gaussian distribution is disallowed as a source prior in ICA is that it does not permit unique recovery of the sources.

Assumptions

Theory $\mathbb{P}(\boldsymbol{z}_t) = \prod_{j=1}^{L} \mathbb{P}_j(z_{tj})$

Strengths

Weaknesses

Relationships with other methods

Examples of application

10.1.8 DBSCAN: Density-Based Spatial Clustering of Applications with Noise

Purpose Given a set of points in some space $(p_i)_{1\leq i\leq n} \in \mathcal{S}^n$, it groups together points that are closely packed together, making then as outliers points that lie alone in low density regions.

Assumptions

Theory

Definition Let $\epsilon \in \mathbb{R}$ the radius of a neighborhood with respect to some points, ρ_{min} a threshold of point count. Consider 2 points p and q.

- $\#\{m : \|m-p\| < \epsilon\} > \rho_{min} \Rightarrow p$ is **core point**
- $\begin{cases} p \text{ is a core point} \\ \|q-p\| < \epsilon \end{cases} \Rightarrow q$ is **directly reachable from** p
- Let $n \in \mathbb{N}$. $\exists (m_i)_{1\leq i\leq n} \in \mathcal{S}^n, \begin{cases} (m_1 = p) \wedge (m_n = q) \\ \forall i \in [\![1, n-1]\!] \ m_{i+1} \text{ is directly reachable from } m_i \end{cases} \Rightarrow q$ *is reachable from* p
- Points that are not reachable from a core point are considered as *outliers*
- If there is a point m from which 2 points p and q are reachable from m then p and q are density-connected

Algorithm

1. For each point m, count the number of points at a distance lower than ϵ from m
2. Identify the core points, the ones having a neighborhood count greater than ρ_{min}
3. Delimit clusters with the reachable property of the different core points.
4. For each non-core points assign it to the closer cluster being at a distance lower than ϵ, if no cluster is found consider this point as noise.

Strengths

- Number of cluster does not need to be specified
- Clusters are arbitrary-shaped
- Robust to outliers

Weaknesses

- For a given border point that is reachable by multiple clusters, the cluster with which it will be associated with depends on the database order.
- Due to euclidean distance to measure distance between 2 points, DBSCAN can suffer from "curse of dimensionality"
- Cannot cluster well data sets with large differences in densities, since ρ_{min} and ϵ would not be set appropriately for each cluster.
- If the data is not well understood, choosing meaningful ρ_{min} and ϵ is difficult.

Relationships with other methods

Examples of application

10.1.9 Affinity Propagation

Purpose The idea is that each point must choose another data point as its exemplar or centroid some data points will choose themselves as centroids and this will automatically determine the number of clusters

Assumptions

Theory

Strengths

Weaknesses

Relationships with other methods

Examples of application

10.1.10 Dirichlet Process

Purpose

Assumptions

Theory

Strengths

Weaknesses

Relationships with other methods

Examples of application

10.1.11 Spectral Clustering

Purpose

Assumptions

Theory

Strengths

Weaknesses

Relationships with other methods

Examples of application

10.1.12 Agglomerative Clustering

Purpose

Assumptions

Theory

Strengths

Weaknesses

Relationships with other methods

Examples of application

10.1.13 Bayesian Hierarchical Clustering

Purpose

Assumptions

Theory

Strengths

Weaknesses

Relationships with other methods

Examples of application

10.2 Association

10.2.1 Directed graphical models (Bayes nets)

Purpose Relevant to compactly represent the joint distribution $\mathbb{P}(\boldsymbol{x}|\boldsymbol{\theta})$ and then *infer* one set of variables given another, and how *learn* the parameters of this distribution.

Assumptions

- Conditional independence

Theory

Graph terminology let's consider a *graph* $G = (\mathcal{V}, \mathcal{E})$ consisting of a set of *vertices* or nodes $\mathcal{V} = v_{1 \leq v \leq V}$ and *edges*, $\mathcal{E} = \{(s,t) : s,t \in \mathcal{V}^2\}$

- *Cycle*: series a cycle to be a series of nodes such that we can get back to where we started by following edges.
- *DAG*: Directed Acyclic Graph is a directed graph without cycles.
- *Tree*: Undirected graph without cycles.
- *Forest*: set of trees

A **Directed Graphical Model (DGM)** are more commonly known as *Bayesian Networks*.

Inference In partitioning the data into *visible variables* $\boldsymbol{x}_v$ and *hidden variables* $\boldsymbol{x}_h$, inference refers to computing the posterior distribution of the unknown given the knows:

$$\mathbb{P}(\boldsymbol{x}_h|\boldsymbol{x}_v, \boldsymbol{\theta}) = \frac{p(\boldsymbol{x}_h, \boldsymbol{x}_v|\boldsymbol{\theta})}{p(\boldsymbol{x}_v|\boldsymbol{\theta})} = \frac{p(\boldsymbol{x}_h, \boldsymbol{x}_v|\boldsymbol{\theta})}{\sum_{\boldsymbol{x}'_h} p(\boldsymbol{x}'_h, \boldsymbol{x}_v|\boldsymbol{\theta})}$$

Learning The aim is to compute a MAP estimate of the parameters given data: $\hat{\boldsymbol{\theta}} = \arg\max_{\boldsymbol{\theta}} \sum_{i=1}^{n} \log(\boldsymbol{x}_{i,v}|\boldsymbol{\theta}) + \log(\mathbb{P}(\boldsymbol{\theta}))$.
In adopting a Bayesian view, the parameter are unknown variables and should also be inferred, thus to a Bayesian there is no distinction between inference and learning.

Strengths

Weaknesses

Relationships with other methods

Examples of application

10.2.2 Kernels

Purpose Measuring the similarity between 2 objects that does not require preprocessing them into feature vector format.

Assumptions

Theory

- *Radial Basis Function (RBF)*, or Gaussian kernel: $k(\boldsymbol{x}, \boldsymbol{x}') = e^{\frac{1}{2}(\boldsymbol{x}-\boldsymbol{x}')^T \boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{x}')}$
- *Mercer* kernel: kernel for which the *Gram matrix* $K = \begin{pmatrix} k(\boldsymbol{x}_1, \boldsymbol{x}_1) & \cdots & k(\boldsymbol{x}_1, \boldsymbol{x}_n) \\ & \vdots & \\ k(\boldsymbol{x}_n, \boldsymbol{x}_1) & \cdots & k(\boldsymbol{x}_n, \boldsymbol{x}_n) \end{pmatrix}$ is positive definite
- *Linear* kernel: $k(\boldsymbol{x}, \boldsymbol{x}') = \boldsymbol{x}^T \boldsymbol{x}'$, useful if original data is already high dimensional and features individually informative
- *Matern* kernels: $k(r) = \frac{2^{1-\nu}}{\Gamma(\nu)} \left(\frac{\sqrt{2\nu r}}{l}\right)^{\nu} \boldsymbol{K}_{\nu} \left(\frac{\sqrt{2\nu r}}{l}\right)$ where $r = \|\boldsymbol{x} - \boldsymbol{x}'\|$, $\nu > 0$, $l > 0$ and $\boldsymbol{K}_{\nu}$ a modified Bessel function.
- *Probability product* kernels: $k(\boldsymbol{x}_i, \boldsymbol{x}_j) = \int \mathbb{P}(\boldsymbol{x}|\boldsymbol{x}_i)^{\rho} \mathbb{P}(\boldsymbol{x}|\boldsymbol{x}_j)^{\rho} d\boldsymbol{x}$ where $\rho > 0$, relevant for a probabilistic generative model.
- *Fisher* kernels: $k(\boldsymbol{x}, \boldsymbol{x}') = \boldsymbol{g}(\boldsymbol{x})^T \boldsymbol{F}^{-1} \boldsymbol{g}(\boldsymbol{x}')^T$ where g is the gradient of the log-likelihood and $\boldsymbol{F}$ is the Fisher information matrix.
- *Epanechnikov* kernel: $k(x) \triangleq \frac{3}{4}(1-x^2)\mathbb{1}_{\{|x|\leq 1\}}$, compact support (useful for efficiently reasons since one can use fast nearest neighbor methods to evaluate density) but not continuous derivatives at the boundaries of its support
- *Tri-cube* kernel: $k(x) \triangleq \frac{3}{4}(1-|x|^3)^3\mathbb{1}_{\{|x|\leq 1\}}$, having compact support and two continuous derivatives at the boundary of its support.

Kernel machines is a Generalized Linear Model where the input feature vector has the form $\phi(\boldsymbol{x}) = (k(\boldsymbol{x}, \boldsymbol{\mu}_k))_{1\leq k\leq K}$ where $\boldsymbol{\mu}_k$ being the kth centroid. We can use any of the sparsity-promoting priors for $\boldsymbol{w}$ to efficiently select a subset of the training exemplars, it is sparse vector machine. We can get even greater sparsity by using *ARD/SBL* resulting in a method called the Relevance Vector Machine (RVM).

Kernel trick Rather than defining our feature vector in terms of kernels, $\phi(\boldsymbol{x}) = (k(\boldsymbol{x}, \boldsymbol{x}_i))_{1\leq i\leq n}$, we can instead work with the original feature vectors $\boldsymbol{x}$ but modify algorithm so that we replace all inner product $\langle \boldsymbol{x} | \boldsymbol{x}' \rangle$ with a call to the kernel function $k(\boldsymbol{x}, \boldsymbol{x}')$.

Kernalized model

- *Nearest Neighbor (Classification)*: in using $\|\boldsymbol{x}_i - \boldsymbol{x}_{i'}\|_2^2 = \langle \boldsymbol{x}_i | \boldsymbol{x}_i \rangle + \langle \boldsymbol{x}_{i'} | \boldsymbol{x}_{i'} \rangle - 2\langle \boldsymbol{x}_i | \boldsymbol{x}_{i'} \rangle$
- *K-medoids (Clustering)*: similar to K-means, but instead of representing each cluster's centroid by the mean of all data vectors assigned to this cluster, we make each centroid be one of the data vectors themselves. When we update the centroids, we look at each object that belong to the cluster and measure the sum of its distance to all others in the same cluster with: $m_k = \arg\min_{i:z_i=k} \sum_{i':z_{i'}=k} \|\boldsymbol{x}_i - \boldsymbol{x}_{i'}\|_2^2$ where $z_i = \arg\min_k d(i, m_k)$
- *Ridge Regression*: using the matrix inversion lemme, $\boldsymbol{w}_{Ridge} = \boldsymbol{X}_T \left(\boldsymbol{X}\boldsymbol{X}^T + \lambda \boldsymbol{I}_n\right)^{-1} \boldsymbol{y}$, we can partially kernalize this by replacing $\boldsymbol{X}\boldsymbol{X}^T$ with the *Gram matrix*. Let's define **dual variables** $\boldsymbol{\alpha} \triangleq (\boldsymbol{K} + \lambda \boldsymbol{I}_n)\, \boldsymbol{y}$ then we rewrite the *primal variables* $\boldsymbol{w} = \boldsymbol{X}^T \alpha = \sum_{i=1}^{n} \alpha_i \boldsymbol{x}_i^T \boldsymbol{x}_i = \sum_{i=1}^{n} \alpha_i k(\boldsymbol{x}_i, \boldsymbol{x})$
- *PCA*: from the eigenvectors of the inner product matrix $\boldsymbol{X}\boldsymbol{X}^T$, allowing to produce a nonlinear embedding using kernel trick. The mathematical demonstration is long and not important.

Kernel Density Estimation (KDE) Instead of estimating $\boldsymbol{\mu}_k$ we can allocate one cluster center per data point so $\boldsymbol{\mu}_i = \boldsymbol{x}_i$ then the model becomes: $\mathbb{P}(\boldsymbol{x}|\mathcal{D}) = \frac{1}{n}\sum_{i=1}^{n}\mathcal{N}(\boldsymbol{x}|\boldsymbol{x}_i, \sigma^2\boldsymbol{I})$ That we can generalize with:

$$\hat{p}(\boldsymbol{x}) = \frac{1}{n}\sum_{i=1}^{n} k_h(\boldsymbol{x} - \boldsymbol{x}_i)$$

This is called *Parzen window density estimator* or *Kernel Density Estimator*.

Strengths

Weaknesses

Relationships with other methods

Examples of application

- comparing documents

10.2.3 Latent Dirichlet Allocation

Purpose

Assumptions

Theory

Strengths

Weaknesses

Relationships with other methods

Examples of application

10.3 Dimensionality Reduction

10.3.1 Principal Components Analysis

Purpose Consider the *FA* model where we constrain $\Phi = \sigma^2\boldsymbol{I}$ and $\boldsymbol{W}$ to be orthonormal. It can be shown that when σ^2 tends to 0, this model reduces to classical PCA, otherwise it will be *Probabilistic PCA*.

Assumptions

Theory Suppose we want to find an orthogonal set of L linear basis vectors $\boldsymbol{w}_j \in \mathbb{R}^D$ and the corresponding scores $\boldsymbol{z}_i \in \mathbb{R}^L$ such that we minimize the average reconstruction error:

$$\boldsymbol{J}(\boldsymbol{W}, \boldsymbol{Z}) = \frac{1}{n}\sum_{i=1}^{n} \|\boldsymbol{x}_i - \hat{\boldsymbol{x}}_i\|^2$$

where $\hat{\boldsymbol{x}}_i = \boldsymbol{W}\boldsymbol{z}_i$ subject to the constraint that $\boldsymbol{W}$ is orthonormal.

Model selection We can plot the *scree plot* : $E(\mathcal{D}_{train}, L)$ vs L, with E being the error reconstruction. A related quantity is the fraction of variance explained defined as $F(\mathcal{D}_{train}, L) = \frac{\sum_{j=1}^{L} \lambda_j}{\sum_{j'=1}^{L_{max}} \lambda_{j'}}$

Strengths

- *PCA* is the best low rank approximation to the data

Weaknesses

- It is not a proper generative model of the data, in providing more latent dimensions it will be able to better approximate the test data

Relationships with other methods

Examples of application

10.3.2 PCA for categorical data

Purpose Useful when data is categorical rather than real-valued

Assumptions

Theory Let $\boldsymbol{X} \in \mathcal{M}_{np}(\llbracket 1, c \rrbracket)$ and we assume that each x_{ij} is generated from a latent variable $\boldsymbol{z}_i \in \mathbb{R}^L$.

$$\begin{cases} \mathbb{P}(\boldsymbol{z}_i) = \mathcal{N}(\boldsymbol{0}, \boldsymbol{I}) \\ \mathbb{P}(\boldsymbol{x}_i | \boldsymbol{z}_i, \boldsymbol{\theta}) = \prod_{j=1}^{p} Cat\left(x_{ij} | \mathcal{S}\left(\boldsymbol{W}_j^T \boldsymbol{z}_i + \boldsymbol{w}_{0j}\right)\right) \end{cases}$$

where $\boldsymbol{W} \in \mathbb{R}^{L \times M}$ is the factor loading matrix for response j and $\boldsymbol{w}_{0j} \in \mathbb{R}^M$ is the offset term for response j. To fit the model in using a modified version of EM: infer a Gaussian approximation to the posterior $\mathbb{P}(\boldsymbol{z}_i | \boldsymbol{x}_i, \boldsymbol{\theta})$ in the E step and then to maximize $\boldsymbol{\theta} = (\boldsymbol{W}_j, \boldsymbol{w}_{0j})_{1 \leq j \leq p}$ in M step.

Strengths

Weaknesses

Relationships with other methods

Examples of application

Chapter 11

Semi-supervised Learning

Chapter 12

Reinforcement Learning

12.1 Markov Decision Processes

12.1.1 Setting

Purpose In supervised learning the correct output is given for each input. In reinforcement learning (RL) an agent interacts with an environment: at each step it observes a state, chooses an action, and receives a scalar reward. No one tells it the right action, it has to discover which sequences of actions give the most reward, and a reward may come long after the action that caused it (*credit assignment*).

Markov Decision Process An MDP is a tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$:

- $\mathcal{S}$ the states and $\mathcal{A}$ the actions;
- $P(s' \mid s, a) = \mathbb{P}(S_{t+1} = s' | S_t = s, A_t = a)$ the transition probabilities, with the Markov assumption: the future depends on the past only through (s, a);
- $R(s, a)$ the expected reward;
- $\gamma \in [0, 1[$ the discount factor.

It is a Markov chain whose transitions are controlled by the actions (see Markov models), and with a single action it reduces to a Markov chain.

Policy and return A **policy** $\pi(a \mid s)$ is the probability of choosing the action a in the state s. Following π generates a trajectory $\tau = (s_0, a_0, r_1, s_1, a_1, \dots)$. The goal is to maximize the expected discounted **return**

$$G_t = \sum_{k=0}^{\infty} \gamma^k \, r_{t+k+1}, \qquad J(\pi) = \mathbb{E}_{\tau \sim \pi}\left[G_0\right]$$

$\gamma < 1$ makes the sum finite and favors immediate rewards. Note that $G_t = r_{t+1} + \gamma G_{t+1}$.

12.1.2 Value Functions and Bellman Equations

Definitions

$$V^\pi(s) = \mathbb{E}_\pi\left[G_t \mid S_t = s\right], \qquad Q^\pi(s, a) = \mathbb{E}_\pi\left[G_t \mid S_t = s, A_t = a\right]$$

The state value V^π is the expected return from s, the action value Q^π is the expected return when taking a first and then following π. They are linked by $V^\pi(s) = \sum_a \pi(a|s) Q^\pi(s, a)$, and the **advantage** $A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$ tells how much better a is than the average action of π.

Bellman expectation equation Using $G_t = r_{t+1} + \gamma G_{t+1}$:

$$V^\pi(s) = \sum_a \pi(a|s) \left[R(s,a) + \gamma \sum_{s'} P(s'|s,a) V^\pi(s') \right]$$

The value of a state is the immediate reward plus the discounted value of the next state: a recursive definition, a linear system for a fixed π.

Bellman optimality equation The optimal value functions V^*, Q^* satisfy

$$Q^*(s,a) = R(s,a) + \gamma \sum_{s'} P(s'|s,a) \max_{a'} Q^*(s',a'), \qquad V^*(s) = \max_a Q^*(s,a)$$

and the optimal policy is greedy with respect to Q^*: $\pi^*(s) = \arg\max_a Q^*(s,a)$. The max makes the equation non-linear.

Dynamic programming If the model (P, R) is known, *value iteration* repeats the update below until convergence:

$$V(s) \leftarrow \max_a \left[R(s,a) + \gamma \sum_{s'} P(s'|s,a) V(s') \right]$$

In RL the model is usually unknown and the agent has to learn from sampled interactions, which is what the next methods do.

Exploration versus exploitation To learn, the agent must try actions that look suboptimal (*explore*), but to collect reward it must use what it knows (*exploit*). The simplest rule is ε-greedy: take a random action with probability ε, the greedy one otherwise.

12.2 Value-based Methods

12.2.1 Purpose

Learn V^π or Q^* from sampled experience (s, a, r, s'), and derive the policy from the values.

12.2.2 Monte Carlo and Temporal Difference

Monte Carlo Wait for the end of the episode, then move $V(s_t)$ towards the observed return G_t. Unbiased, but high variance, and it needs complete episodes.

Temporal difference (TD(0)) Update after each step, using the current estimate of the next state instead of the full return (*bootstrapping*):

$$V(s_t) \leftarrow V(s_t) + \alpha\, \delta_t, \qquad \delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$$

δ_t is the **TD error**. Compared to Monte Carlo, TD has lower variance and learns online, but is biased because $V(s_{t+1})$ is itself an estimate.

12.2.3 SARSA and Q-learning

- **SARSA** (on-policy) uses the action a_{t+1} actually chosen by the current policy:
  $Q(s_t,a_t) \leftarrow Q(s_t,a_t) + \alpha\,[r_{t+1} + \gamma Q(s_{t+1},a_{t+1}) - Q(s_t,a_t)]$
- **Q-learning** (off-policy) uses the best next action, whatever was done:

  $$Q(s_t,a_t) \leftarrow Q(s_t,a_t) + \alpha\,[r_{t+1} + \gamma \max_{a'} Q(s_{t+1},a') - Q(s_t,a_t)]$$

  It is the sampled version of the Bellman optimality equation, and it can learn the optimal policy while behaving with an ε-greedy one.

On-policy methods learn about the policy that generates the data, off-policy methods can learn from data generated by another policy.

Function approximation A table with one entry per (s, a) is impossible for large state spaces. **Deep Q-networks** (DQN) approximate $Q_{\boldsymbol{\theta}}(s, a)$ by a neural network, trained by minimizing the squared TD error. Two ingredients stabilize it: an *experience replay* buffer (random past transitions break the correlation between consecutive samples) and a slowly updated *target network* for the bootstrapped term.

Weaknesses

- they need a max or an arg max over the actions, which is impractical for huge or continuous action spaces (for a language model, an action is a token among $V \approx 10^5$, and a whole answer is a sequence of them);
- bootstrapping with function approximation and off-policy data can be unstable.

12.3 Policy Gradient Methods

12.3.1 Purpose

Instead of learning values and deriving a policy, parameterize the policy itself, $\pi_{\boldsymbol{\theta}}(a|s)$ (a neural network with a softmax output for discrete actions), and do gradient ascent on the expected return $J(\boldsymbol{\theta})$. It works with large or continuous action spaces, and the policy can be stochastic.

12.3.2 Policy Gradient Theorem

Log-derivative trick With $p_{\boldsymbol{\theta}}(\tau)$ the probability of a trajectory and $R(\tau)$ its return:

$$\nabla_{\boldsymbol{\theta}} J = \nabla_{\boldsymbol{\theta}} \int p_{\boldsymbol{\theta}}(\tau) R(\tau) d\tau = \int p_{\boldsymbol{\theta}}(\tau)\, \nabla_{\boldsymbol{\theta}} \log p_{\boldsymbol{\theta}}(\tau)\, R(\tau) d\tau = \mathbb{E}_{\tau \sim \pi_{\boldsymbol{\theta}}} \left[\nabla_{\boldsymbol{\theta}} \log p_{\boldsymbol{\theta}}(\tau)\, R(\tau) \right]$$

using $\nabla p = p \nabla \log p$. The trajectory probability factorizes as $p_{\boldsymbol{\theta}}(\tau) = p(s_0) \prod_t \pi_{\boldsymbol{\theta}}(a_t|s_t) P(s_{t+1}|s_t, a_t)$, and the unknown dynamics P do not depend on $\boldsymbol{\theta}$: their terms vanish in the gradient of the log, so the model is not needed.

REINFORCE

$$\nabla_{\boldsymbol{\theta}} J = \mathbb{E}_{\tau \sim \pi_{\boldsymbol{\theta}}} \left[\sum_t \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(a_t|s_t)\, G_t \right]$$

Here the full return is replaced by the *reward-to-go* G_t, since an action cannot change the rewards that came before it. It is estimated by sampling trajectories with the current policy, which makes the method on-policy.
Interpretation: it is the gradient of the log-likelihood of the sampled actions, weighted by the return. Good trajectories become more probable, bad ones less probable. With $G_t = 1$ for the data we want to imitate, it is exactly the maximum likelihood gradient of supervised learning.

12.3.3 Variance Reduction

Baseline The estimator is unbiased but very noisy. Subtracting a baseline $b(s_t)$ that does not depend on the action leaves the expectation unchanged:

$$\mathbb{E}_{a \sim \pi_{\boldsymbol{\theta}}} \left[\nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(a|s)\, b(s) \right] = b(s) \sum_a \nabla_{\boldsymbol{\theta}} \pi_{\boldsymbol{\theta}}(a|s) = b(s) \nabla_{\boldsymbol{\theta}} \sum_a \pi_{\boldsymbol{\theta}}(a|s) = b(s) \nabla_{\boldsymbol{\theta}} 1 = 0$$

The natural choice is $b(s) = V^\pi(s)$, and $G_t - V^\pi(s_t)$ estimates the **advantage** $A^\pi(s_t, a_t)$: the action is reinforced only if it is better than average.

Actor-critic Learn a second network, the **critic** $V_{\boldsymbol{\phi}}(s)$, by regression on the returns or by TD, and use it to compute the advantage for the **actor** $\pi_{\boldsymbol{\theta}}$:

$$\hat{A}_t = \delta_t = r_{t+1} + \gamma V_{\boldsymbol{\phi}}(s_{t+1}) - V_{\boldsymbol{\phi}}(s_t)$$

Generalized advantage estimation (GAE) averages the TD errors of several steps, $\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}$, and λ trades bias (small λ, like TD) against variance (large λ, like Monte Carlo).

Weaknesses

- high variance and sample inefficiency, as the data is used for one update then discarded;
- a step in parameter space can change the policy much more than expected, and a bad update can collapse performance: this motivates PPO.

12.4 Proximal Policy Optimization (PPO)

Purpose Policy gradient methods take noisy steps, and one step that is too large may ruin the policy, after which the data collected is bad too. PPO (Schulman et al., 2017) lets us reuse the same batch for several gradient steps while keeping the new policy close to the policy that collected the data, with a simple first-order objective.

Theory Let π_{old} be the policy that generated the batch, and the probability ratio

$$r_t(\boldsymbol{\theta}) = \frac{\pi_{\boldsymbol{\theta}}(a_t|s_t)}{\pi_{old}(a_t|s_t)}$$

(importance weight, $r_t = 1$ at the start). The clipped objective to maximize is

$$\boxed{L^{CLIP}(\boldsymbol{\theta}) = \mathbb{E}_t \left[\min \left(r_t(\boldsymbol{\theta})\hat{A}_t,\ \text{clip}\left(r_t(\boldsymbol{\theta}), 1-\varepsilon, 1+\varepsilon\right)\hat{A}_t \right) \right]}$$

with $\varepsilon \approx 0.2$.

- if $\hat{A}_t > 0$ (good action), the objective grows with r_t only up to $1+\varepsilon$: no incentive to make the action much more probable;
- if $\hat{A}_t < 0$ (bad action), it is the symmetric case, the gradient vanishes once $r_t < 1-\varepsilon$;
- the min makes the objective a pessimistic bound: the clipping only removes the incentive to move further, it never hides a harmful move.

Algorithm

1. collect trajectories with π_{old} and compute the advantages $\hat{A}_t$ (critic and GAE);
2. for several epochs, on mini-batches, maximize $L^{CLIP} - c_1 L^{value} + c_2 H[\pi_{\boldsymbol{\theta}}]$, with L^{value} the squared error of the critic and H an entropy bonus that keeps some exploration;
3. set $\pi_{old} \leftarrow \pi_{\boldsymbol{\theta}}$ and repeat.

Strengths and weaknesses

- simple, stable, and the default policy-gradient algorithm for a decade, including the training of language models from human feedback;
- it needs a critic (a second network of the size of the policy for LLMs) and several hyper-parameters, and it remains sensitive to implementation details.

From RL to language models A language model is a policy: the *state* is the prompt and the tokens generated so far, the *action* is the next token, the episode ends at the end-of-text token, and a reward arrives for the complete answer (see the alignment chapter).

Chapter 13

Optimization Methods

13.1 Optimization of Loss Functions

13.1.1 Gradient Descent

Purpose Simplest algorithm for unconstrained optimization.

Theory

Principle

$$\boxed{\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta_k \boldsymbol{g}_k}$$

where η_k is the *step size* or *learning rate*. For a given function to optimize $f : \mathbb{R}^p \longrightarrow \mathbb{R}$ being 2 times derivable with *Taylor's theorem*: $f(\boldsymbol{\theta} + \eta \boldsymbol{d}) \approx f(\boldsymbol{\theta}) + \eta \boldsymbol{g}^T \boldsymbol{d}$ where $\boldsymbol{d}$ is our descent directions.
If η is chosen small enough then $f(\boldsymbol{\theta} + \eta \boldsymbol{d}) < f(\boldsymbol{\theta})$ since the gradient will be negative. But we don't want to choose the step size η to small, so let's pick η to minimize $\phi(\eta) = f(\boldsymbol{\theta}_k + \eta \boldsymbol{d}_k)$ this is called *line minimization* or *line search*

Zig-zag behavior exact line search satisfies $\eta_k = \underset{\eta > 0}{\arg\min}\, \phi(\eta)$. A necessary condition for the optimum is $\phi'(\eta) = 0$, by the chain rule $\phi'(\eta) = \boldsymbol{g}^T \boldsymbol{d}$ where $\boldsymbol{g} = f'(\boldsymbol{\theta}, \eta \boldsymbol{d})$ is the gradient at the end of the step. So we either have $\boldsymbol{g} = \boldsymbol{0}$ meaning that we found a stationary point or $\boldsymbol{g} \perp \boldsymbol{d}$ meaning that exact search stops at a point where the local gradient is perpendicular to the search direction.

Weaknesses

- setting the *step size*: too large steps can induce failing to converge, too small can as well fail to converge because of too long time

13.1.2 Newton's method

Purpose Takes the curvature of the space, the Hessian matrix, into account, it is belong to the family of *second order* optimization methods

Theory It consists to update:

$$\boxed{\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta_k \boldsymbol{H}_k^{-1} \boldsymbol{g}_k}$$

Consider making a second-order Taylor series approximation of $f(\boldsymbol{\theta})$ around $\boldsymbol{\theta}_k$: $f_{quad}(\boldsymbol{\theta}) = f_k + \boldsymbol{g}_k^T(\boldsymbol{\theta} - \boldsymbol{\theta}_k) + \frac{1}{2}(\boldsymbol{\theta} - \boldsymbol{\theta}_k)\boldsymbol{H}_k(\boldsymbol{\theta} - \boldsymbol{\theta}_k)$.
That can be rewrite as

$$f_{quad}(\boldsymbol{\theta}) = \boldsymbol{\theta}^T \boldsymbol{A} \boldsymbol{\theta} + \boldsymbol{b}^T \boldsymbol{\theta} + c \begin{cases} \boldsymbol{A} & = \frac{1}{2}\boldsymbol{H}_k \\ \boldsymbol{b} & = \boldsymbol{g}_k - \boldsymbol{H}_k \boldsymbol{\theta}_k \\ \boldsymbol{c} & = f_k - \boldsymbol{g}_k^T \boldsymbol{\theta}_k + \frac{1}{2}\boldsymbol{\theta}_k^T \boldsymbol{H}_k \boldsymbol{\theta}_k \end{cases}$$

The minimum of f_{quad} is at $\boldsymbol{\theta} = -\frac{1}{2}\boldsymbol{A}^{-1}\boldsymbol{b} = \boldsymbol{\theta}_k - \boldsymbol{H}_k^{-1}\boldsymbol{g}_k$ Thus the Newton step $\boldsymbol{d}_k = -\boldsymbol{H}_k^{-1}\boldsymbol{g}_k$ is what should be added to $\boldsymbol{\theta}_k$ to minimize the second order approximation of around $\boldsymbol{\theta}_k$

Strengths

- Faster than usual gradient descent

13.1.3 Online learning and stochastic optimization

Purpose Instead of of minimizing regret with respect to the past, we want to minimize expected loss in the future.

Theory $f(\boldsymbol{\theta}) = \mathbb{E}\left(f(\boldsymbol{\theta}, \boldsymbol{z})\right)$ where the expectation is taken over future data.
One way to optimize this *stochastic objective* such as the above equation, is to perform the update $\boldsymbol{\theta}_{k+1} = proj_{\Theta}\left(\boldsymbol{\theta}_k - \eta_k \boldsymbol{g}_k\right)$ where $proj_{\mathcal{V}}(\boldsymbol{v}) = \underset{\boldsymbol{w}\in\mathcal{V}}{\arg\min} \|\boldsymbol{w} - \boldsymbol{v}\|^2$ that is the projection of vector $\boldsymbol{v}$ onto space $\mathcal{V}$.
In practice it is usually better to randomly permute the data and sample without replacement, and then to repeat. A single such pass over the entire data set is called and ***epoch***.
In this offline cases, it is often better to compute the gradient of a *mini-batch* of B data cases. If $B = 1$ this is standard Stochastic Gradient Descent, and if $B = n$ this is standard Gradient Descent typically $B \approx 100$ is used.

Strengths

- online learning, useful for streaming data
- main way to train large model on large dataset
- as the number of examples in the training set approaches infinity the model will eventually converge to its best best possible test errors before SGD has sampled every example in the training set.

13.1.4 EM Algorithm

Purpose Gradient-based optimizer used to find a local minimum of the *Negative Log Likelihood* can be stuck with the imposed constraint like positive definite covariance matrices, mixing weights having to sum to one.
In a few words *Expectation Maximization (EM)* which alternates between inferring the missing values given the parameters (*E step*), and then optimizing the parameters given the filled in data (M step).
The goal is to maximize the log likelihood of the observed data:

$$l(\theta) = \sum_{i=1}^{n} \log(\mathbb{P}\left(\boldsymbol{x}_i|\boldsymbol{\theta}\right)) = \sum_{i=1}^{n} \log\left(\sum_{z_i} \mathbb{P}\left(\boldsymbol{x}_i, \boldsymbol{z}_i|\boldsymbol{\theta}\right)\right)$$

since the log cannot be pushed inside the sum, it is difficult to optimize, EM gets around this problem in defining the *complete data log likelihood* to be $l_c(\boldsymbol{\theta}) \triangleq \sum_{i=1}^{n} \log\left(\mathbb{P}\left(\boldsymbol{x}_i, \boldsymbol{z}_i|\boldsymbol{\theta}\right)\right)$. But it cannot be computed since $\boldsymbol{z}_i$ is unknown. So let's define the **expected complete data log likelihood**:

$$Q(\boldsymbol{\theta}, \boldsymbol{\theta}^{t-1}) = \mathbb{E}\left(l_c(\boldsymbol{\theta})|\mathcal{D}, \boldsymbol{\theta}^{t-1}\right)$$

where t is the current iteration number, Q is called the **auxiliary function**. The goal of the *E step* is to compute $Q(\boldsymbol{\theta}, \boldsymbol{\theta}^{t-1})$, in the *M step* we optimize the Q function.

Assumptions

Theory

Strengths

Weaknesses

Relationships with other methods

Examples of application

13.1.5 Backpropagation Algorithm

Purpose Useful for Neural Network's loss function is non-convex function of its parameters. It is common to use first-order online methods such as stochastic gradient descent.

Assumptions

Theory Let's assume a model with just one hidden layer, it is helpful to distinguish the pre- and post-synaptic values of a neuron, that is before and after applying the non-linearity.
Then $\boldsymbol{x}_n$, the nth input, $\boldsymbol{a}_n = \boldsymbol{V}\boldsymbol{x}_n$ be the pre-synaptic hidden layer and $\boldsymbol{z}_n = g(\boldsymbol{a}_n)$ be the pro-synaptic hidden layer where g is some transfer function. More succinctly:

$$\boldsymbol{x}_n \xrightarrow{\boldsymbol{V}} \boldsymbol{a}_n \xrightarrow{g} \boldsymbol{z}_n \xrightarrow{\boldsymbol{W}} \boldsymbol{b}_n \xrightarrow{h} \hat{\boldsymbol{y}}$$

$\boldsymbol{\theta} = (\boldsymbol{V}, \boldsymbol{W})$ the first and second layer weight matrices.
With K outputs the negative linear loss is given by

- *regression*: $J(\boldsymbol{\theta}) = -\sum_n \sum_k (\hat{y}_{nk}(\boldsymbol{\theta}) - y_{nk})$
- *classification*: $J(\boldsymbol{\theta}) = -\sum_n \sum_k y_{nk} \log(\hat{y}_{nk}(\boldsymbol{\theta}))$

Our task is to compute $\Delta_{\boldsymbol{\theta}} J$, we will derive this for each n separately, the overall gradient is obtained by summing over n

1. *output layer*: $\nabla_{\boldsymbol{w}_k} J_n = \dfrac{\partial J_n}{\partial b_{nk}} \nabla_{\boldsymbol{w}_k} = \dfrac{\partial J_n}{\partial b_{nk}} \boldsymbol{z}_n$ since $b_{nk} = \boldsymbol{w}_k^T \boldsymbol{z}_n$
2. considering J_n as a squared penalty: $\dfrac{\partial J_n}{\partial b_{nk}} \triangleq \sigma_{nk}^w = (\hat{y}_{nk} - y_{nk})$ being the error signal, so the overall gradient $\nabla_{\boldsymbol{w}_k} J_n = \sigma_{nk}^w \boldsymbol{z}_n$
3. *input layer*: $\nabla_{v_j} J_n = \dfrac{\partial J_n}{\partial a_{nj}} \nabla_{v_j} a_{nj} \triangleq \delta_{nj}^v \boldsymbol{x}_n$

Strengths

Weaknesses

Relationships with other methods

Examples of application

13.2 Variational Inference

13.2.1 Basics

Suppose $p^*(\boldsymbol{x})$ is our true but intractable distribution and $q(\boldsymbol{x})$ is some approximation chosen from some tractable family, such as a multivariate Gaussian.
In using:

$$\mathbb{KL}(p^* || q) = \sum_{\boldsymbol{x}} q(\boldsymbol{x}) \log\left(\frac{q(\boldsymbol{X})}{p^*(\boldsymbol{x})}\right)$$

Mean field method We assume the posterior is fully factorized approximation of the form: $q(\boldsymbol{x}) = \prod_i q_i(\boldsymbol{x}_i)$ The goal is to solve the optimization problem $\min \mathbb{KL}(q||p)$

13.2.2 Variational Bayes

When we want to infer the parameters themselves, if we make a fully factorized approximation $\mathbb{P}(\boldsymbol{\theta}|\mathcal{D}) \approx \prod_k q(\boldsymbol{\theta}_k)$

13.2.3 Variational Bayes EM

TO COMPLETE

The proposal distribution

13.2.4 Gibbs sampling

TO COMPLETE

13.3 Numerical Computation

13.3.1 Numerical Computation

Poor conditioning Conditioning refers to how rapidly a function changes with respect to small changes in its inputs.

$$\max_{i,j} |\frac{\lambda_i}{\lambda_j}|$$

13.3.2 Jacobian and Hessian matrices

Jacobian If we have a function $f : \mathbb{R}^m \to \mathbb{R}^n$, then the Jocobian matrix $\boldsymbol{J} \in \mathbb{R}^{n \times m}$ such that

$$J_{i,j} = \frac{\partial}{\partial x_j} f(\boldsymbol{x}_i)$$

Hessian

13.3.3 Optimization technique

Gradient-Based Optimization

Part IV

Deep Learning

Chapter 14

Optimization

14.1 Back-propagation

14.1.1 Purpose

The back-propagation algorithm allows the information from the cost to then flow backwards through the network in order to compute the gradient.
Actually it refers only to the method for computing the gradient, while another algorithm such as stochastic gradient descent is used to perform learning using this gradient.

14.1.2 Modeling

Chain rule of calculus Let $(\boldsymbol{x}, \boldsymbol{y}) \in \mathbb{R}^m \times \mathbb{R}^n$ and $z = f(\boldsymbol{y})$ with $f : \mathbb{R}^n \longrightarrow \mathbb{R}$ we have $\nabla_{\boldsymbol{x}} z = \begin{bmatrix} \frac{dz}{dx_1} \\ \vdots \\ \frac{dz}{dx_m} \end{bmatrix}$,

$\nabla_{\boldsymbol{y}} z = \begin{bmatrix} \frac{dz}{dy_1} \\ \vdots \\ \frac{dz}{dy_n} \end{bmatrix}$ and $\frac{\partial \boldsymbol{y}}{\partial \boldsymbol{x}} = \begin{bmatrix} \frac{dy_1}{x_1} & \dots & \frac{dy_1}{x_m} \\ \vdots & \ddots & \vdots \\ \frac{dy_n}{x_1} & \dots & \frac{dy_n}{x_m} \end{bmatrix}$ we have :

$$\nabla_{\boldsymbol{x}} z = \left(\frac{\partial \boldsymbol{y}}{\partial \boldsymbol{x}} \right)^T \nabla_{\boldsymbol{y}} z$$

Back-propagation in fully connected MLP Consider a network of depth l, and the matrices of the model $(\boldsymbol{W}_j)_{1 \leq j \leq l}$ and the bias parameters $(\boldsymbol{b}_j)_{1 \leq j \leq l}$ then $\boldsymbol{x}$ the input vector and $\boldsymbol{y}$ the target vector.

Forward propagation Consider a $l-1$ hidden layer model, assume that we use an unique activation function f:

1. $\boldsymbol{h}_0 = \boldsymbol{x}$ *consider the input as the first layer*
2. for $k \in [\![1, l]\!]$
   - $\boldsymbol{a}_k = \boldsymbol{b}_k + \boldsymbol{W}_k \boldsymbol{h}_{k-1}$ *vector containing all the resulting weighted sum and bias of inputs* k
   - $\boldsymbol{h}_k = f(\boldsymbol{a}_k)$ *vector containing all the neuron outputs of the considered layer by applying the activation function*
3. $\hat{\boldsymbol{y}} = \boldsymbol{h}_l$ *output layer*
4. $J = L(\hat{\boldsymbol{y}}, \boldsymbol{y}) + \lambda \Omega(\theta)$ *finally compute loss function with a potential regulation term*

Backward

1. $\boldsymbol{g} \leftarrow \nabla_{\hat{\boldsymbol{y}}} J = \nabla_{\hat{\boldsymbol{y}}} L(\hat{\boldsymbol{y}}, \boldsymbol{y})$ *initial gradient vector*
2. for $k \in [\![l, 1]\!]$ (from the output layer l down to the first layer)
   - $\boldsymbol{g} \leftarrow \nabla_{\boldsymbol{a}_k} J = \boldsymbol{g} \odot f'(\boldsymbol{a}_k)$ *compute derivative with respect to* $\boldsymbol{a}_k$*, Hadamard product is used to imitate the rule:* $(h \circ f)(x)' = h'(f(x)) f'(x)$
   - $\nabla_{\boldsymbol{b}_k} J = \boldsymbol{g} + \lambda \nabla_{\boldsymbol{b}_k} \Omega(\boldsymbol{\theta})$
   - $\nabla_{\boldsymbol{W}_k} J = \boldsymbol{g} \boldsymbol{h}_{k-1}^T + \lambda \nabla_{\boldsymbol{W}_k} \Omega(\boldsymbol{\theta})$ *outer product of the error signal* $\boldsymbol{g}$ *and the layer input* $\boldsymbol{h}_{k-1}$
   - $\boldsymbol{g} \leftarrow \nabla_{\boldsymbol{h}_{k-1}} J = \boldsymbol{W}_k^T \boldsymbol{g}$ *propagate the gradient to the previous layer*

14.2 Optimizers

14.2.1 Purpose

Back-propagation only computes the gradient $\boldsymbol{g}_t = \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_t)$ on a mini-batch. The optimizer decides how to use it to update the parameters. Plain stochastic gradient descent (see the optimization methods of the classical learning part) is

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \, \boldsymbol{g}_t$$

with η the learning rate. Its weaknesses: a single learning rate for all the parameters, slow progress in ravines (steep in one direction, flat in another), and noisy mini-batch gradients.

14.2.2 Momentum

Keep an exponential moving average of the past gradients, which damps the oscillations and accelerates along consistent directions:

$$\boldsymbol{v}_t = \mu \boldsymbol{v}_{t-1} + \boldsymbol{g}_t, \qquad \boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \, \boldsymbol{v}_t \qquad (\mu = 0.9 \text{ typically})$$

14.2.3 Adaptive learning rates: RMSProp and Adam

RMSProp divides the step of each parameter by a running average of its recent squared gradients, so that parameters with large gradients take smaller steps and parameters with small gradients take larger ones: $\boldsymbol{s}_t = \beta \boldsymbol{s}_{t-1} + (1-\beta) \boldsymbol{g}_t^2$, then $\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \, \boldsymbol{g}_t / \left(\sqrt{\boldsymbol{s}_t} + \varepsilon\right)$ (all operations are element-wise).

Adam combines momentum (first moment) and RMSProp (second moment):

$$\begin{aligned}
\boldsymbol{m}_t &= \beta_1 \boldsymbol{m}_{t-1} + (1-\beta_1) \boldsymbol{g}_t \\
\boldsymbol{v}_t &= \beta_2 \boldsymbol{v}_{t-1} + (1-\beta_2) \boldsymbol{g}_t^2 \\
\hat{\boldsymbol{m}}_t &= \frac{\boldsymbol{m}_t}{1-\beta_1^t}, \qquad \hat{\boldsymbol{v}}_t = \frac{\boldsymbol{v}_t}{1-\beta_2^t} \\
\boldsymbol{\theta}_{t+1} &= \boldsymbol{\theta}_t - \eta \, \frac{\hat{\boldsymbol{m}}_t}{\sqrt{\hat{\boldsymbol{v}}_t} + \varepsilon}
\end{aligned}$$

Defaults: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$. The moments are initialized at $\boldsymbol{0}$, so they are biased towards zero in the first steps: the bias correction (division by $1-\beta^t$) fixes it. Adam is the usual choice for Transformers (the original paper uses $\beta_2 = 0.98$ instead of 0.999).

AdamW With SGD, adding the L^2 penalty $\frac{\lambda}{2} \|\boldsymbol{\theta}\|^2$ to the loss is the same as weight decay. With Adam it is not, because the penalty gradient $\lambda \boldsymbol{\theta}$ goes through the adaptive rescaling. AdamW decouples the decay from the gradient step:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \left(\frac{\hat{\boldsymbol{m}}_t}{\sqrt{\hat{\boldsymbol{v}}_t} + \varepsilon} + \lambda \boldsymbol{\theta}_t \right)$$

It is the standard optimizer for training LLMs.

14.2.4 Learning-rate schedules

The learning rate is not kept constant:

- **warmup**: increase linearly from 0 during the first steps. At the beginning the second-moment estimates of Adam are unreliable and the gradients are large, so full-size steps can destabilize training;
- **decay** afterwards: inverse square root (original Transformer), or *cosine* decay down to a small fraction of the peak value (common for LLMs).

14.2.5 Gradient clipping

Against exploding gradients (see the long-term dependencies of recurrent networks), the gradient is rescaled when its norm exceeds a threshold c:

$$\boldsymbol{g} \leftarrow \boldsymbol{g} \cdot \min\left(1, \frac{c}{\|\boldsymbol{g}\|_2}\right)$$

The direction is kept, only the step size is limited. A threshold $c = 1$ is typical for LLMs.

Chapter 15

Regularization

15.1 Norm Penalties

We denote the regularized objective function by:

$$\tilde{J}(\boldsymbol{\theta}, \boldsymbol{X}, \boldsymbol{y}) = J(\boldsymbol{\theta}, \boldsymbol{X}, \boldsymbol{y}) + \alpha\Omega(\boldsymbol{\theta}) \text{ with}$$
$$\begin{cases} \alpha \in [0, \infty[\text{ hyper-parameters weighting the impact of norm penalty} \\ \Omega \text{ relative to standard objective function } J \end{cases}$$

15.1.1 L^2 parameter regularization

Purpose : This penalty is as well knows as weight decay, it allows to drives the weight closer to the origin.

Theory $\Omega(\boldsymbol{\theta}) = \frac{1}{2}\|\boldsymbol{w}\|_2^2$
Then $\nabla_{\boldsymbol{w}}\tilde{J} = \alpha\boldsymbol{w} + \nabla_{\boldsymbol{w}}J$ and a gradient step of size ε becomes

$$\boldsymbol{w} \leftarrow (1 - \varepsilon\alpha)\,\boldsymbol{w} - \varepsilon\nabla_{\boldsymbol{w}}J$$

that is, the weights are shrunk by a constant factor before each usual update. Only the weights are penalized, the biases usually are not.
This is the MAP estimate with a Gaussian prior $\mathcal{N}\left(\mathbf{0}, \frac{1}{\alpha}\boldsymbol{I}\right)$ on the weights.

15.1.2 L^1 parameter regularization

Purpose : it yields sparse solutions, many weights being driven exactly to zero.

Theory $\Omega(\boldsymbol{\theta}) = \|\boldsymbol{w}\|_1 = \sum_i |w_i|$, so that $\nabla_{\boldsymbol{w}}\tilde{J} = \alpha\,\text{sign}(\boldsymbol{w}) + \nabla_{\boldsymbol{w}}J$: the penalty gradient has a constant magnitude, whatever the size of the weight.
This is the MAP estimate with a Laplace prior on the weights.

15.2 Data Augmentation

Purpose The best way to make a model generalize better is to train it on more data. When data is limited, create fake data by transforming the existing training examples.

Theory Apply to each training input $\boldsymbol{x}$ a transformation T that does not change its label y, and add the pair $(T(\boldsymbol{x}), y)$ to the training set.

- **Images**: translations, flips, crops, small rotations, color jitter.
- **Text**: synonym replacement, back-translation (translate to another language and back).
- **Any input**: injecting random noise (see noise robustness).

Weaknesses The transformation must be label-preserving for the task: rotating a "6" by 180° gives a "9", and flipping "b" gives "d". Augmentation is applied to the training set only, never to the validation or test sets.

15.3 Early Stopping

Purpose With a model large enough to overfit, the training error keeps decreasing while the validation error eventually starts to rise again. Stop training when the validation error stops improving and return the parameters of the best epoch.

Theory

1. evaluate the validation error every few steps;
2. keep a copy of the parameters each time it improves;
3. stop after patience evaluations without improvement and return the saved parameters.

The number of training steps is treated as a hyper-parameter chosen on the validation set. For a quadratic objective with gradient descent, it can be shown to be equivalent to L^2 regularization, the number of steps playing the role of $\frac{1}{\alpha}$.

Strengths Very cheap, requires no change to the model or to the objective.

Weaknesses Requires held-out data. Once the number of steps is known, the model can be retrained on the training and validation sets together for the same number of steps.

15.4 Drop out

Purpose It provides an inexpensive approximation to training and evaluating a *bagged ensemble* of exponentially many neural networks.

Theory It trains the ensemble consisting of **all sub-networks that can be formed by removing non-output units** from an underlying base network.
At each training step, every unit of a layer is kept independently with probability p (*keep probability*): with a mask $\boldsymbol{\mu}$ whose entries are i.i.d. $\mathcal{B}(p)$, the layer output becomes $\tilde{\boldsymbol{h}} = \boldsymbol{\mu} \odot \boldsymbol{h}$. Typical values are $p = 0.5$ for hidden units and $p = 0.8$ for input units.

Inference At test time all units are kept, so the expected input of the next layer would be $\frac{1}{p}$ times larger than during training. Two equivalent fixes:

- **weight scaling inference rule**: multiply the weights by p at test time;
- **inverted dropout** (the common implementation): divide the kept activations by p during training, and leave the network unchanged at test time.

Differences with bagging In bagging the models are independent and trained to convergence. With dropout the sub-networks share parameters and each one is trained for a single step, if it is even sampled. Dropout is also used in Transformers, where it is applied to the output of each sub-layer before the residual connection (original paper: $P_{drop} = 0.1$).

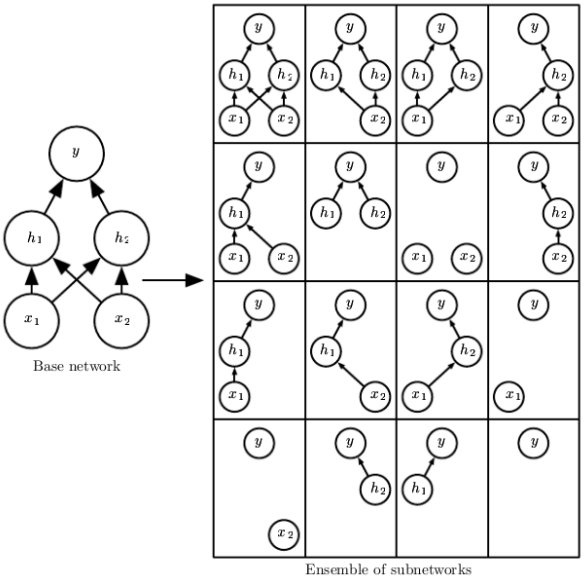


Figure 15.1: Drop out illustration

Chapter 16

Types of Neural Networks

16.1 Feed Forward Neural Networks

Purpose A feed-forward network (or *multi-layer perceptron*, MLP) defines a mapping $\boldsymbol{y} = f(\boldsymbol{x}; \boldsymbol{\theta})$ and learns the parameters $\boldsymbol{\theta}$ that give the best approximation of an unknown function f^*. Information flows from the input to the output with no feedback connection, otherwise the network would be recurrent.

Theory The network is a composition of l layers, $f = f_l \circ \cdots \circ f_1$, each one being an affine map followed by a non-linear activation function g applied element-wise:

$$\boldsymbol{h}_k = g\left(\boldsymbol{W}_k \boldsymbol{h}_{k-1} + \boldsymbol{b}_k\right), \quad \boldsymbol{h}_0 = \boldsymbol{x}, \quad \hat{\boldsymbol{y}} = \boldsymbol{h}_l$$

Without the non-linearity, the composition of affine maps collapses into a single affine map, so depth would bring nothing.

Universal approximation theorem A feed-forward network with one hidden layer, a non-polynomial activation function and enough hidden units can approximate any continuous function on a compact set (Cybenko 1989, Hornik 1991). The theorem says nothing about how many units are needed, nor about whether gradient descent can find the right parameters.

Activation functions

Name	$g(x)$	Remark
Sigmoid	$\sigma(x) = \dfrac{1}{1+e^{-x}}$	saturates, $\sigma' \leq \frac{1}{4}$: gradients vanish
Tanh	$\tanh(x)$	zero-centered, but also saturates
ReLU	$\max(0, x)$	gradient 0 or 1, cheap, units can "die"
GELU	$x\,\Phi(x)$	smooth ReLU, used in GPT and BERT

(Φ is the cdf of the standard normal distribution.) Saturating activations multiply gradients by factors smaller than one at each layer in the backward pass, which is the same phenomenon as in the long-term dependencies of recurrent networks.

Output layer and loss The last layer and the loss are chosen from the probabilistic model of the target, the loss being the negative log-likelihood:

Task	Output layer	Loss
Regression	linear	squared error (Gaussian likelihood)
Binary classification	sigmoid	binary cross-entropy (Bernoulli)
Multi-class classification	softmax	cross-entropy (categorical)

Weaknesses

- no parameter sharing: the number of parameters grows with the input size;
- fixed input size and no notion of order or locality, which motivates convolutional networks (grids) and recurrent networks or Transformers (sequences).

Relationships with other methods

- with no hidden layer, a sigmoid (resp. softmax) output gives logistic regression;
- the position-wise feed-forward sub-layer of the Transformer is a two-layer MLP applied independently to each token.

16.2 Convolutional Neural Networks

16.2.1 Components of CNN

There are neural networks using *convolution* operation instead of general matrix multiplication in at least on of their layers.

1. Apply several convolution in parallel to produce a set of activation functions
2. The previous activation functions are run through nonlinear activation function (*detector stage*)
3. Modify the output of the layer further with **pooling functions**

Convolution

Convolution operation allows to reduce the noise coming from observation by considering multiple observations and averaging them. However we want to have the ability to give more weight to some observations, the more recent ones for example. Let's assume we observe a signal $x(t)$ (*input*) and we have a weighting function $w(a)$(*kernel*) where a is the age of a measurement the resulting signal issued from convolution (*feature map*) is:

$$s(t) = (x * w)(t) = \int x(a)w(t-a)da$$

or in matrix convention

$$\boldsymbol{S}(n,p) = (\boldsymbol{I} * \boldsymbol{K})(n,p) = \sum_i \sum_j \boldsymbol{I}(i,j)\boldsymbol{K}(n-i,p-j) = \sum_i \sum_j \boldsymbol{I}(n-i,p-j)\boldsymbol{K}(i,j) = (\boldsymbol{K} * \boldsymbol{I})(n,p)$$

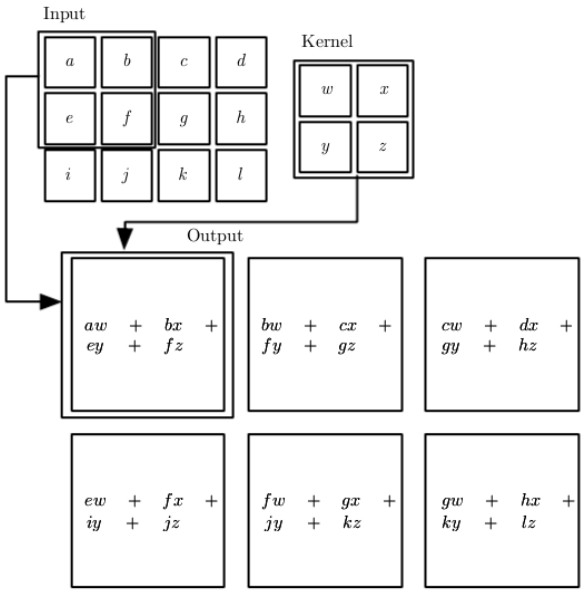


Figure 16.1: 2-D convolution without kernel-flipping

Motivation

- **sparse interactions** (due to reduced size of kernel compare with the input one)
- **parameter sharing** (rather than learning a separate set of parameters, for every location we learn only one set, that are the kernel's ones)
- **equi-variant representations**: $f(x)$ is equi-variant to a function g if $f(g(x)) = g(f(x))$. For example if g is any function that translates the input then the convolution is equi-variant to g
  - for time series, convolution produces a sort of timeline showing when different features appear in the input.
    By moving an event later in time in the input will induce that the same representation of the event will appear in the output
  - for images, convolution creates a 2-D map of where certain features appear in the input.
    By moving the object in the input, its representation will move the same amount in the output
- **ability to work with inputs of variable size**

Pooling

Purpose of pooling is to make representation become approximately *invariant* to small translations of the input. Such a priority can be very useful when we are searching for the presence of some feature rather than their location.
It can be viewed as adding an infinitely strong prior to the weight resulting in layer invariance to small translations

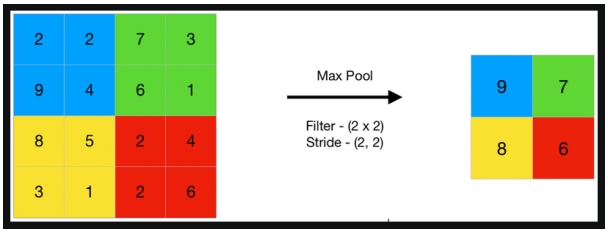


Figure 16.2: Max pooling

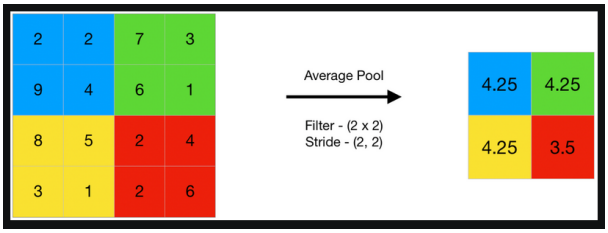


Figure 16.3: Average pooling

Examples of pooling

16.3 Recurrent and Recursive Neural Networks

16.3.1 Recurrent Neural Networks

Architecture Let's consider a *computational graph* to compute the training loss of a recurrent neural network that maps an input sequence of $\boldsymbol{x}$ values to a corresponding sequence of output $\boldsymbol{o}$ values. A loss $\boldsymbol{L}$ measures how far each $\boldsymbol{o}$ is from the corresponding training target $\boldsymbol{y}$

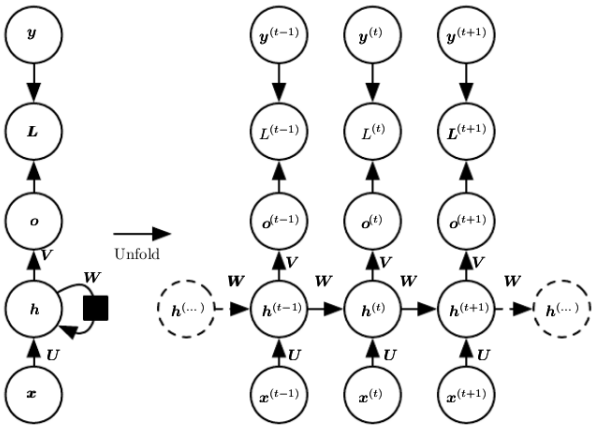


Figure 16.4: Recurrent Neural Networks

Theory With input-to-hidden weights $\boldsymbol{U}$, hidden-to-hidden weights $\boldsymbol{W}$ and hidden-to-output weights $\boldsymbol{V}$ (as in Figure 16.4), at each time step t:

$$\boldsymbol{h}^{(t)} = \tanh\left(\boldsymbol{b} + \boldsymbol{W}\boldsymbol{h}^{(t-1)} + \boldsymbol{U}\boldsymbol{x}^{(t)}\right), \qquad \boldsymbol{o}^{(t)} = \boldsymbol{c} + \boldsymbol{V}\boldsymbol{h}^{(t)}, \qquad \hat{\boldsymbol{y}}^{(t)} = \text{softmax}\left(\boldsymbol{o}^{(t)}\right)$$

The total loss is the sum of the losses over time steps. The same parameters $\boldsymbol{U}, \boldsymbol{V}, \boldsymbol{W}$ are shared across all time steps, which allows to process sequences of any length. The hidden state $\boldsymbol{h}^{(t)}$ is a lossy summary of the past inputs $\boldsymbol{x}^{(1:t)}$.
Training applies back-propagation to the graph unrolled over time (*back-propagation through time*, BPTT): the gradient with respect to a shared parameter is the sum of its contributions at each time step. As the computation of $\boldsymbol{h}^{(t)}$ needs $\boldsymbol{h}^{(t-1)}$, it is inherently sequential and cannot be parallelized over time.

16.3.2 Encoder-Decoder Sequence-to-Sequence Architectures

It is composed of an encoder RNN that reads the input sequence and a decoder RNN that generates the output sequence.

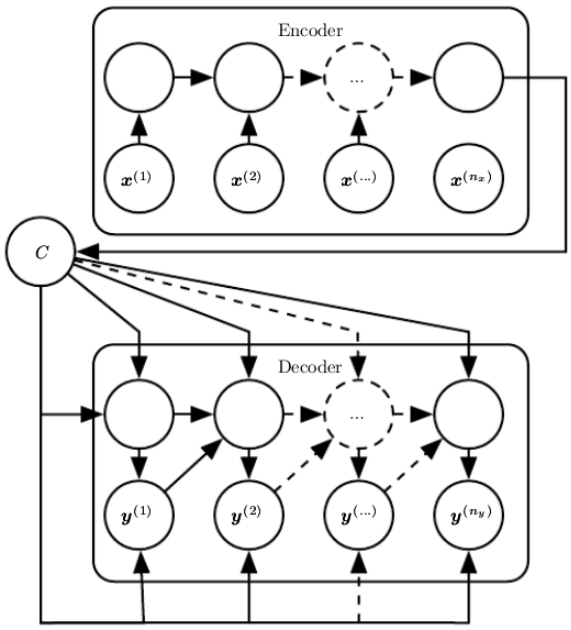


Figure 16.5: Encoder-Decoder (or Sequence-to-Sequence RNN) architecture

The final hidden state of the encoder RNN is used to compute a generally fixed-size context variable C representing a semantic summary of the input sequence, then is given as input to the decoder RNN. Only C is given to the decoder, the intermediate encoder states being discarded: this fixed-size vector is a bottleneck, information is lost for long sequences and the decoder cannot look back at specific input words. This is the motivation of the attention mechanism (see Transformer networks).

16.3.3 The Challenge of Long-Term Dependencies

The basic problem is that gradients propagated over many stages tend to either vanish or explode. Consider a very simple recurrent neural network network lacking a nonlinear function: $\boldsymbol{h}_t = \boldsymbol{W}^T \boldsymbol{h}_{t-1}$, if we admits an eigen-decomposition of the form $\boldsymbol{W} = \boldsymbol{Q}\boldsymbol{\Lambda}\boldsymbol{Q}^T$ with orthogonal $\boldsymbol{Q}$ the recurrence may be simplified further to $\boldsymbol{h}_t = \boldsymbol{Q}\boldsymbol{\Lambda}^t\boldsymbol{Q}^T\boldsymbol{h}_0$. The eigenvalues are raised to the power of t causing the eigenvalues with magnitude lesser than one to decay to zero and eigenvalues with magnitude greater than one to explode.
The same repeated product appears in the backward pass of BPTT, where the gradient is multiplied at each step by $\boldsymbol{W}^T$ and by the derivative of the activation function.

16.3.4 Leaky Units

Purpose dealing with long-term dependencies by designing a model operating at multiple time scales, allowing to let some parts of the model operating at fine-grained time scales to handle small details while other parts operate at coarse time scales and transfer information from the distant past to the present more efficiently.

Adding Skip Connections through Time consists in adding direct connections from variables in the distant past to variables in the present.
By introducing recurrent connections with a time-delay of d to mitigate, Gradients now diminish exponentially as a function of $\frac{\tau}{d}$ rather than τ

Leaky Units Another way to obtain paths on which the product of derivatives is close to one is to have units with linear self-connections and a weight near one on these connections. When we accumulate a running average μ_t of some value v_t by applying $\mu_t \leftarrow \alpha\mu_{t-1} + (1-\alpha)v_t$ the α parameter is an example of linear self connection from μ_{t-1} to μ_t.
When α is near one, the running average remembers information about the past for a long time, and when α is near zero, information about the past is rapidly discarded.

16.3.5 The Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU)

Purpose going further with the idea of self-loops, by making the weight of the self-loop conditioned on the context rather than fixed.
Self-loops produce paths where the gradient can flow for long duration.

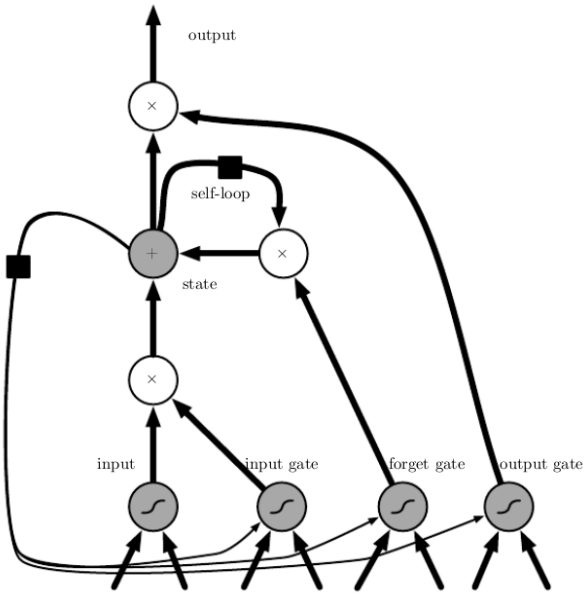


Figure 16.6: Block diagram of the LSTM recurrent neural network *cell*

Cells are connected recurrently to each other replacing the usual hidden units of ordinary recurrent networks.

Theory The **forget gate** unit $f_i^{(t)}$ for time step t and cell i that sets the weight to a value between 0 and 1

$$f_i^{(t)} = \sigma\left(b_i^f + \sum_j U_{i,j}^f x_j^{(t)} + \sum_j W_{i,j}^f h_j^{(t-1)}\right)$$

with $\boldsymbol{x}^{(t)}$ the current input vector, $\boldsymbol{h}^{(t)}$ the current hidden layer vector containing the outputs of all the LSTM cells, and $\boldsymbol{b}^f$, $\boldsymbol{U}^f$, $\boldsymbol{W}^f$ are respectively biases, input weights and recurrent weights for the forget gates.

The **external input gate** $g_i^{(t)}$ and the **output gate** $q_i^{(t)}$ are computed the same way, with their own parameters $\boldsymbol{b}^g, \boldsymbol{U}^g, \boldsymbol{W}^g$ and $\boldsymbol{b}^o, \boldsymbol{U}^o, \boldsymbol{W}^o$:

$$g_i^{(t)} = \sigma\left(b_i^g + \sum_j U_{i,j}^g x_j^{(t)} + \sum_j W_{i,j}^g h_j^{(t-1)}\right), \qquad q_i^{(t)} = \sigma\left(b_i^o + \sum_j U_{i,j}^o x_j^{(t)} + \sum_j W_{i,j}^o h_j^{(t-1)}\right)$$

The internal **state** $s_i^{(t)}$ has a linear self-loop whose weight is the forget gate, and the cell output is gated by $q_i^{(t)}$:

$$s_i^{(t)} = f_i^{(t)} s_i^{(t-1)} + g_i^{(t)} \sigma\left(b_i + \sum_j U_{i,j} x_j^{(t)} + \sum_j W_{i,j} h_j^{(t-1)}\right), \qquad h_i^{(t)} = \tanh\left(s_i^{(t)}\right) q_i^{(t)}$$

The state is updated additively: when $f_i^{(t)} \approx 1$ and $g_i^{(t)} \approx 0$ the information is copied unchanged, and the gradient flows through s_i without being multiplied by a weight matrix at each step. This mitigates, but does not remove, the vanishing gradient problem.

Gated Recurrent Unit (GRU) A simpler variant with no separate state, and two gates: the **update gate** $\boldsymbol{z}^{(t)}$ and the **reset gate** $\boldsymbol{r}^{(t)}$:

$$\boldsymbol{z}^{(t)} = \sigma\left(\boldsymbol{U}^z \boldsymbol{x}^{(t)} + \boldsymbol{W}^z \boldsymbol{h}^{(t-1)} + \boldsymbol{b}^z\right), \qquad \boldsymbol{r}^{(t)} = \sigma\left(\boldsymbol{U}^r \boldsymbol{x}^{(t)} + \boldsymbol{W}^r \boldsymbol{h}^{(t-1)} + \boldsymbol{b}^r\right)$$

$$\tilde{\boldsymbol{h}}^{(t)} = \tanh\left(\boldsymbol{U}\boldsymbol{x}^{(t)} + \boldsymbol{W}\left(\boldsymbol{r}^{(t)} \odot \boldsymbol{h}^{(t-1)}\right) + \boldsymbol{b}\right), \qquad \boldsymbol{h}^{(t)} = \boldsymbol{z}^{(t)} \odot \boldsymbol{h}^{(t-1)} + \left(1 - \boldsymbol{z}^{(t)}\right) \odot \tilde{\boldsymbol{h}}^{(t)}$$

A single gate $\boldsymbol{z}$ both forgets and writes (it plays the role of the forget and input gates together), so the GRU has fewer parameters than the LSTM and similar performance on many tasks.

16.4 Transformer Networks

16.4.1 Motivations

- RNNs were the state of the art for sequence in sequence modeling, as they rely on the generation of hidden states $h^{(t)}$ as a function of the previous one $h^{(t-1)}$ and the input $x^{(t)}$ they cannot be trained in with parallelization paradigm.
- Attention mechanism gained a lot in relevancy as it allows modeling dependencies regardless the position in input and output sequences.

Finally why do not build an architecture relying only on this interesting attention mechanism ?

16.4.2 Encoder and Decoder stacks

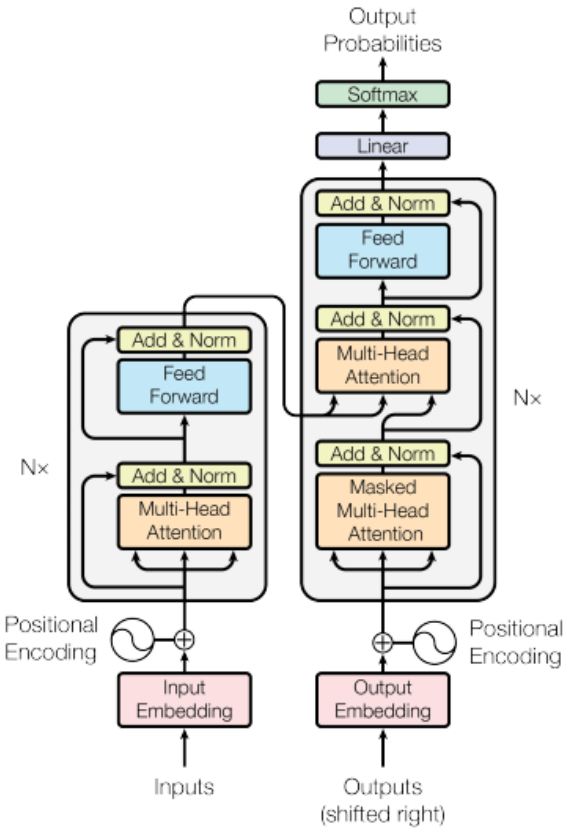


Figure 16.7: Transformer model architecture

Encoder It is composed of N (initially $N = 6$) identical layers, each one has two sub-layers:

1. multi-head self-attention mechanism
2. simple position-wise (meaning independently applied to each element of the sequence) $FFN(x) = \max(0, xW_1 + b_1)W_2 + b_2$

A residual connection is employed around each of these two sub-layers followed by a layer normalization, that is each of them has as output: $LayerNorm(x + Sublayer(x))$

Decoder Decoder relies on the same principle except that it contains a third sub-layer performing multi-head attention over the output of the encoder stack. It is worth noting that self-attention sub-layer has been modified to prevent positions from attending to subsequent positions, more precisely it is a masking. The output embedding are offset by one position ensuring that position i can only depends on position j such that $j \leq i$.

16.4.3 Attention

An *attention mechanism* can be described as mapping a query and a set of key-value pairs to an output, note that query, keys, values and output are all vectors. The output is a weighting sum of the values, while the weight associate to each of these values reflects the compatibility between the query and the key of the considered value. Consider a sequence of length n, and Let d_k, d_v and d_{model} be the dimensions of the keys, the values and the vector (usually 512) representation.

Scaled Dot-Product Attention We compute the matrix:

$$Attention\left(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}\right) = \text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^T}{\sqrt{d_k}}\right)\boldsymbol{V}$$

Step by step:

- For an input $X \in \mathcal{M}(\mathbb{R})_{n,d_{model}}$ then:
  - $Q = XW_Q$ projection matrix $W_Q \in \mathcal{M}(\mathbb{R})_{d_{model},d_k}$
  - $K = XW_K$ projection matrix $W_K \in \mathcal{M}(\mathbb{R})_{d_{model},d_k}$
  - $V = XW_V$ projection matrix $W_V \in \mathcal{M}(\mathbb{R})_{d_{model},d_v}$
- $QK^T \in \mathcal{M}(\mathbb{R})_{n,n}$ where the element located at (i,j) represents how much position i *attends to* position j.
- scaling by $\sqrt{d_k}$ is justified by the fact for large values of d_k the dot products grow large in magnitude pushing the softmax into regions where it has extremely small gradients
- applying softmax function row-wise to transform each row into a probability distribution
- multiplying by V resulting in a matrix $\mathcal{M}(\mathbb{R})_{n,d_v}$, where the element located at (i,l) represents the weighted average of all the positions at the lth representation dimension (of matrix V) from the perspective of the ith position. We talk about weighted average and not just weighted sum because the weights constitute a probability distribution of attention score associated with the ith position.

Multi-Head Attention It has been found more beneficial to avoid using a single attention function (or head) with d_{model} dimensional keys, values and queries and thus repeat the process of scaled-dot product attention h (usually $h = 8$) times in parallel with different projection matrices. Originally $d_v = d_k = \frac{d_{model}}{h} = 64$

$$MultiHead\left(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}\right) = Concat\left(\left(head_i\right)_{1\leq i\leq h}\right)P^O$$

Note that $head_i = Attention\left(\boldsymbol{Q}\boldsymbol{P}_i^Q, \boldsymbol{K}\boldsymbol{P}_i^K, \boldsymbol{V}\boldsymbol{P}_i^V\right)$ and $\begin{cases} \boldsymbol{P}_i^Q \in \mathcal{M}(\mathbb{R})_{d_{model},d_k} \\ \boldsymbol{P}_i^K \in \mathcal{M}(\mathbb{R})_{d_{model},d_k} \\ \boldsymbol{P}_i^V \in \mathcal{M}(\mathbb{R})_{d_{model},d_v} \\ \boldsymbol{P}^O \in \mathcal{M}(\mathbb{R})_{h\times d_v,d_{model}} \end{cases}$

The final projection allows the heads to interact and combine their information.
Each head works in a different subspace of dimension d_k, so the model can attend to different kinds of relations at the same time, for the same total cost as one head of dimension d_{model}.

Three uses of attention in the Transformer

- **encoder self-attention**: $\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}$ all come from the previous encoder layer, each position attends to all the input positions;
- **masked decoder self-attention**: same, but with a causal mask (see below);
- **encoder-decoder (cross) attention**: $\boldsymbol{Q}$ comes from the decoder, $\boldsymbol{K}$ and $\boldsymbol{V}$ from the encoder output, so that each target position can look at every source position. It is the attention of Bahdanau et al., which removes the fixed-size bottleneck of the RNN encoder-decoder.

16.4.4 Masking

Causal mask To generate text, position i must not see the positions $j > i$, otherwise the model could copy the answer during training. Before the softmax, a mask $\boldsymbol{M} \in \mathbb{R}^{n \times n}$ is added to the scores, with $M_{ij} = 0$ if $j \leq i$ and $M_{ij} = -\infty$ otherwise:

$$\textit{Attention}\,(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^T}{\sqrt{d_k}} + \boldsymbol{M}\right)\boldsymbol{V}$$

As $e^{-\infty} = 0$, the attention weights on future positions are exactly zero and each row still sums to one. Thanks to the mask, all the n next-token predictions of a sequence are computed and trained in parallel in a single pass (teacher forcing), whereas generation at inference is still one token at a time.

Padding mask Sequences of a batch are padded to the same length, and the padded positions are masked the same way, so that no position attends to them.

16.4.5 Positional Encoding

Purpose Self-attention is permutation-equivariant: without extra information, shuffling the tokens only shuffles the outputs the same way, so "the cat sat" and "sat cat the" are not distinguished. The order is injected by adding a position vector to each token embedding, at the bottom of the encoder and decoder stacks.

Sinusoidal encoding In the original paper, for position pos and dimension index $i \in [\![0, \frac{d_{model}}{2} - 1]\!]$:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right), \qquad PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

Each dimension is a sinusoid whose wavelength goes from 2π to $10000 \cdot 2\pi$ (geometric progression). For a fixed offset k, PE_{pos+k} is a linear function of PE_{pos} (a rotation of each $(\sin, \cos)$ pair), which makes relative positions easy to learn, and the encoding is defined for lengths not seen in training.

Alternatives

- **learned** position embeddings, one trainable vector per position: the paper reports nearly identical results, but the length is then limited to the training maximum;
- **relative** methods used in recent LLMs: *rotary* embeddings (RoPE) rotate $\boldsymbol{q}$ and $\boldsymbol{k}$ by an angle proportional to their position so that $\boldsymbol{q}^T\boldsymbol{k}$ depends on the distance $i - j$, and ALiBi adds a penalty proportional to the distance to the scores.

16.4.6 Residual Connections and Layer Normalization

Residual connection $\boldsymbol{x} \mapsto \boldsymbol{x} + \textit{Sublayer}(\boldsymbol{x})$ gives the gradient a direct path to the lower layers, the same idea as the additive state of the LSTM. It is what makes stacks of dozens of layers trainable. All the sub-layers output vectors of dimension d_{model} so that the sum is defined.

Layer normalization For one token with representation $\boldsymbol{x} \in \mathbb{R}^d$, the statistics are computed over the feature dimension of that single token:

$$\boxed{LayerNorm(\boldsymbol{x}) = \boldsymbol{\gamma} \odot \frac{\boldsymbol{x} - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \boldsymbol{\beta}}, \qquad \mu = \frac{1}{d}\sum_{k=1}^{d} x_k, \quad \sigma^2 = \frac{1}{d}\sum_{k=1}^{d} (x_k - \mu)^2$$

with learned scale $\boldsymbol{\gamma}$ and shift $\boldsymbol{\beta}$. Unlike batch normalization, it does not depend on the other examples of the batch nor on the sequence length, which suits variable-length sequences and one-token-at-a-time generation.

Post-LN and Pre-LN

- **Post-LN** (original paper): $LayerNorm\,(\boldsymbol{x} + Sublayer(\boldsymbol{x}))$. It needs a learning-rate warmup to train stably.
- **Pre-LN** (most recent LLMs): $\boldsymbol{x} + Sublayer\,(LayerNorm(\boldsymbol{x}))$. The residual path stays an identity from the input to the output, gradients are better behaved and training is more stable, often with a final LayerNorm before the output layer. RMSNorm, a cheaper variant without mean and shift, is common.

16.4.7 Feed-Forward Sub-layer

The position-wise $FFN(\boldsymbol{x}) = \max(0, \boldsymbol{x}\boldsymbol{W}_1 + \boldsymbol{b}_1)\,\boldsymbol{W}_2 + \boldsymbol{b}_2$ is a two-layer MLP applied identically to every token, with an inner dimension $d_{ff} = 4\,d_{model}$ (2048 for $d_{model} = 512$). Attention mixes information between positions, the FFN transforms each position independently, and it holds about two thirds of the parameters of a layer. Recent models replace ReLU by GELU (GPT, BERT) or by a gated variant, **SwiGLU**:

$$FFN_{SwiGLU}(\boldsymbol{x}) = (\text{Swish}(\boldsymbol{x}\boldsymbol{W}_1) \odot \boldsymbol{x}\boldsymbol{W}_3)\,\boldsymbol{W}_2, \qquad \text{Swish}(u) = u\,\sigma(u)$$

16.4.8 Embeddings and Output Layer

Input tokens are mapped to vectors of $\mathbb{R}^{d_{model}}$ by a learned embedding matrix (multiplied by $\sqrt{d_{model}}$ in the paper). At the top, a linear layer followed by a softmax over the vocabulary gives the probability of the next token. The paper shares the weight matrix between the input embedding and this pre-softmax layer (*weight tying*), which saves parameters.

16.4.9 Why Self-Attention instead of Recurrence

n is the sequence length, d the representation dimension.

Layer type	Complexity per layer	Sequential operations	Max path length
Self-attention	$O(n^2 d)$	$O(1)$	$O(1)$
Recurrent	$O(nd^2)$	$O(n)$	$O(n)$

- the path length between any two positions is constant, so long-range dependencies are as easy to learn as short ones;
- the $O(1)$ sequential operations allow the parallelization over positions that RNNs lack;
- the price is the quadratic cost $O(n^2)$ in the sequence length, in time and in memory for the score matrix, which limits the context length.

16.4.10 Training Recipe of the Original Paper

- **Adam** with $\beta_1 = 0.9$, $\beta_2 = 0.98$, $\varepsilon = 10^{-9}$ (see the optimizers section);
- learning rate with *warmup* then inverse-square-root decay, with $warmup = 4000$ steps:

$$lr = d_{model}^{-0.5} \cdot \min\left(step^{-0.5},\ step \cdot warmup^{-1.5}\right)$$

- dropout $P_{drop} = 0.1$ on each sub-layer output (before the residual sum) and on the sums of embeddings and positional encodings;
- **label smoothing** $\varepsilon_{ls} = 0.1$: the target distribution puts $1 - \varepsilon_{ls}$ on the true token and spreads the rest over the others, which hurts perplexity but improves accuracy.

16.4.11 Model Families

The same blocks give three families, depending on the stacks kept and the mask:

Family	Stacks and attention	Examples
Encoder-decoder	encoder + decoder with cross-attention	original Transformer, T5
Encoder-only	bidirectional self-attention, no mask	BERT
Decoder-only	causal self-attention only	GPT, LLaMA

Large language models are decoder-only models trained to predict the next token.

16.5 Autoencoders

16.6 Generative Adversarial Networks

16.7 Graph Neural Networks

Part V

Large Language Models

Chapter 17

Tokens and Embeddings

17.1 Tokenization

Purpose A neural network takes numbers as input. The text is cut into tokens taken from a fixed vocabulary of size V, and each token is replaced by its integer id.

- **characters**: tiny vocabulary, but very long sequences, and attention costs $O(n^2)$ in the length n;
- **words**: short sequences, but a huge vocabulary, out-of-vocabulary words, and no sharing between "run" and "running";
- **subwords**: the compromise used by LLMs. Frequent words are one token, rare words are split into several pieces, and nothing is out of vocabulary.

Byte Pair Encoding (BPE)

1. start from a vocabulary of base symbols (characters, or the 256 bytes);
2. count all the pairs of adjacent symbols in the training corpus;
3. merge the most frequent pair into a new symbol and add it to the vocabulary;
4. repeat until the vocabulary has the desired size.

Example on the string `aaabdaaabac`: the most frequent pair `aa` becomes `Z` (`ZabdZabac`), then `ab` becomes `Y` (`ZYdZYac`), then `ZY` becomes `X` (`XdXac`).
To encode a new text, the learned merges are applied in the order in which they were learned. With **byte-level** BPE the base alphabet is the 256 bytes, so any string can be encoded (GPT-2: $V = 50\,257$, that is 256 bytes, 50 000 merges and one special token).

Consequences

- the model sees tokens, not letters: spelling, counting letters or reversing a word are harder than they look;
- numbers are split into pieces, which complicates arithmetic;
- code and languages that are rare in the corpus need more tokens per word: higher cost and shorter effective context;
- **special tokens** mark the beginning or the end of a text, or the roles in a chat;
- the context length and the prices are counted in tokens (for English, a token is roughly three quarters of a word).

17.2 Embeddings

Purpose Token ids are categorical values with no notion of similarity. An embedding maps each token to a dense vector of $\mathbb{R}^{d_{model}}$ in which similar tokens can be close.

Theory The embedding matrix $\boldsymbol{E} \in \mathcal{M}(\mathbb{R})_{V,d_{model}}$ has one row per token: the vector of the token i is $\boldsymbol{e}_i^T \boldsymbol{E}$, where $\boldsymbol{e}_i$ is the one-hot vector of i, that is simply a table lookup of the row i.
$\boldsymbol{E}$ is learned jointly with the rest of the network by back-propagation, the gradient being non-zero only for the rows of the tokens present in the batch.
The idea comes from the *distributional hypothesis*: the meaning of a word is given by the contexts in which it appears (word2vec, 2013, trained such vectors from co-occurrences).

Static versus contextual The lookup does not depend on the context: "bank" has the same vector in "river bank" and "bank account". The layers of the Transformer transform these static vectors into contextual representations through attention. The position is added separately (see positional encoding), and the same matrix $\boldsymbol{E}$ can be reused as output layer (weight tying).

Chapter 18

Language Modeling

18.1 Autoregressive Factorization

Purpose A language model is a probability distribution over sequences of tokens $\boldsymbol{x} = x_{1:n}$. By the chain rule of probability, with no approximation:

$$\mathbb{P}(x_{1:n}) = \prod_{t=1}^{n} \mathbb{P}(x_t \mid x_{1:t-1})$$

so it is enough to model the distribution of the next token given the previous ones. The network outputs a vector of logits $\boldsymbol{z}_t \in \mathbb{R}^V$ and $\mathbb{P}_{\boldsymbol{\theta}}(x_t = k \mid x_{<t}) = \text{softmax}(\boldsymbol{z}_t)_k$.

Relationship with Markov models An n-gram model assumes that the next token depends only on the $n-1$ previous ones, that is a Markov chain of order $n-1$, and estimates the probabilities by counting. The table has V^n entries: this is the curse of dimensionality, most contexts are never seen. A neural language model conditions on the whole context and shares its parameters across positions, and similar contexts get similar representations, which is how it generalizes.

18.2 Training Objective

Theory For a corpus of N token positions, the maximum likelihood estimate minimizes the cross-entropy (the average negative log-likelihood of the next token):

$$\mathcal{L}(\boldsymbol{\theta}) = -\frac{1}{N} \sum_{t=1}^{N} \log \mathbb{P}_{\boldsymbol{\theta}}(x_t \mid x_{<t})$$

This is the multi-class classification loss of the feed-forward networks (softmax and cross-entropy) applied at every position, with a vocabulary of V classes.

- it is **self-supervised**: the labels are the text itself, shifted by one position, so no annotation is needed;
- with the causal mask, the inputs $x_{1:n-1}$ give the $n-1$ predictions of $x_{2:n}$ in one parallel pass (*teacher forcing*: the true prefix is fed, not the model's own predictions);
- for the data distribution p: $H(p, q_{\boldsymbol{\theta}}) = H(p) + KL(p \,\|\, q_{\boldsymbol{\theta}})$, so minimizing the cross-entropy minimizes the KL divergence to the data, and its minimum is the entropy of the text, which is not zero.

18.3 Perplexity

Theory The standard evaluation metric is the exponential of the cross-entropy (in nats):

$$PPL = \exp(\mathcal{L}) = \left(\prod_{t=1}^{N} \mathbb{P}_{\boldsymbol{\theta}}(x_t \mid x_{<t}) \right)^{-1/N}$$

Interpretation: the model is as uncertain as if it chose uniformly among PPL tokens at each step. A uniform model over the vocabulary has $PPL = V$ and a perfect one has $PPL = 1$.

Weaknesses

- it depends on the tokenizer: two models with different vocabularies cannot be compared directly (use *bits per byte* instead);
- it depends on the domain of the evaluation text;
- a low perplexity does not mean the answers are helpful or true.

Chapter 19

Decoder-only Transformers (GPT)

19.1 Architecture

Purpose A GPT-style model keeps only the decoder stack of the Transformer, without the cross-attention (there is no encoder to attend to), and is trained for next-token prediction.

Theory Let $\boldsymbol{H}^{(0)} \in \mathcal{M}(\mathbb{R})_{n,d_{model}}$ be the token embeddings plus the positional information of the n input tokens. Each of the N identical blocks applies, in the pre-LN form:

$$\boxed{\boldsymbol{H} \leftarrow \boldsymbol{H} + MHA_{causal}\left(LayerNorm(\boldsymbol{H})\right), \qquad \boldsymbol{H} \leftarrow \boldsymbol{H} + FFN\left(LayerNorm(\boldsymbol{H})\right)}$$

After the last block, a final layer normalization and the *unembedding* give the logits of every position, and the softmax the next-token distribution:

$$\boldsymbol{Z} = LayerNorm\left(\boldsymbol{H}^{(N)}\right)\boldsymbol{E}^T \in \mathcal{M}(\mathbb{R})_{n,V}, \qquad \mathbb{P}\left(x_{t+1} \mid x_{\leq t}\right) = \text{softmax}\left(\boldsymbol{Z}_t\right)$$

where $\boldsymbol{E}$ is the embedding matrix (weight tying). The hyper-parameters are the number of layers N, the width d_{model}, the number of heads h (with $d_k = d_{model}/h$), the inner size of the FFN ($d_{ff} = 4\,d_{model}$) and the context length n_{ctx}, that is the maximum number of tokens the model can read.

19.2 Number of Parameters

Per block, with $h\,d_k = d_{model} = d$:

- attention: $\boldsymbol{W}_Q, \boldsymbol{W}_K, \boldsymbol{W}_V, \boldsymbol{W}_O$, four matrices of size $d \times d$, that is $4d^2$;
- FFN: $d \times 4d$ and $4d \times d$, that is $8d^2$;
- the biases and the layer-norm parameters are negligible.

$$\boxed{P \approx 12\,N\,d^2 + V\,d}$$

(plus $n_{ctx}\,d$ if the positions are learned). The block terms dominate for large models.

Model	N	d_{model}	V	P (formula)
GPT-2 small	12	768	50 257	$\approx$ 124M
GPT-3	96	12 288	50 257	$\approx$ 175B

For GPT-2 small: $12 \cdot 12 \cdot 768^2 \approx 85$M for the blocks and $50\,257 \cdot 768 \approx 39$M for the embeddings. The attention scores add no parameter but cost $O(n^2)$ in time and memory.

19.3 Parameters versus Context

The parameters P are fixed after training, and they are shared by all the positions. The context window n_{ctx} limits how many tokens the model can attend to at once, and anything outside it is invisible to the model. The cost of attention grows quadratically with the number of tokens in the context.

Chapter 20

Text Generation

20.1 Autoregressive Decoding

Theory At inference there is no target text, the model's own output is fed back:

1. feed the prompt $x_{1:n}$ and read the logits $\boldsymbol{z}_n$ of the last position;
2. choose a token x_{n+1} from $\text{softmax}(\boldsymbol{z}_n)$ (strategies below);
3. append it to the sequence and repeat, until an end-of-text token or a maximum length.

Contrary to training, each token needs the previous one: generation is inherently sequential.

20.2 Decoding Strategies

Greedy and beam search

- **greedy**: always take $\arg\max_k p_k$. Fast and deterministic, but not the most probable sequence overall;
- **beam search**: keep the B partial sequences with the highest total log-probability. Good when there is a single right answer (translation), but for open-ended text it gives repetitive and dull outputs.

Sampling Draw the next token at random from the distribution, which gives varied outputs.

- **temperature** $T > 0$ rescales the logits before the softmax:

$$p_k = \frac{\exp\left(z_k / T\right)}{\sum_j \exp\left(z_j / T\right)}$$

  $T = 1$ is the model's distribution, $T \to 0$ tends to greedy decoding, and $T \to \infty$ tends to the uniform distribution. A small T is more deterministic, a large T more random;

- **top-k**: keep only the k most probable tokens, renormalize, then sample;
- **top-p (nucleus)**: keep the smallest set of tokens whose cumulative probability reaches p, renormalize, then sample. The size of the set adapts: a few tokens when the model is confident, many when it is not.

These are often combined, and they make the output non-deterministic: the same prompt can give different answers.

20.3 KV Cache

Purpose In the causal self-attention, the keys and values of the previous positions do not change when a token is appended. Recomputing them at every step would repeat the same work.

Theory For each layer, the matrices $\boldsymbol{K}$ and $\boldsymbol{V}$ of the past positions are stored. To generate a new token, only its own $\boldsymbol{q}, \boldsymbol{k}, \boldsymbol{v}$ are computed, $\boldsymbol{k}$ and $\boldsymbol{v}$ are appended to the cache, and $\boldsymbol{q}$ attends to the whole cache. The attention cost of a new token falls from $O(n^2)$ to $O(n)$.
The cache memory per sequence is about

$$2 \times N \times n \times d_{model} \times (\text{bytes per value})$$

(keys and values, for each layer and each position): it grows linearly with the context and with the batch size, and becomes the memory bottleneck for long contexts. Multi-query and grouped-query attention reduce it by sharing the keys and values among several heads.

Two phases

- **prefill**: the prompt is processed in one parallel pass, which fills the cache and is limited by computation;
- **decode**: the tokens are produced one at a time, each step reading the whole cache and all the weights, which is limited by memory bandwidth.

Chapter 21

Training Large Language Models

21.1 Pretraining

Purpose Train the model from scratch on a very large corpus (web pages, books, code) with the next-token objective of the language modeling chapter. It is self-supervised, and the result is a base model: it continues a text very well, but it is not yet an assistant that follows instructions.

Ingredients

- **data**: collection, filtering and deduplication. The quality of the data matters as much as its quantity;
- the texts are tokenized and cut into windows of n_{ctx} tokens, batches contain millions of tokens;
- **AdamW** with warmup, cosine decay and gradient clipping (see the optimizers section), in reduced precision (bfloat16);
- the model and the batches are spread over thousands of accelerators (data, tensor and pipeline parallelism).

Compute For a model with N parameters trained on D tokens, the forward pass costs about $2ND$ FLOPs and the backward pass about twice more:

$$\boxed{C \approx 6\,N\,D \text{ FLOPs}}$$

Example: GPT-3, $N = 175 \cdot 10^9$ and $D \approx 300 \cdot 10^9$, gives $C \approx 3 \cdot 10^{23}$ FLOPs.

21.2 Scaling Laws

Observation (Kaplan et al., 2020) The test loss decreases as a power law of the number of parameters N, of the data D and of the compute C, smoothly over many orders of magnitude, as long as none of them is the bottleneck. It makes the loss of a large training run predictable from small runs.

Compute-optimal training (Hoffmann et al., "Chinchilla", 2022) For a fixed budget $C = 6ND$, there is a best split between N and D. They found that the parameters and the tokens should grow together, with about 20 tokens per parameter. Previous large models were undertrained (GPT-3: about 2 tokens per parameter): a 70B model trained on 1.4T tokens outperformed larger models trained with the same compute.

Caveats

- the optimum above only accounts for the *training* cost. A model that will answer a very large number of requests costs more at inference when it is bigger, so it is worth training a smaller model on more tokens than the Chinchilla rule;
- the laws predict the loss, not directly the performance on a given downstream task;
- the data is finite, repeating it many times brings less and less.

21.3 Supervised Fine-Tuning (SFT)

Purpose Turn a base model into an assistant by training it on demonstrations: pairs (prompt, desired answer) written or curated by humans (*instruction tuning*).

Theory It is the same next-token cross-entropy as in pretraining, with two differences:

- the loss is computed only on the tokens of the answer, the prompt tokens being masked out;
- the conversation is formatted with a template of special tokens marking the roles (system, user, assistant).

The dataset is orders of magnitude smaller than the pretraining corpus, and the learning rate is lower. The model mostly learns the *format and behavior* expected, using capabilities that pretraining already gave.

Weaknesses It is imitation: the model can only be as good as the demonstrations, writing good ones is expensive, and it cannot express that one correct answer is *better* than another.

21.4 Parameter-Efficient Fine-Tuning: LoRA

Purpose Updating all the parameters requires a gradient and the two Adam moments for each of them, a few times the memory of the weights, and a full copy of the model for each task. LoRA (Hu et al., 2021) freezes the pretrained weights and learns a low-rank correction.

Theory For a frozen weight matrix $\boldsymbol{W}_0 \in \mathcal{M}(\mathbb{R})_{d,k}$:

$$\boxed{\boldsymbol{h} = \boldsymbol{W}_0\boldsymbol{x} + \frac{\alpha}{r}\boldsymbol{B}\boldsymbol{A}\boldsymbol{x}, \qquad \boldsymbol{B} \in \mathcal{M}(\mathbb{R})_{d,r},\ \boldsymbol{A} \in \mathcal{M}(\mathbb{R})_{r,k},\ r \ll \min(d,k)}$$

$\boldsymbol{A}$ is initialized randomly (Gaussian) and $\boldsymbol{B}$ at zero, so that the update $\Delta\boldsymbol{W} = \boldsymbol{B}\boldsymbol{A}$ is zero at the start. Only $\boldsymbol{A}$ and $\boldsymbol{B}$ are trained: $r(d+k)$ parameters instead of dk. For $d = k = 4096$ and $r = 8$: 65 536 instead of about 16.8M, that is 0.4%. It is typically applied to the attention matrices.

Strengths

- far less memory for the gradients and the optimizer states;
- after training, $\boldsymbol{W} = \boldsymbol{W}_0 + \frac{\alpha}{r}\boldsymbol{B}\boldsymbol{A}$ can be merged: no extra latency at inference;
- one small adapter per task on top of a shared base model.

Weaknesses The hypothesis is that the useful update has a low intrinsic rank. For a task far from the pretraining distribution, it can learn less than full fine-tuning.

Chapter 22

Alignment with Human Feedback

22.1 Why Human Feedback

Purpose For open-ended tasks, it is hard to write the best answer, but easy to say which of two answers is better. Preference data is a set of triples (x, y_w, y_l): a prompt x and two answers sampled from the model, with y_w preferred to y_l by a human annotator. RLHF (reinforcement learning from human feedback) has three stages: SFT, a reward model learned from preferences, then RL against this reward model.

22.2 Reward Model

Theory A network $r_{\boldsymbol{\phi}}(x, y) \in \mathbb{R}$ is built from the SFT model, replacing the output layer by a linear layer that gives one scalar from the last token. The preferences are modeled by **Bradley-Terry**:

$$\mathbb{P}\left(y_w \succ y_l \,|\, x\right) = \sigma\left(r_{\boldsymbol{\phi}}(x, y_w) - r_{\boldsymbol{\phi}}(x, y_l)\right)$$

and it is trained by maximum likelihood:

$$\boxed{\mathcal{L}(\boldsymbol{\phi}) = -\mathbb{E}_{(x, y_w, y_l)}\left[\log \sigma\left(r_{\boldsymbol{\phi}}(x, y_w) - r_{\boldsymbol{\phi}}(x, y_l)\right)\right]}$$

It is a logistic regression on the difference of the scores, so the reward is defined up to a constant for each prompt.

22.3 RL Fine-Tuning with PPO

Theory The policy $\pi_{\boldsymbol{\theta}}$ is the language model, initialized from the SFT model, and π_{ref} is a frozen copy of it (see the RL chapter, with the prompt and the tokens generated as state, and the next token as action). The objective is

$$\boxed{\max_{\boldsymbol{\theta}} \; \mathbb{E}_{x \sim \mathcal{D},\, y \sim \pi_{\boldsymbol{\theta}}(\cdot|x)}\left[r_{\boldsymbol{\phi}}(x, y)\right] - \beta\, \mathbb{E}_x\left[\mathrm{KL}\left(\pi_{\boldsymbol{\theta}}(\cdot|x) \,\|\, \pi_{ref}(\cdot|x)\right)\right]}$$

Why the KL penalty: the reward model is only reliable close to the data it was trained on. Without the penalty, the policy finds answers that exploit its errors (*reward hacking*) and drifts to text that scores high but is bad or incoherent. The penalty keeps the model close to the SFT one, and β sets the trade-off.

Implementation

- the KL is estimated on the sampled tokens, with $\log \frac{\pi_{\boldsymbol{\theta}}(y_t|x, y_{<t})}{\pi_{ref}(y_t|x, y_{<t})}$, and is usually given as a *per-token reward* $-\beta \log \frac{\pi_{\boldsymbol{\theta}}}{\pi_{ref}}$, while $r_{\boldsymbol{\phi}}(x, y)$ is added at the last token;
- the loop is the PPO one: sample prompts, generate answers, score them, compute the advantages with a critic, and update with the clipped objective;
- four models must be held in memory: the policy, the reference, the reward model and the critic, and generation makes each step expensive.

22.4 Direct Preference Optimization (DPO)

Purpose Obtain the result of the KL-regularized objective without a reward model and without RL, with a loss on the preference data only (Rafailov et al., 2023).

Theory The objective above has a closed-form optimal policy: $\pi^*(y|x) = \frac{1}{Z(x)}\pi_{ref}(y|x)\exp\left(\frac{r(x,y)}{\beta}\right)$. Inverting it:

$$r(x,y) = \beta \log \frac{\pi^*(y|x)}{\pi_{ref}(y|x)} + \beta \log Z(x)$$

In the Bradley-Terry model only the difference of rewards appears, and the term $Z(x)$ cancels. Replacing π^* by $\pi_{\boldsymbol{\theta}}$ and maximizing the likelihood of the preferences:

$$\mathcal{L}_{DPO}(\boldsymbol{\theta}) = -\mathbb{E}_{(x,y_w,y_l)}\left[\log \sigma\left(\beta \log \frac{\pi_{\boldsymbol{\theta}}(y_w|x)}{\pi_{ref}(y_w|x)} - \beta \log \frac{\pi_{\boldsymbol{\theta}}(y_l|x)}{\pi_{ref}(y_l|x)}\right)\right]$$

It increases the probability of the preferred answer relative to the reference and decreases that of the rejected one. The language model plays the role of the reward model, with the implicit reward $\hat{r} = \beta \log \frac{\pi_{\boldsymbol{\theta}}}{\pi_{ref}}$.

Strengths and weaknesses

- simple: a supervised-like loss, two models (policy and frozen reference), no sampling and no critic;
- it is offline: it learns from a fixed dataset of answers, not from the model's own new samples, so it explores less than PPO;
- sensitive to the quality of the preferences and to β.

22.5 Verifiable Rewards and GRPO

Purpose When the answer can be checked automatically (the final result of a math problem, the unit tests of a program), the reward is computed without a learned reward model, for example $r = 1$ if the answer is correct and 0 otherwise.

GRPO (Group Relative Policy Optimization, DeepSeekMath 2024) For each prompt, sample a group of G answers with rewards $r_{1:G}$, and use as advantage of answer i its reward normalized within the group, shared by all its tokens:

$$\hat{A}_i = \frac{r_i - \text{mean}(r_{1:G})}{\text{std}(r_{1:G})}$$

The update is the clipped PPO objective plus a KL penalty to the reference. The group mean plays the role of the baseline, so no critic network is needed, which reduces the memory compared to PPO. This kind of training is used to obtain models that reason with long chains of thought.

22.6 Limitations

- **reward hacking** (Goodhart's law): optimizing a proxy too long degrades the real quality, hence the KL term and early stopping on held-out evaluations;
- the preferences are noisy and reflect the biases of the annotators;
- **sycophancy** and **verbosity**: humans tend to prefer answers that agree with them and longer answers, and the model learns it;
- the alignment shapes the behavior but gives no guarantee on the truthfulness of the answers.

Chapter 23

Retrieval-Augmented Generation

23.1 Principle

Purpose The knowledge of an LLM is frozen in its weights at the end of training: it is outdated, it does not contain private documents, and the model can state false facts with confidence (*hallucination*). **Retrieval-Augmented Generation** (RAG, Lewis et al., 2020) retrieves the relevant passages at query time and puts them in the prompt, so that the model answers from them. The knowledge is changed by updating the index, with no retraining, and the sources can be cited.

Pipeline

1. **indexing** (offline): cut the documents into *chunks*, compute an embedding vector for each chunk, store them in a vector index;
2. **retrieval** (online): embed the query, find the k nearest chunks;
3. **generation**: build a prompt with the instruction, the retrieved chunks and the question, and let the model answer.

23.2 Embeddings for Retrieval

Similarity A text embedding model (a Transformer encoder, with pooling of the token vectors into one vector) maps a query $\boldsymbol{q}$ and a chunk $\boldsymbol{d}$ to $\mathbb{R}^m$, and their relevance is the **cosine similarity**

$$\cos(\boldsymbol{q}, \boldsymbol{d}) = \frac{\boldsymbol{q}^T \boldsymbol{d}}{\|\boldsymbol{q}\| \, \|\boldsymbol{d}\|}$$

which equals the dot product when the vectors are normalized.

Contrastive training The model is trained so that a query is close to its relevant chunk d^+ and far from the others. With a batch of pairs, the other chunks of the batch are used as negatives (*InfoNCE*) and τ is a temperature:

$$\mathcal{L} = -\log \frac{\exp\left(\cos(\boldsymbol{q}, \boldsymbol{d}^+)/\tau\right)}{\sum_j \exp\left(\cos(\boldsymbol{q}, \boldsymbol{d}_j)/\tau\right)}$$

It is the softmax cross-entropy where the classes are the chunks of the batch.

23.3 Vector Search

Exact search compares the query with the N vectors, in $O(Nm)$: too slow for millions of chunks. **Approximate nearest-neighbor** (ANN) indexes trade a little recall for a large speed-up:

- **IVF**: partition the vectors with k-means, and search only the few clusters closest to the query;

- **HNSW**: a multi-layer proximity graph, the search walks greedily from coarse to fine layers;
- **product quantization**: compress the vectors to save memory.

23.4 Chunking

The size of the chunks is a trade-off:

- too small: a chunk loses the context needed to be understood, for example the subject of a sentence;
- too large: the embedding averages several topics and is less precise, and the chunks use up the context window.

Typical sizes are a few hundred tokens, with an *overlap* between consecutive chunks, or a cut at the structure of the document (sections, paragraphs).

23.5 Hybrid Search and Reranking

BM25 (lexical) Dense embeddings capture meaning but can miss exact terms (names, identifiers, codes). BM25 scores the words shared by the query and the document d:

$$\text{BM25}(q,d) = \sum_{t \in q} IDF(t) \cdot \frac{f(t,d)\,(k_1+1)}{f(t,d) + k_1 \left(1 - b + b\frac{|d|}{avgdl}\right)}$$

$f(t,d)$ is the frequency of the term t in d, IDF gives more weight to rare terms, and the denominator saturates the frequency and normalizes by the length ($k_1 \approx 1.2$ to 2, $b = 0.75$).

Hybrid Run both searches and merge the rankings, for example with *reciprocal rank fusion*: $score(d) = \sum_i \frac{1}{c + rank_i(d)}$ ($c \approx 60$), which needs no calibration between the different scores.

Reranking A **cross-encoder** reads the query and a chunk together in a Transformer and outputs a relevance score. It is more accurate than comparing two independent embeddings but too slow for the whole collection, so it is applied to the top 100 of the first stage to keep the best few.

23.6 Evaluation

- **retrieval**: *recall@k* (share of the relevant chunks found in the top k), *precision@k*, MRR and nDCG for the quality of the ranking;
- **generation**: *faithfulness* (is the answer supported by the retrieved chunks?) and *relevance* (does it answer the question?).

The two stages must be evaluated separately, to know whether a wrong answer comes from a retrieval miss or from the generation.

23.7 Failure Modes

- the relevant chunk is not retrieved (bad chunking, vocabulary mismatch);
- irrelevant chunks distract the model, which may follow them;
- *lost in the middle*: models use information at the beginning and at the end of a long context better than in the middle, so the order of the chunks matters;
- **prompt injection**: a retrieved document can contain instructions, which the model may treat as if they came from the user.

23.8 RAG, Fine-tuning and Long Context

- **RAG**: for facts that change or are private, with citations. The model does not learn a new style or skill;
- **fine-tuning**: for behavior, format and style, not for storing many facts;
- **long context**: put all the documents in the prompt when they fit, without an index, at a cost that grows with the length (attention is $O(n^2)$) and with the risk of lost-in-the-middle.

Chapter 24

LLM Agents

24.1 From a Model to an Agent

Purpose A language model only produces text. An **agent** is an LLM placed in a loop with tools (search, code execution, files, APIs) that acts on an environment over several steps to reach a goal, using the results of its actions to decide the next one.
Distinction: in a **workflow** the steps are fixed by the code (the LLM fills each step), in an **agent** the LLM itself decides the steps and the tools. The agent is more flexible but slower, more expensive and less predictable, so the simplest solution (a single call, possibly with retrieval) should be tried first.

24.2 Tool Use

Theory Each tool is described to the model by a *name*, a *description* and a *schema* of its parameters (JSON), placed in the prompt.

1. the model decides to call a tool and outputs a structured call: the name and the arguments;
2. the application, not the model, validates and executes it;
3. the result is appended to the conversation, and the model continues.

The model never executes anything: this is the point where permissions, validation and sandboxing are enforced. The behavior is learned by fine-tuning on examples of calls (Toolformer trained a model to insert API calls in text by itself), and standards such as the Model Context Protocol (MCP) describe how tools and data sources are connected to models.

Weaknesses The model may call a tool that does not exist, give invalid or invented arguments, or misread the result: calls must be validated, and errors returned to the model so that it can correct itself.

24.3 The Agent Loop: ReAct

Theory ReAct (Yao et al., 2022) interleaves *reasoning* and *acting*: the model writes a thought, an action, receives an observation, and repeats.

```
context ← goal + tool descriptions
repeat until a final answer or a maximum number of steps:
  output ← LLM(context)
  if output is a tool call: result ← execute it; append output and result to context
  else: return output
```

Reinforcement learning view This is the setting of the RL chapter: the *state* is the history in the context, the *action* is a text or a tool call, the *observation* is the tool output, and the *policy* is the LLM. The state is only partially observed (the world is not entirely in the context), and the reward arrives at the end (the task is solved or not). Training the model with verifiable rewards on such tasks is an application of the GRPO or PPO methods.

24.4 Planning, Reflection and Multiple Agents

- **planning**: decompose the goal in sub-tasks, either first (plan then execute) or step by step, and revise the plan from the observations;
- **reflection**: the model critiques its own result or the error message, and tries again;
- **orchestrator and workers**: a main agent delegates sub-tasks to other calls, each one with its own short context, and collects their results.

24.5 Memory and Context

- **short-term memory**: the context window, limited to n_{ctx} tokens. Every step sends the whole history again, and the KV cache of the common prefix avoids recomputing it;
- **long-term memory**: notes or documents stored outside and retrieved when needed (see RAG);
- when the window fills, the old steps are *summarized* or the large tool outputs are removed: a bad summary loses information the agent will need.

24.6 Reliability and Evaluation

Errors compound If each step succeeds with probability p independently, a task of n steps succeeds with probability about p^n: for $p = 0.95$ and $n = 20$ it is $0.95^{20} \approx 0.36$. Long horizons need reliable steps, checks and recovery.

Metrics

- the **success rate** on tasks with an automatic check (tests that pass, the final state of the environment);
- *pass@k*: the probability that at least one of k attempts succeeds. From n attempts of which c succeed, the unbiased estimate is $1 - \binom{n-c}{k} \Big/ \binom{n}{k}$;
- the **cost**: tokens, number of steps and tool calls, latency.

Typical failure modes are loops without progress, a wrong tool or arguments, a hallucinated result, and an error early in the run that propagates.

24.7 Safety

- **indirect prompt injection**: instructions hidden in a web page, a file or a tool output are read by the model as if they were legitimate, and can make it take actions the user did not want. The text of tool results must be treated as data, not as instructions;
- **least privilege**: give the agent only the tools and the permissions the task needs, and run the code in a sandbox;
- **human in the loop** before irreversible or external actions (sending, paying, deleting);
- limits on the number of steps and on the budget, and logs of every call to audit what the agent did.

Part VI

Overall Issues

Chapter 25

High Dimensions

25.0.1 Dimension Increase

25.0.2 Curse of dimensionality

Purpose When the dimensionality increases, the volume of the space increases so fast that the available data become sparse.

Main issues

Combinatorics Let's consider $(d, k) \in \mathbb{N}^*$ and $(x_i)_{1 \leq i \leq d} \in [\![1, K]\!]^d$, the range of possible values for the families $(x_i)_{1 \leq i \leq d}$ is K^d which increases obviously exponentially.